POLITECHNIKA LUBELSKA

Materiały do przedmiotu:

GENERATYWNA SZTUCZNA INTELIGENCJA I PROMPT ENGINEERING

Instrukcje laboratoryjne

Autor:

dr inż. Krzysztof Dziedzic

dr Marcin Barszcz

Katedra Informatyki

Fundusze Europejskie dla Rozwoju Społecznego

Rzeczpospolita Polska

Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

POLITECHNIKA LUBELSKA

Spis treści

Laboratorium nr 1. Konfiguracja środowiska laboratoryjnego. Wybór zintegrowanego środowiska programistycznego (IDE). Wybór modelu ---------- 3

Laboratorium nr 2. Testowanie lokalnych i zewnętrznych modeli. Tworzenie promptów – kwestia kreatywności, kontekstu, personalizacji, iteracji. Analiza przypadków ---------- 18

Laboratorium nr 3. -- Analiza wpływu różnych parametrów dla modelu na jakość generowanych odpowiedzi. Analiza losowości odpowiedzi -- 47

Laboratorium nr 4. Praktyczne tworzenie promptów z wykorzystaniem podstawowych metod promptowania ---------- 71

Laboratorium nr 5. ---------- Praktyczne tworzenie promptów z wykorzystaniem zaawansowanych metod promptowania ---------- 97

Laboratorium nr 6. ---------- Tworzenie promptów dla różnych zadań, takich jak generowanie tekstu, obrazu, wideo lub prezentacji ---------- 119

Laboratorium nr 7. Ocena wyników i optymalizacja promptów. Test A/B ------- 139

Laboratorium nr 8. ---------- Tworzenie UI w Streamlit: wejście, wybór modelu, temperatura, zachowanie pamięci rozmowy. Generowanie i użycie API key ---------- 155

Laboratorium nr 9. Tworzenie chatbota wykorzystującego AI ---------- 171

Laboratorium nr 10. ---------- Tworzenie asystenta AI do wspomagania nauki ---------- 187

Laboratorium nr 11. ---------- Tworzenie asystenta AI do programowania ---------- 205

Laboratorium nr 12. ---- Tworzenie aplikacji wspomaganych AI do automatyzacji zadań ---------- 220

Fundusze Europejskie dla Rozwoju Społecznego | Rzeczpospolita Polska | Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

Materiały do przedmiotu:

GENERATYWNA SZTUCZNA INTELIGENCJA I PROMPT ENGINEERING

Laboratorium nr 1. Konfiguracja środowiska laboratoryjnego. Wybór zintegrowanego środowiska programistycznego (IDE). Wybór modelu

Autor:

dr inż. Krzysztof Dziedzic

dr Marcin Barszcz

Katedra Informatyki

Fundusze Europejskie dla Rozwoju Społecznego

Rzeczpospolita Polska

Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

Spis treści

Wprowadzenie, tematyka zajęć ---------- 5
Cel ćwiczenia/laboratorium ---------- 6
Instrukcja laboratoryjna/ćwiczeniowa ---------- 7
Ćwiczenia samodzielne ---------- 14
Pytania pomocnicze ---------- 15
Podsumowanie ---------- 16
Literatura ---------- 17

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

Wprowadzenie, tematyka zajęć

Laboratorium z przedmiotu „Generatywna sztuczna inteligencja i prompt engineering" wprowadza w praktyczne wykorzystanie dużych modeli językowych (LLM) szczególnie w pracy inżyniera oprogramowania. Uczy tworzyć i optymalizować zapytania kierowane do modeli generatywnej sztucznej inteligencji.

Generatywna sztuczna inteligencja (Generative AI, Gen AI) to klasa modeli uczenia maszynowego, które na podstawie danych wejściowych (promptu) tworzą nowe treści (tekst, obraz, dźwięk lub kod) zamiast jedynie klasyfikować lub przewidywać istniejące dane. Modele uczą się statystycznych zależności w ogromnych zbiorach danych treningowych, dzięki czemu potrafią generować odpowiedzi przypominające ludzkie wytwory. W obszarze przetwarzania języka naturalnego najważniejszą grupą modeli generatywnych są duże modele językowe (LLM), które przewidują kolejne tokeny na podstawie kontekstu i w ten sposób tworzą spójne wypowiedzi [1, 2].

Modele językowe LLM (Large Language Models) to zaawansowane modele sztucznej inteligencji wytrenowane na ogromnych zbiorach danych tekstowych. Najpopularniejsze modele to GPT (OpenAI), Claude (Anthropic), Gemini (Google) oraz modele open source takie jak Llama (Meta) czy Mistral. Dostęp do modeli odbywa się poprzez interfejsy webowe (np. ChatGPT, Claude.ai) lub API (interfejs programistyczny aplikacji) [1, 3].

API (Application Programming Interface) to zestaw protokołów i narzędzi umożliwiających programistyczną interakcję z modelami LLM. Klucz API (API key) to unikalny identyfikator umożliwiający uwierzytelnienie i autoryzację żądań do serwisu. W przypadku pracy z modelami komercyjnymi klucz API należy przechowywać bezpiecznie i nie udostępniać go publicznie [4].

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

Środowisko programistyczne (IDE) to zintegrowane narzędzie służące do tworzenia, edycji, testowania i debugowania kodu. W kontekście pracy z generatywną sztuczną inteligencją najczęściej wykorzystywane są środowiska takie jak Visual Studio Code (VS Code), PyCharm lub Google Colab. Umożliwiają one pisanie kodu w języku Python, który jest standardem w dziedzinie AI [5].

Python to język programowania powszechnie wykorzystywany w dziedzinie sztucznej inteligencji. Biblioteki takie jak openai, Anthropic, Google Generative AI oraz framework LangChain umożliwiają łatwą integrację z modelami językowymi i tworzenie zaawansowanych aplikacji AI [6].

Google Colab to darmowe środowisko notebooków Jupyter działające w chmurze Google. Nie wymaga instalacji oprogramowania na komputerze lokalnym i oferuje bezpłatny dostęp do zasobów obliczeniowych, w tym do GPU [7].

Ollama to narzędzie umożliwiające pobieranie i uruchamianie dużych modeli językowych (LLM) lokalnie, na własnym komputerze, bez korzystania z usług chmurowych ani z klucza API. Umożliwia pracę z modelami open source (m.in. Llama, Mistral, Gemma) w trybie offline, co jest przydatne, gdy istotna jest prywatność danych lub gdy zależy nam na uniknięciu kosztów wywołań API. Wydajność zależy jednak od zasobów sprzętowych komputera (ilości pamięci RAM oraz obecności karty graficznej).

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

Cel ćwiczenia/laboratorium

Głównym celem laboratorium jest **samodzielne przygotowanie i weryfikacja środowiska programistycznego do pracy z modelami generatywnymi** oraz wykonanie pierwszej interakcji programistycznej z modelem językowym. Po ukończeniu zajęć student potrafi:

- zainstalować i skonfigurować środowisko Python (3.10+) z edytorem Visual Studio Code lub równoważnie przygotować środowisko chmurowe Google Colab;
- utworzyć izolowane środowisko wirtualne (venv) i zarządzać zależnościami projektu za pomocą menedżera pakietów pip oraz pliku requirements.txt;
- wygenerować w usłudze GroqCloud (console.groq.com) klucz API i bezpiecznie przechować go poza kodem źródłowym, stosując zmienne środowiskowe;
- zainstalować oficjalny pakiet groq i wykonać pierwsze żądanie do modelu poprawnie odczytując odpowiedź usługi;
- wyjaśnić rolę kluczowych pojęć: IDE, LLM, API, klucz API, token oraz parametry generacji (temperatura, top-p, top-k, max_output_tokens).

Instrukcja laboratoryjna

Zasady pracy na laboratorium i oznaczenia w sprawozdaniu

Przygotowanie do zajęć laboratoryjnych może być realizowane przez studenta na podstawie wykładów oraz zalecanej literatury. Dodatkowo w każdym laboratorium umieszczono krótkie wprowadzenie dotyczące realizowanej tematyki.

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

Podczas zajęć studenci wykonują poszczególne zadania laboratoryjne. Każdy student pracuje indywidualnie. Instrukcje zawierają szczegółowy opis zadań, które mają zostać wykonane podczas danego laboratorium. Instrukcje zawierają również zadania do samodzielnego wykonania. Po ukończeniu instrukcji student odpowiada na pytania lub wykonuje polecenia oznaczone literą **P** w formie sprawozdania. Formę oddawania sprawozdań i wykonanych zadań ustala prowadzący.

Zadanie 1.1. Instalacja i konfiguracja środowiska Python + VS Code

Podstawowym środowiskiem kursu jest Python w wersji 3.10 lub nowszej wraz z edytorem Visual Studio Code. Studenci mogą wybrać pracę lokalną (instalacja na własnym komputerze lub komputerze laboratoryjnym) albo chmurową (Zadanie 1.2).

Treść zadania:

1. Pobierz i zainstaluj Python 3.10+ z oficjalnej strony `python.org`. W instalatorze Windows zaznacz opcję **„Add Python to PATH"**.
2. Zainstaluj edytor Visual Studio Code ze strony `code.visualstudio.com`.
3. W VS Code zainstaluj rozszerzenie **Python** (wydawca: Microsoft) z panelu Extensions (`Ctrl+Shift+X`).
4. Utwórz katalog roboczy, np. `C:\GenAI_Lab` lub `~/genai_lab` i otwórz go w VS Code (**File ▸ Open Folder**).
5. Otwórz wbudowany terminal (**Terminal ▸ New Terminal**) i utwórz oraz aktywuj środowisko wirtualne, a następnie zweryfikuj wersję interpretera:

```
# utworzenie izolowanego środowiska wirtualnego
python -m venv .venv

# aktywacja  (Windows PowerShell)
.venv\Scripts\Activate.ps1

# aktywacja  (Linux / macOS)
source .venv/bin/activate

# weryfikacja wersji interpretera
python --version
```

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

6. Otwórz panel rozszerzeń Extensions (Ctrl+Shift+X). Wyszukaj i zainstaluj dwa rozszerzenia „Python” oraz „Jupyter”.

Środowisko wirtualne (venv) izoluje zależności projektu od systemowej instalacji Pythona, co jest standardem profesjonalnej pracy i zapobiega konfliktom wersji bibliotek między projektami.

Zadanie 1.2. Alternatywa chmurowa – Google Colab

Google Colab to środowisko notatników Jupyter działające w przeglądarce z zarządzanym runtime'em. Nie wymaga instalacji oprogramowania ani konfiguracji lokalnej. Jest to zalecana ścieżka dla osób pracujących na komputerach bez uprawnień administratora.

Treść zadania:

1. Otwórz stronę `colab.research.google.com` i zaloguj się kontem Google.
2. Utwórz nowy notatnik (**File ▸ New notebook**) i zmień jego nazwę na `Lab1_Prompt_Engineering`.
3. W pierwszej komórce umieść i uruchom (`Shift+Enter`) kod diagnostyczny:

```
import sys, platform
print('Python:', sys.version)
print('Platforma:', platform.platform())
```

Zadanie 1.3. Instalacja bibliotek do pracy z modelami LLM

Do pracy z modelami zainstalujemy oficjalny pakiet klienta Groq oraz bibliotekę pomocniczą do wczytywania zmiennych środowiskowych. Opcjonalnie, na potrzeby alternatywnej ścieżki Gemini, można zainstalować pakiet Google GenAI SDK.

Treść zadania:

1. W terminalu VS Code (lub w komórce Colab z przedrostkiem `!`) zainstaluj wymagane pakiety:

```
pip install -U groq python-dotenv
```


**Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00**

```
pip install -U google-genai          # opcjonalnie, dla alternatywnej ścieżki Gemini
```

2. Utrwal listę zależności projektu w pliku `requirements.txt`:

```
pip freeze > requirements.txt
```

3. Zweryfikuj instalację, tworząc plik `test_imports.py` o treści i uruchamiając go w terminalu poleceniem `python test_imports.py`:

```
try:
    from groq import Groq
    import dotenv
    print('OK - wszystkie biblioteki zaimportowane poprawnie.')
except ImportError as e:
    print(f'Blad importu: {e}')
```

Zadanie 1.4. Konfiguracja dostępu do modelu i uwierzytelnianie

W tym zadaniu skonfigurujemy programistyczny dostęp do modelu. Dostęp przez API wymaga uwierzytelnienia przy użyciu klucza API wygenerowanego w usłudze GroqCloud. Punkt końcowy udostępnia modele open source (m.in. Llama 3.3 70B) – w zadaniach laboratoryjnych wykorzystywany będzie m.in. model llama-3.3-70b-versatile, zapewniający niską latencję i wysoką przepustowość.

Opcja A (zalecana): GroqCloud – dostęp do modeli w chmurze

GroqCloud to chmurowa usługa udostępniająca modele open source (m.in. Llama 3.3 70B oraz Llama 3.1 8B) przez API o bardzo niskiej latencji. Dostęp uzyskuje się po wygenerowaniu klucza API i zainstalowaniu dedykowanego pakietu klienta.

1. Wejdź na console.groq.com i zaloguj się kontem Google lub GitHub (albo utwórz nowe konto).

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

2. Wybierz zakładkę API Keys ▸ Create API Key, nadaj kluczowi nazwę i skopiuj go – pełna wartość klucza wyświetlana jest tylko raz.
3. W katalogu roboczym w pliku .env zapisz klucz (bez cudzysłowów, bez spacji wokół znaku =):

```
GROQ_API_KEY=twoj_wygenerowany_klucz_api
```

4. Zainstaluj oficjalny pakiet klienta Groq w aktywnym środowisku wirtualnym:

```
pip install groq
```

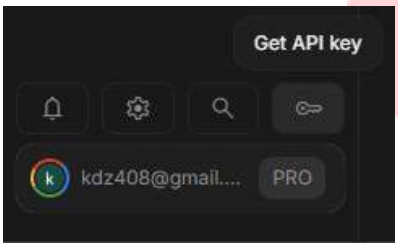
5. Zabezpiecz klucz – utwórz plik `.gitignore` i dodaj do niego wpisy, aby klucz nie trafił do repozytorium:

```
.env
.venv/
__pycache__/
```

Opcja B (alternatywna): Google AI Studio – generowanie klucza API

1. Wejdź na `aistudio.google.com` i zaloguj się kontem Google.
2. Wybierz **Get API key** i wygeneruj nowy klucz (rysunek 1.1), a następnie skopiuj go.
3. W katalogu roboczym utwórz plik `.env` i zapisz w nim klucz (bez cudzysłowów, bez spacji wokół znaku =). Zapisz plik .env: File > Save.

```
GEMINI_API_KEY=twoj_wygenerowany_klucz_api
```



Rysunek 1.1. API key.

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

4. Zabezpiecz klucz – utwórz plik `.gitignore` i dodaj do niego wpisy, aby klucz nie trafił do repozytorium:

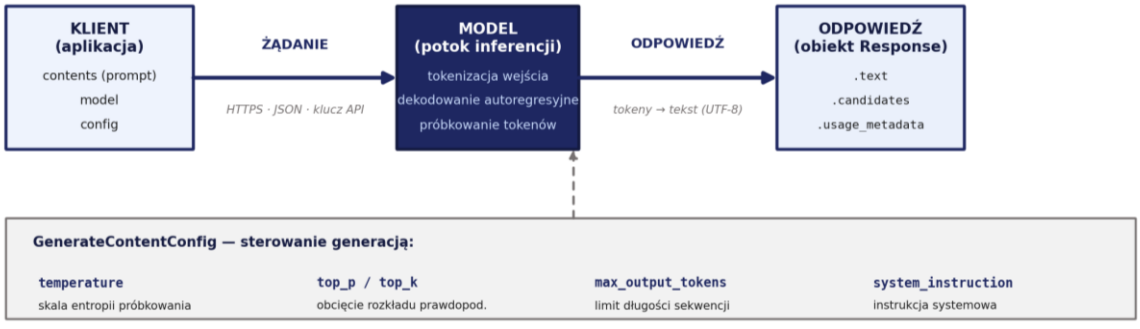
```
.env
.venv/
__pycache__/
```

Opcja C (bez API): interfejs webowy Gemini / ChatGPT / Claude

Jeżeli z przyczyn technicznych nie uda się wygenerować klucza API, dopuszczalne jest przeprowadzenie pierwszych testów przez interfejs webowy: `gemini.google.com`, `chat.openai.com` lub `claude.ai`. Ścieżka webowa nie wymaga kodu, ale nie umożliwia programistycznego sterowania parametrami generacji. W kolejnych zadaniach kursu wykorzystywany będzie dostęp przez API.

Zadanie 1.5. Programistyczna interakcja z modelem

Zadanie polega na wykonaniu pierwszego żądania do modelu za pomocą pakietu klienta Groq (dla ścieżki alternatywnej – Google GenAI SDK). Schemat pojedynczego żądania (od aplikacji, przez warstwę uwierzytelnienia, po model i odpowiedź) wraz z parametrami sterującymi generacją przedstawia rysunek 1.2.



Rysunek 1.2. Budowa pojedynczego żądania do API modelu językowego oraz ustawień sterujących generacją odpowiedzi.

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

Treść zadania (ścieżka API – Groq; alternatywnie Gemini):

1. Utwórz plik test_groq.py o następującej treści:

```
import os
from dotenv import load_dotenv
from groq import Groq

# wczytanie klucza z pliku .env do zmiennej srodowiskowej
load_dotenv()

# odczyt klucza API z pliku .env
api_key = os.getenv('GROQ_API_KEY')
if not api_key:
    raise SystemExit('Nie znaleziono klucza. Sprawdz, czy plik
.env znajduje sie w tym katalogu i czy zawiera GROQ_API_KEY.')

client = Groq(api_key=api_key)

response = client.chat.completions.create(
    model='openai/gpt-oss-120b',
    messages=[
        {'role': 'system', 'content': 'Odpowiadaj zwiezle, w
jezyku polskim.'},
        {'role': 'user', 'content': 'Witaj! Kim jesteś? Opisz
krótko swoje możliwości.'},
    ],
    temperature=0.7,
    max_tokens=1000,
)
print(response.choices[0].message.content)
```

2. Utwórz plik `test_gemini.py` o następującej treści (wariant alternatywny dla usługi Gemini):

```
import os
from dotenv import load_dotenv
from google import genai
from Google. genai import types
```

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

```
# wczytanie klucza z pliku .env do zmiennej srodowiskowej
load_dotenv()

# odczyt klucza API z pliku .env
api_key = os.getenv('GEMINI_API_KEY')
if not api_key:
    raise SystemExit('Nie znaleziono klucza. Sprawdź, czy plik
.env znajduje się w tym katalogu i czy zawiera GEMINI_API_KEY.')

client = genai.Client(api_key=api_key)

response = client.models.generate_content(
    model='gemini-2.5-flash',
    contents='Witaj! Kim jesteś? Opisz krótko swoje
możliwości.GenerateContentConfig(
        system_instruction='Odpowiadaj zwiezle, w jezyku
polskim.',
        temperature=0.7,
        max_output_tokens=1000,
        thinking_config=types.ThinkingConfig(thinking_budget=0),
    ),
)
print(response.text)
```

3. Uruchom skrypt poleceniem `python test_groq.py` lub `python test_gemini.py` i obserwuj odpowiedź modelu w terminalu.

Parametry sterujące generacją działają analogicznie w obu usługach: system_instruction (w Groq przekazywana jako wiadomość o roli „system") wyznacza rolę i kontekst modelu, temperature reguluje losowość odpowiedzi, a max_output_tokens (w Groq: max_tokens) ogranicza jej długość. Parametry te będą szczegółowo analizowane w Laboratorium 3.

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

Treść zadania (ścieżka webowa – alternatywnie):

1. Zaloguj się do wybranego interfejsu webowego i wpisz prompt: „Witaj! Kim jesteś? Opisz swoje możliwości w kontekście pomocy w nauce prompt engineeringu.”
2. Zaobserwuj i przeanalizuj odpowiedź modelu.

Zadanie 1.6. Instalacja Ollama – lokalne uruchamianie modeli

Treść zadania:

1. Pobierz i zainstaluj Ollama z oficjalnej strony ollama.com (dostępne wersje dla systemów Windows, macOS i Linux).
2. Zweryfikuj poprawność instalacji, sprawdzając wersję narzędzia w terminalu:

```
ollama --version
```

3. Pobierz i uruchom wybrany model, np. model Llama 3.1. Przy pierwszym uruchomieniu model zostanie automatycznie pobrany na dysk:

```
ollama run llama3.1
```

4. Po pojawieniu się znaku zachęty wpisz przykładowy prompt „Witaj! Kim jesteś? Opisz krótko swoje możliwości.” i obserwuj odpowiedź modelu generowaną lokalnie. Aby zakończyć sesję, wpisz polecenie /bye.
5. Wyświetl listę modeli pobranych i dostępnych lokalnie:

```
ollama list
```

Ollama uruchamia lokalny serwer dostępny domyślnie pod adresem http://localhost:11434. Dzięki temu z modelem można komunikować się także programistycznie za pomocą biblioteki Ollama dla języka Python lub bezpośrednio przez REST API. W odróżnieniu od dostępu przez API usługi chmurowej, praca lokalna nie wymaga klucza API i działa w trybie offline.

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

Ćwiczenia samodzielne

Ćwiczenie 1.1 (podstawowe). Parametryzacja generacji

Zmodyfikuj skrypt `test_groq.py` tak, aby ten sam prompt został wywołany trzykrotnie z różnymi wartościami parametru `temperature` (np. 0.0, 0.7 oraz 1.5). Zapisz uzyskane odpowiedzi i porównaj ich zróżnicowanie.

P: Zamieść w sprawozdaniu trzy odpowiedzi i sformułuj wniosek dotyczący wpływu temperatury na determinizm oraz kreatywność wyników.

Ćwiczenie 1.2 (średnie). Interaktywny skrypt z pętlą i obsługą błędów

Napisz program, który w pętli pobiera prompt od użytkownika (funkcja `input()`), wysyła go do modelu i wypisuje odpowiedź, aż użytkownik wpisze „koniec”. Zadbaj o obsługę wyjątków – przechwyć błąd `APIError` z modułu `google.genai.errors` i wyświetl czytelny komunikat zamiast przerywać działanie programu.

P: Dołącz kod programu oraz zrzut ekranu z przykładową sesją (co najmniej 3 zapytania i poprawne zakończenie pętli).

Ćwiczenie 1.3 (trudne). Porównanie strategii promptowania

Wykorzystując parametr `system_instruction`, przygotuj dwie różne „persony” modelu (np. lakoniczny ekspert techniczny vs. nauczyciel tłumaczący prostym językiem) i zadaj im to samo pytanie o wybrane pojęcie z informatyki. Zmierz długość odpowiedzi w tokenach, odczytując pole `usage_metadata` z obiektu odpowiedzi.

P: Zestaw obie odpowiedzi w tabeli (persona, liczba tokenów wyjściowych, ocena przydatności) i skomentuj, jak instrukcja systemowa wpływa na styl odpowiedzi oraz na liczbę generowanych tokenów.

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

**Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00**

Pytania pomocnicze

1. Czym jest zintegrowane środowisko programistyczne (IDE) i jakie są najczęściej wykorzystywane IDE w pracy z modelami generatywnymi?
2. Wymień co najmniej trzy duże modele językowe (LLM) wraz z ich producentami. Które z nich są dostępne w wersji open source?
3. Czym jest API oraz klucz API i jaką rolę pełnią w programistycznym dostępie do modeli LLM? Dlaczego klucza API nie wolno umieszczać w kodzie źródłowym?
4. Jakie są zalety środowiska Google Colab w porównaniu z lokalną instalacją Pythona? Kiedy wybór ścieżki lokalnej jest korzystniejszy?
5. Do czego służy środowisko wirtualne (venv) oraz plik requirements.txt w projekcie wykorzystującym biblioteki AI?
6. Wyjaśnij rolę pliku .env oraz wpisu .env w pliku .gitignore w kontekście bezpiecznego przechowywania danych dostępowych.
7. Jaki jest wpływ parametru temperature na charakter odpowiedzi modelu? Jakie wartości wybrać dla zadań wymagających powtarzalności, a jakie dla zadań kreatywnych?

Podsumowanie

Po ukończeniu pierwszego laboratorium student dysponuje kompletnym, powtarzalnym środowiskiem programistycznym do pracy z modelami generatywnymi. W szczególności:

- ma skonfigurowane środowisko programistyczne (Python z VS Code lub Google Colab) wraz z izolowanym środowiskiem wirtualnym i utrwaloną listą zależności;
- ma zainstalowany aktualny pakiet google-genai oraz skonfigurowany dostęp do modelu gemini-2.5-flash przez klucz API z Google AI Studio;

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

- potrafi wykonać programistyczne żądanie do modelu i poprawnie odczytać odpowiedź, a także świadomie stosuje podstawowe parametry sterujące generacją;
- stosuje dobre praktyki bezpieczeństwa: rozdziela konfiguracje od kodu (.env) i chroni sekretne dane przed publikacją (.gitignore);
- rozumie i potrafi wyjaśnić pojęcia: IDE, LLM, API, klucz API.

Zbudowane stanowisko laboratoryjne stanowi podstawę do realizacji kolejnych zajęć laboratoryjnych. Dodatkowe narzędzia i biblioteki niezbędne do realizacji kolejnych zadań będą dodawane w kolejnych laboratoriach.

UWAGA dotyczqca bezpieczeństwa: klucza API nie wolno udostępniać publicznie (np. w repozytorium GitHub bez zabezpieczeń). Plik `.env` musi być zawsze ujęty w pliku `.gitignore`. W przypadku ujawnienia klucza należy niezwłocznie unieważnić go w Google AI Studio i wygenerować nowy klucz.

Literatura

[1]. Alto V., Moch W., Generatywna sztuczna inteligencja z ChatGPT i modelami OpenAI. Podnieś swoją produktywność i innowacyjność za pomocą GPT-3 i GPT-4, Helion, Gliwice 2024.
[2]. Kaplan J., Generatywna AI, Wydawnictwo Naukowe PWN, Warszawa 2025.
[3]. Tunstall L., von Werra L., Wolf T., Przetwarzanie języka naturalnego z wykorzystaniem transformerów, Helion, Gliwice 2023.
[4]. Google, Gemini API – Quickstart, dokumentacja oficjalna, https://ai.google.dev/gemini-api/docs/quickstart (dostęp: 2026).
[5]. Microsoft, Visual Studio Code – Python in VS Code, https://code.visualstudio.com/docs/languages/python (dostęp: 2026).
[6]. Python Software Foundation, Python 3 – dokumentacja i pobieranie, https://www.python.org/ (dostęp: 2026).

Fundusze Europejskie dla Rozwoju Społecznego | Rzeczpospolita Polska | Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

[7]. Google Research, Google Colaboratory – środowisko notatników, https://colab.research.google.com/ (dostęp: 2026).
[8]. Google, Gemini API Libraries – Google GenAI SDK, https://ai.google.dev/gemini-api/docs/libraries (dostęp: 2026).
[9]. Google, Strategie promptowania (Prompting strategies), https://ai.google.dev/gemini-api/docs/prompting-strategies (dostęp: 2026).
[10]. Google, Gemini API – Rate limits (limity i przydziały zapytań), https://ai.google.dev/gemini-api/docs/rate-limits (dostęp: 2026).
[11]. Google AI Studio, panel generowania kluczy API, https://aistudio.google.com/app/apikey (dostęp: 2026).
[12]. Google GenAI Python SDK – repozytorium i dokumentacja pakietu, https://github.com/googleapis/python-genai (dostęp: 2026).

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

POLITECHNIKA LUBELSKA

Materiały do przedmiotu:

GENERATYWNA SZTUCZNA INTELIGENCJA I PROMPT ENGINEERING

Laboratorium nr 2.
Testowanie lokalnych i zewnętrznych modeli. Tworzenie promptów – kwestia kreatywności, kontekstu, personalizacji, iteracji. Analiza przypadków

Autor:

dr inż. Krzysztof Dziedzic

dr Marcin Barszcz

Katedra Informatyki

Fundusze Europejskie dla Rozwoju Społecznego

Rzeczpospolita Polska

Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

POLLUB z nami nowoczesne technologie

POLITECHNIKA LUBELSKA

Fundusze Europejskie dla Rozwoju Społecznego

Rzeczpospolita Polska

Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

POLITECHNIKA LUBELSKA

Spis treści

Wprowadzenie, tematyka zajęć ---------- 20
Cel ćwiczenia/laboratorium ---------- 23
Instrukcja laboratoryjna/ćwiczeniowa ---------- 24
Ćwiczenia samodzielne ---------- 42
Pytania pomocnicze ---------- 43
Podsumowanie ---------- 44
Literatura ---------- 45

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

Wprowadzenie, tematyka zajęć

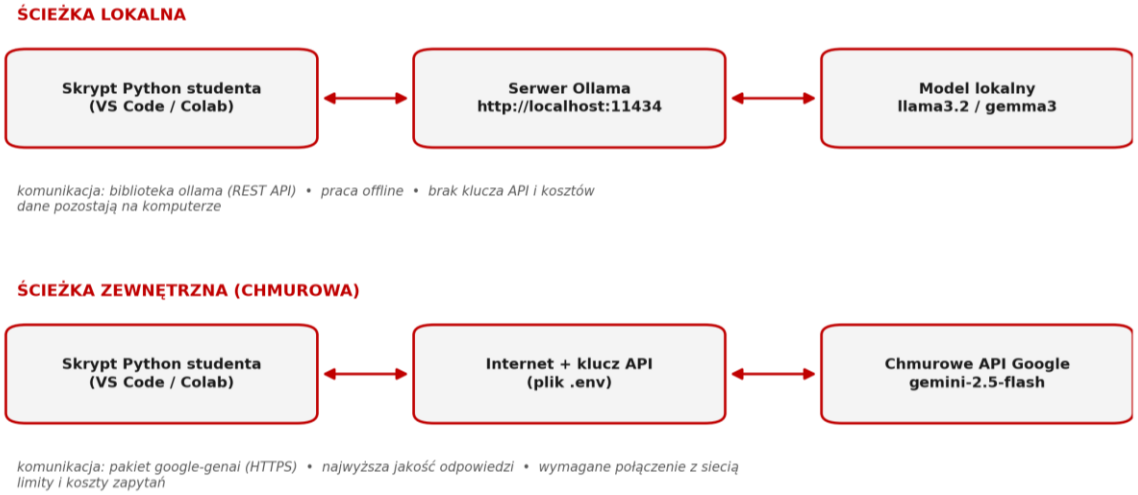
Niniejsze laboratorium stanowi kontynuację Laboratorium 1, w którym przygotowano kompletne stanowisko pracy: interpreter Python z edytorem Visual Studio Code (lub środowisko Google Colab), pakiet groq z dostępem do modelu openai/gpt-oss-120b za pomocą klucza API oraz narzędzie Ollama umożliwiające lokalne uruchamianie modeli językowych. Na tak przygotowanym stanowisku przeprowadzone zostaną testy modeli lokalnych i zewnętrznych, a następnie ćwiczenia z projektowania promptów, uwzględniające cztery kluczowe aspekty: kreatywność, kontekst, personalizację oraz iteracyjne doskonalenie [1, 2].

Modele lokalne to modele językowe uruchamiane bezpośrednio na komputerze użytkownika. Narzędzie Ollama pobiera skompresowane (skwantyzowane) wersje modeli open source (m.in. Llama, Gemma, Mistral) i udostępnia je zarówno w trybie interaktywnym w terminalu, jak i programistycznie przez bibliotekę `ollama` dla języka Python lub interfejs REST działający domyślnie pod adresem `http://localhost:11434`. Zaletami pracy lokalnej są: pełna kontrola nad danymi (treść promptów nie opuszcza komputera), brak kosztów wywołań API, niezależność od połączenia z internetem oraz brak limitów zapytań. Ograniczeniami są wymagania sprzętowe (ilość pamięci RAM, obecność karty GPU) oraz niższa jakość odpowiedzi mniejszych modeli w porównaniu z największymi modelami komercyjnymi [3].

Modele zewnętrzne (chmurowe) działają na serwerach dostawców (Google, OpenAI, Anthropic, Groq) i są udostępniane przez interfejsy webowe (Gemini, ChatGPT, Claude.ai) lub programistycznie za pośrednictwem API. Ich zaletami są dostęp do najnowszych, najbardziej sprawnych modeli, brak wymagań sprzętowych po stronie użytkownika oraz regularne aktualizacje. Ograniczenia to: konieczność połączenia z internetem, limity i koszty zapytań oraz przesyłanie danych do zewnętrznego dostawcy. Aspekt ten jest istotny wszędzie tam, gdzie przetwarzane są dane osobowe, dane firmowe lub dane objęte tajemnicą. Obie ścieżki dostępu wykorzystywane w niniejszym laboratorium przedstawia rysunek 2.1 [4].

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00



Rysunek 2.1. Dwie ścieżki dostępu do modeli językowych wykorzystywane w laboratorium: lokalna (Ollama) oraz zewnętrzna (API w chmurze).

Porównywanie modeli. Rzetelne porównanie modeli wymaga zdefiniowania kryteriów oceny oraz użycia identycznych promptów testowych. Do typowych kryteriów należą: poprawność merytoryczna, zgodność z instrukcją (np. z limitem długości lub zadanym formatem), klarowność i dostosowanie do odbiorcy, jakość języka polskiego, czas odpowiedzi (latencja) oraz koszt. W badaniach porównawczych stosuje się sformalizowane benchmarki (np. MMLU dla wiedzy ogólnej czy HumanEval dla generowania kodu, a w nowszych zestawieniach m.in. MMLU-Pro i LiveCodeBench), jednak w zastosowaniach praktycznych podstawą oceny jest test na własnych, reprezentatywnych zadaniach. Pojedynczy prompt nigdy nie wystarcza do wiarygodnej oceny modelu [11].

Tokeny i koszt użycia modeli. Token to podstawowa jednostka tekstu przetwarzana przez model językowy. Najczęściej jest to fragment słowa lub całe krótkie słowo. W języku angielskim jeden token odpowiada średnio około 4 znakom. Teksty w języku polskim tokenizują się zwykle mniej efektywnie, tzn. ta sama treść wymaga więcej tokenów. Liczba tokenów ma trzy praktyczne konsekwencje: określa koszt użycia komercyjnego API (dostawcy rozliczają oddzielnie tokeny

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

wejściowe – prompt – oraz wyjściowe – odpowiedź, podając ceny za milion tokenów), podlega limitom zapytań oraz musi mieścić się w oknie kontekstu modelu. Groq API zwraca w każdej odpowiedzi pole usage ze statystykami użycia (prompt_tokens, completion_tokens, total_tokens). Ollama zwraca analogiczne pola prompt_eval_count i eval_count. Każdy model korzysta z własnego tokenizatora, dlatego ten sam tekst może mieć różną liczbę tokenów w różnych modelach [5, 6].

Struktura skutecznego promptu. Prompt to tekst wejściowy kierowany do modelu, który definiuje zadanie i kontekst interakcji. Skuteczny prompt zwykle zawiera cztery elementy: rolę (perspektywę, z której model ma odpowiadać, np. „Jesteś doświadczonym nauczycielem programowania"), zadanie (jednoznaczne polecenie, np. „Wyjaśnij pojęcie rekurencji"), kontekst (niezbędne informacje, np. „dla studenta pierwszego roku informatyki") oraz format (oczekiwaną formę odpowiedzi, np. „w punktach, z przykładem kodu"). Dokumentacja IBM rozróżnia dodatkowo instrukcje bezpośrednie, otwarte i zadaniowe – im bardziej doprecyzowana jest instrukcja, tym większa jest zgodność wyniku z oczekiwaniami. Dane wejściowe (tekst do streszczenia, kod do analizy, treść zgłoszenia) warto oddzielać od polecenia ogranicznikami (ang. delimiters), np. potrójnym cudzysłowem """ lub znacznikami ###, co jednoznacznie rozdziela instrukcję od przetwarzanego materiału [1, 7].

Kreatywność w promptach. Kreatywne podejście polega na eksperymentowaniu z formą polecenia, zachowując przy tym ten sam cel. Do sprawdzonych technik należą: analogie („wyjaśnij działanie sieci neuronowej przez analogię do pracy zespołu ekspertów"), storytelling (osadzenie wyjaśnienia w narracji), zmiana perspektywy („jak opisałby ten problem administrator sieci, a jakby opisał go audytor bezpieczeństwa?") oraz wizualizacja słowna. Kreatywne sformułowanie promptu potrafi radykalnie zmienić przystępność i zapamiętywalność odpowiedzi przy niezmienionej treści merytorycznej [8].

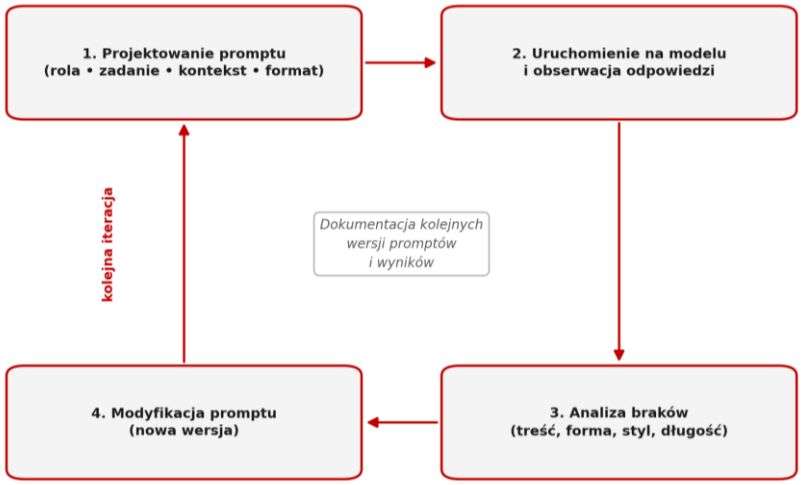
Personalizacja. Personalizacja polega na dostosowaniu promptu do konkretnego odbiorcy: jego poziomu wiedzy, słownictwa, celu oraz kontekstu branżowego lub

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

**Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00**

kulturowego. Ten sam temat można wyjaśnić menedżerowi bez wykształcenia technicznego, studentowi pierwszego roku i doświadczonemu inżynierowi – każdorazowo w innym języku, z innymi przykładami i na innym poziomie szczegółowości. W dostępie programistycznym personalizację najwygodniej realizuje instrukcja systemowa (parametr `system_instruction`), która wyznacza trwałą rolę i styl modelu, niezależnie od treści kolejnych pytań użytkownika [1].

Iteracyjne doskonalenie promptów oraz analiza przypadków. Pierwsza wersja promptu rzadko daje optymalny wynik. Iteracyjne doskonalenie to cykl: (1) projektowanie promptu, (2) uruchomienie go na modelu i obserwacja odpowiedzi, (3) analiza braków (treść, forma, styl, długość), (4) modyfikacja promptu i ponowny test (rysunek 2.2). Dobrą praktyką inżynierską jest dokumentowanie kolejnych wersji promptów wraz z oceną ich wyników. W ten sposób powstaje własna biblioteka sprawdzonych rozwiązań. Analiza przypadków (case study), czyli praca na realnych scenariuszach z praktyki informatycznej i biznesowej, pozwala połączyć wszystkie powyższe aspekty w kompletny proces inżynierii promptów [1, 8].



Rysunek 2.2. Cykl iteracyjnego doskonalenia promptu.

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

Cel ćwiczenia/laboratorium

Głównym celem laboratorium jest nabycie praktycznych umiejętności testowania i porównywania modeli językowych (lokalnych i zewnętrznych) oraz świadomego projektowania promptów z wykorzystaniem kreatywności, kontekstu, personalizacji i iteracyjnego doskonalenia. Po ukończeniu zajęć student potrafi:

1. uruchomić lokalny model językowy za pomocą narzędzia Ollama i komunikować się z nim programistycznie z poziomu języka Python;
2. przeprowadzić systematyczne porównanie odpowiedzi modelu lokalnego i zewnętrznego według zdefiniowanych kryteriów (jakość merytoryczna, zgodność z instrukcją, czas odpowiedzi);
3. zliczać tokeny promptu i odpowiedzi (pole usage: prompt_tokens i completion_tokens) oraz szacować koszt użycia komercyjnego API na podstawie cennika dostawcy;
4. konstruować prompty zawierające rolę, zadanie, kontekst i format oraz oddzielać dane wejściowe ogranicznikami;
5. stosować technikę few-shot, dobierając spójne przykłady par wejście–wyjście w celu poprawy trafności i powtarzalności formatu odpowiedzi;
6. stosować kreatywne techniki formułowania promptów (analogia, storytelling, zmiana perspektywy) oraz personalizować odpowiedzi dla różnych odbiorców, m.in. za pomocą instrukcji systemowej;
7. prowadzić iteracyjne doskonalenie promptu, dokumentować jego kolejne wersje i weryfikować wygenerowany kod testami jednostkowymi;
8. przeprowadzić kompleksową analizę przypadku – od zdefiniowania problemu po ocenę przydatności uzyskanego rozwiązania.

Instrukcja laboratoryjna/ćwiczeniowa

Zadanie 2.1. Rozbudowa środowiska – programistyczny dostęp do modelu lokalnego

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

W Laboratorium 1 model lokalny obsługiwany był wyłącznie w trybie interaktywnym w terminalu. W zadaniach porównawczych potrzebny będzie dostęp programistyczny z poziomu skryptu w Pythonie. Ollama udostępnia w tym celu oficjalną bibliotekę `ollama`, która komunikuje się z lokalnym serwerem pod adresem `http://localhost:11434`.

Treść zadania:

1. Upewnij się, że usługa Ollama działa, a model z Laboratorium 1 jest dostępny – wykonaj w terminalu polecenia:

```
ollama --version
ollama list
```

2. Aktywuj środowisko wirtualne projektu (`.venv`), doinstaluj bibliotekę kliencką i zaktualizuj listę zależności projektu:

```
pip install -U ollama
pip freeze > requirements.txt
```

3. Pobierz drugi, lekki model lokalny, który posłuży do porównań w ćwiczeniach samodzielnych, a następnie ponownie wyświetl listę modeli:

```
ollama pull gemma3:1b
ollama list
```

4. Utwórz plik `test_ollama.py` o poniższej treści i uruchom go poleceniem `python test_ollama.py`:

```
import ollama

response = ollama.chat(
    model='llama3.1',
    messages=[
        {'role': 'system','content':
         'Odpowiadaj zwięźle, w języku polskim.'},
        {'role': 'user',
         'content': 'Czym jest prompt engineering? '
                    'Odpowiedz w trzech zdaniach.'},
    ],
)
```

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

```
print(response['message']['content'])
```

Zwróć uwagę, że struktura wywołania – lista wiadomości z rolami `system` i `user` – jest analogiczna do interfejsów modeli chmurowych. Dzięki temu ten sam prompt można w prosty sposób kierować do różnych modeli i porównywać wyniki.

Zadanie 2.2. Porównanie modelu lokalnego i zewnętrznego jednolitym promptem

Zadanie polega na wysłaniu identycznego promptu do modelu zewnętrznego (openai/gpt-oss-120b przez Groq API) oraz lokalnego (llama3.1 przez Ollama), zmierzeniu czasu odpowiedzi i ocenie wyników według wspólnych kryteriów. Prompt testowy zawiera już elementy struktury, tj.: rolę, zadanie, kontekst oraz ograniczenie długości.

Treść zadania:

1. W katalogu roboczym utwórz plik `porownanie_modeli.py` o następującej treści:

```
import os
import sys
import time
from dotenv import load_dotenv
from groq import Groq
from groq import APIError
import ollama

# poprawne wyświetlanie polskich znaków w konsoli Windows
sys.stdout.reconfigure(encoding='utf-8')

load_dotenv()
client = Groq(api_key=os.getenv('GROQ_API_KEY'))

# nazwy modeli w jednym miejscu - latwiej je podmienic, gdy
dostawca
# wycofa model (patrz uwaga pod skryptem)
MODEL = 'openai/gpt-oss-120b'
```

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

```
MODEL_LOKALNY = 'llama3.1'

PROMPT = ('Jesteś ekspertem od sztucznej inteligencji. '
          'Wyjaśnij w 3-4 zdaniach, czym jest transformer '
          'w kontekście modeli językowych. Użyj prostego języka, '
          'zrozumiałego dla studenta pierwszego roku
informatyki.')

# --- model zewnetrzny: Groq (API) ---
print(f'=== {MODEL} (model zewnetrzny) ===')
try:
    start = time.perf_counter()
    response = client.chat.completions.create(
        model=MODEL,
        messages=[{'role': 'user', 'content': PROMPT}],
        temperature=0.7,
        max_tokens=1000,
    )
    t_groq = time.perf_counter() - start
    print(response.choices[0].message.content)
    print(f'Czas odpowiedzi: {t_groq:.2f} s')
except APIError as e:
    # np. blad 429 (przekroczony limit zapytan) lub problem z
kluczem/siecia
    print(f'[Blad Groq API] {e}')
    print('Pomijam Groq i przechodze do modelu lokalnego.')

# --- model lokalny: Ollama ---
print(f'=== {MODEL_LOKALNY} (model lokalny) ===')
try:
    start = time.perf_counter()
    local = ollama.chat(model=MODEL_LOKALNY,
                        messages=[{'role': 'user', 'content':
PROMPT}])
    t_llama = time.perf_counter() - start
    print(local['message']['content'])
    print(f'Czas odpowiedzi: {t_llama:.2f} s')
```

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

```
except Exception as e:
    # np. serwer Ollama nie dziala lub brak pobranego modelu
    print(f'[Blad Ollama] {e}')
    print(f'Upewnij sie, ze serwer Ollama dziala i model
{MODEL_LOKALNY} '
          f'jest pobrany (ollama pull {MODEL_LOKALNY}).')
```

Uwaga: modele udostępniane w chmurze mają ograniczony cykl życia – dostawcy okresowo wycofują starsze modele i zastępują je nowszymi. Dlatego nazwy modeli są przechowywane w stałych MODEL i MODEL_LOKALNY na początku skryptu. Po wycofaniu modelu wystarczy zmienić wartość stałej. Aktualny harmonogram wyłączeń modeli Groq: https://console.groq.com/docs/deprecations.

2. Pamiętaj o zapisaniu pliku (Ctrl+S). Uruchom skrypt poleceniem `python porownanie_modeli.py` i zapisz obie odpowiedzi oraz zmierzone czasy. Skrypt uruchom dwukrotnie: pierwszy pomiar modelu lokalnego obejmuje także czas ładowania modelu do pamięci (tzw. zimny start), dlatego do tabeli 2.1 wpisz czasy z drugiego uruchomienia.
3. Ten sam prompt wprowadź w co najmniej jednym interfejsie webowym (`claude.ai`, `chatgpt.com` lub `gemini.google.com`) i zapisz odpowiedź.
4. Oceń wszystkie uzyskane odpowiedzi w skali 1–5 według kryteriów przedstawionych w tabeli 2.1.

Tabela 2.1. Kryteria porównania odpowiedzi modeli (ocena w skali 1–5; czas w sekundach)

Kryterium	**openai/gpt-oss-120b'**	**llama3.1 (lokalnie)**	**model webowy (nazwa)**
Poprawność merytoryczna			
Zgodność z limitem długości (3–4 zdania)			
Dostosowanie języka do odbiorcy			
Poprawność językowa (polszczyzna)			

Kryterium	openai/gpt-oss-120b'	llama3.1 (lokalnie)	model webowy (nazwa)
Czas odpowiedzi [s]			–

P: Dodaj komentarz: który model poradził sobie najlepiej i dlaczego? Czy niższa jakość odpowiedzi modelu lokalnego jest w tym zadaniu akceptowalna, biorąc pod uwagę prywatność danych i brak kosztów?

Zadanie 2.3. Zliczanie tokenów i szacowanie kosztów użycia modeli

Rozliczenia komercyjnych API oraz limity zapytań opierają się na tokenach, dlatego umiejętność ich zliczania jest niezbędna przy szacowaniu kosztów wdrożeń. W zadaniu odczytasz liczbę tokenów promptu i odpowiedzi z pola usage zwracanego przez API, obliczysz koszt pojedynczego zapytania według cennika dostawcy oraz porównasz tokenizację tekstu polskiego i angielskiego. Uwaga: w darmowej warstwie Groq API nie ponosisz opłat; obowiązują natomiast limity liczby zapytań. Poniższe obliczenia dotyczą płatnej warstwy, stosowanej w zastosowaniach produkcyjnych.

Treść zadania:

1. Sprawdź aktualne ceny modelu openai/gpt-oss-120b w cenniku Groq API i w razie potrzeby zaktualizuj stałe CENA_WE i CENA_WY w poniższym skrypcie (ceny podano w USD za 1 mln tokenów, stan na 2026 r.).
2. Utwórz plik `licz_tokeny.py` o następującej treści i uruchom go poleceniem `python licz_tokeny.py`

```
import os
import sys
from dotenv import load_dotenv
from groq import Groq
from groq import APIError

# poprawne wyswietlanie polskich znakow w konsoli Windows
sys.stdout.reconfigure(encoding='utf-8')
```

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

**Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00**

```
load_dotenv()
client = Groq(api_key=os.getenv('GROQ_API_KEY'))

MODEL = 'openai/gpt-oss-120b'
# ceny platnej warstwy API w USD za 1 mln tokenow (stan na 2026
r.)
CENA_WE = 0.15
CENA_WY = 0.60

PROMPT = ('Jesteś ekspertem od sztucznej inteligencji. '
          'Wyjaśnij w 3-4 zdaniach, czym jest tokenizacja '
          'w kontekście modeli językowych. Użyj prostego języka, '
          'zrozumiałego dla studenta pierwszego roku
informatyki.')

try:
    # wyslanie zadania i odczyt statystyk z pola usage
    response = client.chat.completions.create(
        model=MODEL,
        messages=[{'role': 'user', 'content': PROMPT}],
        temperature=0.7,
        max_tokens=1000,
    )
    print(response.choices[0].message.content)
    tok_we = response.usage.prompt_tokens
    tok_wy = response.usage.completion_tokens
    print(f'Tokeny wejsciowe: {tok_we} | tokeny wyjsciowe:
{tok_wy}')

    # szacunkowy koszt pojedynczego zapytania
    koszt = (tok_we * CENA_WE + tok_wy * CENA_WY) / 1_000_000
    print(f'Szacunkowy koszt zapytania: {koszt:.6f} USD')
except APIError as e:
    # np. blad 429 (przekroczony limit zapytan) lub problem z
kluczem/siecia
    print(f'[Blad Groq API] {e}')
```

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

3. Rozszerz skrypt o porównanie tokenizacji języka polskiego i angielskiego – dopisz na końcu i uruchom ponownie:

```
# porownanie tokenizacji: polski vs angielski
zdania = {
    'PL': 'Sztuczna inteligencja zmienia współczesną
informatykę.',
    'EN': 'Artificial intelligence is changing modern computing.',
}
for jezyk, tekst in zdania.items():
    r = client.chat.completions.create(model=MODEL,
        messages=[{'role': 'user', 'content': tekst}],
max_tokens=1)
    print(f'{jezyk}: {r.usage.prompt_tokens} tokenów |
{len(tekst)} znaków')
```

4. Model lokalny również raportuje zużycie tokenów. Pola prompt_eval_count i eval_count w odpowiedzi Ollama. Dopisz poniższy fragment i porównaj wynik z tokenizacją modelu w Groq (każdy model używa własnego tokenizatora):

```
# tokeny w modelu lokalnym (Ollama)
import ollama
local = ollama.chat(model='llama3.1',
                    messages=[{'role': 'user', 'content':
PROMPT}])
print('Tokeny wejściowe (Ollama):', local['prompt_eval_count'])
print('Tokeny wyjściowe (Ollama):', local['eval_count'])
```

Uwaga: przy ponownym wysłaniu identycznego promptu Ollama korzysta z pamięci podręcznej i może pominąć pole `prompt_eval_count` w odpowiedzi. Wystarczy wtedy nieznacznie zmienić treść promptu albo ponownie załadować model.

5. Uzupełnij tabelę 2.2, wykonując pomiary dla trzech promptów: promptu o transformerze z zadania 2.2, promptu o tokenizacji z tego zadania oraz własnego promptu o długości co najmniej 100 słów.

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

**Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00**

Tabela 2.2. Zestawienie liczby tokenów i szacunkowych kosztów zapytań (model openai/gpt-oss-120b, płatna warstwa API)

Prompt testowy	Tokeny wejściowe	Tokeny wyjściowe	Koszt [USD]
Prompt o transformerze (zadanie 2.2)			
Prompt o tokenizacji (licz_tokeny.py)			
Własny prompt (min. 100 słów)			

6. **Oszacowanie kosztów wdrożenia.** Przyjmij scenariusz: automatyczna klasyfikacja zgłoszeń działu IT obsługuje 500 zgłoszeń miesięcznie, a każde zapytanie zawiera około 250 tokenów wejściowych (prompt few-shot wraz ze zgłoszeniem) i około 10 tokenów wyjściowych. Oblicz miesięczny koszt. Następnie powtórz obliczenie dla scenariusza generowania raportów: 500 zapytań po 150 tokenów wejściowych i 1000 tokenów wyjściowych. Porównaj oba wyniki.

P: Sformułuj wnioski: (1) który język – polski czy angielski – tokenizuje się efektywniej i jakie ma to konsekwencje dla kosztów? (2) dlaczego liczba tokenów tego samego tekstu może różnić się między modelem w Groq a modelem lokalnym w Ollama (różne tokenizatory)? (3) który składnik kosztu dominuje przy generowaniu długich odpowiedzi i jak można go kontrolować za pomocą parametru max_tokens?

Zadanie 2.4. Struktura promptu: rola – zadanie – kontekst – format

Ćwiczenie pokazuje, jak dodanie do promptu pełnej struktury wpływa na jakość odpowiedzi. Scenariusz zadania: przygotowujesz dokumentację projektu zespołowego realizowanego w Katedrze Informatyki, dotyczącego zastosowania uczenia maszynowego do wykrywania ataków w sieci komputerowej (system klasy IDS). Potrzebne jest zwięzłe wprowadzenie do raportu technicznego.

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

Treść zadania:

1. Krok 1 – prompt bez struktury. W katalogu roboczym utwórz plik struktura_promptu.py o następującej treści:

```
import os
import sys
import time
from dotenv import load_dotenv
from groq import Groq
from groq import APIError
import ollama

# poprawne wyswietlanie polskich znakow w konsoli Windows
sys.stdout.reconfigure(encoding='utf-8')

load_dotenv()
client = Groq(api_key=os.getenv('GROQ_API_KEY'))

# testujemy dwa modele: zewnetrzny (API) i lokalny (Ollama)
MODEL = 'openai/gpt-oss-120b'
MODEL_LOKALNY = 'llama3.1'

# Krok 1 - prompt bez struktury
PROMPT_BEZ_STRUKTURY = 'Napisz wprowadzenie do raportu o
wykrywaniu ataków sieciowych.'

# Kroki 2-3 - dekompozycja i prompt ze struktura (rola + zadanie +
kontekst + format z tabeli 2.3)
ROLA = ('Jesteś doświadczonym inżynierem cyberbezpieczeństwa '
        'i autorem dokumentacji technicznej.')
PROMPT_ZE_STRUKTURA = (
    'Zadanie: napisz wprowadzenie do raportu technicznego '
    'z projektu studenckiego. '
    'Kontekst: projekt dotyczy zastosowania uczenia '
    'maszynowego do wykrywania ataków w sieci komputerowej '
    '(system klasy IDS); porównywane są klasyczne metody '
    'sygnaturowe z klasyfikatorami uczonymi na zbiorze '
    'CIC-IDS2017; odbiorcą jest nauczyciel przedmiotu '
```

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

```
    'Projekt zespołowy na kierunku informatyka. '
    'Format: około 250 słów, trzy akapity (motywacja '
    'i znaczenie problemu; krótki przegląd podejść; cel '
    'i zakres projektu); styl formalny, bez wypunktowań.'
)


def uruchom(opis, prompt, rola=None):
    print(f'\n\n{"#" * 64}\n###  ODPOWIEDZ -> {opis}\n{"#" * 64}')
    # model zewnetrzny: Groq (API)
    print(f'\n--- [{opis}] model zewnetrzny: {MODEL} ---')
    try:
        wiadomosci_api = [{'role': 'user', 'content': prompt}]
        if rola:
            wiadomosci_api.insert(0, {'role': 'system', 'content':
rola})
        start = time.perf_counter()
        response = client.chat.completions.create(
            model=MODEL,
            messages=wiadomosci_api,
            temperature=0.7,
            max_tokens=1000,
        )
        print(response.choices[0].message.content)
        print(f'Czas odpowiedzi: {time.perf_counter() - start:.2f}
s')
    except APIError as e:
        print(f'[Blad Groq API] {e}')
    # model lokalny: Ollama (rola trafia do wiadomosci systemowej)
    print(f'\n--- [{opis}] model lokalny: {MODEL_LOKALNY} ---')
    try:
        wiadomosci = []
        if rola:
            wiadomosci.append({'role': 'system', 'content': rola})
        wiadomosci.append({'role': 'user', 'content': prompt})
        start = time.perf_counter()
```

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

```
        local = ollama.chat(model=MODEL_LOKALNY,
messages=wiadomosci)
        print(local['message']['content'])
        print(f'Czas odpowiedzi: {time.perf_counter() - start:.2f}
s')
    except Exception as e:
        print(f'[Blad Ollama] {e}')
        print(f'Upewnij sie, ze serwer Ollama dziala i model
{MODEL_LOKALNY} '
              f'jest pobrany (ollama pull {MODEL_LOKALNY}).')


# Krok 1 - bez struktury (bez roli systemowej)
uruchom('KROK 1: prompt bez struktury', PROMPT_BEZ_STRUKTURY)

# Krok 3 - ze struktura; rola jako instrukcja systemowa
uruchom('KROK 3: prompt ze struktura', PROMPT_ZE_STRUKTURA,
rola=ROLA)
```

2. Zapisz plik (Ctrl+S) i uruchom skrypt poleceniem python struktura_promptu.py. Model lokalny uruchom dwukrotnie – pierwszy przebieg obejmuje ładowanie modelu do pamięci (zimny start), dlatego notuj czas z drugiego uruchomienia. Uwaga: pojedyncze uruchomienie skryptu generuje od razu odpowiedzi dla Kroku 1 (prompt bez struktury) i Kroku 3 (prompt ze strukturą) – na tym etapie analizuj wyłącznie Krok 1, do Kroku 3 wrócisz w punkcie 4. Zapisz wszystkie odpowiedzi, a następnie oceń odpowiedzi z Kroku 1 (prompt bez struktury): jaka jest ich długość i poziom szczegółowości, do jakiego odbiorcy są kierowane oraz czy nadają się do raportu bez poprawek.
3. Krok 2 – dekompozycja promptu. Cztery elementy struktury (rola, zadanie, kontekst, format) z tabeli 2.3 zostały odwzorowane w skrypcie w zmiennych ROLA i PROMPT_ZE_STRUKTURA. Prześledź to odwzorowanie; możesz zmodyfikować treść tych zmiennych, zachowując wszystkie cztery elementy.

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

Tabela 2.3. Dekompozycja promptu na elementy składowe

Element	Przykładowa treść
Rola	Jesteś doświadczonym inżynierem cyberbezpieczeństwa i autorem dokumentacji technicznej.
Zadanie	Napisz wprowadzenie do raportu technicznego z projektu studenckiego.
Kontekst	Projekt dotyczy zastosowania uczenia maszynowego do wykrywania ataków w sieci komputerowej (system klasy IDS). Porównywane są klasyczne metody sygnaturowe z klasyfikatorami uczonymi na zbiorze danych CIC-IDS2017. Odbiorcą raportu jest nauczyciel przedmiotu „Projekt zespołowy" na kierunku informatyka.
Format	Około 250 słów, trzy akapity: (1) motywacja i znaczenie problemu, (2) krótki przegląd podejść, (3) cel i zakres projektu. Styl formalny, bez wypunktowań.

4. Krok 3 – prompt ze strukturą. Odpowiedzi ze strukturą pochodzą z drugiego wywołania funkcji uruchom() w skrypcie, wykonanego na tych samych dwóch modelach co w Kroku 1. Rola została przekazana jako wiadomość o roli 'system' – zarówno dla modelu openai/gpt-oss-120b (Groq), jak i dla modelu llama3.1 (Ollama). Jeśli zmodyfikowałeś treść promptu, uruchom skrypt ponownie.
5. Porównaj odpowiedzi bez struktury (Krok 1) i ze strukturą (Krok 3) pod kątem kompletności, zgodności z zadanym formatem (liczba akapitów, długość) oraz przydatności w raporcie. Osobno dla modelu zewnętrznego (np. openai/gpt-oss-120b) oraz lokalnego (llama3.1).

P: Zamieść oba prompty oraz uzyskane odpowiedzi (dla obu modeli), a następnie porównaj je w formie krótkiej tabeli. Wskaż, który element struktury (rola, kontekst czy format) najbardziej zmienił jakość wyniku i uzasadnij swoją odpowiedź.

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

Zadanie 2.5. Kreatywne techniki formułowania promptów

Wybierz jedno trudne pojęcie informatyczne. Proponowane: rekurencja, złożoność obliczeniowa (notacja O) lub mechanizm uwagi w transformerach. Wygeneruj jego wyjaśnienie w 3 kreatywnych wariantach promptu. Wszystkie warianty uruchom na tym samym modelu. Poniższe przykłady dotyczą rekurencji; dla innego pojęcia zmodyfikuj treść, zachowując tę technikę.

Treść zadania:

1. **Wariant A – analogia:**

> *„Wyjaśnij, czym jest rekurencja, używając analogii do rosyjskich matrioszek. Zakończ jednym zdaniem, wskazując, co w tej analogii odpowiada warunkowi stopu. Maksymalnie 120 słów."*

2. **Wariant B – storytelling:**

> *„Opowiedz krótką historię (do 150 słów) o studencie informatyki, który podczas rodzinnego obiadu tłumaczy dziadkowi, czym jest rekurencja. Historia ma zawierać poprawne merytorycznie wyjaśnienie pojęcia."*

3. **Wariant C – zmiana perspektywy:**

> *„Jesteś stosem wywołań programu. Opisz w pierwszej osobie, co dzieje się z tobą podczas rekurencyjnego obliczania silni z liczby 4. Maksymalnie 120 słów."*

4. Porównaj trzy odpowiedzi pod kątem: zrozumiałości dla osoby początkującej, poprawności merytorycznej oraz atrakcyjności i zapamiętywalności wyjaśnienia.

W katalogu roboczym utwórz plik kreatywne_prompty.py o następującej treści. Skrypt uruchamia wszystkie trzy warianty (analogia, storytelling, zmiana perspektywy) na tym samym modelu i wypisuje odpowiedzi wraz z czasem:

```
import os
import sys
import time
from dotenv import load_dotenv
```

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

```
from groq import Groq
from groq import APIError
import ollama

# poprawne wyswietlanie polskich znakow w konsoli Windows
sys.stdout.reconfigure(encoding='utf-8')

load_dotenv()
client = Groq(api_key=os.getenv('GROQ_API_KEY'))

# wszystkie warianty uruchamiamy na tym samym modelu
MODEL = 'openai/gpt-oss-120b'
# aby uzyc modelu lokalnego, ustaw UZYJ_OLLAMA = True
UZYJ_OLLAMA = False
MODEL_LOKALNY = 'llama3.1'

# trzy kreatywne warianty promptu (te same, co w krokach recznych)
PROMPTY = {
    'A - analogia': (
        'Wyjaśnij, czym jest rekurencja, używając analogii '
        'do rosyjskich matrioszek. Zakończ jednym zdaniem, '
        'wskazując, co w tej analogii odpowiada warunkowi '
        'stopu. Maksymalnie 120 słów.'
    ),
    'B - storytelling': (
        'Opowiedz krótką historię (do 150 słów) o studencie '
        'informatyki, który podczas rodzinnego obiadu tłumaczy '
        'dziadkowi, czym jest rekurencja. Historia ma zawierać '
        'poprawne merytorycznie wyjaśnienie pojęcia.'
    ),
    'C - zmiana perspektywy': (
        'Jesteś stosem wywołań programu. Opisz w pierwszej '
        'osobie, co dzieje się z tobą podczas rekurencyjnego '
        'obliczania silni z liczby 4. Maksymalnie 120 słów.'
    ),
}
```

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

```python
def uruchom_groq(prompt):
    r = client.chat.completions.create(
        model=MODEL,
        messages=[{'role': 'user', 'content': prompt}],
        temperature=0.9,
        max_tokens=400,
        reasoning_effort='low',
    )
    return r.choices[0].message.content


def uruchom_ollama(prompt):
    r = ollama.chat(
        model=MODEL_LOKALNY,
        messages=[{'role': 'user', 'content': prompt}],
    )
    return r['message']['content']


for opis, prompt in PROMPTY.items():
    print(f'\n\n{"#" * 64}\n###  WARIANT {opis}\n{"#" * 64}')
    print(prompt)
    print('-' * 64)
    try:
        start = time.perf_counter()
        if UZYJ_OLLAMA:
            odpowiedz = uruchom_ollama(prompt)
        else:
            odpowiedz = uruchom_groq(prompt)
        print(odpowiedz)
        print(f'Czas odpowiedzi: {time.perf_counter() - start:.2f}
s')
    except APIError as e:
        print(f'[Blad Groq API] {e}')
    except Exception as e:
        print(f'[Blad] {e}')
```

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

Uruchom skrypt poleceniem python kreatywne_prompty.py. Dla każdego z trzech wariantów zobaczysz wygenerowane wyjaśnienie. Aby użyć modelu lokalnego zamiast Groq, ustaw w skrypcie UZYJ_OLLAMA = True; wszystkie warianty muszą działać na tym samym modelu.

P: Zamieść trzy prompty oraz trzy odpowiedzi modelu. Oceń, który wariant najlepiej tłumaczy pojęcie osobie uczącej się go po raz pierwszy i dlaczego. Czy któryś z wariantów wprowadził nieścisłość merytoryczną? Jeśli tak, wskaż ją.

Zadanie 2.6. Personalizacja promptu dla różnych odbiorców

Temat ćwiczenia: konteneryzacja aplikacji (Docker). Przygotujesz trzy wersje promptu wyjaśniającego to samo pojęcie dla trzech różnych odbiorców, a następnie porównasz uzyskane odpowiedzi.

Treść zadania:

1. **Wersja A – odbiorca biznesowy:**

> „Wyjaśnij właścicielowi małej firmy, bez wykształcenia technicznego, czym jest konteneryzacja aplikacji (Docker) i jakie korzyści biznesowe daje jej wdrożenie. Użyj codziennej analogii i unikaj żargonu technicznego. Maksymalnie 100 słów."

2. **Wersja B – student pierwszego roku informatyki:**

> „Wyjaśnij studentowi pierwszego roku informatyki, czym jest kontener Dockera i czym różni się od maszyny wirtualnej. Użyj podstawowych pojęć z systemów operacyjnych i podaj jedno przykładowe polecenie docker run. Około 150 słów."

3. **Wersja C – doświadczony administrator systemów:**

> „Przedstaw doświadczonemu administratorowi systemów zwięzłe porównanie kontenerów i maszyn wirtualnych na poziomie izolacji zasobów (przestrzenie nazw, cgroups). Wskaż dwa typowe zastosowania produkcyjne i jedno ryzyko bezpieczeństwa. Około 150 słów, forma zwięzłych akapitów."

4. Uruchom wszystkie trzy prompty na tym samym modelu i porównaj odpowiedzi pod kątem słownictwa, użytych przykładów oraz poziomu szczegółowości.

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

5. **Ścieżka API (opcjonalnie)** – zrealizuj personalizację pętlą po „personach” zdefiniowanych jako instrukcje systemowe:

W katalogu roboczym utwórz plik personalizacja.py. Na podstawie skryptów z Zadań 2.1–2.5 dopisz nagłówek pliku (importy, wczytanie klucza API przez dotenv oraz utworzenie klienta Groq), a następnie umieść poniższy kod:

```
MODEL = 'openai/gpt-oss-120b'

PERSONY = {
    'biznes':  'Jesteś doradcą biznesowym. Tłumaczysz technologie
'
               'językiem korzyści, bez żargonu technicznego.',
    'student': 'Jesteś asystentem dydaktycznym na kierunku
informatyka. '
               'Tłumaczysz precyzyjnie, z prostymi przykładami.',
    'admin':   'Jesteś starszym inżynierem DevOps. '
               'Odpowiadasz technicznie i zwięźle.',
}
PYTANIE = 'Czym jest konteneryzacja aplikacji (Docker)?'

for nazwa, instrukcja in PERSONY.items():
    response = client.chat.completions.create(
        model=MODEL,
        messages=[
            {'role': 'system', 'content': instrukcja},
            {'role': 'user', 'content': PYTANIE},
        ],
        max_tokens=1000,
    )
    print(f'=== {nazwa.upper()} ===')
    print(response.choices[0].message.content, end='\n\n')
```

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

Uruchom skrypt poleceniem python personalizacja.py i porównaj odpowiedzi wygenerowane dla trzech osób.

P: Zamieść trzy odpowiedzi oraz analizę różnic między nimi (słownictwo, przykłady, struktura, poziom szczegółowości). Oceń w 2–3 zdaniach, czy personalizacja była skuteczna dla każdego z odbiorców.

Zadanie 2.7. Analiza przypadku: iteracyjne doskonalenie promptu

Studium przypadku łączy poznane elementy w spójny proces inżynierii promptów. Scenariusz: dział IT uczelni potrzebuje funkcji w języku Python, która będzie walidować studenckie adresy e-mail w domenie uczelnianej. Zadaniem jest doprowadzenie (w co najmniej trzech iteracjach) do promptu, który generuje kompletny, przetestowany kod.

Treść zadania:

1. **Iteracja 1 – prompt bazowy.** Uruchom na wybranym modelu prompt:

„Napisz funkcję Python do walidacji adresu e-mail."

2. Oceń odpowiedź: czy funkcja uwzględnia domenę uczelnianą? Czy zawiera testy? Jakie przypadki brzegowe pominięto?
3. **Iteracja 2 – doprecyzowanie wymagań.** Uruchom prompt:

„Napisz funkcję Python walidującą studencki adres e-mail. Adres jest poprawny wyłącznie w domenie pollub.edu.pl. Funkcja ma używać wyrażeń regularnych i zwracać True lub False. Dodaj testy jednostkowe dla przypadków: adres poprawny, zła domena, brak znaku @, pusty łańcuch znaków."

4. Oceń ponownie: co się poprawiło, a czego nadal brakuje (np. reguły dla części lokalnej adresu, styl kodu, liczba i jakość testów)?
5. **Iteracja 3 – wersja finalna z pełną strukturą.** Uruchom prompt:

„Jesteś doświadczonym programistą Python w dziale IT uczelni. Napisz funkcję validate_student_email(email: str) -> bool z użyciem modułu re. Wymagania:
– adres jest poprawny wyłącznie w domenie pollub.edu.pl;
– część lokalna to imie.nazwisko (małe litery łacińskie, bez polskich znaków) albo sześciocyfrowy numer albumu;

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

> *– dodaj docstring z trzema przykładami użycia;*
> *– przygotuj co najmniej 8 testów jednostkowych w pytest, obejmujących m.in.: poprawny adres z nazwiskiem, poprawny adres z numerem albumu, złą domenę, brak znaku @, wielkie litery, polskie znaki, pusty łańcuch, dwie kropki w części lokalnej;*
> *– kod ma być zgodny z PEP 8 i zawierać komentarz wyjaśniający wyrażenie regularne."*

Ścieżka API – w katalogu roboczym utwórz plik doskonalenie_promptu.py i na podstawie skryptów z Zadań 2.1–2.5 (importy, wczytanie klucza API, utworzenie klienta Groq oraz funkcja uruchamiająca model) wywołaj kolejno trzy powyższe wersje promptu i wypisz odpowiedzi modelu.

6. Zapisz wygenerowaną funkcję do pliku `validate_email.py`, a testy do pliku `test_validate_email.py`. Zainstaluj framework testowy i uruchom testy:

```
pip install pytest
pytest -v
```

7. Udokumentuj cały proces w tabeli 2.4. Dla każdej iteracji wpisz zidentyfikowane braki w odpowiedziach oraz zmianę wprowadzoną w prompcie.

Tabela 2.4. Dokumentacja iteracyjnego doskonalenia promptu (przykładowy wpis w pierwszym wierszu)

Iteracja	Zidentyfikowane braki odpowiedzi	Zmiana wprowadzona w prompcie
1	np. brak wymagania domeny uczelnianej, brak testów, dowolna sygnatura funkcji	doprecyzowanie domeny, wymaganie testów jednostkowych
2		
3		– (wersja finalna)

P: Zamieść trzy wersje promptu, kluczowe fragmenty odpowiedzi modelu oraz uzupełnioną tabelę 2.4. Wskaż, które elementy dodane do promptu przyniosły największą poprawę i dlaczego.

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

Ćwiczenia samodzielne

Ćwiczenie 2.1 (podstawowe). Drugi model lokalny

Wykorzystując model `gemma3:1b` pobrany w zadaniu 2.1, powtórz porównanie z zadania 2.2 (ten sam prompt dotyczący transformera) i rozszerz tabelę 2.1 o dodatkową kolumnę z ocenami tego modelu oraz zmierzonym czasem odpowiedzi.

Ćwiczenie 2.2 (średnie). Personalizowany komunikat o przerwie technicznej

Zaprojektuj prompt o pełnej strukturze (rola, zadanie, kontekst, format), który generuje komunikat o planowanej przerwie technicznej uczelnianej platformy e-learningowej w trzech wariantach: dla studentów, wykładowców oraz administratorów IT. Każdy wariant powinien mieć inny poziom szczegółowości technicznej, ale powinien zawierać te same informacje o terminie i czasie trwania przerwy.

Ćwiczenie 2.3 (trudne). Własne studium przypadku

Zidentyfikuj rzeczywisty problem z własnej praktyki (projekt zaliczeniowy, praca dyplomowa, działalność koła naukowego, praca zawodowa), który można rozwiązać za pomocą inżynierii promptów. Przeprowadź kompletny proces: opisz problem, zaprojektuj prompt o pełnej strukturze, wykonaj co najmniej trzy udokumentowane iteracje (tabela wzorowana na tabeli 2.4), a finalny prompt przetestuj zarówno na modelu zewnętrznym, jak i lokalnym.

Pytania pomocnicze

1. Jakie są zasadnicze różnice między modelem lokalnym a zewnętrznym (chmurowym)? W jakich sytuacjach uzasadniony jest wybór modelu lokalnego mimo jego niższej jakości?

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

2. Pod jakim adresem dostępny jest lokalny serwer Ollama i jakie są sposoby programistycznej komunikacji z nim?
3. Jakie kryteria można stosować przy porównywaniu modeli językowych? Dlaczego pojedynczy prompt nie wystarcza do wiarygodnej oceny modelu?
4. Czym są token i proces tokenizacji? Dlaczego ten sam tekst ma zwykle więcej tokenów w języku polskim niż w angielskim i dlaczego liczba tokenów różni się między modelami?
5. Jak na podstawie pola usage (prompt_tokens, completion_tokens) i cennika dostawcy obliczyć koszt pojedynczego zapytania do API? W jaki sposób parametr max_tokens pomaga kontrolować koszty?
6. Wymień i scharakteryzuj cztery elementy skutecznego promptu: rolę, zadanie, kontekst i format. Podaj przykład promptu zawierającego wszystkie elementy.
7. Czym różni się podejście zero-shot od few-shot? Jaką rolę pełnią przykłady w prompcie i dlaczego powinny mieć spójny format?
8. Do czego służą ograniczniki (delimitery) w promptach i jakie znaki są w tej roli najczęściej stosowane?
9. Jakie kreatywne techniki formułowania promptów znasz (analogia, storytelling, zmiana perspektywy) i kiedy warto je stosować?
10. Na czym polega personalizacja promptu i jaką rolę w niej pełni instrukcja systemowa (wiadomość o roli 'system')?
11. Opisz cykl iteracyjnego doskonalenia promptu. Dlaczego warto dokumentować kolejne wersje promptów i ich wyniki?

Podsumowanie

Laboratorium łączy dwa wątki: testowanie modeli językowych oraz świadome projektowanie promptów. Po jego ukończeniu student:

1. potrafi komunikować się programistycznie zarówno z modelem zewnętrznym, jak i lokalnym oraz rozumie kompromis między jakością, kosztem, czasem odpowiedzi i prywatnością danych;
2. porównuje modele według zdefiniowanych kryteriów, mierząc m.in. czas odpowiedzi i zgodność z instrukcją;

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

3. zlicza tokeny wejściowe i wyjściowe (pole usage: prompt_tokens i completion_tokens, pola raportowane przez Ollama) oraz szacuje koszt zapytań do komercyjnego API, uwzględniając wpływ języka i tokenizatora na liczbę tokenów;
4. projektuje prompty o pełnej strukturze (rola, zadanie, kontekst, format) i oddziela dane wejściowe ogranicznikami;
5. stosuje technikę few-shot do wymuszania spójnego formatu odpowiedzi i poprawy trafności klasyfikacji;
6. wykorzystuje kreatywne sformułowania (analogia, storytelling, zmiana perspektywy) oraz personalizację (instrukcja systemowa) do dostosowania odpowiedzi do odbiorcy;
7. Prowadzi i dokumentuje iteracyjne doskonalenie promptu, weryfikując wygenerowany kod za pomocą testów jednostkowych.

WSKAZÓWKA: warto od początku kursu prowadzić własną bibliotekę promptów – plik (np. `prompts.md`) lub użyć programu **AI Prompt Lab**, w którym zapisywane są sprawdzone prompty wraz z datą, używanym modelem, numerem wersji oraz krótką oceną wyniku. W praktyce inżynierskiej taka dokumentacja może być równie cenna jak dobrze opisany kod.

Literatura

Materiały źródłowe:

[1]. Phoenix J., Taylor M., Skuteczna inżynieria promptów. Przyszłościowe rozwiązania dla rzetelnych wyników generatywnej AI, Helion, Gliwice 2025.

[2]. Korzyński P. i in., Artificial Intelligence Prompt Engineering as a New Digital Competence: Analysis of Generative AI Technologies Such as ChatGPT, Entrepreneurial Business and Economics Review 11(3), 2023, s. 25–38.

[3]. Ollama – oficjalna dokumentacja: https://ollama.com oraz https://github.com/ollama/ollama (dostęp: 2026).

[4]. Alto V., Generatywna sztuczna inteligencja z ChatGPT i modelami OpenAI. Podnieś swoją produktywność i innowacyjność za pomocą GPT-3 i GPT-4, Helion, Gliwice 2024.

Fundusze Europejskie dla Rozwoju Społecznego | Rzeczpospolita Polska | Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

[5]. Groq, Text/Chat Generation – dokumentacja API (pole usage), https://console.groq.com/docs/text-chat (dostęp: 2026).
[6]. Groq, Pricing (cennik), https://groq.com/pricing/ (dostęp: 2026).
[7]. Gadesha V., Prompt Engineering Techniques, IBM Think, https://www.ibm.com/think/topics/prompt-engineering-techniques (dostęp: 2026).
[8]. Mizrahi G., Inżynieria promptów bez tajemnic. Sztuka kreatywnego generowania tekstów, Helion, Gliwice 2025.
[9]. Google Cloud, Include few-shot examples – Gemini, projektowanie promptów, https://docs.cloud.google.com/gemini-enterprise-agent-platform/models/prompts/few-shot-examples (dostęp: 2026).
[10]. Google, Prompting strategies – Gemini API, https://ai.google.dev/gemini-api/docs/prompting-strategies (dostęp: 2026).
[11]. Ollama Python Library, https://github.com/ollama/ollama-python (dostęp: 2026).
[12]. Groq, Quickstart – pierwsze żądanie do API, https://console.groq.com/docs/quickstart (dostęp: 2026).
[13]. Groq, Model deprecations (harmonogram wyłączania modeli), https://console.groq.com/docs/deprecations (dostęp: 2026).

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

Materiały do przedmiotu:

GENERATYWNA SZTUCZNA INTELIGENCJA I PROMPT ENGINEERING

Laboratorium nr 3.
Analiza wpływu różnych parametrów dla modelu na jakość generowanych odpowiedzi. Analiza losowości odpowiedzi

Autor:

dr inż. Krzysztof Dziedzic

dr Marcin Barszcz

Katedra Informatyki

Fundusze Europejskie dla Rozwoju Społecznego

Rzeczpospolita Polska

Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

POLITECHNIKA LUBELSKA

Spis treści

Wprowadzenie, tematyka zajęć ---------- 49

Cel ćwiczenia/laboratorium ---------- 51

Instrukcja laboratoryjna/ćwiczeniowa ---------- 52

Ćwiczenia samodzielne ---------- 67

Pytania pomocnicze ---------- 68

Podsumowanie ---------- 69

Literatura ---------- 70

Fundusze Europejskie dla Rozwoju Społecznego

Rzeczpospolita Polska

Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

Wprowadzenie, tematyka zajęć

W niniejszym laboratorium przedstawiono drugi, równie ważny mechanizm sterowania modelem: parametry generacji. Ten sam prompt, uruchomiony z różnymi ustawieniami parametrów, może dawać odpowiedzi skrajnie różne, od w pełni powtarzalnych i zachowawczych po zróżnicowane i kreatywne. Celem zajęć jest zbadanie tego wpływu w systematycznych eksperymentach oraz analiza losowości odpowiedzi modeli językowych.

Słownik (ang. vocabulary). Zbiór wszystkich tokenów, które model językowy rozpoznaje i którymi posługuje się podczas generowania tekstu. Tokeny nie są tożsame z pełnymi wyrazami, to jednostki podwyrazowe (fragmenty słów, pojedyncze znaki lub całe krótkie wyrazy) wyznaczone na etapie uczenia przez tokenizator. Słownik ma skończony rozmiar (dla współczesnych modeli rzędu od dziesiątek do setek tysięcy pozycji), a to właśnie nad jego elementami model wyznacza rozkład prawdopodobieństwa, z którego losowany jest kolejny token. Rozmiar i konstrukcja słownika wpływają na sposób dzielenia tekstu na tokeny, a pośrednio na koszt generacji (liczbę tokenów) oraz na jakość obsługi poszczególnych języków.

Skąd bierze się losowość odpowiedzi. Model językowy nie „wybiera słów" jednoznacznie. Dla danego kontekstu wyznacza rozkład prawdopodobieństwa dla wszystkich tokenów słownika. Każdemu możliwemu następnemu tokenowi przypisuje odpowiednie prawdopodobieństwo. Kolejny token jest następnie losowany (próbkowany) z tego rozkładu, a procedura powtarza się aż do zakończenia odpowiedzi. To właśnie ten etap losowania sprawia, że dwa uruchomienia tego samego promptu mogą dawać różne wyniki. Parametry generacji nie zmieniają wiedzy modelu. Modyfikują sposób losowania: kształt rozkładu (temperatura) albo zbiór tokenów, spośród których można losować (top_p, top_k).

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

Temperatura (temperature). Parametr skaluje rozkład prawdopodobieństwa przed losowaniem. W Groq API przyjmuje wartości z zakresu 0,0–2,0. Przy temperaturze bliskiej zeru rozkład „wyostrza się", prawie całe prawdopodobieństwo przypada na jeden, najbardziej prawdopodobny token, a przy temperature=0 model działa tak, że wybiera zawsze token najbardziej prawdopodobny. Wysoka temperatura „spłaszcza" rozkład. Tokeny mniej typowe zyskują realne szanse wyboru, co zwiększa różnorodność, ale też ryzyko niespójności i błędów merytorycznych [1]. Orientacyjnie: 0,0–0,2 stosuje się do zadań wymagających precyzji i powtarzalności (analiza i poprawianie kodu, ekstrakcja danych, klasyfikacja, tłumaczenia), 0,3–0,7 do zastosowań ogólnych (wyjaśnienia, e-maile, dokumentacja), a 0,8–1,3 do zadań kreatywnych (burza mózgów, slogany, historie). Wartości powyżej ok. 1,5 dają odpowiedzi coraz bardziej chaotyczne i rzadko bywają użyteczne.

Parametry top_p i top_k. Parametry te zamiast zmieniać kształt rozkładu, ograniczają zbiór tokenów dopuszczonych do losowania. Parametr top_p (próbkowanie jądrowe, ang. nucleus sampling) sortuje tokeny malejąco według prawdopodobieństwa i dopuszcza tylko najmniejszy zbiór, którego łączne prawdopodobieństwo osiąga próg p – np. przy top_p=0,9 losowanie odbywa się wyłącznie w obrębie „jądra" obejmującego 90% masy prawdopodobieństwa. Parametr top_k dopuszcza stałą liczbę k najbardziej prawdopodobnych tokenów, np. top_k=40. Szczególny przypadek top_k=1 oznacza wybór „zachłanny", równoważny w skutkach temperaturze 0. W praktyce inżynierskiej zaleca się modyfikowanie jednego parametru losowości naraz (temperature albo top_p). Jednoczesne strojenie obu elementów utrudnia interpretację wyników eksperymentalnych [2].

Długość odpowiedzi: max_tokens i sekwencje stopu. Parametr max_tokens wyznacza twardy limit długości odpowiedzi wyrażony w tokenach. Ma trzy praktyczne zastosowania: kontrolę kosztów (tokeny wyjściowe są w płatnej warstwie API najdroższym składnikiem, wymuszanie zwięzłości oraz ochronę aplikacji przed nadmiernie długimi generacjami. Zbyt niski limit ucina odpowiedź w pół zdania. Model nie „planuje" treści pod limit, lecz jest przerywany po jego

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

osiągnięciu. Informację o przyczynie zakończenia zwraca pole finish_reason odpowiedzi ("stop" – naturalny koniec, "length" – ucięcie). Uzupełnieniem jest parametr stop: lista sekwencji znaków, których wygenerowanie natychmiast kończy odpowiedź [2].

Powtarzalność i ziarno losowości (seed). Losowanie tokenów korzysta z generatora liczb pseudolosowych, którego stan początkowy określa ziarno (seed). Ustawienie stałego ziarna w Ollama (opcja seed) zapewnia pełną powtarzalność. Ten sam prompt, model i parametry wygenerują identyczną odpowiedź przy każdym uruchomieniu, nawet przy wysokiej temperaturze. Groq API również udostępnia parametr seed, jednak dla modeli chmurowych powtarzalność ma charakter „najlepszych starań" (ang. best effort). Odpowiedzi mogą się nieznacznie różnić, m.in. z powodu zmian infrastruktury po stronie dostawcy. Z tego samego powodu nawet temperature=0 nie gwarantuje w chmurze stuprocentowej identyczności [2]. Kontrola powtarzalności jest istotna np. w testach automatycznych aplikacji wykorzystujących modele oraz przy odtwarzaniu wyników eksperymentów.

Kary za powtórzenia. Parametry frequency_penalty i presence_penalty (Groq API) obniżają prawdopodobieństwo wystąpienia tokenów, które już pojawiły się w generowanym tekście. Pierwszy proporcjonalnie do liczby wystąpień (ogranicza powtarzanie tych samych słów), drugi jednorazowo za sam fakt wystąpienia (zachęca do wprowadzania nowych wątków). W Ollama zbliżoną funkcję pełni opcja repeat_penalty. Parametry te bywają przydatne w dłuższych tekstach generowanych przy wysokiej temperaturze [2, 3].

Gdzie ustawia się parametry. W dostępie programistycznym do Groq parametry przekazuje się bezpośrednio jako argumenty metody client.chat.completions.create() (m.in. temperature, top_p, max_tokens, stop, seed, frequency_penalty, presence_penalty); interfejs Groq, zgodny z API OpenAI, nie udostępnia parametru top_k [2]. W bibliotece ollama analogiczną rolę pełni słownik `options` (m.in. temperature, top_p, top_k, num_predict, stop, seed, repeat_penalty)

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

[3], a w trybie interaktywnym terminala polecenie `/set parameter`. Wygodne eksperymenty „na suwakach” umożliwia GroqCloud Playground (console.groq.com/playground) – interfejs webowy z panelem ustawień uruchomienia (temperature, top_p, limit długości) [4]. Dla modeli lokalnych podobną rolę może pełnić bezpłatna aplikacja LM Studio z graficznym panelem parametrów i możliwością zapisywania konfiguracji jako presetów. Korzystanie z niej w tym laboratorium jest opcjonalne w przypadku problemów z innymi aplikacjami.

Cel ćwiczenia/laboratorium

Głównym celem laboratorium jest zrozumienie mechanizmu losowości w generowaniu tekstu oraz nabycie praktycznej umiejętności doboru parametrów generacji tekstu do rodzaju zadania. Po ukończeniu zajęć student potrafi:

1. wyjaśnić, dlaczego odpowiedzi modelu językowego są losowe, oraz opisać rolę rozkładu prawdopodobieństwa i próbkowania tokenów;
2. ustawiać parametry generacji w Groq API (`argumenty metody chat.completions.create`), w bibliotece ollama (słownik `options`) oraz w interfejsie GroqCloud Playground;
3. zbadać eksperymentalnie wpływ temperatury na powtarzalność i różnorodność odpowiedzi oraz dobrać jej wartość do zadań analitycznych, faktograficznych i kreatywnych;
4. stosować parametry top_p i top_k do kontroli różnorodności wyjścia i wskazać różnicę między nimi a temperaturą;
5. kontrolować długość odpowiedzi parametrem max_tokens i sekwencjami stopu oraz interpretować pole finish_reason, uwzględniając wpływ limitu na koszty;
6. zapewniać powtarzalność generacji za pomocą ziarna losowości (seed) i wskazać różnice gwarancji między modelem lokalnym a chmurowym;
7. zaprojektować, przeprowadzić i udokumentować eksperyment porównawczy (wiele wartości parametru, wiele prób) oraz dobrać konfigurację parametrów do konkretnego zastosowania.

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

Instrukcja laboratoryjna/ćwiczeniowa

Uwaga o modelach: komercyjne modele chmurowe mają ograniczony cykl życia – dostawcy okresowo wycofują starsze modele i zastępują je nowszymi, a aktualny harmonogram wyłączeń modeli Groq publikowany jest pod adresem console.groq.com/docs/deprecations [5]. Dlatego we wszystkich skryptach nazwy modeli są definiowane w jednym miejscu (stałe `MODEL` i `MODEL_LOKALNY`), co ułatwia ich zmianę w przyszłości. Przed przystąpieniem do laboratorium należy sprawdzić, który model jest obecnie wspierany.

Zadanie 3.1. Parametry generacji w interfejsach: GroqCloud Playground i Ollama

Zanim zaczniesz sterować parametrami z poziomu kodu, poznasz je w dwóch interfejsach: webowym GroqCloud Playground (model chmurowy, suwaki ustawień) [4] oraz terminalowym Ollama (model lokalny, polecenie `/set parameter`). Tabela 3.1 zestawia nazwy najważniejszych parametrów w obu środowiskach.

Tabela 3.1. Najważniejsze parametry generacji i ich nazwy w Groq API oraz Ollama

Parametr (rola)	Groq API (chat.completions.create)	Ollama (options)	Typowy zakres
losowość – kształt rozkładu	temperature	temperature	0,0–2,0
losowość – próg skumulowany	top_p	top_p	0,0–1,0
losowość – liczba tokenów	– (nieobsługiwane)	top_k	1–100
limit długości odpowiedzi	max_tokens	num_predict	10–8192
sekwencje kończące generację	stop	stop	lista tekstów

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

Parametr (rola)	Groq API (chat.completions.create)	Ollama (options)	Typowy zakres
ziarno losowości (powtarzalność)	seed	seed	liczba całkowita
kara za powtórzenia	frequency_penalty / presence_penalty	repeat_penalty	0,0–2,0 / ok. 1,1

Treść zadania:

1. **GroqCloud Playground.** Zaloguj się na console.groq.com i otwórz zakładkę Playground z modelem llama-3.3-70b-versatile. W panelu ustawień uruchomienia (po prawej stronie) odszukaj parametry: temperature, a w ustawieniach zaawansowanych – top_p i limit długości odpowiedzi (max_tokens).
2. Ustaw temperature = 0 i dwukrotnie wyślij ten sam prompt:

> „Zaproponuj chwytliwą nazwę i jednozdaniowy slogan dla studenckiego koła naukowego sztucznej inteligencji na Politechnice Lubelskiej."

3. Powtórz eksperyment przy temperature = 1,8 (również dwukrotnie). Porównaj cztery odpowiedzi pod kątem: powtarzalności, oryginalności propozycji i poprawności językowej.
4. **Ollama (terminal).** Uruchom model lokalny w trybie interaktywnym i ustaw parametry poleceniami wydawanymi wewnątrz sesji:

```
ollama run llama3.1
>>> /set parameter temperature 0
>>> /set parameter seed 42
>>> Zaproponuj chwytliwą nazwę i jednozdaniowy slogan dla
studenckiego koła naukowego sztucznej inteligencji na Politechnice
Lubelskiej.
```

5. Wyślij ten sam prompt dwukrotnie (strzałka w górę przywołuje poprzednią wiadomość) i sprawdź, czy odpowiedzi są identyczne. Następnie ustaw `/set parameter temperature 1.5` oraz `/set parameter seed 0` i ponów próbę. Sesję kończy polecenie `/bye`.
6. **Opcjonalnie – LM Studio.** Jeżeli chcesz eksperymentować z parametrami modeli lokalnych w interfejsie graficznym, zainstaluj bezpłatną aplikację LM

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

Studio (`lmstudio.ai`). Panel boczny czatu udostępnia suwaki temperature, top_p i top_k, a zestawy ustawień można zapisywać jako presety.

P: Porównaj zachowanie modeli przy skrajnych temperaturach w obu interfejsach: czy przy temperature = 0 odpowiedzi były identyczne? Co zmieniło ustawienie ziarna (seed) w Ollama?

Zadanie 3.2. Temperatura a losowość i powtarzalność odpowiedzi

Pierwszy eksperyment programistyczny: ten sam prompt zostanie uruchomiony po trzy razy dla trzech wartości temperatury, a skrypt policzy, ile spośród trzech odpowiedzi jest unikalnych. Liczba unikalnych odpowiedzi jest prostą, obiektywną miarą losowości: 1 oznacza pełną powtarzalność, 3 – każda odpowiedź jest inna.

Treść zadania:

1. W katalogu roboczym utwórz plik `temperatura_losowosc.py` o następującej treści:

```
import os
import sys
from dotenv import load_dotenv
from groq import Groq
from groq import APIError

# poprawne wyswietlanie polskich znakow w konsoli Windows
sys.stdout.reconfigure(encoding='utf-8')

load_dotenv()
client = Groq(api_key=os.getenv('GROQ_API_KEY'))

# nazwy modeli w jednym miejscu - latwiej je podmienic
MODEL = 'llama-3.3-70b-versatile'
MODEL_LOKALNY = 'llama3.2'   # model zapasowy (Ollama), gdy Groq
nie odpowiada

PROMPT = ('Podaj pięć zastosowań sztucznej inteligencji '
          'w przemyśle. Odpowiedz krótką listą.')
```


Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

```
TEMPERATURY = [0.0, 0.7, 1.5]
PROBY = 3

for temp in TEMPERATURY:
    print(f'\n\n{"#" * 64}')
    print(f'###  TEMPERATURE = {temp}')
    print('#' * 64)
    odpowiedzi = []
    for i in range(1, PROBY + 1):
        try:
            r = client.chat.completions.create(
                model=MODEL,
                messages=[{'role': 'user', 'content': PROMPT}],
                temperature=temp,
                max_tokens=300,
            )
            tekst = r.choices[0].message.content.strip()
            odpowiedzi.append(tekst)
            print(f'\n--- proba {i} ---')
            print(tekst)
        except APIError as e:
            # np. blad 429 (limit zapytan) lub problem z kluczem
            print(f'[Blad Groq API] {e}')
            # brak odpowiedzi z Groq -> awaryjne uzycie modelu
lokalnego
            import ollama
            lok = ollama.generate(model=MODEL_LOKALNY,
prompt=PROMPT,
                                  options={'temperature': temp,
'num_predict': 300})
            tekst = lok['response'].strip()
            odpowiedzi.append(tekst)
            print(f'--- proba {i} (model lokalny: {MODEL_LOKALNY})
---')
            print(tekst)
    unikalne = len(set(odpowiedzi))
```

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

```
    print(f'\n>>> Unikalne odpowiedzi: {unikalne} z
{len(odpowiedzi)}')
```

2. Zapisz plik (Ctrl+S) i uruchom skrypt poleceniem `python temperatura_losowosc.py`. Wykonanie potrwa chwilę. Skrypt wysyła 9 zapytań do API. W przypadku braku dostępu do modelu chmurowego zadanie zostanie wykonane na modelu lokalnym.
3. Na podstawie wydruku uzupełnij tabelę 3.2. W kolumnie obserwacji zanotuj charakter różnic: czy zmieniają się tylko sformułowania, czy również dobór przykładów i struktura listy?

Tabela 3.2. Wpływ temperatury na powtarzalność odpowiedzi (prompt o zastosowaniach AI, po 3 próby)

Temperatura	Unikalne odpowiedzi (z 3)	Obserwowane różnice między próbami
0,0		
0,7		
1,5		

P: Zamieść uzupełnioną tabelę 3.2 i sformułuj wnioski: (1) czy przy temperature = 0 odpowiedzi były w pełni identyczne, a jeśli nie – jak to wyjaśnić w przypadku modelu chmurowego? (2) jak rośnie zróżnicowanie odpowiedzi wraz z temperaturą? (3) dla jakich zastosowań wybierzesz temperature = 0, a dla jakich wartości wyższe?

Zadanie 3.3. Dobór temperatury do typu zadania

Optymalna temperatura zależy od charakteru zadania. W tym ćwiczeniu ten sam zestaw trzech temperatur zostanie zastosowany do trzech skrajnie różnych zadań: analitycznego (znajdowanie błędów w kodzie), faktograficznego (tłumaczenie techniczne) oraz kreatywnego (generowanie pomysłu).

Treść zadania:

1. W katalogu roboczym utwórz plik `temperatura_zadania.py`. Nagłówek pliku (importy, konfiguracja konsoli, wczytanie klucza API, utworzenie klienta oraz stała MODEL) przygotuj na podstawie skryptu z zadania 3.2, a następnie dopisz część merytoryczną:

```
KOD_Z_BLEDAMI = '''
def suma_listy(liczby):
    suma = 0
    for i in range(len(liczby)
        suma += liczby[i]
    return suma
'''

SCENARIUSZE = {
    'A - analiza kodu': (
        'Przeanalizuj poniższy kod Python i wypisz '
        'wszystkie błędy:\n' + KOD_Z_BLEDAMI
    ),
    'B - tlumaczenie': (
        'Przetłumacz na język angielski: "Uczenie '
        'maszynowe to dziedzina sztucznej inteligencji '
        'zajmująca się tworzeniem algorytmów, które uczą '
        'się na podstawie danych."'
    ),
    'C - zadanie kreatywne': (
        'Wymyśl jedno nietypowe, kreatywne zastosowanie '
        'sztucznej inteligencji ułatwiające życie studentom '
        'Politechniki Lubelskiej. Opisz je w 2-3 zdaniach.'
    ),
}

TEMPERATURY = [0.0, 0.7, 1.2]

for nazwa, prompt in SCENARIUSZE.items():
    print(f'\n\n{"#" * 64}')
    print(f'###  SCENARIUSZ {nazwa}')
    print('#' * 64)
    for temp in TEMPERATURY:
```

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

```
        print(f'\n--- temperature = {temp} ---')
        try:
            r = client.chat.completions.create(
                model=MODEL,
                messages=[{'role': 'user', 'content': prompt}],
                temperature=temp,
                max_tokens=400,
            )
            print(r.choices[0].message.content.strip())
        except APIError as e:
            print(f'[Blad Groq API] {e}')
```

2. Zapisz plik (Ctrl+S) i uruchom skrypt poleceniem `python temperatura_zadania.py`.
3. Dla scenariusza A sprawdź poprawność merytoryczną każdej odpowiedzi. Kod zawiera dokładnie dwa błędy: brak nawiasu zamykającego w wierszu z `range` oraz brak dwukropka na końcu tego wiersza. Czy wysoka temperatura wprowadziła błędne lub nadmiarowe „poprawki”?
4. Dla scenariusza B porównaj wierność i terminologię tłumaczeń, a dla scenariusza C oryginalność i zróżnicowanie pomysłów.
5. Uzupełnij tabelę 3.3, wskazując optymalną temperaturę dla każdego typu zadania.

Tabela 3.3. Rekomendowana temperatura dla różnych typów zadań (na podstawie eksperymentu)

Typ zadania	Optymalna temperatura	Uzasadnienie (1–2 zdania)
analiza kodu		
tłumaczenie techniczne		
zadanie kreatywne		

P: Zamieść po jednej charakterystycznej odpowiedzi z każdego scenariusza dla skrajnych temperatur oraz uzupełnij tabelę 3.3. Odpowiedz: w którym scenariuszu wysoka temperatura była najbardziej szkodliwa i dlaczego?

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

Zadanie 3.4. Parametry top_p i top_k – kontrola różnorodności

Eksperyment analogiczny do zadania 3.2, ale losowość będzie sterowana parametrami top_p i top_k przy stałej temperaturze 1,0. Ponieważ interfejs Groq (zgodny z API OpenAI) nie udostępnia parametru top_k, parametr ten testujemy na modelu lokalnym (Ollama), który go obsługuje. Dzięki temu zaobserwujesz, że różnorodność odpowiedzi można ograniczać nie tylko poprzez temperaturę. Zrozumiesz również zasadę modyfikowania jednego parametru losowości naraz.

Treść zadania:

1. W katalogu roboczym utwórz plik `top_p_top_k.py` (nagłówek pliku jak w zadaniu 3.2) i dopisz część merytoryczną:

```
PROMPT = ('Opisz jednym zdaniem, czym jest głębokie '
          'uczenie (deep learning).')
PROBY = 3
import ollama

# uwaga: interfejs Groq (zgodny z API OpenAI) obsluguje top_p,
# ale nie udostepnia parametru top_k - dlatego top_k testujemy
# na modelu lokalnym (Ollama), ktory ten parametr obsluguje
MODEL_LOKALNY = 'llama3.1'

def testuj_groq(opis, **parametry):
    print(f'\n\n{"#" * 64}')
    print(f'###  {opis}  (Groq: {MODEL})')
    print('#' * 64)
    odpowiedzi = []
    for i in range(1, PROBY + 1):
        try:
            r = client.chat.completions.create(
                model=MODEL,
                messages=[{'role': 'user', 'content': PROMPT}],
                temperature=1.0,
                max_tokens=120,
                **parametry,
            )
```

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

```
            tekst = r.choices[0].message.content.strip()
            odpowiedzi.append(tekst)
            print(f'proba {i}: {tekst}')
        except APIError as e:
            print(f'[Blad Groq API] {e}')
    print(f'>>> Unikalne odpowiedzi: {len(set(odpowiedzi))} '
          f'z {len(odpowiedzi)}')

def testuj_ollama(opis, **opcje):
    print(f'\n\n{"#" * 64}')
    print(f'###  {opis}  (Ollama: {MODEL_LOKALNY})')
    print('#' * 64)
    odpowiedzi = []
    for i in range(1, PROBY + 1):
        try:
            r = ollama.chat(
                model=MODEL_LOKALNY,
                messages=[{'role': 'user', 'content': PROMPT}],
                options={'temperature': 1.0, 'num_predict': 120,
                         **opcje},
            )
            tekst = r['message']['content'].strip()
            odpowiedzi.append(tekst)
            print(f'proba {i}: {tekst}')
        except Exception as e:
            print(f'[Blad Ollama] {e}')
    print(f'>>> Unikalne odpowiedzi: {len(set(odpowiedzi))} '
          f'z {len(odpowiedzi)}')

# Test A: rozne wartosci top_p na modelu zewnetrznym (temp. = 1.0)
for p in [0.1, 0.5, 0.95]:
    testuj_groq(f'TOP_P = {p}', top_p=p)

# Test B: rozne wartosci top_k na modelu lokalnym (temp. = 1.0)
for k in [1, 5, 40]:
    testuj_ollama(f'TOP_K = {k}', top_k=k)
```

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

2. Zapisz plik (Ctrl+S) i uruchom skrypt poleceniem `python top_p_top_k.py`. Skrypt wykona 18 zapytań (9 do modelu Groq dla top_p oraz 9 do modelu lokalnego Ollama dla top_k). Obserwuj liczbę unikalnych odpowiedzi dla każdego ustawienia.
3. Zwróć uwagę na przypadek top_k = 1: mimo temperatury 1,0 losowanie ogranicza się do jednego, najbardziej prawdopodobnego tokenu. Wynik powinien przypominać zachowanie przy temperature = 0.

P: Zamieść zestawienie liczby unikalnych odpowiedzi dla wszystkich sześciu ustawień oraz po jednym przykładzie odpowiedzi dla wartości skrajnych. Sformułuj wnioski: (1) jak top_p wpływa na różnorodność w porównaniu z temperaturą z zadania 3.2? (2) dlaczego top_k = 1 „wyłącza" losowość? (3) dlaczego zaleca się strojenie jednego parametru losowości naraz?

Zadanie 3.5. Kontrola długości: max_tokens i sekwencje stopu

Limit długości odpowiedzi wpływa jednocześnie na kompletność treści i koszt zapytania. W tym zadaniu zbadasz skutki trzech różnych limitów oraz zatrzymasz generację sekwencją stopu, a pole finish_reason wskaże przyczynę zakończenia generacji.

Treść zadania:

1. W katalogu roboczym utwórz plik `dlugosc_odpowiedzi.py` (nagłówek pliku jak w zadaniu 3.2) i dopisz część merytoryczną:

```
# ceny platnej warstwy API w USD za 1 mln tokenow (stan na 2026 r.)
CENA_WY = 0.79

PROMPT = ('Wyjaśnij, czym jest metoda gradientu prostego '
          '(gradient descent) w uczeniu maszynowym. '
          'Uwzględnij definicję, zastosowanie i przykład.')

# czesc 1: rozne limity dlugosci odpowiedzi
for limit in [50, 150, 500]:
    print(f'\n\n{"#" * 64}')
```

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

```
    print(f'###  MAX_TOKENS = {limit}')
    print('#' * 64)
    try:
        r = client.chat.completions.create(
            model=MODEL,
            messages=[{'role': 'user', 'content': PROMPT}],
            temperature=0.3,
            max_tokens=limit,
        )
        print(r.choices[0].message.content)
        tok_wy = r.usage.completion_tokens
        powod = r.choices[0].finish_reason
        print(f'\n>>> tokeny wyjsciowe: {tok_wy} | '
              f'finish_reason: {powod} | '
              f'koszt: {tok_wy * CENA_WY / 1e6:.6f} USD')
    except APIError as e:
        print(f'[Blad Groq API] {e}')

# czesc 2: sekwencja stopu konczy generacje
print(f'\n\n{"#" * 64}')
print('###  STOP = ["###"]')
print('#' * 64)
PROMPT_STOP = ('Wypisz trzy kroki instalacji interpretera '
               'Python w systemie Windows. Po trzecim kroku '
               'napisz ###, a następnie dodaj szczegółowy '
               'opis każdego kroku.')
try:
    r = client.chat.completions.create(
        model=MODEL,
        messages=[{'role': 'user', 'content': PROMPT_STOP}],
        temperature=0.3,
        max_tokens=500,
        stop=['###'],
    )
    print(r.choices[0].message.content)
    print(f'\n>>> finish_reason: {r.choices[0].finish_reason}')
except APIError as e:
```

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

```
    print(f'[Blad Groq API] {e}')
```

2. Zapisz plik (Ctrl+S) i uruchom skrypt poleceniem `python dlugosc_odpowiedzi.py`.
3. Porównaj trzy odpowiedzi z części 1: która wartość limitu zapewnia komplet wymaganych elementów (definicja, zastosowanie, przykład)? Przy którym limicie odpowiedź została ucięta w pół zdania i jaki finish_reason temu towarzyszył ("length" zamiast "stop")?
4. W części 2 sprawdź, czy szczegółowy opis kroków – zapowiedziany w prompcie po znaczniku ### – w ogóle pojawił się w odpowiedzi.

P: Zamieść trzy odpowiedzi z części 1 wraz z liczbą tokenów wyjściowych, wartością finish_reason i kosztem oraz odpowiedź z części 2. Sformułuj wnioski: (1) jaka wartość limitu jest optymalna dla tego pytania i dlaczego? (2) o ile limit 50 tokenów obniżył koszt względem limitu 500 i jakim kosztem jakościowym? (3) do czego w praktyce można wykorzystać sekwencje stopu?

Zadanie 3.6. Powtarzalność kontrolowana: ziarno losowości (seed)

Wysoka temperatura i powtarzalność nie muszą się wykluczać. Stałe ziarno losowości pozwala odtworzyć identyczną „losową" odpowiedź. W tym ćwiczeniu porównasz gwarancje powtarzalności modelu lokalnego (Ollama) i chmurowego (Groq API).

Treść zadania:

1. W katalogu roboczym utwórz plik `seed_powtarzalnosc.py` (nagłówek pliku jak w zadaniu 3.2, dodatkowo `import ollama` oraz stała `MODEL_LOKALNY = 'llama3.1'`) i dopisz część merytoryczną:

```
PROMPT = ('Zaproponuj oryginalną nazwę robota '
          'sprzątającego laboratorium komputerowe. '
          'Odpowiedz samą nazwą.')

def proby_ollama(opis, opcje, n=3):
    print(f'\n--- Ollama: {opis} ---')
    wyniki = []
```

```
    for i in range(n):
        try:
            r = ollama.chat(
                model=MODEL_LOKALNY,
                messages=[{'role': 'user', 'content': PROMPT}],
                options=opcje,
            )
            wyniki.append(r['message']['content'].strip())
            print(f'proba {i + 1}: {wyniki[-1]}')
        except Exception as e:
            print(f'[Blad Ollama] {e}')
    print(f'>>> Unikalne: {len(set(wyniki))} z {len(wyniki)}')

# czesc 1: model lokalny - pelna kontrola ziarna
proby_ollama('temperature=0.8, seed=42',
             {'temperature': 0.8, 'seed': 42})
proby_ollama('temperature=0.8, seed=7',
             {'temperature': 0.8, 'seed': 7})
proby_ollama('temperature=0.8, bez seed',
             {'temperature': 0.8})

# czesc 2: model chmurowy - seed w trybie best effort
print('\n--- Groq: temperature=0.8, seed=42 ---')
wyniki = []
for i in range(3):
    try:
        r = client.chat.completions.create(
            model=MODEL,
            messages=[{'role': 'user', 'content': PROMPT}],
            temperature=0.8,
            seed=42,
            max_tokens=60,
        )
        wyniki.append(r.choices[0].message.content.strip())
        print(f'proba {i + 1}: {wyniki[-1]}')
    except APIError as e:
        print(f'[Blad Groq API] {e}')
```

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

```
print(f'>>> Unikalne: {len(set(wyniki))} z {len(wyniki)}')
```

2. Zapisz plik (Ctrl+S) i uruchom skrypt poleceniem `python seed_powtarzalnosc.py`. Model lokalny uruchom dwukrotnie – pierwszy przebieg obejmuje ładowanie modelu do pamięci (zimny start).
3. Przeanalizuj wyniki: czy przy stałym ziarnie model lokalny wygenerował trzy identyczne nazwy mimo temperatury 0,8? Czy zmiana ziarna (42 → 7) dała inną, ale również powtarzalną odpowiedź? Jak zachował się model chmurowy?

P: Zamieść wyniki wszystkich czterech serii prób. Sformułuj wnioski: (1) czym różni się gwarancja powtarzalności modelu lokalnego od chmurowego i skąd bierze się ta różnica? (2) podaj dwa praktyczne zastosowania stałego ziarna losowości (np. testy automatyczne aplikacji AI, odtwarzalne eksperymenty).

Zadanie 3.7. Analiza przypadku: dobór konfiguracji parametrów do zastosowania

Scenariusz zadania: studenckie koło naukowe buduje narzędzie do streszczeń abstraktów prac inżynierskich. Streszczenia muszą być spójne i powtarzalne (to samo wejście powinno dawać to samo wyjście), rzeczowe, bez ozdobników oraz nie dłuższe niż 3 zdania. Zadaniem jest eksperymentalny dobór konfiguracji parametrów spełniającej te wymagania.

Treść zadania:

1. W katalogu roboczym utwórz plik `konfiguracja_streszczen.py` (nagłówek pliku jak w zadaniu 3.2) i dopisz część merytoryczną:

```
ABSTRAKT = (
    'Praca przedstawia system wykrywania ataków w sieci '
    'komputerowej (IDS) oparty na uczeniu maszynowym. '
    'Porównano klasyczne metody sygnaturowe z trzema '
    'klasyfikatorami uczonymi na zbiorze CIC-IDS2017: '
    'lasem losowym, maszyną wektorów nośnych oraz siecią '
    'neuronową. Najlepszą skuteczność (98,2% F1) osiągnął '
    'las losowy przy najniższym czasie klasyfikacji. '
    'Zaproponowano architekturę wdrożenia systemu w sieci '
```

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

**Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00**

```
    'laboratoryjnej uczelni oraz omówiono ograniczenia '
    'metody, w tym podatność na ataki dnia zerowego.'
)

PROMPT = ('Streść poniższy abstrakt pracy inżynierskiej '
          'w maksymalnie trzech zdaniach, rzeczowo i bez '
          'ozdobników:\n"""' + ABSTRAKT + '"""')

KONFIGURACJE = {
    'A (temperature=0.0)': dict(temperature=0.0),
    'B (temperature=0.3, top_p=0.8)': dict(temperature=0.3,
                                           top_p=0.8),
    'C (temperature=1.0)': dict(temperature=1.0),
}

for nazwa, parametry in KONFIGURACJE.items():
    print(f'\n\n{"#" * 64}')
    print(f'###  KONFIGURACJA {nazwa}')
    print('#' * 64)
    for i in (1, 2):
        try:
            r = client.chat.completions.create(
                model=MODEL,
                messages=[{'role': 'user', 'content': PROMPT}],
                max_tokens=200,
                **parametry,
            )
            print(f'\n--- proba {i} ---')
            print(r.choices[0].message.content.strip())
        except APIError as e:
            print(f'[Blad Groq API] {e}')
```

2. Zapisz plik (Ctrl+S) i uruchom skrypt poleceniem `python konfiguracja_streszczen.py`.
3. Oceń każdą konfigurację względem wymagań scenariusza: powtarzalność (czy dwie próby dały ten sam tekst?), rzeczowość i zwięzłość (limit trzech

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

zdań, brak ozdobników), zachowanie kluczowych informacji (metody, wnioski).

4. Wybierz konfigurację finalną. Zastanów się, czy warto dodać do niej ziarno losowości (seed) lub obniżyć max_tokens – uzasadnij.

P: Zamieść wyniki wszystkich sześciu prób (3 konfiguracje × 2 próby), wskaż konfigurację finalną z uzasadnieniem w świetle wymagań. Zaproponuj (bez uruchamiania i testowania) konfiguracje parametrów dla trzech innych zastosowań: chatbota odpowiadającego na najczęstsze pytania studentów, generatora kreatywnych opisów projektów na targi kół naukowych oraz asystenta poprawiającego błędy w kodzie.

Ćwiczenia samodzielne

Ćwiczenie 3.1 (podstawowe). Losowość małego modelu lokalnego

Powtórz eksperyment z zadania 3.2 na modelu `gemma3:1b (lub innym)`. Wywołanie `ollama.chat()` z parametrem `options={'temperature': temp}`). W sprawozdaniu przedstaw tabelę wzorowaną na tabeli 3.2 oraz wniosek: czy mały model lokalny jest przy tych samych temperaturach bardziej, czy mniej stabilny od modelu chmurowego i jak oceniasz jakość jego odpowiedzi?

Ćwiczenie 3.2 (średnie). Generator nazw i sloganów – niska vs wysoka temperatura

Przygotuj prompt generujący dziesięć propozycji nazw wraz ze sloganem dla studenckiego projektu informatycznego (np. aplikacji mobilnej dla studentów Politechniki Lubelskiej). Uruchom go dwukrotnie: przy temperature = 0,2 oraz temperature = 1,2. W sprawozdaniu przedstaw obie listy. Porównaj ich oryginalność i poprawność językową. Wskaż najlepszą propozycję oraz uzasadnij, która temperatura lepiej sprawdza się w tym zastosowaniu i jaką wartość top_p warto rozważyć zamiast zmiany temperatury.

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

**Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00**

Ćwiczenie 3.3 (średnie). Stabilność formatu klasyfikacji przy różnych temperaturach

Wykorzystaj prompt few-shot do klasyfikacji zgłoszeń działu IT z zadania 2.5 (Laboratorium 2). Dla jednego wybranego zgłoszenia wykonaj po pięć prób przy temperature = 0,0 oraz temperature = 1,0 i policz, w ilu próbach odpowiedź zachowała dokładnie format KATEGORIA | PRIORYTET [6]. W sprawozdaniu przedstaw zestawienie (2 × 5 prób), odsetek odpowiedzi w poprawnym formacie dla każdej temperatury oraz wniosek: dlaczego w zadaniach klasyfikacji, których wynik przetwarza dalej inny program, stosuje się temperaturę bliską zeru?

Ćwiczenie 3.4 (trudne). Własny mini-eksperyment badawczy

Sformułuj własną hipotezę dotyczącą wpływu jednego wybranego parametru generacji na odpowiedzi modelu (np. „powyżej pewnej wartości temperatury odpowiedzi na pytania faktograficzne zaczynają zawierać błędy merytoryczne" albo „top_p poniżej 0,3 eliminuje różnorodność równie skutecznie jak temperature = 0"). Zaprojektuj eksperyment weryfikujący hipotezę: co najmniej 3 wartości parametru, co najmniej 3 próby na wartość, ten sam prompt, oba modele (chmurowy i lokalny). W sprawozdaniu przedstaw kompletny raport: hipotezę, opis metodyki, skrypt, tabelę wyników oraz wniosek, czy hipoteza została potwierdzona?

Pytania pomocnicze

1. Skąd bierze się losowość odpowiedzi modelu językowego? Jaką rolę odgrywają rozkład prawdopodobieństwa kolejnego tokenu i próbkowanie?
2. Jak działa parametr temperature i jaki jest jego typowy zakres w Groq API? Co dzieje się z rozkładem prawdopodobieństwa przy temperaturze bliskiej zeru, a co przy wysokiej?

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

3. Dla jakich typów zadań zaleca się temperaturę bliską zeru, a dla jakich wartości wyższe? Podaj po dwa przykłady.
4. Czy temperature = 0 gwarantuje identyczne odpowiedzi modelu chmurowego przy każdym uruchomieniu? Uzasadnij.
5. Na czym polega próbkowanie jądrowe (top_p) i czym różni się ono od parametru top_k? Dlaczego top_k = 1 daje efekt zbliżony do temperature = 0?
6. Dlaczego zaleca się modyfikowanie jednego parametru losowości naraz (temperature albo top_p)?
7. Do czego służy parametr max_tokens i jakie są skutki ustawienia zbyt niskiej wartości? Jak pole finish_reason pozwala odróżnić naturalne zakończenie odpowiedzi od ucięcia?
8. Czym są sekwencje stopu (stop) i podaj przykład ich praktycznego zastosowania.
9. Czym jest ziarno losowości (seed) i czym różnią się gwarancje powtarzalności modelu lokalnego (Ollama) od modelu chmurowego (Groq API)?
10. Jak działają parametry frequency_penalty i presence_penalty? W jakich sytuacjach warto je stosować?
11. Jak zaprojektować rzetelny eksperyment badający wpływ parametru na odpowiedzi modelu? Dlaczego pojedyncza próba nie wystarcza?

Podsumowanie

Laboratorium poświęcone było drugiemu mechanizmowi sterowania modelem językowym: parametrom generacji. Po jego ukończeniu student:

1. rozumie mechanizm losowości odpowiedzi (rozkład prawdopodobieństwa tokenów i próbkowanie) oraz wpływ parametrów na ten proces;
2. dobiera temperaturę do typu zadania i potwierdza wybór eksperymentem z wieloma próbami;
3. stosuje parametry top_p i top_k do kontroli różnorodności oraz przestrzega zasady strojenia jednego parametru losowości naraz;
4. kontroluje długość i koszt odpowiedzi parametrem max_tokens i sekwencjami stopu oraz interpretuje pole finish_reason;

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

5. zapewnia powtarzalność generacji ziarnem losowości (seed) i zna różnicę gwarancji między modelem lokalnym a chmurowym;
6. projektuje i dokumentuje eksperymenty porównawcze oraz dobiera kompletne konfiguracje parametrów do konkretnych zastosowań.

UWAGA: rozszerz bibliotekę promptów prowadzoną od Laboratorium 2 (plik `prompts.md` lub program AI Prompt Lab) o kolumnę z konfiguracją parametrów. Sprawdzony prompt bez zapisanych ustawień temperatury, top_p i limitu długości jest niekompletny, bo ten sam tekst przy innej konfiguracji może dawać zupełnie inne wyniki.

Literatura

[1]. Gadesha V., Prompt Engineering Techniques, IBM Think, https://www.ibm.com/think/topics/prompt-engineering-techniques (dostęp: 2026).
[2]. Groq, Text Generation – dokumentacja API (parametry generacji: temperature, top_p, max_tokens, stop, seed), https://console.groq.com/docs/text-chat (dostęp: 2026).
[3]. Ollama – Modelfile i parametry generacji (temperature, top_p, top_k, num_predict, seed, repeat_penalty), https://github.com/ollama/ollama/blob/main/docs/modelfile.md (dostęp: 2026).
[4]. GroqCloud Playground, https://console.groq.com/playground (dostęp: 2026).
[5]. Groq, Model deprecations (harmonogram wyłączania modeli), https://console.groq.com/docs/deprecations (dostęp: 2026).
[6]. Google Cloud, Include few-shot examples – Gemini, projektowanie promptów, https://docs.cloud.google.com/gemini-enterprise-agent-platform/models/prompts/few-shot-examples (dostęp: 2026).
[7]. Google, Prompting strategies – Gemini API, https://ai.google.dev/gemini-api/docs/prompting-strategies (dostęp: 2026).
[8]. LM Studio – aplikacja do lokalnego uruchamiania modeli językowych, https://lmstudio.ai (dostęp: 2026).
[9]. Ollama – dokumentacja oficjalna, https://ollama.com oraz https://github.com/ollama/ollama (dostęp: 2026).

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

[10]. Ollama Python Library, https://github.com/ollama/ollama-python (dostęp: 2026).
[11]. Groq, Pricing (cennik), https://groq.com/pricing/ (dostęp: 2026).
[12]. Phoenix J., Taylor M., Skuteczna inżynieria promptów. Przyszłościowe rozwiązania dla rzetelnych wyników generatywnej AI, Helion, Gliwice 2025.
[13]. Mizrahi G., Inżynieria promptów bez tajemnic. Sztuka kreatywnego generowania tekstów, Helion, Gliwice 2025.

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

POLITECHNIKA LUBELSKA

Materiały do przedmiotu:

GENERATYWNA SZTUCZNA INTELIGENCJA I PROMPT ENGINEERING

Laboratorium nr 4.
Praktyczne tworzenie promptów z wykorzystaniem podstawowych metod promptowania

Autor:

dr inż. Krzysztof Dziedzic

dr Marcin Barszcz

Katedra Informatyki

Fundusze Europejskie dla Rozwoju Społecznego

Rzeczpospolita Polska

Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

POLLUB z nami nowoczesne technologie

POLITECHNIKA LUBELSKA

Fundusze Europejskie dla Rozwoju Społecznego

Rzeczpospolita Polska

Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

POLITECHNIKA LUBELSKA

Spis treści

Wprowadzenie, tematyka zajęć ---------- 73
Cel ćwiczenia/laboratorium ---------- 75
Instrukcja laboratoryjna/ćwiczeniowa ---------- 76
Ćwiczenia samodzielne ---------- 93
Pytania pomocnicze ---------- 94
Podsumowanie ---------- 95
Literatura ---------- 95

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

Wprowadzenie, tematyka zajęć

W niniejszym laboratorium przedstawiono podstawowe techniki prompt engineeringu: zero-shot, few-shot, chain-of-thought (CoT) oraz self-consistency. Techniki te tworzą naturalną hierarchię: na poziomie bazowym model otrzymuje wyłącznie instrukcję (zero-shot, także z frazą wyzwalającą rozumowanie), na poziomie przykładowym do instrukcji dołącza się pary wejście–wyjście (one-shot, few-shot), a na poziomie rozumowania model prowadzi jawny tok myślenia lub głosuje nad wieloma ścieżkami (CoT, self-consistency) [1]. Wraz z poziomem rośnie jakość odpowiedzi w zadaniach złożonych, ale również koszt (liczba tokenów i wywołań). Praktyczna reguła jest następująca: zaczynaj od najprostszej techniki i dodawaj złożoność tylko wtedy, gdy jakość odpowiedzi mierzalnie rośnie wraz ze zwiększaniem jej poziomu.

Zero-shot prompting. Technika, w której poprawna odpowiedź powstaje wyłącznie na podstawie wiedzy nabytej przez model podczas pretrenowania. Prompt nie zawiera żadnych przykładów oczekiwanego wyniku [1]. Skuteczny prompt zero-shot składa się z instrukcji zadania, niezbędnego kontekstu, specyfikacji formatu odpowiedzi oraz ograniczeń. Obowiązuje zasada: im precyzyjniejsza instrukcja, tym mniej potrzebnych przykładów. Zero-shot sprawdza się w zadaniach prostych i jednoznacznych (klasyfikacja, tłumaczenie, proste pytania), zapewniając najniższy koszt i najkrótszy czas odpowiedzi.

Warianty wzmacniające zero-shot. Jakość odpowiedzi zero-shot można podnieść bez dodawania przykładów: rola (np. „Jesteś doświadczonym inżynierem bezpieczeństwa…") dostarcza modelowi perspektywy eksperckiej; ograniczenia i instrukcje negatywne („Odpowiedz wyłącznie etykietą.", „Nie dodawaj wyjaśnień.") redukują zmienność formatu; samoweryfikacja („Po udzieleniu odpowiedzi sprawdź jej zgodność z treścią; jeśli trzeba, popraw.") zmniejsza liczbę błędów w pojedynczym wywołaniu; dekompozycja („najpierw wskaż kluczowe fakty, potem wyciągnij wniosek") daje efekty zbliżone do CoT [1].

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

Few-shot prompting. Przed właściwym zadaniem podaje się modelowi od jednego (one-shot) do kilku (zwykle 2–8) przykładów par wejście–wyjście. Przykłady pełnią rolę wzorca. Określają one format i styl odpowiedzi, zawężają interpretację zadania i działają jak „mini-trening" w obrębie jednego wywołania, bez zmiany wag modelu. Przykłady powinny mieć spójny, jednolity format; o skuteczności decyduje ich jakość i reprezentatywność (a nie liczba), a kolejność ma znaczenie. Model silniej wzoruje się na przykładach położonych bliżej końca promptu [2, 3]. Few-shot jest techniką pierwszego wyboru, gdy wymagany jest konkretny format wyjścia, którego zero-shot nie zapewnia.

Chain-of-thought (CoT). Technika, w której model przedstawia kolejne kroki rozumowania, zamiast od razu podawać gotową odpowiedź. Zamiast schematu pytanie–odpowiedź powstaje ślad: pytanie, krok 1, krok 2, ..., odpowiedź [4]. Pośrednie kroki pełnią rolę pamięci roboczej. Każdy wygenerowany token to dodatkowy krok obliczeń, więc CoT „daje modelowi czas na myślenie", choć nie dodaje mu wiedzy. Najprostszą odmianą jest zero-shot CoT. Do polecenia dopisuje się frazę wyzwalającą, np. „Myślmy krok po kroku" [5]. Odmiana few-shot CoT umieszcza w prompcie przykłady wraz z pełnym tokiem rozumowania. Przykład staje się wzorcem stylu myślenia [4]. CoT poprawia wyniki w zadaniach wieloetapowych (arytmetyka, logika, analiza kodu). W prostych zadaniach nie daje zysku, a wydłuża i podraża odpowiedź. Uwaga praktyczna: model llama-3.3-70b-versatile nie prowadzi ukrytego, wbudowanego rozumowania, dzięki czemu efekt frazy CoT obserwujesz w czystej formie.

Self-consistency. Zamiast jednej odpowiedzi generuje się N odpowiedzi na ten sam prompt przy temperaturze większej od zera (zwykle około 0,7), każda z pełnym tokiem rozumowania, a następnie wybiera się wynik najczęstszy (głosowanie większościowe) [6]. Technika działa, ponieważ ścieżki błędne prowadzą do różnych wyników i rozpraszają się w głosowaniu, podczas gdy odpowiedź poprawna powtarza się najczęściej. Warunkiem jest temperatura większa od zera (przy temperaturze 0 wszystkie ścieżki byłyby identyczne) oraz wyodrębnianie odpowiedzi końcowej w stałym formacie ułatwiającym zliczanie

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

**Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00**

głosów. Koszt rośnie N-krotnie, dlatego self-consistency rezerwuje się dla zadań, w których liczy się wiarygodność wyniku.

Dobór techniki. Wybór techniki zależy od charakteru zadania oraz dostępnych zasobów. Orientacyjne kryteria zestawiono w tabeli 4.1. W praktyce techniki łączy się: few-shot z CoT to najczęstszy standard produkcyjny, a dodanie self-consistency daje najwyższą jakość przy najwyższym koszcie [1].

Tabela 4.1. Dobór podstawowej techniki promptowania do zadania (zestawienie referencyjne)

Technika	Kiedy stosować	Warunek / zasoby
zero-shot	proste, jednoznaczne zadania (klasyfikacja, tłumaczenie, proste pytania)	priorytetem szybkość i niski koszt tokenów
few-shot	wymagany konkretny format lub styl wyjścia, którego zero-shot nie utrzymuje	kilka reprezentatywnych przykładów w spójnym formacie
chain-of-thought	rozumowanie wieloetapowe (arytmetyka, logika, analiza kodu)	wynik zależy od więcej niż jednego kroku
self-consistency	zadania krytyczne, wymagające wysokiej wiarygodności wyniku	budżet na N = 5–10 wywołań modelu

Cel ćwiczenia/laboratorium

Głównym celem laboratorium jest opanowanie podstawowych technik promptowania oraz umiejętność świadomego doboru techniki do zadania. Po ukończeniu zajęć student potrafi:

1. konstruować precyzyjne prompty zero-shot (instrukcja, kontekst, format, ograniczenia) oraz wzmacniać je rolą, instrukcjami negatywnymi i samoweryfikacją;

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

2. projektować prompty few-shot zgodnie z zasadami spójności formatu, jakości i kolejności przykładów oraz porównywać je z podejściem zero-shot;
3. wymuszać strukturalny format wyjścia (np. JSON) i weryfikować programistycznie jego poprawność;
4. stosować zero-shot CoT (frazę wyzwalającą rozumowanie) oraz few-shot CoT (przykłady z tokiem rozumowania) w zadaniach wieloetapowych;
5. zaimplementować technikę self-consistency: wielokrotne generowanie przy temperaturze większej od zera, ekstrakcję odpowiedzi i głosowanie większościowe;
6. dobierać technikę do zadania według kryteriów jakości i kosztu oraz stosować regułę stopniowej eskalacji złożoności;
7. porównywać eksperymentalnie skuteczność technik na modelu chmurowym i lokalnym oraz dokumentować wyniki tych porównań.

Instrukcja laboratoryjna/ćwiczeniowa

Zadanie 4.1. Zero-shot: precyzja instrukcji i warianty wzmacniające

Scenariusz zadania: dziekanat analizuje opinie studentów z ankiet ewaluacyjnych i chce automatycznie oznaczać ich wydźwięk. W jednym zadaniu (klasyfikacja wydźwięku opinii) porównasz cztery warianty promptu zero-shot: od podstawowej instrukcji po wariant z rolą i samoweryfikacją [1].

Treść zadania:

1. W katalogu roboczym utwórz plik `zero_shot_warianty.py` o następującej treści:

```
import os
import sys
from dotenv import load_dotenv
from groq import Groq
from groq import APIError

# poprawne wyswietlanie polskich znakow w konsoli Windows
```

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

```
sys.stdout.reconfigure(encoding='utf-8')

load_dotenv()
client = Groq(api_key=os.getenv('GROQ_API_KEY'))

# nazwy modeli
MODEL = 'llama-3.3-70b-versatile'
MODEL_LOKALNY = 'llama3.2'   # model zapasowy (Ollama)

OPINIE = [
    'Świetnie prowadzone laboratoria, wszystko było '
    'jasno wyjaśnione.',
    'Wykład jak wykład, da się przeżyć.',
    'Materiały były nieaktualne, a terminy oddania '
    'projektów zmieniano bez uprzedzenia.',
    'Prowadzący był bardzo pomocny i zawsze '
    'odpowiadał na pytania.',
    'Zajęcia bywały chaotyczne, ale zaliczenie '
    'przebiegło bez problemów.',
]

FORMAT = ('Odpowiedz wyłącznie jednym słowem: '
          'POZYTYWNA, NEUTRALNA albo NEGATYWNA.')
ROLA = ('Jesteś analitykiem ankiet ewaluacyjnych '
        'na uczelni technicznej. ')

WARIANTY = {
    'A - bazowy': 'Sklasyfikuj wydźwięk poniższej opinii
studenta.',
    'B - z formatem': 'Sklasyfikuj wydźwięk poniższej opinii '
                      'studenta. ' + FORMAT,
    'C - z rolą': ROLA + 'Sklasyfikuj wydźwięk poniższej '
                  'opinii studenta. ' + FORMAT,
    'D - z samoweryfikacją': ROLA + 'Sklasyfikuj wydźwięk '
                  'poniższej opinii studenta. Zanim odpowiesz, '
                  'sprawdź zgodność etykiety z treścią opinii. '
                  'W razie potrzeby ją popraw. ' + FORMAT,
```

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

```
}

for nazwa, instrukcja in WARIANTY.items():
    print(f'\n\n{"#" * 64}')
    print(f'###  WARIANT {nazwa}')
    print('#' * 64)
    for i, opinia in enumerate(OPINIE, start=1):
        prompt = f'{instrukcja}\nOpinia: """{opinia}"""'
        try:
            r = client.chat.completions.create(
                model=MODEL,
                messages=[{'role': 'user', 'content': prompt}],
                temperature=0.0,
                max_tokens=30,
            )
            odp =
r.choices[0].message.content.strip().replace('\n', ' ')
        except APIError as e:
            # np. blad 429 (limit zapytan) lub problem z kluczem
            print(f'[Blad Groq API] {e}')
            # awaryjne uzycie modelu lokalnego (Ollama)
            import ollama
            lok = ollama.generate(model=MODEL_LOKALNY,
prompt=prompt,
                                  options={'temperature': 0.0,
                                           'num_predict': 30})
            odp = lok['response'].strip().replace('\n', ' ')
            odp += f'   [model lokalny: {MODEL_LOKALNY}]'
        print(f'opinia {i}: {odp}')
```

2. Zapisz plik (Ctrl+S) i uruchom skrypt poleceniem `python zero_shot_warianty.py`. W przypadku braku dostępu do modelu chmurowego odpowiedź wygeneruje lokalny model.
3. Oceń wyniki pod dwoma względami: stabilność formatu (czy odpowiedź jest zawsze dokładnie jednym słowem z zadanej listy?) oraz trafność. Opinia druga jest celowo niejednoznaczna. Obserwuj, jak kolejne warianty radzą sobie z nią.

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

P: Zamieść zestawienie odpowiedzi. Sformułuj wnioski: (1) który wariant najstabilniej trzymał format jednego słowa i dlaczego? (2) Jak rola i samoweryfikacja wpłynęły na klasyfikację opinii jako niejednoznacznej? (3) Który element instrukcji dał największą poprawę w porównaniu z wariantem bazowym?

Zadanie 4.2. Kontekst i przykłady: zero-shot vs few-shot

Scenariusz zadania: dział wsparcia IT wydziału chce automatycznie klasyfikować zgłoszenia użytkowników do jednej z kategorii: SIEĆ, SPRZĘT, OPROGRAMOWANIE, KONTO – wraz z określeniem priorytetu (WYSOKI lub NORMALNY). W ćwiczeniu porównasz skuteczność podejścia zero-shot i few-shot oraz zastosujesz ograniczniki, aby oddzielić dane od instrukcji.

Treść zadania:

1. **Krok 1 – wariant zero-shot.** Dla każdego z trzech zgłoszeń testowych (poniżej) uruchom na modelu zewnętrznym llama-3.3-70b-versatile (Groq API) lub lokalnym llama3.1 (Ollama) prompt:

> *„Zaklasyfikuj poniższe zgłoszenie do jednej z kategorii: SIEĆ, SPRZĘT, OPROGRAMOWANIE, KONTO oraz określ priorytet (WYSOKI lub NORMALNY). Zgłoszenie: """{treść zgłoszenia}"""”*

Zgłoszenia testowe: (1) „W sali 305 nie działa rzutnik – komputer nie wykrywa urządzenia.”; (2) „Od rana nie mogę zalogować się do poczty uczelnianej mimo zresetowania hasła, a jutro mija termin zapisów na zajęcia.”; (3) „W laboratorium 102 co kilka minut przerywa się połączenie Wi-Fi podczas zajęć.”

2. Zanotuj odpowiedzi. Zwróć szczególną uwagę na format: czy jest jednolity we wszystkich trzech przypadkach i czy byłby łatwy do automatycznego przetworzenia (np. w skrypcie)?
3. **Krok 2 – wariant few-shot.** Rozbuduj prompt o trzy przykłady w spójnym formacie i ścisłe określenie formy odpowiedzi:

> *„Klasyfikujesz zgłoszenia serwisowe działu IT. Odpowiadaj wyłącznie w formacie: KATEGORIA | PRIORYTET.*

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

**Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00**

Zgłoszenie: """Monitor w pokoju 112 nie włącza się mimo wymiany kabla zasilającego."""
Odpowiedź: SPRZĘT | NORMALNY

Zgłoszenie: """Nie mogę uzyskać dostępu do systemu USOS – konto zostało zablokowane, a dziś kończy się rejestracja na seminaria."""
Odpowiedź: KONTO | WYSOKI

Zgłoszenie: """Po instalacji aktualizacji pakiet Office przestał otwierać pliki .docx."""
Odpowiedź: OPROGRAMOWANIE | NORMALNY

Zgłoszenie: """{treść nowego zgłoszenia}"""
Odpowiedź:"

4. Uruchom wariant few-shot dla tych samych trzech zgłoszeń testowych, na tym samym modelu co w kroku 1.
5. **Krok 3** – porównaj oba warianty: trafność klasyfikacji oraz powtarzalność formatu odpowiedzi.

W katalogu roboczym utwórz plik `klasyfikacja_zgloszen.py` o następującej treści. Skrypt uruchamia oba warianty (zero-shot i few-shot) dla wszystkich trzech zgłoszeń na obu modelach – llama-3.3-70b-versatile (Groq) i llama3.1 (Ollama) – i wypisuje wyniki w formie tabeli:

```
import os
import sys
import time
from dotenv import load_dotenv
from groq import Groq
from groq import APIError
import ollama

# poprawne wyswietlanie polskich znakow w konsoli Windows
sys.stdout.reconfigure(encoding='utf-8')
```

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

```
load_dotenv()
client = Groq(api_key=os.getenv('GROQ_API_KEY'))

# testujemy dwa modele: zewnetrzny (API) i lokalny (Ollama)
MODEL = 'llama-3.3-70b-versatile'
MODEL_LOKALNY = 'llama3.1'

# trzy zgloszenia testowe (te same, co w krokach recznych)
ZGLOSZENIA = [
    'W sali 305 nie działa rzutnik - komputer '
    'nie wykrywa urządzenia.',
    'Od rana nie mogę zalogować się do poczty '
    'uczelnianej mimo zresetowania hasła, a jutro '
    'mijają terminy zapisów na zajęcia.',
    'W laboratorium 208 co kilka minut zrywa się '
    'połączenie Wi-Fi podczas zajęć.',
]


def prompt_zero_shot(zgloszenie):
    return ('Zaklasyfikuj poniższe zgłoszenie do jednej '
            'z kategorii: SIEĆ, SPRZĘT, OPROGRAMOWANIE, '
            'KONTO oraz określ priorytet (WYSOKI lub '
            'NORMALNY). Odpowiadaj wyłącznie w formacie: '
            'KATEGORIA | PRIORYTET.\n'
            f'Zgłoszenie: """{zgloszenie}"""')


def prompt_few_shot(zgloszenie):
    return ('Klasyfikujesz zgłoszenia serwisowe działu '
            'IT. Odpowiadaj wyłącznie w formacie: '
            'KATEGORIA | PRIORYTET.\n'
            'Zgłoszenie: """Monitor w pokoju 112 nie '
            'włącza się mimo wymiany kabla."""\n'
            'Odpowiedź: SPRZĘT | NORMALNY\n'
            'Zgłoszenie: """Nie mogę uzyskać dostępu do '
```

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

```
            'USOS - konto zablokowane, dziś kończy się '
            'rejestracja."""\n'
            'Odpowiedź: KONTO | WYSOKI\n'
            'Zgłoszenie: """Po aktualizacji pakiet '
            'Office nie otwiera plików .docx."""\n'
            'Odpowiedź: OPROGRAMOWANIE | NORMALNY\n'
            f'Zgłoszenie: """{zgloszenie}"""\n'
            'Odpowiedź:')


def klasyfikuj_groq(prompt):
    try:
        r = client.chat.completions.create(
            model=MODEL,
            messages=[{'role': 'user', 'content': prompt}],
            temperature=0.0,
            max_tokens=20,
        )
        return r.choices[0].message.content.strip().replace('\n',
' ')
    except APIError as e:
        return '[Blad Groq API]'


def klasyfikuj_ollama(prompt):
    try:
        msgs = [{'role': 'user', 'content': prompt}]
        r = ollama.chat(model=MODEL_LOKALNY,
                        messages=msgs)
        txt = r['message']['content']
        return txt.strip().replace('\n', ' ')
    except Exception as e:
        return f'[Blad Ollama: {e}]'


# naglowek tabeli wynikow
print(f'{"NR":<3}{"WARIANT":<11}{"MODEL":<10}ODPOWIEDŹ')
```

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

```
print('-' * 60)

for i, zgl in enumerate(ZGLOSZENIA, start=1):
    warianty = [('zero-shot', prompt_zero_shot),
                ('few-shot', prompt_few_shot)]
    for opis, buduj in warianty:
        p = buduj(zgl)
        wg = klasyfikuj_groq(p)
        wl = klasyfikuj_ollama(p)
        print(f'{i:<3}{opis:<11}{"groq":<10}{wg}')
        print(f'{i:<3}{opis:<11}{"llama3.1":<10}{wl}')
    print('-' * 60)
```

Uruchom skrypt poleceniem `python klasyfikacja_zgloszen.py`. W tabeli, dla każdego zgłoszenia i wariantu (zero-shot, few-shot), zobaczysz odpowiedź obu modeli. Porównaj powtarzalność formatu KATEGORIA | PRIORYTET między wariantami.

P: Zamieść oba prompty oraz odpowiedzi dla trzech zgłoszeń w obu wariantach. Sformułuj wnioski: Jak przykłady wpłynęły na format i trafność odpowiedzi?

Zadanie 4.3. Few-shot ze strukturą wyjścia: ekstrakcja danych do formatu JSON

Scenariusz zadania: koło naukowe kataloguje ogłoszenia o praktykach studenckich publikowane w formie swobodnego tekstu. Aby zasilić bazę danych, każde ogłoszenie ma zostać zamienione na rekord JSON o stałej strukturze. Porównasz warianty zero-shot i few-shot pod kątem najważniejszego kryterium integracji: czy odpowiedź modelu można sparsować za pomocą programu [2].

Treść zadania:

1. W katalogu roboczym utwórz plik `ekstrakcja_json.py` (nagłówek pliku jak w zadaniu 4.1, dodatkowo `import json`) i dopisz część merytoryczną:

```
OGLOSZENIA = [
    'Firma NetSoft z Lublina poszukuje studenta na '
    'Praktyki jako junior tester. Mile widziane: '
    'Python i SQL. Wymiar: 2 dni w tygodniu.',
    'Praktyki wakacyjne w dziale IT szpitala '
    'wojewódzkiego - wsparcie helpdesk, wymagana '
    'znajomość systemu Windows. Pełny etat w lipcu.',
    'Startup Medi42 (praca zdalna) przyjmie '
    'praktykanta do zespołu frontend: React oraz '
    'TypeScript. Elastyczne godziny.',
]

POLA = ('firma, miasto, stanowisko, technologie '
        '(lista), wymiar')

# WARIANT NAIWNY (zero-shot): luzna instrukcja, bez slowa
"wylacznie"
# i bez przykladow. Model czesto opakowuje JSON w blok ```json
albo
# dodaje komentarz, przez co json.loads sie wywala.
PROMPT_ZERO = ('Wyodrębnij z ogłoszenia dane: ' + POLA +
               '. Jeśli danej brak, wpisz null. '
               'Podaj wynik w formacie JSON.\n'
               'Ogłoszenie: """{tekst}"""')

PROMPT_FEW = (
    'Zamieniasz ogłoszenia o praktykach na obiekty '
    'JSON z polami: ' + POLA + '. Jeśli danej brak, '
    'wpisz null. Zwracaj wyłącznie JSON.\n'
    'Ogłoszenie: """Biuro projektowe KOMPAS (Lublin) '
    'szuka praktykanta do działu GIS; wymagany '
    'QGIS; 3 dni w tygodniu."""\n'
    'JSON: {"firma": "KOMPAS", "miasto": "Lublin", '
    '"stanowisko": "praktykant GIS", "technologie": '
    '["QGIS"], "wymiar": "3 dni w tygodniu"}\n'
    'Ogłoszenie: """Zdalne praktyki w zespole DevOps '
    'firmy CloudPol: Docker, Linux."""\n'
```

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

```
    'JSON: {"firma": "CloudPol", "miasto": null, '
    '"stanowisko": "praktykant DevOps", '
    '"technologie": ["Docker", "Linux"], '
    '"wymiar": null}\n'
    'Ogłoszenie: """{tekst}"""\n'
    'JSON:'
)


def wyodrebnij(prompt):
    try:
        r = client.chat.completions.create(
            model=MODEL,
            messages=[{'role': 'user', 'content': prompt}],
            temperature=0.0,
            max_tokens=200,
        )
        return r.choices[0].message.content
    except APIError as e:
        # np. blad 429 (limit zapytan) lub problem z kluczem
        print(f'[Blad Groq API] {e}')
        # awaryjne uzycie modelu lokalnego (Ollama)
        import ollama
        lok = ollama.generate(model=MODEL_LOKALNY, prompt=prompt,
                              options={'temperature': 0.0,
                                       'num_predict': 200})
        return lok['response']


for nazwa, szablon in [('zero-shot', PROMPT_ZERO),
                       ('few-shot', PROMPT_FEW)]:
    print(f'\n\n{"#" * 64}')
    print(f'###  WARIANT {nazwa}')
    print('#' * 64)
    poprawne = 0
    for i, tekst in enumerate(OGLOSZENIA, start=1):
        odp = wyodrebnij(szablon.replace('{tekst}', tekst))
```

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

```
        # UWAGA: celowo NIE usuwamy blokow ```json ani prozy wokol
JSON.
        # Chcemy zobaczyc, ktory wariant zwraca czysty, parsowalny
JSON,
        # a ktory psuje parsowanie dodatkowymi
znacznikami/tekstem.
        odp = odp.strip()
        try:
            dane = json.loads(odp)
            status = 'OK'
            poprawne += 1
        except json.JSONDecodeError:
            status = 'BLAD PARSOWANIA'
        print(f'\nogloszenie {i}: {status}')
        print(odp)
    print(f'\n>>> Parsowalne odpowiedzi: {poprawne} '
          f'z {len(OGLOSZENIA)}')
```

2. Zapisz plik (Ctrl+S) i uruchom skrypt poleceniem `python ekstrakcja_json.py`. Skrypt wysyła 6 zapytań i dla każdej próbuje wykonać `json.loads()` – licznik na końcu każdego wariantu pokazuje, ile odpowiedzi dało się sparsować.
3. Porównaj oba warianty: liczbę parsowalnych odpowiedzi, kompletność pól (czy technologie są listą, czy brakuje miasta tam, gdzie go nie podano) oraz zgodność nazw pól z wymaganiami.

P: Zamieść wyniki obu wariantów wraz z licznikami parsowalności. Sformułuj wnioski: (1) który wariant zapewnił poprawny JSON we wszystkich przypadkach i co najczęściej psuło parsowanie w drugim? (2) dlaczego w integracji z innym oprogramowaniem parsowalność bywa ważniejsza od elegancji odpowiedzi?

Zadanie 4.4. Chain-of-thought: fraza „Myślmy krok po kroku”

Na dwóch zadaniach-pułapkach porównasz odpowiedzi bez rozumowania i z zero-shot CoT [4, 5]. Ponieważ model llama-3.3-70b-versatile nie prowadzi ukrytego,

wbudowanego rozumowania, różnica między wariantami pokazuje czysty efekt frazy wyzwalającej rozumowanie.

Treść zadania:

1. W katalogu roboczym utwórz plik `cot_rozumowanie.py` (nagłówek pliku jak w zadaniu 4.1) i dopisz część merytoryczną:

```
ZADANIA = {
    'A - zlozonosc algorytmu': (
        'Wyznacz złożoność czasową poniższego kodu:\n'
        'for i in range(n):\n'
        '    for j in range(i):\n'
        '        op()'
    ),
    'B - zadanie tekstowe': (
        'Do pracowni zamówiono 4 zestawy po 6 '
        'komputerów. Pięć komputerów okazało się '
        'wadliwych i je zwrócono, a następnie '
        'domówiono połowę liczby zwróconych '
        '(zaokrąglając w górę). Ile komputerów '
        'trafiło ostatecznie do pracowni?'
    ),
}

WARIANTY = {
    'bez CoT': 'Podaj samą odpowiedź, bez wyjaśnień.',
    'zero-shot CoT': ('Myślmy krok po kroku. Na końcu '
                      'podaj odpowiedź w osobnej linii '
                      'w formacie: ODPOWIEDŹ: ...'),
}

for nazwa, tresc in ZADANIA.items():
    print(f'\n\n{"#" * 64}')
    print(f'###  ZADANIE {nazwa}')
    print('#' * 64)
    for opis, dopisek in WARIANTY.items():
        prompt = f'{tresc}\n{dopisek}'
        print(f'\n--- wariant: {opis} ---')
```

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

```
        try:
            r = client.chat.completions.create(
                model=MODEL,
                messages=[{'role': 'user', 'content': prompt}],
                temperature=0.0,
                max_tokens=400,
            )
            print(r.choices[0].message.content.strip())
            tok_wy = r.usage.completion_tokens
            print(f'>>> tokeny wyjsciowe: {tok_wy}')
        except APIError as e:
            # np. blad 429 (limit zapytan) lub problem z kluczem
            print(f'[Blad Groq API] {e}')
            # awaryjne uzycie modelu lokalnego (Ollama)
            import ollama
            lok = ollama.generate(model=MODEL_LOKALNY,
prompt=prompt,
                                  options={'temperature': 0.0,
                                           'num_predict': 400})
            print(f'--- odpowiedz modelu lokalnego:
{MODEL_LOKALNY} ---')
            print(lok['response'].strip())
            print(f'>>> tokeny wyjsciowe: {lok["eval_count"]}')
```

2. Zapisz plik (Ctrl+S) i uruchom skrypt poleceniem `python cot_rozumowanie.py`.
3. Zweryfikuj poprawność odpowiedzi. W zadaniu A poprawny wynik to O(n²) – granica pętli wewnętrznej zależy od i, więc liczba operacji to suma 0 + 1 + … + (n−1). W zadaniu B poprawny wynik to 22 komputery (24 − 5 + 3). Sprawdź, czy wariant bez rozumowania popełnił błąd (np. odpowiedź O(n)) i czy fraza CoT go wyeliminowała.
4. Porównaj liczbę tokenów wyjściowych w obu wariantach. Jest to bezpośrednia cena jawnego rozumowania.

P: Zamieść odpowiedzi dla obu wariantów w obu zadaniach wraz z liczbą tokenów wyjściowych. Sformułuj wnioski: (1) w którym zadaniu fraza CoT zmieniła poprawność wyniku i dlaczego właśnie tam? (2) ile razy wzrosła

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

długość odpowiedzi po włączeniu rozumowania? (3) dla jakich zadań stosowanie CoT nie ma uzasadnienia?

Zadanie 4.5. Self-consistency: głosowanie większościowe

Pojedyncza ścieżka rozumowania przy temperaturze większej od zera bywa błędna. W tym zadaniu zaimplementujesz technikę self-consistency. Wygenerujesz pięć niezależnych ścieżek rozumowania, wyodrębnisz z nich odpowiedzi końcowe i wybierzesz wynik większością głosów [6].

Treść zadania:

1. W katalogu roboczym utwórz plik `self_consistency.py` (nagłówek pliku jak w zadaniu 4.1, dodatkowo `import re` oraz `from collections import Counter`) i dopisz część merytoryczną:

```
ZADANIE = ('Trzej studenci testują aplikację. Każdy '
           'z nich zgłosił 8 błędów, ale 5 błędów '
           'powtórzyło się u dokładnie dwóch osób, '
           'a 2 błędy zgłosili wszyscy trzej. Ile '
           'unikalnych błędów zgłoszono łącznie?')

PROMPT = (ZADANIE + ' Myślmy krok po kroku. Na końcu '
          'podaj wynik w osobnej linii w formacie: '
          'WYNIK: <liczba>')

N = 5
glosy = []

for i in range(1, N + 1):
    try:
        r = client.chat.completions.create(
            model=MODEL,
            messages=[{'role': 'user', 'content': PROMPT}],
            temperature=0.7,
            max_tokens=400,
        )
```

```
        odp = r.choices[0].message.content.strip()
    except APIError as e:
        # np. blad 429 (limit zapytan) lub problem z kluczem
        print(f'[Blad Groq API] {e}')
        # awaryjne uzycie modelu lokalnego (Ollama)
        import ollama
        lok = ollama.generate(model=MODEL_LOKALNY, prompt=PROMPT,
                              options={'temperature': 0.7,
                                       'num_predict': 800})
        odp = lok['response'].strip()
        odp += f'\n[model lokalny: {MODEL_LOKALNY}]'
    m = re.search(r'WYNIK:\s*(-?\d+)', odp)
    wynik = m.group(1) if m else 'brak'
    glosy.append(wynik)
    print(f'\n--- sciezka {i} -> wynik: {wynik} ---')
    print(odp[:300])

print(f'\n{"#" * 64}')
print('###  GLOSOWANIE WIEKSZOSCIOWE')
print('#' * 64)
rozklad = Counter(glosy)
print('Rozklad glosow:', dict(rozklad))
finalny, liczba = rozklad.most_common(1)[0]
print(f'Odpowiedz finalna: {finalny} ({liczba} z {N} glosow)')
```

2. Zapisz plik (Ctrl+S) i uruchom skrypt poleceniem `python self_consistency.py`. Skrypt celowo używa temperatury 0,7. Przy temperaturze 0 wszystkie ścieżki byłyby identyczne, a głosowanie nie miałoby sensu [6].
3. Zweryfikuj wynik: poprawna odpowiedź to 15 unikalnych błędów (24 zgłoszenia − 5 duplikatów z par − 2·2 duplikaty z trójki). Sprawdź w wydruku, czy któraś pojedyncza ścieżka podała błędny wynik i czy głosowanie go „przegłosowało".
4. Uruchom skrypt ponownie i porównaj rozkłady głosów obu uruchomień. Czy ostateczna odpowiedź jest stabilna mimo różnych ścieżek rozumowania?

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

P: Zamieść rozkłady głosów z dwóch uruchomień oraz odpowiedź końcową. Sformułuj wnioski: (1) czy zdarzyła się błędna ścieżka i jak zachowało się głosowanie? (2) dlaczego technika wymaga temperatury większej od zera? (3) koszt wzrósł pięciokrotnie – dla jakich zastosowań taka cena jest uzasadniona, a dla jakich nie?

Zadanie 4.6. Analiza przypadku: eskalacja technik przy generowaniu SQL

Scenariusz zadania: system raportowy dziekanatu generuje zapytania SQL na podstawie poleceń w języku naturalnym. Baza zawiera tabele: `studenci(id, imie, nazwisko, rok_studiow)` oraz `oceny(id, student_id, przedmiot, ocena)`. Polecenie testowe wymaga podzapytania, przez co proste techniki często się nie sprawdzają. Zastosujesz regułę eskalacji: od zero-shot, przez zero-shot CoT i few-shot, po few-shot CoT i sprawdzisz, na którym poziomie zapytanie staje się poprawne [1].

Treść zadania:

1. W katalogu roboczym utwórz plik `eskalacja_sql.py` (nagłówek pliku jak w zadaniu 4.1) i dopisz część merytoryczną:

```
SCHEMAT = ('Tabele: studenci(id, imie, nazwisko, '
           'rok_studiow); oceny(id, student_id, '
           'przedmiot, ocena).')

POLECENIE = ('Podaj imiona i nazwiska studentów, '
             'których średnia ocen jest wyższa niż '
             'średnia ocen wszystkich studentów.')

PRZYKLADY = (
    'Polecenie: Podaj nazwiska studentów trzeciego '
    'roku.\n'
    'SQL: SELECT nazwisko FROM studenci WHERE '
    'rok_studiow = 3;\n'
    'Polecenie: Podaj średnią ocen każdego studenta.\n'
    'SQL: SELECT s.imie, s.nazwisko, AVG(o.ocena) '
    'FROM studenci s JOIN oceny o ON s.id = '
```

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

```
    'o.student_id GROUP BY s.id, s.imie, s.nazwisko;\n'
)

COT = ('Myślmy krok po kroku: najpierw zaplanuj '
       'potrzebne obliczenia, a na końcu podaj gotowe '
       'zapytanie SQL.')

POZIOMY = {
    '1 - zero-shot': (SCHEMAT + ' Napisz zapytanie SQL. '
                      + POLECENIE + ' Zwróć wyłącznie SQL.'),
    '2 - zero-shot CoT': (SCHEMAT + ' Napisz zapytanie '
                          'SQL. ' + POLECENIE + ' ' + COT),
    '3 - few-shot': (SCHEMAT + ' Piszesz zapytania SQL.\n'
                     + PRZYKLADY + 'Polecenie: '
                     + POLECENIE + '\nSQL:'),
    '4 - few-shot CoT': (SCHEMAT + ' Piszesz zapytania '
                         'SQL.\n' + PRZYKLADY
                         + 'Polecenie: ' + POLECENIE
                         + '\n' + COT),
}

for nazwa, prompt in POZIOMY.items():
    print(f'\n\n{"#" * 64}')
    print(f'###  POZIOM {nazwa}')
    print('#' * 64)
    try:
        r = client.chat.completions.create(
            model=MODEL,
            messages=[{'role': 'user', 'content': prompt}],
            temperature=0.0,
            max_tokens=350,
        )
        print(r.choices[0].message.content.strip())
    except APIError as e:
        # np. blad 429 (limit zapytan) lub problem z kluczem
        print(f'[Blad Groq API] {e}')
        # awaryjne uzycie modelu lokalnego (Ollama)
```

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

```
        import ollama
        lok = ollama.generate(model=MODEL_LOKALNY, prompt=prompt,
                              options={'temperature': 0.0,
                                       'num_predict': 350})
        print(f'--- odpowiedz modelu lokalnego: {MODEL_LOKALNY} ---')
        print(lok['response'].strip())
```

2. Zapisz plik (Ctrl+S) i uruchom skrypt poleceniem `python eskalacja_sql.py`.
3. Oceń zapytanie z każdego poziomu według trzech kryteriów poprawności: (a) średnia ocen studenta jest obliczana po złączeniu tabel i grupowaniu (JOIN + GROUP BY), (b) porównanie następuje z podzapytaniem wyznaczającym średnią wszystkich ocen (AVG w podzapytaniu, a nie bezpośrednio w WHERE), (c) wynik zawiera imię i nazwisko. Uzupełnij tabelę 4.2.

Tabela 4.2. Ocena zapytań SQL wygenerowanych na kolejnych poziomach eskalacji techniki

Poziom techniki	Zapytanie poprawne? (T/N)	Uwagi (które kryterium zawiodło)
1 – zero-shot		
2 – zero-shot CoT		
3 – few-shot		
4 – few-shot CoT		

P: Zamieść cztery wygenerowane zapytania oraz uzupełnioną tabelę 4.2. Sformułuj wnioski: (1) od którego poziomu zapytanie spełnia wszystkie kryteria? (2) który składnik (przykłady czy rozumowanie) okazał się w tym zadaniu ważniejszy?

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

Ćwiczenia samodzielne

Ćwiczenie 4.1 (podstawowe). Warianty zero-shot na modelu lokalnym

Powtórz eksperyment z zadania 4.1 na modelu lokalnym `Llama 3.2` (wywołanie `ollama.generate()` z parametrem `options={'temperature': 0}`). W sprawozdaniu przedstaw zestawienie odpowiedzi (4 warianty na 3 opinie) oraz wniosek: czy mały model lokalny reaguje na rolę, wymuszenie formatu i samoweryfikację równie wyraźnie jak model chmurowy? Który wariant okazał się dla niego niezbędny, aby zachować format jednego słowa?

Ćwiczenie 4.2 (średnie). Few-shot CoT: klasyfikacja komunikatów o błędach

Dział utrzymania systemów mapuje komunikaty o błędach na wewnętrzne kody kategorii (np. NET_TIMEOUT, AUTH_DENIED, STORAGE_FULL, DB_DEADLOCK). Przygotuj prompt few-shot CoT zawierający trzy przykłady z krótkim tokiem rozumowania (komunikat, analiza przyczyny, kod kategorii), a następnie przetestuj go na czterech nowych komunikatach, z których co najmniej jeden nie pasuje wprost do żadnego wzorca. Porównaj wyniki z wariantem few-shot bez rozumowania (same pary komunikat–kod). Przedstaw oba prompty, odpowiedzi oraz wniosek: w których przypadkach ślad rozumowania poprawił trafność klasyfikacji?

Ćwiczenie 4.3 (średnie). Few-shot: klasyfikacja komunikatów commit

Przygotuj prompt few-shot do klasyfikacji opisów commitów Git zgodnie z konwencją Conventional Commits (typy: `feat`, `fix`, `docs`, `refactor`, `test`). Umieść w prompcie 3–4 przykłady w spójnym formacie, a następnie przetestuj klasyfikację na 5 własnych opisach commitów – w wariancie zero-shot oraz few-shot. Policz odsetek poprawnych klasyfikacji w obu wariantach.

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

Ćwiczenie 4.4 (trudne). Wpływ liczby ścieżek N na self-consistency

Zbadaj, jak liczba ścieżek wpływa na skuteczność i koszt techniki self-consistency. Dla zadania 4.5 (lub własnego zadania tekstowego z pułapką) uruchom głosowanie dla N równego 1, 3, 5 i 9, powtarzając każdy eksperyment 3 razy. Dla każdej wartości N wyznacz odsetek poprawnych odpowiedzi końcowych oraz łączną liczbę tokenów wyjściowych. Eksperyment wykonaj na obu modelach (chmurowym i lokalnym). Przedstaw kompletny raport: skrypt, tabelę wyników, wykres lub zestawienie koszt–jakość oraz wniosek, od której wartości N dalsze zwiększanie liczby ścieżek przestaje się opłacać.

Pytania pomocnicze

1. Jak dzieli się podstawowe techniki promptowania według złożoności (poziomy: bazowe, przykładowe, rozumowanie) i jak wraz z poziomem zmieniają się jakość oraz koszt?
2. Z jakich elementów składa się skuteczny prompt zero-shot i dla jakich zadań ta technika jest wystarczająca?
3. Wymień i krótko opisz trzy warianty wzmacniające zero-shot (np. rola, instrukcje negatywne, samoweryfikacja). Kiedy warto po nie sięgnąć?
4. Jakie zasady konstruowania przykładów few-shot (wytyczne Google): co jest ważniejsze od liczby przykładów i dlaczego ich kolejność ma znaczenie?
5. Dlaczego few-shot działa, mimo że nie zmienia wag modelu? Jaką rolę pełnią przykłady w obrębie pojedynczego wywołania?
6. Na czym polega mechanizm chain-of-thought i dlaczego mówi się, że CoT „daje modelowi czas na myślenie", ale nie dodaje mu wiedzy? W jakich zadaniach CoT nie przynosi korzyści?
7. Czym jest zero-shot CoT i jaka fraza go wywołuje? Dlaczego warto narzucić jednocześnie stały format odpowiedzi końcowej?

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

8. Czym few-shot CoT różni się od zwykłego few-shot i w jakich zadaniach ta różnica jest największa?
9. Opisz algorytm self-consistency (generowanie, ekstrakcja, głosowanie). Dlaczego technika wymaga temperatury wyższej niż 0°C?

Podsumowanie

Laboratorium poświęcone było podstawowym technikom promptowania oraz ich świadomemu doborowi. Po jego ukończeniu student:

1. rozróżnia poziomy technik promptowania (bazowe, przykładowe, rozumowanie) oraz zna zależność między ich złożonością, jakością i kosztem;
2. konstruuje precyzyjne prompty zero-shot i wzmacnia je rolą, ograniczeniami, instrukcjami negatywnymi oraz samoweryfikacją;
3. projektuje prompty few-shot zgodnie z zasadami spójności formatu i jakości przykładów oraz wymusza strukturalne, parsowalne wyjście (np. JSON);
4. stosuje zero-shot CoT i few-shot CoT w zadaniach wieloetapowych, uwzględniając tokenowy koszt jawnego rozumowania;
5. implementuje technikę self-consistency: wielokrotne generowanie, ekstrakcję odpowiedzi końcowej i głosowanie większościowe;
6. dobiera technikę do zadania według kryteriów z tabeli 4.1 i stosuje regułę stopniowej eskalacji złożoności.

UWAGA: rozszerz bibliotekę promptów prowadzoną od Laboratorium 2 (plik `prompts.md` lub program AI Prompt Lab) o kolumnę z zastosowaną techniką promptowania. Obok konfiguracji parametrów (Laboratorium 3) technika jest drugą informacją, bez której trudno odtworzyć działanie promptu. Ten sam tekst zadania działa inaczej jako zero-shot, few-shot czy z frazą wyzwalającą rozumowanie.

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

Literatura

[1]. Gadesha V., Prompt Engineering Techniques, IBM Think, https://www.ibm.com/think/topics/prompt-engineering-techniques (dostęp: 2026).

[2]. Google Cloud, Include few-shot examples – Gemini, projektowanie promptów, https://docs.cloud.google.com/gemini-enterprise-agent-platform/models/prompts/few-shot-examples (dostęp: 2026).

[3]. Google, Prompting strategies – Gemini API, https://ai.google.dev/gemini-api/docs/prompting-strategies (dostęp: 2026).

[4]. Wei J. I in., Chain-of-Thought Prompting Elicits Reasoning in Large Language Models, arXiv:2201.11903, 2022.

[5]. Kojima T. I., Large Language Models are Zero-Shot Reasoners, arXiv:2205.11916, 2022.

[6]. Wang X. I in., Self-Consistency Improves Chain of Thought Reasoning in Language Models, arXiv:2203.11171, 2023.

[7]. Ollama – dokumentacja oficjalna: https://ollama.com oraz https://github.com/ollama/ollama (dostęp: 2026).

[8]. Ollama Python Library, https://github.com/ollama/ollama-python (dostęp: 2026).

[9]. Groq, Text/Chat Generation – dokumentacja API, https://console.groq.com/docs/text-chat (dostęp: 2026).

[10]. Phoenix J., Taylor M., Skuteczna inżynieria promptów. Przyszłościowe rozwiązania dla rzetelnych wyników generatywnej AI, Helion, Gliwice 2025.

[11]. Mizrahi G., Inżynieria promptów bez tajemnic. Sztuka kreatywnego generowania tekstów, Helion, Gliwice 2025.

[12]. Groq, Model deprecations (harmonogram wyłączania modeli), https://console.groq.com/docs/deprecations (dostęp: 2026).

Fundusze Europejskie dla Rozwoju Społecznego

Rzeczpospolita Polska

Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

Materiały do przedmiotu:

GENERATYWNA SZTUCZNA INTELIGENCJA I PROMPT ENGINEERING

Laboratorium nr 5.
Praktyczne tworzenie promptów z wykorzystaniem zaawansowanych metod promptowania

Autor:

dr inż. Krzysztof Dziedzic

dr Marcin Barszcz

Katedra Informatyki

Fundusze Europejskie dla Rozwoju Społecznego

Rzeczpospolita Polska

Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

POLLUB z nami nowoczesne technologie

POLITECHNIKA LUBELSKA

Fundusze Europejskie dla Rozwoju Społecznego

Rzeczpospolita Polska

Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

POLITECHNIKA LUBELSKA

Spis treści

Wprowadzenie, tematyka zajęć ---------- 99
Cel ćwiczenia/laboratorium ---------- 102
Instrukcja laboratoryjna/ćwiczeniowa ---------- 102
Ćwiczenia samodzielne ---------- 115
Pytania pomocnicze ---------- 116
Podsumowanie ---------- 117
Literatura ---------- 118

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

Wprowadzenie, tematyka zajęć

W niniejszym laboratorium przedstawiono zaawansowane techniki prompt engineeringu. Techniki zaawansowane budują z wywołań całe procesy. Dzielą złożone zadanie na etapy, każą modelowi oceniać i poprawiać własne odpowiedzi, porównują konkurencyjne pomysły, a nawet pozwalają mu korzystać z zewnętrznych narzędzi [1]. Wspólnym mianownikiem jest sterowanie modelem z poziomu programu. Skrypt w Pythonie staje się orkiestratorem, który buduje prompty, przekazuje wyniki między wywołaniami i podejmuje decyzje na podstawie otrzymanych odpowiedzi.

Łańcuch promptów (prompt chaining). Zadanie złożone dzieli się na podzadania realizowane sekwencyjnie. Wynik jednego wywołania staje się wejściem dla następnego, a wynik ostatniego kroku jest wynikiem końcowym. Mniejsze prompty poprawiają sterowalność, ułatwiają diagnozowanie błędów (widać, który krok zawiódł) i podnoszą dokładność; spójny format wyjścia w każdym kroku ułatwia łączenie [2, 3]. Gdy podzadania są od siebie niezależne, można je uruchamiać równolegle i na końcu scalić odpowiedzi (agregacja) [3]. Pokrewną techniką jest least-to-most: model najpierw sam dekomponuje problem na podproblemy od najprostszego do najtrudniejszego, a potem rozwiązuje je kolejno, korzystając z wcześniejszych rozwiązań [4].

Generowanie wiedzy i cofnięcie do zasady (step-back). Zamiast pytać wprost, można najpierw poprosić model o wypisanie kluczowych faktów lub reguł dotyczących tematu, a dopiero w drugim kroku o odpowiedź opartą na tej wygenerowanej wiedzy [5]. Wariant step-back każe modelowi „cofnąć się” do ogólnej zasady (np. reguły protokołu, prawa fizyki), a następnie zastosować ją do konkretnego przypadku. Obie techniki poprawiają odpowiedzi na pytania podchwytliwe i zmniejszają ryzyko zmyśleń, ponieważ rozdzielają przywoływanie wiedzy od wnioskowania.

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

Samodoskonalenie (self-refinement). Model potrafi poprawiać własne odpowiedzi, gdy proces zostanie podzielony na role: generatora, krytyka i korektora. W pętli samodoskonalenia ta sama treść przechodzi cykl: wygeneruj, oceń względem jawnej listy kryteriów, popraw zgodnie z krytyką [6]. Kluczem są konkretne, sprawdzalne kryteria oceny. Bez nich krytyka sprowadza się do ogólników. W praktyce 1–2 iteracje dają największy zysk. Kolejne poprawiają coraz mniej, a kosztują tyle samo.

Drzewo myśli (tree of thoughts). Zamiast jednej ścieżki rozwiązania model generuje kilka konkurencyjnych koncepcji (gałęzi), każda z nich jest oceniana, najczęściej przez model w roli sędziego z jawnymi kryteriami i stałym formatem oceny, a najlepiej oceniona gałąź zostaje rozwinięta w pełne rozwiązanie [7]. Technika sprawdza się w zadaniach projektowych i otwartych, gdzie istnieje wiele sensownych podejść. Różnorodność gałęzi uzyskuje się wysoką temperaturą, a obiektywność oceny temperaturą zerową sędziego.

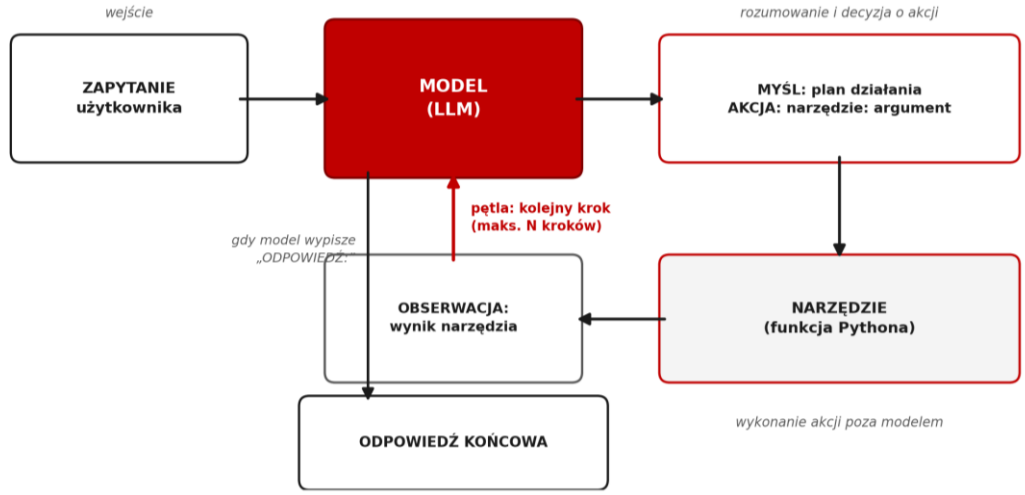
Meta-prompting. Technika, w której model nie rozwiązuje zadania, lecz generuje prompt dla innego (lub tego samego) modelu [8]. Meta-prompt opisuje wymagania wobec promptu docelowego: rolę, punkty kontrolne, format wyniku i ograniczenia. Podejście to automatyzuje pracę inżyniera promptów. Pozwala tworzyć całe rodziny wyspecjalizowanych promptów z jednego szablonu (np. recenzent kodu dla różnych języków i aspektów) a jakość wygenerowanego promptu weryfikuje się, uruchamiając go na próbce danych.

Agent ReAct: rozumowanie i działanie. ReAct (ang. reasoning and acting) łączy rozumowanie modelu z wykonywaniem akcji poza modelem [9]. Model działa w pętli: formułuje myśl (MYŚL), wybiera akcję z listy dostępnych narzędzi (AKCJA), a program wykonuje narzędzie (np. funkcję Pythona) i dokleja wynik jako obserwację (OBSERWACJA), po czym model kontynuuje, aż wypisze odpowiedź końcową (rysunek 5.1). Narzędzia dają modelowi dostęp do danych i obliczeń, których nie ma w wagach (bazy danych, kalkulacje, wyszukiwanie), a ścisły format komunikatów

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

pozwala programowi niezawodnie parsować decyzje modelu. Jest to podstawa współczesnych agentów AI.



Rysunek 5.1. Pętla agenta ReAct: model na przemian rozumuje i zleca akcje narzędziom, a ich wyniki wracają jako obserwacje.

Dobór techniki. Techniki zaawansowane kosztują wielokrotność pojedynczego wywołania, dlatego stosuje się je tam, gdzie techniki podstawowe zawodzą. Orientacyjne kryteria zestawia tabela 5.1. Reguła eskalacji z Laboratorium 4 pozostaje aktualna: zwiększaj złożoność procesu tylko wtedy, gdy jakość wyniku rośnie mierzalnie [1].

Tabela 5.1. Zaawansowane techniki promptowania i ich typowe zastosowania (zestawienie referencyjne)

Technika	Idea	Typowe zastosowanie
łańcuch promptów	sekwencja wywołań: wynik kroku wejściem następnego	wieloetapowe przetwarzanie tekstu, automatyzacja procesów
generowanie wiedzy / step-back	najpierw fakty lub zasada ogólna, potem odpowiedź	pytania podchwytliwe, redukcja zmyśleń

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

**Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00**

Technika	Idea	Typowe zastosowanie
samodoskonalenie	pętla: generuj, krytykuj wg kryteriów, popraw	poprawa jakości kodu i dokumentów
drzewo myśli	kilka koncepcji, ocena sędziego, rozwinięcie najlepszej	zadania projektowe i otwarte
meta-prompting	model generuje prompt dla modelu	rodziny wyspecjalizowanych promptów
agent ReAct	pętla MYŚL, AKCJA, OBSERWACJA z narzędziami	dostęp do danych i obliczeń spoza modelu

Cel ćwiczenia/laboratorium

Głównym celem laboratorium jest opanowanie zaawansowanych technik promptowania, w których skrypt w Pythonie orkiestruje wiele wywołań modelu. Po ukończeniu zajęć student potrafi:

1. zaprojektować i zaimplementować łańcuch promptów, w którym wynik jednego wywołania modelu jest wejściem następnego, oraz porównać go z pojedynczym promptem zbiorczym;
2. stosować techniki generowania wiedzy i step-back do poprawy odpowiedzi na pytania podchwytliwe;
3. zbudować pętlę samodoskonalenia (generacja, krytyka według jawnych kryteriów, poprawa) i ocenić zysk z kolejnych iteracji;
4. zaimplementować uproszczone drzewo myśli: generowanie konkurencyjnych koncepcji przy wysokiej temperaturze, ocenę modelem-sędzią w stałym formacie i rozwinięcie najlepszej gałęzi;
5. tworzyć meta-prompty generujące wyspecjalizowane prompty oraz weryfikować jakość wygenerowanych promptów na próbce danych;
6. zbudować prostego agenta ReAct: pętlę MYŚL–AKCJA–OBSERWACJA z narzędziami w Pythonie, parsowaniem akcji i limitem kroków;
7. dobierać technikę zaawansowaną do zadania, z uwzględnieniem kosztu wielokrotnych wywołań, oraz dokumentować przebieg procesów wieloetapowych.

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

POLITECHNIKA LUBELSKA

Instrukcja laboratoryjna/ćwiczeniowa

Zadanie 5.1. Łańcuch promptów: automatyczna obsługa zgłoszenia

Scenariusz zadania: dział wsparcia IT chce zautomatyzować pierwszą reakcję na zgłoszenia. Proces składa się z trzech kroków: streszczenia chaotycznego zgłoszenia, klasyfikacji (format KATEGORIA | PRIORYTET znany z zadania 4.2 [2]) oraz przygotowania uprzejmej odpowiedzi do zgłaszającego. Zaimplementujesz ten proces jako łańcuch promptów [3] i porównasz go z pojedynczym promptem zbiorczym. W tym zadaniu powstaje też funkcja pomocnicza `zapytaj()`, z której będą korzystać wszystkie kolejne skrypty tego laboratorium.

Treść zadania:

1. W katalogu roboczym utwórz plik `lancuch_promptow.py` o następującej treści:

```
import os
import sys
from dotenv import load_dotenv
from groq import Groq
from groq import APIError

# poprawne wyswietlanie polskich znakow w konsoli Windows
sys.stdout.reconfigure(encoding='utf-8')

load_dotenv()
client = Groq(api_key=os.getenv('GROQ_API_KEY'))

# nazwy modeli w jednym miejscu - latwiej je podmienic
MODEL = 'llama-3.3-70b-versatile'
MODEL_LOKALNY = 'llama3.2'   # model zapasowy (Ollama)


def zapytaj(prompt, temp=0.0, limit=300, stop=None):
    """Jedno wywolanie modelu; przy bledzie API uzywa
```

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

```
    modelu lokalnego. Parametr stop przyda sie w zad. 5.6."""
    try:
        r = client.chat.completions.create(
            model=MODEL,
            messages=[{'role': 'user', 'content': prompt}],
            temperature=temp,
            max_tokens=limit,
            stop=stop,
        )
        return r.choices[0].message.content.strip()
    except APIError as e:
        # np. blad 429 (limit zapytan) lub problem z kluczem
        print(f'[Blad Groq API] {e}')
        print(f'(odpowiedz wygeneruje model lokalny: '
              f'{MODEL_LOKALNY})')
        import ollama
        lok = ollama.generate(model=MODEL_LOKALNY, prompt=prompt,
                              options={'temperature': temp,
                                       'num_predict': limit,
                                       'stop': stop})
        return lok['response'].strip()


ZGLOSZENIE = (
    'Dzień dobry, pisałem już wczoraj ale bez odzewu, '
    'więc ponawiam. Od poniedziałku po zalogowaniu do '
    'komputera w sali 208 wszystko strasznie długo się '
    'otwiera, a przy próbie wejścia na dysk sieciowy '
    'P: wyskakuje błąd o braku dostępu. Kolega obok ma '
    'to samo. W piątek oddajemy projekt zaliczeniowy '
    'i muszę dostać się do plików. Pozdrawiam, student '
    'II roku informatyki.'
)

# krok 1: streszczenie
streszczenie = zapytaj(
    'Streść poniższe zgłoszenie w maksymalnie dwóch '
```


Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

```
    'zdaniach, rzeczowo, zachowując informacje istotne '
    'technicznie:\n"""' + ZGLOSZENIE + '"""')
print(f'\n--- KROK 1: streszczenie ---\n{streszczenie}')

# krok 2: klasyfikacja (wynik kroku 1 jest wejsciem)
klasyfikacja = zapytaj(
    'Na podstawie streszczenia zgłoszenia przypisz '
    'kategorię (SIEĆ, SPRZĘT, OPROGRAMOWANIE, KONTO) '
    'i priorytet (WYSOKI lub NORMALNY). Odpowiedz '
    'wyłącznie w formacie: KATEGORIA | PRIORYTET.\n'
    'Streszczenie: """' + streszczenie + '"""',
    limit=20)
print(f'\n--- KROK 2: klasyfikacja ---\n{klasyfikacja}')

# krok 3: odpowiedz do zglaszajacego (uzywa krokow 1 i 2)
odpowiedz = zapytaj(
    'Napisz uprzejmą odpowiedź (3-4 zdania) do studenta '
    'zgłaszającego problem. Potwierdź przyjęcie '
    'zgłoszenia, wskaż nadaną kategorię i priorytet '
    'oraz poinformuj o przekazaniu sprawy technikom.\n'
    'Streszczenie: """' + streszczenie + '"""\n'
    'Klasyfikacja: ' + klasyfikacja)
print(f'\n--- KROK 3: odpowiedz ---\n{odpowiedz}')

# wariant zbiorczy: wszystko w jednym prompcie
zbiorczo = zapytaj(
    'Przeczytaj zgłoszenie, streść je w dwóch zdaniach, '
    'przypisz kategorię (SIEĆ, SPRZĘT, OPROGRAMOWANIE, '
    'KONTO) i priorytet w formacie KATEGORIA | PRIORYTET '
    'oraz napisz uprzejmą odpowiedź do zgłaszającego.\n'
    'Zgłoszenie: """' + ZGLOSZENIE + '"""',
    limit=500)
print(f'\n--- WARIANT ZBIORCZY (1 prompt) ---\n{zbiorczo}')
```

2. Zapisz plik (Ctrl+S) i uruchom skrypt poleceniem python `lancuch_promptow.py`. Skrypt wykonuje 4 wywołania modelu: trzy kroki

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

łańcucha oraz wariant zbiorczy do porównania. W przypadku braku dostępu do modelu chmurowego odpowiedzi wygeneruje model lokalny.

3. Porównaj wyniki łańcucha z wariantem zbiorczym: czy klasyfikacja w wariancie zbiorczym ma dokładnie format KATEGORIA | PRIORYTET i czy dałoby się ją niezawodnie wyciąć za pomocą programu? W którym podejściu łatwiej zlokalizować i poprawić słabe ogniwo?
4. Zmień treść ZGLOSZENIE na inną (np. problem z kontem pocztowym) i uruchom ponownie – sprawdź, czy łańcuch działa bez modyfikacji promptów.

P: Zamieść wydruki trzech kroków łańcucha i wariantu zbiorczego. Sformułuj wnioski: (1) które podejście zapewnia bardziej przewidywalny format wyników pośrednich i dlaczego ma to znaczenie przy integracji z innym oprogramowaniem? (2) jak łańcuch ułatwia diagnozowanie błędów? (3) jaki jest koszt łańcucha (liczba wywołań) względem promptu zbiorczego?

Zadanie 5.2. Generowanie wiedzy i step-back: pytania podchwytliwe

Na pytaniu podchwytliwym z zakresu sieci komputerowych porównasz odpowiedź bezpośrednią z dwiema technikami dwukrokowymi: generowaniem wiedzy (najpierw fakty, potem odpowiedź) [5] oraz step-back (najpierw zasada ogólna, potem zastosowanie do przypadku).

Treść zadania:

1. W katalogu roboczym utwórz plik `wiedza_wygenerowana.py`. Nagłówek pliku oraz funkcję `zapytaj()` przygotuj na podstawie skryptu z zadania 5.1, a następnie dopisz część merytoryczną:

```
PYTANIE = ('Czy protokół HTTPS ukrywa przed moim '
           'dostawcą internetu adres strony, którą '
           'odwiedzam? Odpowiedz TAK lub NIE i uzasadnij '
           'w dwóch zdaniach.')

# wariant A: pytanie zadane wprost
```

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

**Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00**

```
wprost = zapytaj(PYTANIE)
print(f'\n--- A: odpowiedz bezposrednia ---\n{wprost}')

# wariant B: generowanie wiedzy (2 kroki)
fakty = zapytaj(
    'Wypisz cztery kluczowe fakty techniczne o tym, '
    'które elementy komunikacji HTTPS są szyfrowane, '
    'a które pozostają widoczne dla obserwatora sieci '
    '(uwzględnij DNS, SNI oraz ścieżkę URL).')
print(f'\n--- B, krok 1: wygenerowane fakty ---\n{fakty}')
odp_b = zapytaj(
    'Korzystając wyłącznie z poniższych faktów, '
    'odpowiedz na pytanie.\nFakty:\n' + fakty +
    '\nPytanie: ' + PYTANIE)
print(f'\n--- B, krok 2: odpowiedz ---\n{odp_b}')

# wariant C: step-back (najpierw zasada ogolna)
zasada = zapytaj(
    'Sformułuj w 2-3 zdaniach ogólną zasadę: które '
    'warstwy i elementy komunikacji chroni protokół '
    'TLS, a czego nie ukrywa przed operatorem sieci.')
print(f'\n--- C, krok 1: zasada ogolna ---\n{zasada}')
odp_c = zapytaj(
    'Zastosuj poniższą zasadę do pytania i odpowiedz.\n'
    'Zasada: ' + zasada + '\nPytanie: ' + PYTANIE)
print(f'\n--- C, krok 2: odpowiedz ---\n{odp_c}')
```

2. Zapisz plik (Ctrl+S) i uruchom skrypt poleceniem `python wiedza_wygenerowana.py`. Skrypt wykonuje 5 wywołań modelu.
3. Oceń merytorycznie trzy odpowiedzi. Pytanie jest podchwytliwe: HTTPS szyfruje ścieżkę i treść, ale nazwa domeny zwykle pozostaje widoczna (zapytania DNS, pole SNI w nawiązaniu TLS) – pełna odpowiedź powinna rozróżnić te elementy. Sprawdź, czy bezpośrednia odpowiedź uprościła problem oraz czy fakty z wariantu B faktycznie zostały wykorzystane w odpowiedzi końcowej.

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

P: Zamieść wszystkie wydruki. Sformułuj wnioski: (1) która odpowiedź najpełniej rozróżnia elementy widoczne i szyfrowane? (2) czym różni się generowanie wiedzy od step-back i kiedy każda z technik ma przewagę? (3) jaki jest koszt obu technik względem pytania wprost?

Zadanie 5.3. Samodoskonalenie: pętla generuj–krytykuj–popraw

Model poprawi własny kod w pętli samodoskonalenia [6]. Punktem wyjścia jest działająca, lecz niedbała funkcja walidująca numer PESEL; jawna lista kryteriów pełni rolę listy kontrolnej dla recenzenta.

Treść zadania:

1. W katalogu roboczym utwórz plik `samodoskonalenie.py` (nagłówek pliku oraz funkcja `zapytaj()` jak w zadaniu 5.1) i dopisz część merytoryczną:

```
KOD = '''
def czy_pesel_poprawny(pesel):
    suma = 0
    wagi = [1, 3, 7, 9, 1, 3, 7, 9, 1, 3]
    for i in range(10):
        suma += int(pesel[i]) * wagi[i]
    return (10 - suma % 10) % 10 == int(pesel[10])
'''

KRYTERIA = (
    '1) walidacja wejścia: długość 11 znaków i same '
    'cyfry, bez wyjątków dla błędnych danych; '
    '2) docstring z opisem parametru i wartości '
    'zwracanej; 3) czytelne nazwy i stałe zamiast '
    'liczb magicznych; 4) adnotacje typów.'
)

wersja = KOD
for i in (1, 2):
    print(f'\n\n{"#" * 64}')
    print(f'###  ITERACJA {i}')
```

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

```
    print('#' * 64)
    krytyka = zapytaj(
        'Jesteś recenzentem kodu. Oceń poniższą funkcję '
        'wyłącznie względem kryteriów: ' + KRYTERIA +
        ' Wypunktuj konkretne naruszenia (bez '
        'poprawiania kodu).\nKod:\n' + wersja,
        limit=350)
    print(f'--- krytyka ---\n{krytyka}')
    wersja = zapytaj(
        'Popraw funkcję zgodnie z krytyką. Zachowaj '
        'algorytm sumy kontrolnej. Zwróć wyłącznie kod '
        'Python, bez komentarza wokół.\nKod:\n' + wersja +
        '\nKrytyka:\n' + krytyka,
        limit=450)
    print(f'\n--- wersja po iteracji {i} ---\n{wersja}')
```

2. Zapisz plik (Ctrl+S) i uruchom skrypt poleceniem `python samodoskonalenie.py`. Skrypt wykonuje 4 wywołania (dwie iteracje po dwa kroki).
3. Porównaj wersję wyjściową z wersjami po iteracjach: które kryteria zostały spełnione po pierwszej iteracji, a które dopiero po drugiej? Czy druga krytyka wykryła jeszcze istotne naruszenia ?
4. Skopiuj końcową wersję funkcji do osobnego pliku i sprawdź, że działa: wywołaj ją dla poprawnego numeru PESEL, numeru z błędną cyfrą kontrolną oraz łańcucha „abc”.

P: Zamieść wersję wyjściową, obie krytyki oraz obie poprawione wersje funkcji, a także wynik własnego testu trzech wywołań. Sformułuj wnioski: (1) ile realnej poprawy dała pierwsza iteracja, a ile druga? (2) dlaczego jawna lista kryteriów jest kluczowa dla jakości krytyki? (3) kiedy pętlę samodoskonalenia warto zatrzymać?

Zadanie 5.4. Drzewo myśli: projekt modelu danych z sędzią

Scenariusz zadania: wydział potrzebuje modelu danych dla systemu rezerwacji sal dydaktycznych (sale, prowadzący, rezerwacje jednorazowe i cykliczne, wykrywanie

kolizji). Zaimplementujesz uproszczone drzewo myśli [7]: trzy konkurencyjne koncepcje wygenerowane przy wysokiej temperaturze, ocena każdej przez model-sędziego w stałym formacie oraz rozwinięcie najlepszej gałęzi.

Treść zadania:

1. W katalogu roboczym utwórz plik `drzewo_mysli.py` (nagłówek pliku oraz funkcja `zapytaj()` jak w zadaniu 5.1, dodatkowo `import re`) i dopisz część merytoryczną:

```
ZADANIE = ('Zaprojektuj model danych (tabele, kluczowe '
           'pola, relacje) dla systemu rezerwacji sal '
           'dydaktycznych na wydziale. Wymagania: sale '
           'z wyposażeniem, prowadzący, rezerwacje '
           'jednorazowe i cykliczne (co tydzień przez '
           'semestr), wykrywanie kolizji terminów.')

KRYTERIA = ('poprawność relacji i kluczy, obsługa '
            'rezerwacji cyklicznych, łatwość wykrywania '
            'kolizji, prostota schematu')

# faza 1: trzy konkurencyjne koncepcje (wysoka temperatura)
koncepcje = []
for i in range(1, 4):
    k = zapytaj(ZADANIE + ' Przedstaw JEDNĄ koncepcję '
                'w 5-7 zdaniach (bez pełnego SQL): nazwij '
                'tabele i opisz, jak rozwiązujesz '
                'rezerwacje cykliczne oraz kolizje.',
                temp=0.9, limit=350)
    koncepcje.append(k)
    print(f'\n\n{"#" * 64}\n###  KONCEPCJA {i}\n{"#" * 64}')
    print(k)

# faza 2: ocena sedziego (temperatura 0, staly format)
oceny = []
for i, k in enumerate(koncepcje, start=1):
    werdykt = zapytaj(
        'Jesteś sędzią. Oceń koncepcję modelu danych '
```

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

```
        'względem kryteriów: ' + KRYTERIA + '. Zwróć '
        'dokładnie: OCENA: <liczba 1-10> oraz jedno '
        'zdanie uzasadnienia.\nKoncepcja:\n' + k,
        limit=120)
    m = re.search(r'OCENA:\s*(\d+)', werdykt)
    punkty = int(m.group(1)) if m else 0
    oceny.append(punkty)
    print(f'\n--- sedzia o koncepcji {i} ---\n{werdykt}')

# faza 3: rozwiniecie najlepszej galezi
naj = oceny.index(max(oceny))
print(f'\n>>> Najlepsza koncepcja: {naj + 1} '
      f'(oceny: {oceny})')
final = zapytaj(
    'Rozwiń poniższą koncepcję w kompletny schemat: '
    'wypisz tabele z polami i typami, klucze główne '
    'i obce oraz opisz zapytanie wykrywające kolizję '
    'terminów.\nKoncepcja:\n' + koncepcje[naj],
    limit=600)
print(f'\n\n{"#" * 64}\n###  ROZWINIECIE\n{"#" * 64}')
print(final)
```

2. Zapisz plik (Ctrl+S) i uruchom skrypt poleceniem `python drzewo_mysli.py`. Skrypt wykonuje 7 wywołań (3 koncepcje, 3 oceny, 1 rozwinięcie). Wysoka temperatura w fazie 1 jest celowa – zapewnia różnorodność gałęzi; sędzia pracuje przy temperaturze 0.
3. Na podstawie wydruku uzupełnij tabelę 5.2. Oceń też samodzielnie: czy koncepcje rzeczywiście się różnią (np. osobna tabela wystąpień terminów vs reguła cykliczności w rezerwacji), czy sędzia wskazał tę, którą sam uznajesz za najlepszą?

Tabela 5.2. Ocena gałęzi drzewa myśli przez model-sędziego (do uzupełnienia na podstawie wydruku)

Koncepcja	Ocena sędziego (1–10)	Jedno zdanie uzasadnienia sędziego
1		

Koncepcja	Ocena sędziego (1–10)	Jedno zdanie uzasadnienia sędziego
2		
3		

P: Zamieść trzy koncepcje, werdykty sędziego, uzupełnioną tabelę 5.2 oraz rozwinięcie najlepszej gałęzi. Sformułuj wnioski: (1) czy gałęzie były istotnie różne i co o tym zdecydowało? (2) dlaczego sędzia wymaga stałego formatu OCENA: <liczba> i temperatury 0? (3) porównaj koszt drzewa myśli (7 wywołań) z pojedynczym promptem – kiedy taka inwestycja się opłaca?

Zadanie 5.5. Meta-prompting: generator promptów do przeglądu kodu

Model wygeneruje wyspecjalizowane prompty recenzenta kodu na podstawie jednego meta-promptu z parametrami (język, aspekt) [8], a następnie przetestuje jakość wygenerowanego promptu na próbce kodu podatnego na błędy.

Treść zadania:

1. W katalogu roboczym utwórz plik `meta_prompting.py` (nagłówek pliku oraz funkcja `zapytaj()` jak w zadaniu 5.1) i dopisz część merytoryczną:

```
META = ('Jesteś ekspertem prompt engineeringu. Napisz '
        'prompt dla modelu językowego, który będzie '
        'recenzował kod w języku {jezyk} wyłącznie pod '
        'kątem {aspekt}. Prompt ma: nadać rolę '
        'recenzenta, wymienić 4-5 konkretnych punktów '
        'kontrolnych, narzucić format wyniku (lista: '
        'linia lub fragment | problem | proponowana '
        'poprawka) oraz zakazać uwag niezwiązanych '
        'z {aspekt}. Zwróć wyłącznie treść promptu.')

PROBKA = '''
import sqlite3
def znajdz_uzytkownika(nazwa):
    con = sqlite3.connect('baza.db')
```

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

```
    kursor = con.cursor()
    zapytanie = "SELECT * FROM users " \\
        "WHERE name = '" + nazwa + "'"
    kursor.execute(zapytanie)
    wynik = kursor.fetchall()
    return wynik
'''

KOMBINACJE = [('Python', 'bezpieczeństwa'),
              ('Python', 'wydajności')]

for jezyk, aspekt in KOMBINACJE:
    print(f'\n\n{"#" * 64}')
    print(f'###  META-PROMPT: {jezyk} / {aspekt}')
    print('#' * 64)
    # krok 1: model generuje prompt recenzenta
    prompt_recenzenta = zapytaj(
        META.format(jezyk=jezyk, aspekt=aspekt),
        limit=400)
    print(f'--- wygenerowany prompt ---\n{prompt_recenzenta}')
    # krok 2: test wygenerowanego promptu na probce kodu
    recenzja = zapytaj(
        prompt_recenzenta + '\n\nKod do recenzji:\n'
        + PROBKA, limit=450)
    print(f'\n--- recenzja probki kodu ---\n{recenzja}')
```

2. Zapisz plik (Ctrl+S) i uruchom skrypt poleceniem `python meta_prompting.py`. Skrypt wykonuje 4 wywołania (dla każdej kombinacji: generacja promptu i jego test).
3. Oceń wygenerowane prompty względem specyfikacji z meta-promptu: czy mają rolę, 4–5 punktów kontrolnych, narzucony format i zakaz uwag spoza aspektu? Następnie oceń recenzje: wariant „bezpieczeństwo” powinien wykryć podatność SQL injection (sklejanie zapytania z danymi użytkownika) i zaproponować zapytanie parametryzowane; wariant „wydajność” powinien zwrócić uwagę na inne kwestie (np. otwieranie połączenia przy każdym wywołaniu), nie powtarzając uwag o bezpieczeństwie.

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

P: Zamieść oba wygenerowane prompty i obie recenzje. Sformułuj wnioski: (1) czy prompty-produkty spełniły specyfikację meta-promptu? (2) czy recenzje trzymały się swoich aspektów – co to mówi o skuteczności ograniczeń zapisanych w meta-prompcie? (3) podaj dwa inne zastosowania meta-promptingu w pracy zespołu programistycznego.

Zadanie 5.6. Analiza przypadku: agent ReAct z narzędziami

Scenariusz zadania: asystent dziekanatu ma odpowiadać na pytania dotyczące ocen, ale dane znajdują się poza modelem – w strukturach programu. Zbudujesz agenta ReAct [9]: model w pętli MYŚL–AKCJA–OBSERWACJA korzysta z dwóch narzędzi w Pythonie (pobranie ocen, średnia), a skrypt parsuje jego akcje i wykonuje je (por. rysunek 5.1).

Treść zadania:

1. W katalogu roboczym utwórz plik `agent_react.py` (nagłówek pliku oraz funkcja `zapytaj()` jak w zadaniu 5.1, dodatkowo `import re`) i dopisz część merytoryczną:

```
# dane dostepne wylacznie przez narzedzia (poza modelem)
OCENY = {
    'programowanie': [4.5, 5.0, 4.0, 3.5],
    'bazy danych': [3.0, 4.0, 3.5],
    'sieci komputerowe': [5.0, 4.5, 5.0, 4.0],
}

def pobierz_oceny(przedmiot):
    oceny = OCENY.get(przedmiot.strip().lower())
    return str(oceny) if oceny else 'brak takiego przedmiotu'

def srednia(lista_liczb):
    try:
        liczby = [float(x) for x in lista_liczb
                  .replace('[', '').replace(']', '')
                  .split(',')]
        return str(round(sum(liczby) / len(liczby), 2))
```

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

**Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00**

```
    except (ValueError, ZeroDivisionError):
        return 'blad: podaj liczby po przecinku'

NARZEDZIA = {'pobierz_oceny': pobierz_oceny,
             'srednia': srednia}

INSTRUKCJA = (
    'Jesteś agentem odpowiadającym na pytania o oceny. '
    'Masz narzędzia:\n'
    '- pobierz_oceny: <nazwa przedmiotu> -> lista ocen\n'
    '- srednia: <liczby po przecinku> -> średnia\n'
    'Działaj w pętli. W każdej turze wypisz DOKŁADNIE:\n'
    'MYŚL: <krótki plan>\n'
    'AKCJA: <narzędzie>: <argument>\n'
    'i zatrzymaj się. Po otrzymaniu OBSERWACJA wykonaj '
    'kolejną turę. Gdy znasz wynik, wypisz:\n'
    'ODPOWIEDŹ: <pełna odpowiedź>\n'
)

PYTANIE = ('Z którego przedmiotu student ma wyższą '
           'średnią: z programowania czy z baz danych? '
           'Podaj obie średnie.')

historia = INSTRUKCJA + '\nPYTANIE: ' + PYTANIE + '\n'

for krok in range(1, 7):
    tura = zapytaj(historia, limit=200,
                   stop=['OBSERWACJA:'])
    print(f'\n--- krok {krok} ---\n{tura}')
    if 'ODPOWIEDŹ:' in tura:
        break
    m = re.search(r'AKCJA:\s*([a-z_]+)\s*:\s*(.+)', tura)
    if m and m.group(1) in NARZEDZIA:
        obs = NARZEDZIA[m.group(1)](m.group(2))
    else:
        obs = 'blad: nieznana akcja, uzyj narzedzia z listy'
    print(f'OBSERWACJA: {obs}')
```

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

```
    historia += tura + '\nOBSERWACJA: ' + obs + '\n'
else:
    print('\n[przekroczono limit krokow agenta]')
```

2. Zapisz plik (Ctrl+S) i uruchom skrypt poleceniem `python agent_react.py`. Sekwencja stopu OBSERWACJA: (parametr stop funkcji zapytaj, znany z Laboratorium 3) wymusza zatrzymanie modelu po jednej akcji. Obserwację dostarcza wyłącznie program, nie model.
3. Prześledź wydruk: ile kroków wykonał agent i w jakiej kolejności użył narzędzi? Czy wynik liczbowy zgadza się z ręcznym obliczeniem średnich (programowanie: 4,25; bazy danych: 3,5)?
4. Zmień PYTANIE na dotyczące przedmiotu, którego nie ma w danych (np. fizyki), i uruchom ponownie – zaobserwuj, jak agent reaguje na obserwację „brak takiego przedmiotu".

P: Zamieść pełne ślady obu uruchomień agenta (pytanie oryginalne i pytanie o brakujący przedmiot). Sformułuj wnioski: (1) dlaczego ścisły format MYŚL/AKCJA/ODPOWIEDŹ jest warunkiem działania pętli? (2) jaką rolę pełni sekwencja stopu i co by się stało bez niej? (3) po co ustala się limit kroków i jak agent poradził sobie z brakującymi danymi?

Ćwiczenia samodzielne

Ćwiczenie 5.1 (podstawowe). Dwustopniowy łańcuch na modelu lokalnym

Zbuduj na modelu lokalnym `llama3.2` dwustopniowy łańcuch przetwarzania: krok 1 tłumaczy krótki akapit anglojęzycznej dokumentacji technicznej na język polski, a krok 2 weryfikuje i ujednolica terminologię w otrzymanym tłumaczeniu (np. thread jako wątek, commit pozostaje bez tłumaczenia). Porównaj wynik łańcucha z pojedynczym promptem łączącym oba polecenia. W sprawozdaniu przedstaw oba przebiegi oraz wniosek: co zyskuje się dzięki rozdzieleniu tłumaczenia i korekty terminologii na osobne kroki?

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

**Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00**

Ćwiczenie 5.2 (średnie). Generowanie wiedzy dla własnych pytań podchwytliwych

Sformułuj dwa własne pytania podchwytliwe z zakresu sieci komputerowych lub programowania (takie, na które szybka odpowiedź „z głowy" bywa błędna lub niepełna). Dla każdego pytania porównaj bezpośrednią odpowiedź z techniką generowania wiedzy lub step-back (jak w zadaniu 5.2). W sprawozdaniu przedstaw pytania, wydruki wszystkich wariantów oraz wniosek: przy którym pytaniu technika dwukrokowa realnie poprawiła odpowiedź, a przy którym okazała się zbędnym kosztem – i po czym to poznać?

Ćwiczenie 5.3 (trudne). Rozbudowa agenta ReAct

Rozbuduj agenta z zadania 5.6: dodaj trzecie narzędzie (np. `liczba_ocen` albo `najlepszy_przedmiot` zwracające przedmiot o najwyższej średniej), uzupełnij opis narzędzi w instrukcji agenta oraz dodaj obsługę błędnego argumentu narzędzia (komunikat w obserwacji zamiast wyjątku skryptu). Przetestuj agenta na dwóch pytaniach wymagających użycia co najmniej dwóch narzędzi. W sprawozdaniu przedstaw kompletne ślady obu uruchomień (wszystkie kroki MYŚL–AKCJA–OBSERWACJA) oraz krótką analizę: czy po obserwacji komunikatu błędu agent skorygował swoje działanie?

Pytania pomocnicze

1. Na czym polega łańcuch promptów? Dlaczego ułatwia diagnozowanie błędów i integrację z innym oprogramowaniem?
2. Czym różni się sekwencyjny łańcuch promptów od równoległej agregacji odpowiedzi i kiedy każde z podejść jest właściwe?

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

3. Na czym polega technika least-to-most i czym różni się od ręcznie zaprojektowanego łańcucha promptów?
4. Porównaj generowanie wiedzy ze step-back: co jest wynikiem kroku pośredniego w każdej z tych technik?
5. Dlaczego rozdzielenie przywoływania wiedzy od wnioskowania zmniejsza ryzyko zmyśleń w odpowiedziach na pytania podchwytliwe?
6. Jaką rolę w pętli samodoskonalenia pełni jawna lista kryteriów oceny i po czym poznać, że kolejne iteracje przestają się opłacać?
7. Dlaczego w drzewie myśli gałęzie generuje się przy wysokiej temperaturze, a model-sędzia pracuje przy temperaturze 0? Po co sędziemu narzuca się stały format oceny?
8. Czym jest meta-prompting i w jaki sposób weryfikuje się jakość promptu wygenerowanego przez model?
9. Wymień elementy pętli agenta ReAct. Jaką rolę pełnią sekwencja stopu oraz limit liczby kroków?
10. Jak dobrać technikę zaawansowaną do zadania, uwzględniając koszt wielokrotnych wywołań modelu?

Podsumowanie

Laboratorium poświęcone było zaawansowanym technikom promptowania, w których wiele wywołań modelu łączy się w jeden proces. Po jego ukończeniu student:

1. projektuje łańcuchy promptów (wynik kroku wejściem następnego) i potrafi porównać je z pojedynczym promptem zbiorczym;
2. stosuje generowanie wiedzy i step-back do pytań podchwytliwych, rozdzielając przywoływanie wiedzy od wnioskowania;
3. buduje pętlę samodoskonalenia z jawną listą kryteriów i ocenia zysk z kolejnych iteracji;
4. implementuje uproszczone drzewo myśli: różnorodne gałęzie, ocenę modelem-sędzią w stałym formacie i rozwinięcie najlepszej koncepcji;
5. tworzy meta-prompty generujące wyspecjalizowane prompty i weryfikuje prompty-produkty na próbce danych;

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

6. buduje agenta ReAct z narzędziami w Pythonie, parsowaniem akcji, sekwencją stopu i limitem kroków.

UWAGA: rozszerz bibliotekę promptów prowadzoną od Laboratorium 2 (plik `prompts.md` lub program AI Prompt Lab) o zapisy całych schematów wieloetapowych, a nie tylko pojedynczych promptów: kolejność kroków łańcucha, prompt sędziego z formatem oceny, instrukcję agenta z listą narzędzi i formatem akcji. Dopiero technika, konfiguracja parametrów i schemat procesu razem pozwalają odtworzyć działające rozwiązanie.

Literatura

[1]. Gadesha V., Prompt Engineering Techniques, IBM Think, https://www.ibm.com/think/topics/prompt-engineering-techniques (dostęp: 2026).
[2]. Google Cloud, Include few-shot examples – Gemini, projektowanie promptów, https://docs.cloud.google.com/gemini-enterprise-agent-platform/models/prompts/few-shot-examples (dostęp: 2026).
[3]. Google Cloud, Break down complex tasks into simpler prompts – Gemini, projektowanie promptów, https://docs.cloud.google.com/gemini-enterprise-agent-platform/models/prompts/break-down-prompts (dostęp: 2026).
[4]. Zhou D. i in., Least-to-Most Prompting Enables Complex Reasoning in Large Language Models, arXiv:2205.10625, 2022.
[5]. Liu J. i in., Generated Knowledge Prompting for Commonsense Reasoning, arXiv:2110.08387, 2022.
[6]. Madaan A. i in., Self-Refine: Iterative Refinement with Self-Feedback, arXiv:2303.17651, 2023.
[7]. Yao S. i in., Tree of Thoughts: Deliberate Problem Solving with Large Language Models, arXiv:2305.10601, 2023.
[8]. Zhang Y. i in., Meta Prompting for AI Systems, arXiv:2311.11482, 2023.
[9]. Yao S. i in., ReAct: Synergizing Reasoning and Acting in Language Models, arXiv:2210.03629, 2022.
[10]. Ollama – dokumentacja oficjalna, https://ollama.com oraz https://github.com/ollama/ollama (dostęp: 2026).

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

[11]. Ollama Python Library, https://github.com/ollama/ollama-python (dostęp: 2026).
[12]. Phoenix J., Taylor M., Skuteczna inżynieria promptów. Przyszłościowe rozwiązania dla rzetelnych wyników generatywnej AI, Helion, Gliwice 2025.
[13]. Mizrahi G., Inżynieria promptów bez tajemnic. Sztuka kreatywnego generowania tekstów, Helion, Gliwice 2025.
[14]. Groq, Model Deprecation (harmonogram wyłączania modeli), https://console.groq.com/docs/deprecations (dostęp: 2026).

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

POLITECHNIKA LUBELSKA

Materiały do przedmiotu:

GENERATYWNA SZTUCZNA INTELIGENCJA I PROMPT ENGINEERING

Laboratorium nr 6.
Tworzenie promptów dla różnych zadań, takich jak generowanie tekstu, obrazu, wideo lub prezentacji

Autor:

dr inż. Krzysztof Dziedzic

dr Marcin Barszcz

Katedra Informatyki

Fundusze Europejskie dla Rozwoju Społecznego

Rzeczpospolita Polska

Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

Spis treści

Wprowadzenie, tematyka zajęć ----------121
Cel ćwiczenia/laboratorium---------- 123
Instrukcja laboratoryjna/ćwiczeniowa---------- 124
Ćwiczenia samodzielne ---------- 135
Pytania pomocnicze ---------- 136
Podsumowanie---------- 136
Literatura ---------- 137

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

Wprowadzenie, tematyka zajęć

W dotychczasowych laboratoriach inżynieria promptów dotyczyła wyłącznie tekstu: w Laboratorium 2 poznano strukturę skutecznego promptu (rola – zadanie – kontekst – format), a w Laboratorium 5 – techniki wieloetapowe sterowane z poziomu Pythona. W niniejszym laboratorium te umiejętności zostaną przeniesione na inne modalności – obraz, wideo i prezentacje – w sposób **programistyczny**: praca odbywa się w Visual Studio Code, a generowanie treści realizują skrypty Pythona. Głównym modelem zewnętrznym pozostaje usługa GroqCloud (znana z Laboratoriów 1–5), modelem lokalnym – Ollama (Laboratorium 2), a przykładem modelu komercyjnego jest generowanie obrazów modelem Gemini przez darmową warstwę Gemini API (klucz z Google AI Studio, znany z Laboratorium 1). Część zajęć stanowi także realistyczne zadanie inżynierskie: **analiza dokumentacji** modeli wideo (Veo) i muzyki (Lyria) pod kątem tego, czy i jak można z nich skorzystać w projekcie bez budżetu.

Anatomia promptu obrazowego. Prompt dla generatora obrazów to nie polecenie, lecz **opis sceny**. Skuteczny opis zawiera pięć elementów: (1) podmiot – co lub kto jest na obrazie; (2) styl – np. fotorealizm, płaska grafika wektorowa, akwarela; (3) kompozycję i kadr – plan, perspektywa; (4) oświetlenie i nastrój; (5) detale techniczne – np. głębia ostrości, proporcje obrazu. Im bardziej konkretny opis, tym mniej decyzji model podejmuje „za nas" – to ta sama zasada doprecyzowania, którą poznano przy promptach tekstowych. W wykonaniu programistycznym opis sceny jest zwykłym łańcuchem znaków w kodzie, co pozwala budować go dynamicznie – np. sklejać stały opis stylu ze zmiennym podmiotem w pętli.

Generowanie obrazów przez API – wynik analizy dokumentacji. Dokumentacja Google [1] opisuje generowanie obrazów modelami Gemini (m.in. `gemini-2.5-flash-image`) z użyciem pakietu `google-genai` – tego samego, który zainstalowano w Laboratorium 1. Przykłady w dokumentacji korzystają z trybu enterprise (projekt Google Cloud, zmienne `GOOGLE_CLOUD_PROJECT` i uwierzytelnianie kontem chmurowym), jednak ten sam kod działa również z

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

darmowym kluczem Gemini API wygenerowanym w Google AI Studio – wystarczy utworzyć klienta wywołaniem `genai.Client(api_key=...)`. Darmowa warstwa ma dzienny limit generacji, wystarczający na potrzeby laboratorium. Odpowiedź modelu zawiera części tekstowe i binarne – obraz znajduje się w polu `inline_data` i zapisywany jest do pliku PNG (biblioteka Pillow).

Wideo (Veo) i muzyka (Lyria) – wynik analizy dokumentacji. Dokumentacja generowania wideo z tekstu [2] w sekcji „Before you begin" wymaga: konta Google Cloud, utworzenia projektu, włączenia API platformy oraz uwierzytelnienia narzędziem `gcloud` – czyli konta rozliczeniowego (nowi użytkownicy otrzymują czasowe kredyty próbne, ale rejestracja wymaga karty płatniczej). W Gemini API modele Veo dostępne są wyłącznie w warstwie płatnej. Analogicznie generowanie muzyki modelami Lyria [3] dostępne jest wyłącznie w konsoli Google Cloud (Vertex AI Studio, wynik w plikach WAV) lub przez REST z tokenem `gcloud` – bez ścieżki darmowej. **Wniosek inżynierski:** w projekcie bez budżetu wideo generuje się przez darmowy interfejs webowy Gemini (dzienny limit), a prompty projektuje się według oficjalnego przewodnika Veo [4]; generowanie muzyki pozostaje poza zasięgiem darmowych narzędzi programistycznych i w tym laboratorium ograniczone jest do analizy. Umiejętność takiej oceny wykonalności na podstawie dokumentacji jest równie ważna jak samo kodowanie.

Prompt wideo rozszerza opis sceny o **ruch**: co robi podmiot, jak porusza się kamera (najazd, panorama, ujęcie z drona), jakie jest tempo i czas trwania. Obecne modele generują krótkie klipy i miewają problemy z fizyką ruchu, dłońmi i tekstem w kadrze – rozpoznawanie takich artefaktów jest częścią zadania.

Prezentacje programistycznie. Zamiast webowych generatorów prezentacji zastosowane zostanie podejście w pełni programistyczne i dwuetapowe: (1) model językowy (Groq) generuje **konspekt jako JSON** o ustalonych polach – tytuły slajdów, punkty, notatki prelegenta; (2) biblioteka `python-pptx` buduje z konspektu plik `.pptx`. Kontrakt JSON (wymuszony w prompcie i parsowany w kodzie) oddziela treść od formy i daje pełną kontrolę nad każdym slajdem.

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

Rola modelu lokalnego. Ollama (Laboratorium 2) posłuży do porównania: te same prompty tekstowe zostaną wykonane na modelu zewnętrznym (Groq, `llama-3.3-70b-versatile`) i lokalnym (`llama3.1`). Modele lokalne nie generują w tym zestawie obrazów ani wideo, ale w zadaniach tekstowych pozwalają pracować offline i bez wysyłania danych na zewnątrz. Ścieżki dostępu wykorzystywane w laboratorium zestawia tabela 6.1.

Tabela 6.1. Ścieżki dostępu do generowania treści (wynik analizy dokumentacji, stan na 2026 r.)

Modalność	Model / usługa	Darmowy dostęp programistyczny	Ścieżka w laboratorium
Tekst	Groq API (llama-3.3-70b-versatile)	tak – klucz API, limity zapytań	skrypt Python (główna)
Tekst	Ollama (llama3.1, lokalnie)	tak – bez klucza, offline	skrypt Python (porównanie)
Obraz	Gemini API (gemini-2.5-flash-image)	tak – klucz AI Studio, limit dzienny	skrypt Python
Wideo	Veo (Vertex AI / Gemini API)	nie – wymagany projekt GCP z billingiem	analiza + web (gemini.google.com)
Muzyka	Lyria (wyłącznie Vertex AI)	nie – konsola GCP, token gcloud	wyłącznie analiza

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

Cel ćwiczenia/laboratorium

Głównym celem laboratorium jest **programistyczne projektowanie i wykonywanie promptów dla różnych modalności** – tekstu, obrazu i prezentacji – w Visual Studio Code, z Groq jako głównym modelem zewnętrznym, Ollama jako modelem lokalnym i Gemini jako przykładem modelu komercyjnego, oraz ocena dostępności generatorów wideo i muzyki na podstawie dokumentacji. Po ukończeniu zajęć student potrafi:

1. zbudować projekt wykorzystujący równolegle kilka usług generatywnych (dwa klucze API w jednym pliku `.env` oraz model lokalny);
2. wykonywać programistycznie prompty tekstowe o różnych formatach i porównywać wyniki modelu zewnętrznego (Groq) z lokalnym (Ollama);
3. generować obrazy z kodu przez darmową warstwę Gemini API: budować opis sceny z pełną anatomią, odbierać dane binarne z odpowiedzi i zapisywać pliki PNG;
4. utrzymywać spójną serię grafik, sklejając w pętli stały przewodnik stylu ze zmiennym podmiotem;
5. ocenić na podstawie dokumentacji wykonalność użycia modeli wideo (Veo) i muzyki (Lyria) w projekcie bez budżetu oraz zaprojektować prompt wideo z opisem ruchu;
6. wygenerować prezentację dwuetapowo i w pełni programistycznie: konspekt jako kontrakt JSON z modelu, plik `.pptx` z biblioteki python-pptx;
7. wskazać zasady etyczne i licencyjne wykorzystania treści generowanych przez AI.

Laboratorium wymaga: klucza Groq (Laboratorium 1), klucza Gemini API z Google AI Studio (Laboratorium 1, Opcja B – jeśli nie został wygenerowany, zostanie utworzony teraz) oraz działającego narzędzia Ollama z modelem `llama3.1` (Laboratorium 2).

Instrukcja laboratoryjna/ćwiczeniowa

Zadanie 6.1. Nowy projekt multimodalny i konfiguracja kluczy

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

Projekt łączy trzy źródła generacji: Groq (tekst), Gemini API (obrazy) i lokalną Ollamę. Docelowa struktura:

```
multimodal_lab/
|-- src/
|   |-- tekst_formy.py     # Zadanie 6.2
|   |-- obraz.py           # Zadanie 6.3
|   |-- seria.py           # Zadanie 6.4
|   `-- prezentacja.py     # Zadanie 6.6
|-- out/                   # wygenerowane pliki (PNG, PPTX)
|-- .env                   # dwa klucze API
|-- .gitignore
`-- requirements.txt
```

Treść zadania:

1. Utwórz nowy katalog roboczy, np. `C:\GenAI_Lab\multimodal_lab`, otwórz go w VS Code i utwórz podkatalogi `src` oraz `out`.
2. Utwórz i aktywuj środowisko wirtualne, a następnie zainstaluj biblioteki i utrwal zależności:

```
python -m venv .venv
.venv\Scripts\Activate.ps1      # Windows PowerShell
pip install -U groq ollama google-genai pillow python-pptx
pip install -U python-dotenv
pip freeze > requirements.txt
```

3. W pliku .env zapisz dwa klucze: klucz Groq (z Laboratorium 1 lub nowy) oraz klucz Gemini API – jeśli go nie masz, wejdź na aistudio.google.com, wybierz Get API key i wygeneruj klucz (procedura z Laboratorium 1, Opcja B):

```
GROQ_API_KEY=twoj_klucz_groq
GEMINI_API_KEY=twoj_klucz_gemini
```

4. Utwórz .gitignore z wpisami .env, .venv/, __pycache__/, out/.
5. Zweryfikuj model lokalny: w terminalu wykonaj ollama list i upewnij się, że model llama3.1 (Laboratorium 2) jest pobrany; w razie potrzeby wykonaj ollama pull llama3.1.

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

Zadanie 6.2. Prompty tekstowe programistycznie: Groq i Ollama

Ten sam temat zostanie ujęty w trzech formach tekstu (definicja, post edukacyjny, e-mail), a wszystkie prompty wykona skrypt – najpierw na modelu zewnętrznym, potem lokalnym. Zadanie pokazuje, że element **format** struktury promptu steruje stylem i długością wyniku, oraz jak programistycznie porównywać modele.

Treść zadania:

1. W katalogu src utwórz plik tekst_formy.py o następującej treści:

```
import os
import sys

import ollama
from dotenv import load_dotenv
from groq import Groq

sys.stdout.reconfigure(encoding='utf-8')
load_dotenv()

MODEL_GROQ = 'llama-3.3-70b-versatile'
MODEL_LOKALNY = 'llama3.1'
client = Groq(api_key=os.getenv('GROQ_API_KEY'))

FORMY = {
    'definicja': (
        'Jesteś autorem skryptu akademickiego. Napisz zwięzłą '
        'definicję pojęcia "token" w modelach językowych: '
        '3-4 zdania, styl formalny, bez wypunktowań, z jednym '
        'przykładem liczbowym.'),
    'post': (
        'Jesteś popularyzatorem technologii. Napisz post (do 500 '
        'znaków) o tym, czym jest "token" w modelach językowych: '
        'lekki ton, jedna analogia z życia codziennego, na końcu '
        'pytanie angażujące czytelników.'),
    'e-mail': (
```

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

```
        'Jesteś studentem piszącym do wykładowcy. Napisz uprzejmy
'
        'e-mail z prośbą o wyjaśnienie, dlaczego ten sam tekst ma
'
        'różną liczbę tokenów w różnych modelach. Temat '
        'wiadomości, zwroty grzecznościowe, maksymalnie 120 '
        'słów.'),
}


def zapytaj_groq(prompt):
    r = client.chat.completions.create(
        model=MODEL_GROQ,
        messages=[{'role': 'user', 'content': prompt}],
        temperature=0.7,
        max_tokens=600,
    )
    return r.choices[0].message.content


def zapytaj_ollama(prompt):
    r = ollama.chat(
        model=MODEL_LOKALNY,
        messages=[{'role': 'user', 'content': prompt}],
    )
    return r['message']['content']


for nazwa, prompt in FORMY.items():
    print(f'\n{"#" * 60}\n###  FORMA: {nazwa.upper()}')
    print('--- Groq (model zewnętrzny) ---')
    print(zapytaj_groq(prompt))
    print('--- Ollama (model lokalny) ---')
    print(zapytaj_ollama(prompt))
```

2. Uruchom skrypt z katalogu głównego poleceniem python src/tekst_formy.py (pierwsze wywołanie Ollamy obejmuje ładowanie modelu do pamięci – zimny start).

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

3. Porównaj wyniki w dwóch wymiarach: (a) między formami – jak sam format zmienił styl, długość i dobór treści przy tym samym temacie? (b) między modelami – oceń jakość polszczyzny, trzymanie się limitów i kompletność odpowiedzi modelu lokalnego względem Groq.

P: Zamieść wydruki (lub ich kluczowe fragmenty) i porównaj modele w 3–4 zdaniach: w których formach model lokalny był wystarczający, a gdzie ustępował modelowi zewnętrznemu?

Zadanie 6.3. Generowanie obrazów z kodu (Gemini API)

Przykład modelu komercyjnego: generowanie obrazów modelem `gemini-2.5-flash-image` przez darmową warstwę Gemini API. Kod stanowi adaptację przykładu z dokumentacji Google [1]: zamiast uwierzytelniania projektem chmurowym (tryb enterprise) używany jest klucz API z AI Studio przekazany do `genai.Client`. Odpowiedź modelu składa się z części – tekstowych (`part.text`) i binarnych (`part.inline_data`), z których zapisywany jest plik PNG.

Treść zadania:

1. W katalogu `src` utwórz plik `obraz.py` o następującej treści:

```
import os
from io import BytesIO

from dotenv import load_dotenv
from google import genai
from google.genai.types import GenerateContentConfig, Modality
from PIL import Image

load_dotenv()

MODEL_OBRAZU = 'gemini-2.5-flash-image'

# klucz API z Google AI Studio zamiast trybu enterprise
client = genai.Client(api_key=os.getenv('GEMINI_API_KEY'))
```

```
PROMPT_SCENY = (
    'Fotorealistyczne zdjęcie rudego kota śpiącego na stercie '
    'starych książek w bibliotece uniwersyteckiej. Ciepłe '
    'popołudniowe światło z wysokiego okna, kadr z poziomu '
    'blatu, zbliżenie, płytka głębia ostrości, proporcje 16:9.'
)


def generuj_obraz(prompt, plik_wy):
    """Generuje obraz z opisu sceny i zapisuje go do PNG."""
    response = client.models.generate_content(
        model=MODEL_OBRAZU,
        contents=prompt,
        config=GenerateContentConfig(
            response_modalities=[Modality.TEXT, Modality.IMAGE]),
    )
    for part in response.candidates[0].content.parts:
        if part.text:
            print('[model]:', part.text)
        elif part.inline_data:
            obraz = Image.open(BytesIO(part.inline_data.data))
            obraz.save(plik_wy)
            print('Zapisano', plik_wy)


if __name__ == '__main__':
    os.makedirs('out', exist_ok=True)
    # A: prompt ogólny - decyzje podejmuje model
    generuj_obraz('kot w bibliotece', 'out/kot_ogolny.png')
    # B: pełny opis sceny (anatomia promptu obrazowego)
    generuj_obraz(PROMPT_SCENY, 'out/kot_scena.png')
```

2. Uruchom skrypt poleceniem python src/obraz.py i porównaj oba pliki z katalogu out: jakie decyzje (styl, kadr, oświetlenie, otoczenie) model podjął „za Ciebie” przy prompcie ogólnym?

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

3. Zmodyfikuj w prompcie B wyłącznie element stylu (np. na „minimalistyczna płaska grafika wektorowa, trzy kolory"), zmień nazwę pliku wynikowego i uruchom ponownie. Zaobserwuj, jak jeden element anatomii zmienia cały obraz.
4. Uruchom dwukrotnie ten sam prompt B (dwie różne nazwy plików) i porównaj wyniki – generacja obrazu jest niedeterministyczna nawet przy identycznym opisie.

Uwaga: darmowa warstwa Gemini API ma dzienny limit generacji obrazów – po jego przekroczeniu API zwraca błąd 429 (RESOURCE_EXHAUSTED). Generuj rozważnie i nie umieszczaj dużej liczby wywołań w pętlach. Aktualne limity opisuje dokumentacja ai.google.dev [5].

P: Dołącz plik `obraz.py` oraz obrazy: ogólny, pełny opis sceny i wariant stylu. Wskaż, które decyzje przejął model przy prompcie ogólnym i który element anatomii najsilniej zmienia wynik.

Zadanie 6.4. Spójna seria obrazów – przewodnik stylu w pętli

Serię grafik o spójnym stylu (np. ikony do materiałów kursu) uzyskuje się, sklejając stały **przewodnik stylu** ze zmiennym podmiotem – programistycznie jest to pętla po liście podmiotów. Zadanie pokazuje też typową słabość generatorów: tekst w kadrze.

Treść zadania:

1. W katalogu `src` utwórz plik `seria.py` (funkcja `generuj_obraz` importowana jest z zadania 6.3):

```
import os

from obraz import generuj_obraz

STYL = (
    'Minimalistyczna płaska ikona w stylu grafiki wektorowej, '
    'grube zaokrąglone kontury, paleta granatowo-biała z jednym '
    'pomarańczowym akcentem, białe tło, bez tekstu, bez cieni, '
    'kompozycja wyśrodkowana. Podmiot: '
```

```
)

PODMIOTY = ['laptop z otwartym terminalem',
            'chmura obliczeniowa',
            'klucz API']

os.makedirs('out', exist_ok=True)
for i, podmiot in enumerate(PODMIOTY, start=1):
    print(f'[{i}/{len(PODMIOTY)}] {podmiot}...')
    generuj_obraz(STYL + podmiot, f'out/ikona_{i}.png')
```

2. Uruchom skrypt poleceniem python src/seria.py i oceń spójność serii w katalogu out: paleta, grubość konturów, poziom szczegółowości. Jeśli któraś ikona odstaje, wygeneruj ją ponownie pojedynczym wywołaniem.
3. **Problem tekstu na obrazie.** Dopisz do listy czwarty podmiot zawierający napis, np. „tarcza z napisem PROMPT ENGINEERING", uruchom ponownie i przyjrzyj się literom – generatory obrazów często zniekształcają tekst w kadrze.

P: Dołącz plik `seria.py`, trzy ikony serii i obraz z napisem. Oceń spójność serii w skali 1–5 i wyjaśnij, dlaczego stały przewodnik stylu ją poprawia. Skomentuj jakość napisu.

Zadanie 6.5. Wideo i muzyka: analiza dostępności oraz prompt wideo

Realistyczne zadanie inżynierskie: zanim projekt użyje modelu, trzeba na podstawie dokumentacji ocenić, czy dostęp jest możliwy w ramach dostępnego budżetu. Analizie podlegają modele Veo (wideo) [2] i Lyria (muzyka) [3]; następnie zaprojektowany prompt wideo zostanie wykonany darmową ścieżką webową.

Treść zadania:

1. Otwórz stronę dokumentacji „Generate videos from text prompts" [2] i odszukaj sekcję „Before you begin". Wynotuj wymagania: konto Google Cloud, utworzenie projektu, włączenie API platformy, uwierzytelnienie `gcloud`. Oceń: czy da się to

wykonać bez konta rozliczeniowego? Zanotuj też, że w Gemini API modele Veo dostępne są wyłącznie w warstwie płatnej.

2. Analogicznie przejrzyj stronę „Generate music with Lyria” [3]: zwróć uwagę na dostęp wyłącznie przez konsolę Google Cloud (Vertex AI Studio ▸ Generate Media ▸ Music) lub REST z tokenem `gcloud auth print-access-token` i identyfikatorem projektu oraz na format wyniku (pliki WAV). Sformułuj wniosek o dostępności w projekcie bez budżetu.
3. Dla pełnego obrazu przeczytaj poniższy fragment kodu ścieżki **płatnej** (informacyjnie – nie uruchamiaj bez płatnego dostępu). Generacja wideo jest operacją długotrwałą: kod cyklicznie odpytuje o jej stan:

```
# ścieżka PŁATNA (informacyjnie): Veo przez pakiet google-genai
import time

operacja = client.models.generate_videos(
    model='veo-3.0-fast-generate-001',
    prompt='...opis sceny i ruchu...',
)
while not operacja.done:
    time.sleep(10)          # generacja trwa do kilku minut
    operacja = client.operations.get(operacja)
wideo = operacja.response.generated_videos[0]
```

4. Zaprojektuj prompt wideo zawierający: scenę, ruch podmiotu, ruch kamery, styl i czas trwania, np.:

```
Ujęcie filmowe: dron powoli przelatuje nad kampusem
uniwersyteckim o wschodzie słońca. Jesienne drzewa rzucają
długie cienie, studenci idą alejką. Kamera płynnie unosi się
i odsłania panoramę miasta. Ciepła kolorystyka, 8 sekund.
```

5. Wykonaj prompt darmową ścieżką: zaloguj się na gemini.google.com, wybierz tryb generowania wideo (model Veo, dzienny limit darmowych generacji) i wygeneruj klip. Obejrzyj go w zwolnieniu i wynotuj artefakty: nienaturalny ruch, zniekształcone postacie, obiekty pojawiające się i znikające, błędy fizyki.

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

P: Zamieść: (1) wnioski z analizy dostępności Veo i Lyrii (wymagania, ścieżki, koszty) w 4–6 zdaniach, (2) prompt wideo z oznaczeniem jego elementów, (3) kadry z klipu i listę co najmniej dwóch artefaktów. Odpowiedz: dlaczego przed rozpoczęciem implementacji warto czytać sekcje „Before you begin” i cenniki w dokumentacji?

Zadanie 6.6. Prezentacja programistycznie: JSON + python-pptx

Prezentacja powstanie dwuetapowo i w całości z kodu: Groq generuje konspekt jako JSON o ustalonych polach (kontrakt wymuszony w prompcie i sprawdzany przez `json.loads`), a biblioteka python-pptx buduje z niego plik `.pptx` wraz z notatkami prelegenta.

Treść zadania:

1. W katalogu `src` utwórz plik `prezentacja.py` o następującej treści:

```
import json
import os
import sys

from dotenv import load_dotenv
from groq import Groq
from pptx import Presentation

sys.stdout.reconfigure(encoding='utf-8')
load_dotenv()
client = Groq(api_key=os.getenv('GROQ_API_KEY'))

TEMAT = 'Bezpieczne korzystanie z kluczy API'

PROMPT_KONSPEKTU = (
    'Przygotuj konspekt prezentacji edukacyjnej pt. "' + TEMAT +
    '" dla studentów I roku informatyki. Zwróć WYŁĄCZNIE '
    'poprawny JSON, bez żadnego innego tekstu, o strukturze: '
    '{"tytul": "...", "slajdy": [{"tytul": "...", '
    '"punkty": ["...", "..."], "notatka": "..."}]}. '
```

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

**Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00**

```
    'Dokładnie 6 slajdów; 3-4 zwięzłe punkty i 2-zdaniowa '
    'notatka prelegenta na slajd. Zakres: czym jest klucz API, '
    'typowe błędy (klucz w kodzie, publiczne repozytorium), '
    'dobre praktyki (.env, .gitignore, unieważnianie), '
    'podsumowanie.'
)

# etap 1: konspekt z modelu (kontrakt JSON, niska temperatura)
r = client.chat.completions.create(
    model='llama-3.3-70b-versatile',
    messages=[{'role': 'user', 'content': PROMPT_KONSPEKTU}],
    temperature=0.2,
    max_tokens=1500,
)
tekst = r.choices[0].message.content.strip()
konspekt = json.loads(tekst)          # walidacja kontraktu
print('Slajdów w konspekcie:', len(konspekt['slajdy']))

# etap 2: budowa pliku PPTX z konspektu
prezentacja = Presentation()

# slajd tytułowy (układ 0: tytuł + podtytuł)
slajd = prezentacja.slides.add_slide(
    prezentacja.slide_layouts[0])
slajd.shapes.title.text = konspekt['tytul']
slajd.placeholders[1].text = (
    'Generatywna sztuczna inteligencja i prompt engineering')

# slajdy treści (układ 1: tytuł + lista punktów)
for dane in konspekt['slajdy']:
    slajd = prezentacja.slides.add_slide(
        prezentacja.slide_layouts[1])
    slajd.shapes.title.text = dane['tytul']
    pole = slajd.placeholders[1].text_frame
    pole.text = dane['punkty'][0]
    for punkt in dane['punkty'][1:]:
        pole.add_paragraph().text = punkt
```

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

```
    # notatka prelegenta
    slajd.notes_slide.notes_text_frame.text = dane['notatka']

os.makedirs('out', exist_ok=True)
prezentacja.save('out/prezentacja.pptx')
print('Zapisano out/prezentacja.pptx')
```

2. Uruchom skrypt poleceniem python src/prezentacja.py, a następnie otwórz plik out/prezentacja.pptx w programie PowerPoint lub LibreOffice Impress. Sprawdź slajdy oraz notatki prelegenta (okienko notatek pod slajdem).
3. Oceń konspekt merytorycznie. Poprawki wprowadź w prompcie (np. „dodaj na slajdzie 3 przykład z życia studenta", zmiana liczby slajdów) i wygeneruj prezentację ponownie – zaobserwuj, że cała forma odtwarza się automatycznie, bo treść i forma są rozdzielone.
4. Test kontraktu: zmień na próbę temperature na 1.2 i uruchom skrypt 2–3 razy. Jeśli model doda tekst poza JSON-em, json.loads zgłosi błąd – zanotuj wynik i przywróć wartość 0.2.

P: Dołącz plik `prezentacja.py`, zrzuty 2–3 slajdów (w tym jeden z widocznymi notatkami prelegenta) oraz wynik testu kontraktu. Wyjaśnij: (1) co daje rozdzielenie treści (JSON) od formy (python-pptx)? (2) dlaczego konspekt generowany jest przy niskiej temperaturze?

Ćwiczenia samodzielne

Ćwiczenie 6.1 (podstawowe). Logo koła naukowego – warianty w pętli

Rozszerz skrypt `seria.py`: wygeneruj 4 warianty logo koła naukowego, iterując pętlą po liście czterech różnych opisów stylu (przy stałym podmiocie). Wybierz najlepszy wariant i uzasadnij wybór. W sprawozdaniu zamieść kod pętli, wszystkie warianty z promptami i wskazanie zwycięzcy.

Ćwiczenie 6.2 (średnie). Diagram z kodu – Groq + Mermaid

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

Wygeneruj diagram bez generatora obrazów: napisz skrypt, w którym Groq zwraca wyłącznie kod diagramu w składni Mermaid (np. schemat cyklu iteracyjnego doskonalenia promptu z Laboratorium 2), skrypt zapisuje go do pliku `out/diagram.mmd`, a render wykonujesz w darmowym edytorze `mermaid.live`. W sprawozdaniu zamieść prompt, kod Mermaid i zrzut diagramu oraz odpowiedz: kiedy diagram z kodu jest lepszym wyborem niż obraz z generatora (edytowalność, precyzja, tekst)?

Ćwiczenie 6.3 (średnie). Ilustracje w prezentacji

Połącz zadania 6.4 i 6.6: rozszerz `prezentacja.py` tak, aby dla wybranego slajdu generował ilustrację (funkcja `generuj_obraz` z opisem sceny zbudowanym z tytułu slajdu i przewodnika stylu) i wstawiał ją na slajd metodą `add_picture` (jednostki `Inches` z modułu `pptx.util`). W sprawozdaniu zamieść fragment kodu, zrzut slajdu z ilustracją i ocenę trafności grafiki względem treści slajdu.

Ćwiczenie 6.4 (trudne). Potok pakietu materiałów

Napisz jeden skrypt `pakiet.py`, który na zadany temat generuje kompletny, spójny stylistycznie pakiet: (1) tekst przewodni do 150 słów (Groq), (2) grafikę ilustracyjną (Gemini) z przewodnikiem stylu wplecionym w opis sceny, (3) prezentację 5 slajdów (JSON + python-pptx) – wszystko z jednym, wspólnym przewodnikiem stylu (odbiorca, ton, paleta) zdefiniowanym jako stała na początku skryptu i doklejanym do każdego promptu. W sprawozdaniu zamieść skrypt, wygenerowane elementy pakietu oraz ocenę spójności: która modalność najsłabiej trzymała się wytycznych i dlaczego?

Pytania pomocnicze

1. Czym różni się prompt obrazowy od tekstowego i jakie pięć elementów zawiera pełny opis sceny?

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

2. Jak w kodzie zadania 6.3 odbierany jest obraz z odpowiedzi modelu? Czym różnią się części text i inline_data?
3. Czym różni się uwierzytelnianie kluczem API z AI Studio od trybu enterprise opisanego w dokumentacji Google i dlaczego w laboratorium wybrano klucz API?
4. Jak programistycznie uzyskuje się spójną serię grafik? Dlaczego dwa uruchomienia identycznego promptu dają różne obrazy?
5. Jakie wymagania stawia sekcja „Before you begin" dokumentacji Veo i jaki wniosek płynie z niej dla projektu bez budżetu? Jaka jest darmowa ścieżka generowania wideo?
6. Dlaczego generowanie muzyki modelami Lyria ograniczono w laboratorium do analizy dokumentacji?
7. Jakie elementy – poza opisem sceny – zawiera prompt wideo? Wymień dwa typowe artefakty generowanego wideo.
8. Na czym polega dwuetapowe, programistyczne generowanie prezentacji i jaką rolę pełni kontrakt JSON między modelem a biblioteką python-pptx?
9. Dlaczego konspekt prezentacji generuje się przy niskiej temperaturze i co pokazał test kontraktu przy temperaturze 1.2?

Podsumowanie

Po ukończeniu laboratorium student potrafi programistycznie projektować i wykonywać prompty dla różnych modalności oraz oceniać dostępność usług generatywnych. W szczególności:

1. prowadzi projekt łączący kilka usług generatywnych: Groq (tekst), Gemini API (obrazy) i lokalną Ollamę, z kluczami w pliku `.env`;
2. wykonuje z kodu prompty tekstowe o różnych formatach i porównuje model zewnętrzny z lokalnym;
3. generuje i zapisuje obrazy przez darmową warstwę Gemini API, buduje opisy scen z pełną anatomią i spójne serie w pętli;
4. ocenia na podstawie dokumentacji wykonalność użycia modeli wideo i muzyki (Veo – projekt GCP z billingiem lub ścieżka webowa; Lyria – wyłącznie Vertex) oraz projektuje prompty wideo z opisem ruchu;

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

5. generuje prezentacje w pełni programistycznie: konspekt jako kontrakt JSON, forma z python-pptx, z notatkami prelegenta;
6. zna zasady etyczne i licencyjne pracy z treściami generowanymi przez AI.

WSKAZÓWKA: rozszerz bibliotekę promptów (plik `prompts.md`) o sekcję multimodalną: przewodniki stylu, sprawdzone opisy scen, szablon promptu wideo i prompt-kontrakt konspektu prezentacji – z nazwą modelu i oceną wyniku, bo ten sam opis działa różnie w różnych generatorach. Wzorzec „kontrakt JSON + kod budujący dokument" wróci przy automatyzacji zadań w dalszej części kursu.

UWAGA: oba klucze API chroń w pliku `.env` ujętym w `.gitignore`; nie umieszczaj wywołań generowania obrazów w długich pętlach (dzienne limity darmowej warstwy). Nie generuj wizerunków rzeczywistych osób ani treści wprowadzających w błąd; treści wygenerowane przez AI w pracach zaliczeniowych zawsze wyraźnie oznaczaj wraz z nazwą modelu. Przed publicznym użyciem grafiki lub wideo sprawdź warunki licencyjne usługi.

Literatura

[1]. Google Cloud, Generate images with Gemini – dokumentacja, https://docs.cloud.google.com/gemini-enterprise-agent-platform/models/capabilities/image-generation (dostęp: 2026).

[2]. Google Cloud, Generate videos from text prompts (Veo) – dokumentacja, https://docs.cloud.google.com/gemini-enterprise-agent-platform/models/video/generate-videos-from-text (dostęp: 2026).

[3]. Google Cloud, Generate music with Lyria – dokumentacja, https://docs.cloud.google.com/gemini-enterprise-agent-platform/models/music/generate-music (dostęp: 2026).

[4]. Google Cloud, Veo prompt guide – przewodnik projektowania promptów wideo, https://docs.cloud.google.com/gemini-enterprise-agent-platform/models/video/video-gen-prompt-guide (dostęp: 2026).

Fundusze Europejskie dla Rozwoju Społecznego | Rzeczpospolita Polska | Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

[5]. Google, Gemini API – Image generation (darmowa warstwa, limity), https://ai.google.dev/gemini-api/docs/image-generation (dostęp: 2026).
[6]. Groq, Text/Chat Generation – dokumentacja API, https://console.groq.com/docs/text-chat (dostęp: 2026).
[7]. Ollama Python Library – dokumentacja, https://github.com/ollama/ollama-python (dostęp: 2026).
[8]. python-pptx – dokumentacja biblioteki, https://python-pptx.readthedocs.io/ (dostęp: 2026).
[9]. Mizrahi G., Inżynieria promptów bez tajemnic. Sztuka kreatywnego generowania tekstów, Helion, Gliwice 2025.
[10]. Phoenix J., Taylor M., Skuteczna inżynieria promptów. Przyszłościowe rozwiązania dla rzetelnych wyników generatywnej AI, Helion, Gliwice 2025.

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

POLITECHNIKA LUBELSKA

Materiały do przedmiotu:

GENERATYWNA SZTUCZNA INTELIGENCJA I PROMPT ENGINEERING

Laboratorium nr 7.
Ocena wyników i optymalizacja promptów. Test A/B

Autor:

dr inż. Krzysztof Dziedzic

dr Marcin Barszcz

Fundusze Europejskie dla Rozwoju Społecznego

Rzeczpospolita Polska

Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

POLLUB z nami nowoczesne technologie

POLITECHNIKA LUBELSKA

Fundusze Europejskie dla Rozwoju Społecznego

Rzeczpospolita Polska

Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

Spis treści

Wprowadzenie, tematyka zajęć ----------141
Cel ćwiczenia/laboratorium---------- 142
Instrukcja laboratoryjna/ćwiczeniowa---------- 142
Ćwiczenia samodzielne ---------- 152
Pytania pomocnicze ---------- 153
Podsumowanie---------- 153
Literatura ---------- 154

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

Wprowadzenie, tematyka zajęć

Niniejsze laboratorium poświęcone jest ocenie wyników i optymalizacji promptów, w tym testom A/B, realizowanym programistycznie z użyciem biblioteki DSPy w środowisku Visual Studio Code. Ćwiczenie łączy trzy wątki przedmiotu: konstruowanie skutecznych promptów, architekturę RAG (Retrieval-Augmented Generation) oraz automatyczną optymalizację i porównawczą ocenę promptów. Model językowy uruchamiany jest lokalnie (Ollama), dzięki czemu całość działa bez kont w usługach chmurowych i bez kluczy API.

Test A/B w kontekście prompt engineeringu to metoda porównawczej oceny dwóch wariantów promptu (A i B), polegająca na uruchomieniu obu wariantów w identycznych warunkach (na tym samym zbiorze danych i z tą samą metryką), a następnie na wyborze wariantu o wyższej skuteczności. Pozwala to obiektywnie, na podstawie pomiaru ilościowego, wskazać lepszy prompt zamiast polegać na ocenie intuicyjnej. Metoda wywodzi się z eksperymentów prowadzonych w analityce internetowej i badaniach użyteczności, w których porównuje się dwie wersje strony lub interfejsu. W prompt engineeringu warianty mogą różnić się treścią instrukcji, doborem przykładów few-shot, formatem oczekiwanej odpowiedzi, modelem bazowym lub parametrami generowania (np. temperaturą). Warunkiem rzetelnego testu jest zmiana tylko jednego czynnika naraz oraz odpowiednio liczny zbiór ewaluacyjny. Przy małej próbie niewielka różnica trafności może być dziełem przypadku, a wynik należy traktować jako nierozstrzygnięty.

DSPy to framework do programistycznego budowania i optymalizacji aplikacji opartych na dużych modelach językowych. Zamiast ręcznego pisania promptów zadanie definiuje się w nim deklaratywnie (za pomocą sygnatur i modułów), a treść promptu, w tym dobór przykładów few-shot, generuje automatycznie optymalizator na podstawie zdefiniowanej metryki oraz zbioru przykładów. Framework powstał na Uniwersytecie Stanforda, a jego nazwa nawiązuje do idei „programowania" modeli językowych zamiast ręcznego formułowania poleceń (Declarative Self-improving Python). Podstawowe elementy DSPy to: sygnatura (Signature),

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

opisująca wejścia i wyjścia zadania; moduły (np. Predict, ChainOfThought), realizujące strategie generowania odpowiedzi; oraz optymalizatory (np. BootstrapFewShot, MIPROv2), które podczas kompilacji dobierają treść promptu tak, aby maksymalizować wskazaną metrykę. Dzięki warstwie LiteLLM ten sam program może współpracować z różnymi dostawcami modeli (Ollama, OpenAI, watsonx) bez zmiany logiki – zmienia się jedynie konfigurację backendu.

Cel ćwiczenia/laboratorium

Celem laboratorium jest nabycie umiejętności budowy, automatycznej optymalizacji oraz ilościowej oceny aplikacji LLM typu pytanie–odpowiedź (QA) z architekturą RAG, przy użyciu frameworka DSPy w środowisku Visual Studio Code z modelem uruchamianym lokalnie. Po ukończeniu zajęć student potrafi:

- skonfigurować środowisko pracy: Visual Studio Code (rozszerzenia Python i Jupyter), wirtualne środowisko Pythona oraz lokalny serwer modeli Ollama;
- zdefiniować zadanie w DSPy za pomocą sygnatury i modułu oraz uruchomić predyktor;
- zbudować potok RAG łączący wyszukiwanie kontekstu z generowaniem odpowiedzi;
- zastosować optymalizator BootstrapFewShot do automatycznego wygenerowania promptu;
- zmierzyć trafność potoku za pomocą dspy.Evaluate oraz odczytać wygenerowany prompt;
- przeprowadzić test A/B wariantów promptu i wskazać wariant o wyższej trafności;
- przełączyć backend modelu (Ollama) bez zmiany logiki programu.

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

Instrukcja laboratoryjna/ćwiczeniowa

Zadania wykonujemy w oparciu o przygotowany projekt (notatniki DSPy_RAG_VSCode.ipynb oraz DSPy_Test_AB.ipynb wraz z plikami README.md, requirements.txt i .env.example).

Wymagania techniczne:

- **Oprogramowanie** – Python 3.10+, Visual Studio Code z rozszerzeniami Python i Jupyter, Ollama.
- **Biblioteki** – dspy oraz python-dotenv (instalowane z pliku requirements.txt).
- **Model** – lokalny model językowy pobrany przez Ollamę (np. llama3.1).

Zadanie 7.1. Przygotowanie środowiska

W tym laboratorium pracujemy w osobnym folderze projektu z własnym środowiskiem wirtualnym, aby zależności Lab 4 (dspy) były odizolowane. Python 3.10+, VS Code z rozszerzeniami Python i Jupyter oraz Ollama z pobranym modelem są już zainstalowane na komputerze – poniżej jedynie je weryfikujemy.

Treść zadania:

1. Rozpakuj archiwum LAB07.zip do wybranej lokalizacji. Powstanie folder projektu z plikami: DSPy_RAG_VSCode.ipynb, DSPy_Test_AB.ipynb, requirements.txt, .env.example, README.md oraz instrukcją.
2. Otwórz ten folder w VS Code: File → Open Folder i wskaż rozpakowany katalog.
3. Otwórz terminal (Terminal → New Terminal) i zweryfikuj dostępność narzędzi:

```
python --version      # powinno byc 3.10+
ollama list           # powinien byc widoczny np. llama3.1
                      # (w razie braku: ollama pull llama3.1)
```

Sprawdź też, że w panelu Extensions (Ctrl+Shift+X) czy masz rozszerzenia Python i Jupyter.

4. Utwórz i aktywuj nowe środowisko wirtualne wewnątrz folderu projektu:

```
python -m venv .venv
```

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

```
# Windows (PowerShell):
.venv\Scripts\Activate.ps1
# macOS / Linux:
source .venv/bin/activate
```

5. Zainstaluj zależności laboratorium (instaluje dspy oraz python-dotenv):

```
pip install -r requirements.txt
python -m pip install ipykernel
```

6. Otwórz DSPy_RAG_VSCode.ipynb, kliknij Select Kernel i wybierz interpreter z utworzonego .venv.
7. Uruchom pierwszą komórkę (Shift+Enter) – wyświetli się numer wersji DSPy, co potwierdza gotowość środowiska.

Zadanie 7.2. Konfiguracja modelu w DSPy

Połącz DSPy z lokalnym modelem i wykonaj test poprawności konfiguracji (kroki 1–5).

Treść zadania:

1. Uruchom komórkę notatnika (Shift+Enter) – importuje ona DSPy i wczytuje zmienne środowiskowe z pliku .env.

```
import os, dspy
from dotenv import load_dotenv
load_dotenv(override=True)
```

2. Uruchom komórkę, która tworzy obiekt modelu dspy.LM dla Ollamy. Parametry: nazwa modelu w formacie „ollama_chat/<model>”, adres serwera (api_base), pusty klucz (api_key) oraz temperatura sterująca losowością generowania.

```
lm = dspy.LM("ollama_chat/llama3.1",
             api_base="http://localhost:11434",
             api_key="", temperature=0.7)
```

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

3. Uruchom komórkę, która rejestruje model w DSPy, aby wszystkie moduły korzystały z niego automatycznie.

```
dspy.configure(lm=lm)
```

4. Uruchom komórkę, która wysyła proste zapytanie i wyświetla odpowiedź. Wywołanie modelu zwraca listę odpowiedzi, dlatego pobieramy pierwszy element.

```
print(lm("Czym jest inzynieria promptow?")[0])
```

5. Odpowiedź powinna pojawić się lokalnie (przy pierwszym wywołaniu może wystąpić opóźnienie, ładowanie modelu do pamięci). Jeśli pojawi się błąd połączenia, sprawdź w terminalu poleceniem ollama list, czy usługa działa i czy model został pobrany.

P: Wykonaj zrzut ekranu z odpowiedzią modelu na pierwsze zapytanie i zamieść go w sprawozdaniu.

Zadanie 7.3. Podstawowe QA bez kontekstu

Zdefiniuj sygnaturę zadania i zaobserwuj odpowiedź modelu na pytanie spoza jego wiedzy.

Treść zadania:

1. Uruchom komórkę definiującą sygnaturę BasicQA. Docstring sygnatury staje się instrukcją promptu, pole InputField opisuje dane wejściowe, a OutputField – oczekiwane wyjście.

```
class BasicQA(dspy.Signature):
    "Odpowiadaj zwięźle, krótkimi odpowiedziami."
    pytanie = dspy.InputField()
    odpowiedz = dspy.OutputField(desc="1-5 slow")
```

2. Uruchom komórkę, która tworzy wykonywalny predyktor na podstawie sygnatury.

```
qa = dspy.Predict(BasicQA)
```

3. Uruchom komórkę, która zadaje pytanie o fikcyjny framework Helion DataFlow (wymyślony na potrzeby ćwiczenia) i wyświetla odpowiedź.

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

```
print(qa(pytanie="W ktorym roku powstal framework Helion
DataFlow?").odpowiedz)
```

4. Zanotuj odpowiedź. Ponieważ model nie zna tego frameworka, najpewniej ją zmyśli (halucynacja). To pokazuje ograniczenie modelu działającego wyłącznie na wiedzy parametrycznej i uzasadnia zastosowanie RAG.

P: Zamieść w sprawozdaniu odpowiedź modelu na pytanie o framework Helion DataFlow i skomentuj, dlaczego jest to halucynacja.

Zadanie 7.4. Architektura RAG

Dołącz wyszukany kontekst do promptu i porównaj odpowiedź z poprzednią.

Treść zadania:

1. Uruchom komórkę definiującą listę krótkich fragmentów tekstu – lokalną bazę wiedzy. Zawiera ona fakty o Helion DataFlow oraz kilka fragmentów niezwiązanych (dystraktorów).

```
KORPUS = [
    "Helion DataFlow to framework do przetwarzania strumieniowego,
     stworzony w 2021 roku przez Marie Kowalczyk...",
    "Jezyk zapytan Helion DataFlow nazywa sie HQL...",
    # ... kolejne fakty oraz dystraktory
]
```

2. Uruchom komórkę definiującą funkcję retrieve, która zwraca k fragmentów najbardziej dopasowanych do pytania (wg liczby wspólnych słów). To uproszczony mechanizm wyszukiwania (retriever).

```
import re
def retrieve(pytanie, k=4):
    q = set(re.findall(r"\w+", pytanie.lower()))
    return sorted(KORPUS,
                  key=lambda f: len(q & set(re.findall(r"\w+",
f.lower()))),
                  reverse=True)[:k]
```

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

3. Uruchom komórkę definiującą sygnaturę GenerateAnswer, rozszerzoną o pole kontekst – model ma odpowiadać wyłącznie na jego podstawie.

```
class GenerateAnswer(dspy.Signature):
    """Odpowiadaj wylacznie na podstawie kontekstu."""
    kontekst = dspy.InputField()
    pytanie = dspy.InputField()
    odpowiedz = dspy.OutputField(desc="1-5 slow")
```

4. Uruchom komórkę definiującą moduł RAG, który w metodzie forward najpierw wyszukuje kontekst, a następnie generuje odpowiedź z rozumowaniem krok po kroku (ChainOfThought).

```
class RAG(dspy.Module):
    def __init__(self, k=4):
        super().__init__()
        self.k = k
        self.generuj = dspy.ChainOfThought(GenerateAnswer)
    def forward(self, pytanie):
        ctx = retrieve(pytanie, k=self.k)
        return self.generuj(kontekst=ctx, pytanie=pytanie)
```

5. Uruchom komórkę, która zadaje to samo pytanie co w Zadaniu 7.3, tym razem przez moduł RAG, i wyświetla odpowiedź.

```
rag = RAG()
print(rag(pytanie="W ktorym roku powstal framework Helion
DataFlow?").odpowiedz)
```

6. Porównaj wynik z Zadaniem 7.3. Po dołączeniu kontekstu model podaje poprawną odpowiedź (2021, Maria Kowalczyk) – to istota RAG: uziemienie odpowiedzi w dostarczonych faktach.

P: Porównaj w sprawozdaniu odpowiedzi z zadań 7.3 i 7.4 oraz wyjaśnij, skąd bierze się różnica.

Zadanie 7.5. Automatyczna optymalizacja promptu

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

Użyj optymalizatora DSPy, aby automatycznie wygenerować prompt, a następnie odczytaj jego treść (kroki 1–7).

Treść zadania:

1. Uruchom komórkę przygotowującą zbiór przykładów pytanie–odpowiedź. Posłużą one optymalizatorowi do doboru demonstracji few-shot.

```
trening = [
    dspy.Example(pytanie="W ktorym roku powstal Helion DataFlow?",
                 odpowiedz="2021").with_inputs("pytanie"),
    dspy.Example(pytanie="Kto jest autorem Helion DataFlow?",
                 odpowiedz="Maria
Kowalczyk").with_inputs("pytanie"),
    # ... kolejne przyklady
]
```

2. Uruchom komórkę definiującą funkcję oceniającą trafność – odpowiedź jest poprawna, gdy zawiera odpowiedź wzorcową.

```
def metryka(example, pred, trace=None):
    return example.odpowiedz.lower() in (pred.odpowiedz or
"").lower()
```

3. Uruchom komórkę, która tworzy optymalizator BootstrapFewShot, przekazując mu metrykę oraz maksymalną liczbę automatycznie dobieranych przykładów.

```
from dspy.teleprompt import BootstrapFewShot
opt = BootstrapFewShot(metric=metryka, max_bootstrapped_demos=4)
```

4. Uruchom komórkę, która kompiluje moduł RAG na zbiorze danych treningowych. Podczas kompilacji DSPy uruchamia moduł na przykładach i samodzielnie dobiera najlepsze demonstracje few-shot do promptu.

```
rag_opt = opt.compile(RAG(), trainset=trening)
```

5. Uruchom komórkę, która zadaje pytanie zoptymalizowanym modułem i wyświetla odpowiedź.

```
print(rag_opt(pytanie="W ktorym roku powstal Helion
DataFlow?").odpowiedz)
```

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

6. Uruchom komórkę, która wyświetla rzeczywisty prompt wysłany do modelu. Zobaczysz przykłady few-shot dobrane automatycznie podczas kompilacji.

```
dspy.inspect_history(n=1)
```

7. Zwróć uwagę, że inżynierię promptu (dobór i sformatowanie przykładów) wykonał framework, a nie człowiek – to istota podejścia programistycznego w DSPy.

P: Dołącz do sprawozdania fragment promptu wygenerowanego przez DSPy (inspect_history) wraz z krótkim komentarzem.

Zadanie 7.6. Test A/B – porównanie wariantów promptu

Porównaj ilościowo dwa warianty rozwiązania na wspólnym zbiorze ewaluacyjnym i wybierz lepszy.

Test A/B polega na zestawieniu dwóch wariantów (A i B) w identycznych warunkach (ten sam zbiór danych i ta sama metryka) a następnie na wyborze wariantu o wyższej skuteczności. Porównamy: Wariant A – prompt bazowy (RAG bez optymalizacji); Wariant B – ten sam potok z promptem zoptymalizowanym przez BootstrapFewShot. Zadanie korzysta z korpusu, mechanizmu wyszukiwania i modułu RAG z zadań 7.3–7.5 (możesz też użyć gotowego notatnika DSPy_Test_AB.ipynb).

Uwaga metodyczna: aby wynik nie kończył się remisem, stosujemy rygorystyczną metrykę dokładnego dopasowania oraz pomiar przy temperaturze 0 (powtarzalność). Przy łagodnej metryce „zawiera się" oba warianty osiągałyby ~100% (remis); metryka dokładna daje wariantowi B mierzalną przewagę, ponieważ few-shot uczy model podawać zwięzłą, kanoniczną odpowiedź.

Treść zadania:

1. Uruchom komórkę przygotowującą zestaw pytań z poprawnymi odpowiedziami, rozłączną ze zbiorem treningowym – dzięki temu ocena jest uczciwa (model nie był „uczony" na pytaniach testowych).

```
devset = [
    dspy.Example(pytanie="Jakiego jezyka zapytan uzywa Helion
DataFlow?", odpowiedz="HQL").with_inputs("pytanie"),
    dspy.Example(pytanie="W jakim jezyku napisano Helion
DataFlow?", odpowiedz="Scala").with_inputs("pytanie"),
    dspy.Example(pytanie="Na jakiej uczelni powstal Helion
DataFlow?", odpowiedz="Politechnika
Lubelska").with_inputs("pytanie"),
    dspy.Example(pytanie="Z czym integruje sie Helion DataFlow?",
odpowiedz="Kafka").with_inputs("pytanie"),
]
```

2. Uruchom komórkę definiującą rygorystyczną metrykę – odpowiedź jest poprawna tylko wtedy, gdy po normalizacji (małe litery, bez interpunkcji) jest dokładnie równa wzorcowej. To kluczowa zmiana w porównaniu z metryką „zawiera się", która dawałaby remis.

```
import re
def _norm(s):
    s = re.sub(r"[^\w\s]", " ", (s or "").lower())
    return re.sub(r"\s+", " ", s).strip()

def metryka(example, pred, trace=None):
    return _norm(pred.odpowiedz) == _norm(example.odpowiedz)
```

3. Wariant A to potok RAG bez optymalizacji. Wariant B powstaje poprzez kompilację tego samego potoku optymalizatorem BootstrapFewShot na zbiorze treningowym z Zadania 7.5.

```
from dspy.teleprompt import BootstrapFewShot
wariant_A = RAG()                                    # bazowy
opt = BootstrapFewShot(metric=metryka, max_bootstrapped_demos=4)
wariant_B = opt.compile(RAG(), trainset=trening) # zoptymalizowany
```

4. Uruchom komórkę mierzącą skuteczność obu wariantów na zbiorze devset. Dla powtarzalności pomiar wykonywany jest przy temperaturze 0 (bez losowości, wynik jest deterministyczny). Pomiar w identycznych warunkach to istota testu A/B.

```
ewaluator = dspy.Evaluate(devset=devset, metric=metryka,
```

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

```
                              num_threads=1, display_progress=True)
lm_pomiar = dspy.LM("ollama_chat/llama3.1",
api_base="http://localhost:11434",
                    api_key="", temperature=0.0)

def procent(program):
    with dspy.context(lm=lm_pomiar):   # wymuszenie temperature=0
na czas pomiaru
        w = ewaluator(program)
    return getattr(w, "score", w)
wynik_A = procent(wariant_A)
wynik_B = procent(wariant_B)
print("Wariant A:", wynik_A, "%   |   Wariant B:", wynik_B, "%")
```

5. Uruchom komórkę, która oblicza różnicę trafności i wskazuje lepszy wariant. Jeżeli różnica jest minimalna, wynik traktuj jako nierozstrzygnięty i powtórz pomiar na większym zbiorze danych.

```
roznica = wynik_B - wynik_A
if   wynik_B > wynik_A: zwyciezca = "B (zoptymalizowany)"
elif wynik_A > wynik_B: zwyciezca = "A (bazowy)"
else:                   zwyciezca = "remis"
print(f"Roznica B-A: {roznica:+.1f} p.p.  ->  zwyciezca:
{zwyciezca}")
```

6. Uruchom komórkę, która powtarza test dla innej pary wariantów – dwóch wartości temperatury (np. 0,0 i 1,0) – aby zbadać wpływ losowości generowania na trafność. Zestaw w tabeli i sformułuj wniosek.

```
for t in (0.0, 1.0):
    lm_t = dspy.LM("ollama_chat/llama3.1",
api_base="http://localhost:11434",
                   api_key="", temperature=t)
    with dspy.context(lm=lm_t):
        w = ewaluator(wariant_B)
        print(f"temperatura {t}:", getattr(w, "score", w), "%")
```

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

P: Uzupełnij Tabelę 7.1 wynikami pomiarów i sformułuj wniosek, który wariant promptu wybrać do wdrożenia.

Tabela 7.1. Wyniki testu A/B wariantów promptu

Wariant	Opis	Trafność [%]
A	Prompt bazowy (RAG bez optymalizacji)	
B	Prompt zoptymalizowany (BootstrapFewShot)	

Opracowanie wyników całości:

- Zestaw odpowiedzi z zadań 7.3–7.5 (bez kontekstu, RAG, po optymalizacji) oraz porównaj ich trafność.
- Dołącz fragment promptu wygenerowanego przez DSPy (z inspect_history) wraz z krótkim komentarzem.
- Przedstaw wynik testu A/B (Tabela 7.1) i sformułuj wniosek, który wariant wybrać do wdrożenia.

Ćwiczenia samodzielne

Ćwiczenie 7.1 (podstawowe). Zmiana modelu

Pobierz w Ollamie inny model (np. qwen2.5 lub mistral), ustaw go w pliku .env (zmienna OLLAMA_MODEL) i powtórz Zadania 7.3–7.5. Porównaj trafność oraz czas odpowiedzi.

Ćwiczenie 7.2 (podstawowe). Rozbudowa bazy wiedzy

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

Dodaj do korpusu co najmniej 5 nowych faktów oraz odpowiadające im pytania. Sprawdź, czy mechanizm wyszukiwania poprawnie odnajduje właściwe fragmenty i czy odpowiedzi są trafne.

Ćwiczenie 7.3 (podstawowe). Pomiar jakości

Zbuduj zbiór ewaluacyjny (devset) i zmierz trafność potoku za pomocą dspy.Evaluate przed i po optymalizacji. Wyniki przedstaw w tabeli i skomentuj różnicę.

Ćwiczenie 7.4 (średnie). Porównanie optymalizatorów

Zastąp BootstrapFewShot optymalizatorem dspy.MIPROv2 i porównaj jakość odpowiedzi oraz czas kompilacji obu podejść.

Ćwiczenie 7.5 (trudne). Test A/B wariantów promptu

Przeprowadź test A/B dwóch wariantów rozwiązania na tym samym zbiorze ewaluacyjnym (devset): Wariant A – prompt bazowy (bez optymalizacji), Wariant B – prompt zoptymalizowany przez BootstrapFewShot. Dla obu wariantów zmierz trafność za pomocą dspy.Evaluate, zestaw wyniki w tabeli, oceń istotność różnicy i wskaż wariant zwycięski. Następnie powtórz test dla innej pary wariantów (np. dwóch różnych modeli lub dwóch różnych wartości temperatury) i sformułuj wniosek, który wariant wybrać do wdrożenia.

Pytania pomocnicze

1. Czym różni się programistyczne tworzenie promptów w DSPy od ręcznego prompt engineeringu? Jakie są zalety pierwszego podejścia?
2. Jaką funkcję pełnią sygnatura (Signature) oraz jej docstring? Co opisują pola InputField i OutputField?
3. Na czym polega architektura RAG i dlaczego ogranicza ryzyko halucynacji?

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

4. Co robi optymalizator BootstrapFewShot podczas kompilacji? Skąd pochodzą „few-shot” w wynikowym promptcie?
5. Na czym polega test A/B promptów? Dlaczego oba warianty trzeba oceniać na tym samym zbiorze i według tej samej metryki?
6. Czy zwiększenie liczby pobieranych fragmentów (k) zawsze poprawia odpowiedź? Tak/Nie – uzasadnij.
7. W jaki sposób definicja metryki wpływa na wynik oceny i optymalizacji? Co się stanie, gdy metryka będzie zbyt liberalna lub zbyt rygorystyczna?
8. Jak wielkość zbioru ewaluacyjnego wpływa na wiarygodność wyniku testu A/B?

Podsumowanie

W trakcie laboratorium zbudowano w środowisku Visual Studio Code kompletny potok pytanie–odpowiedź z architekturą RAG, automatyczną optymalizacją promptów oraz ich ilościową oceną (test A/B) przy użyciu frameworku DSPy i modelu uruchamianego lokalnie.

- Pokazano różnicę między odpowiedzią modelu bez kontekstu (ryzyko halucynacji) a odpowiedzią opartą na wyszukanym kontekście (RAG).
- Zademonstrowano, że optymalizator DSPy potrafi automatycznie wygenerować skuteczny prompt na podstawie zbioru przykładów oraz metryki.
- Przeprowadzono test A/B, który pozwala obiektywnie wybrać lepszy wariant promptu na podstawie pomiaru trafności.

Literatura

[1]. White J. I in., A Prompt Pattern Catalogue to Enhance Prompt Engineering with ChatGPT, arXiv:2302.11382, 2023.

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

[2]. Khattab O. I in., DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines, ICLR 2024.
[3]. Lewis P. I in., Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks, NeurIPS 2020.
[4]. Liu N. F. i in., Lost in the Middle: How Language Models Use Long Contexts, TACL 2023.
[5]. Noble J., Prompt Engineering with DSPy, IBM Think (tutorial), 2024. https://www.ibm.com/think/tutorials/prompt-engineering-with-dspy
[6]. Dokumentacja DSPy, https://dspy.ai (dostęp: 2026).
[7]. Ollama – lokalne uruchamianie modeli językowych, https://ollama.com (dostęp: 2026).

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

POLITECHNIKA LUBELSKA

Materiały do przedmiotu:

GENERATYWNA SZTUCZNA INTELIGENCJA I PROMPT ENGINEERING

Laboratorium nr 8.
Tworzenie UI w Streamlit: wejście, wybór modelu, temperatura, zachowanie pamięci rozmowy. Generowanie i użycie API key

Autor:

dr inż. Krzysztof Dziedzic

dr Marcin Barszcz

Katedra Informatyki

Fundusze Europejskie dla Rozwoju Społecznego

Rzeczpospolita Polska

Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

Spis treści

Wprowadzenie, tematyka zajęć ---------- 157
Cel ćwiczenia/laboratorium ---------- 158
Instrukcja laboratoryjna/ćwiczeniowa ---------- 159
Ćwiczenia samodzielne ---------- 168
Pytania pomocnicze ---------- 168
Podsumowanie ---------- 169
Literatura ---------- 170

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

Wprowadzenie, tematyka zajęć

Niniejsze laboratorium rozpoczyna cykl zajęć poświęconych budowie kompletnej aplikacji asystenta opartej na dużym modelu językowym. W poprzednich laboratoriach przygotowano środowisko pracy (Python, VS Code, klucz API Groq), poznano parametry sterujące generacją oraz podstawowe i zaawansowane techniki projektowania promptów. Dotychczas komunikacja z modelem odbywała się jednak wyłącznie przez terminal. W tym laboratorium powstanie graficzny interfejs użytkownika (UI) w bibliotece Streamlit: pole wejściowe, wybór modelu, suwak temperatury, tryby asystenta oraz historia interakcji zachowywana w stanie sesji [1].

Streamlit to biblioteka języka Python służąca do szybkiego tworzenia interaktywnych aplikacji webowych bez znajomości HTML, CSS ani JavaScriptu. Aplikacja Streamlit to zwykły skrypt Pythona, który po uruchomieniu poleceniem `streamlit run` jest serwowany w przeglądarce (domyślnie pod adresem `http://localhost:8501`). Streamlit jest standardem przy budowie prototypów aplikacji [2].

Model wykonania (rerun). Kluczową cechą Streamlit jest sposób wykonywania skryptu. Przy każdej interakcji użytkownika (kliknięcie przycisku, zmiana wartości suwaka, wysłanie tekstu) cały skrypt jest wykonywany ponownie od początku do końca. Ten model, nazywany rerun, upraszcza kod (nie trzeba obsługiwać zdarzeń), ale ma ważną konsekwencję: zwykłe zmienne Pythona są tworzone na nowo przy każdym przebiegu i tracą swoje wartości. Dlatego wszystko, co ma przetrwać między interakcjami (na przykład historia rozmowy) musi być przechowywane w specjalnym obiekcie `st.session_state` [3].

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

st.session_state to słownik przypisany do sesji przeglądarki użytkownika, który zachowuje wartości między kolejnymi uruchomieniami skryptu. Zapis `st.session_state.history = []` tworzy trwałą listę, a warunek `if 'history' not in st.session_state` gwarantuje, że inicjalizacja wykona się tylko raz podczas pierwszego uruchomienia. To właśnie w `session_state` aplikacja będzie zachowywać pamięć (historię) rozmowy z asystentem [4].

Widżety i układ strony. Streamlit udostępnia gotowe komponenty interfejsu: `st.title()` (tytuł), `st.text_input()` (pole tekstowe), `st.selectbox()` (lista rozwijana), `st.slider()` (suwak), `st.checkbox()` (pole wyboru), `st.button()` (przycisk), `st.markdown()` (sformatowany tekst). Elementy można umieszczać w panelu bocznym przez przedrostek `st.sidebar`, co pozwala oddzielić ustawienia (wybór modelu, temperaturę, tryb asystenta) od głównej treści strony [4].

Klucz API i bezpieczeństwo. Dostęp do modeli w usłudze GroqCloud wymaga uwierzytelnienia za pomocą klucza API. W tym laboratorium generowany jest nowy, dedykowany klucz dla projektu. To dobra praktyka inżynierska: osobne klucze dla osobnych aplikacji ułatwiają kontrolę użycia i pozwalają unieważnić klucz jednej aplikacji bez wpływu na pozostałe. Klucz przechowywany jest poza kodem źródłowym, w pliku `.env` wczytywanym przez bibliotekę `python-dotenv`, a plik `.gitignore` chroni go przed przypadkową publikacją w repozytorium [5, 6].

Architektura projektu. Aplikacja zostanie podzielona na warstwy: moduł `models.py` odpowiada wyłącznie za komunikację z API (utworzenie klienta, wywołanie modelu, wczytanie promptu systemowego), a moduł `app_ui.py` wyłącznie za interfejs. Dodatkowo prompty systemowe są przechowywane w osobnych plikach tekstowych w katalogu `prompts/`, dzięki czemu tryb pracy asystenta można zmieniać bez modyfikacji kodu. Taka separacja ułatwi rozbudowę i modyfikację aplikacji w laboratoriach 9–11 [1].

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

Cel ćwiczenia/laboratorium

Głównym celem laboratorium jest samodzielne zbudowanie od podstaw aplikacji webowej asystenta w Streamlit, wraz z konfiguracją nowego projektu i wygenerowaniem nowego klucza API. Po ukończeniu zajęć student potrafi:

- utworzyć nowy projekt aplikacji z izolowanym środowiskiem wirtualnym (venv) i listą zależności w pliku `requirements.txt`;
- wygenerować w usłudze GroqCloud nowy klucz API dla projektu i bezpiecznie go przechować w pliku `.env` chronionym wpisem w `.gitignore`;
- wyjaśnić model wykonania aplikacji Streamlit (rerun) oraz rolę obiektu `st.session_state` w zachowywaniu stanu między interakcjami;
- zbudować interfejs asystenta: pole wejściowe, panel boczny z wyborem modelu, suwakiem temperatury i wyborem trybu asystenta;
- zachowywać historię interakcji w `st.session_state` i wyświetlać ją w aplikacji;
- rozdzielić logikę komunikacji z modelem (`models.py`) od warstwy interfejsu (`app_ui.py`) oraz ładować prompty systemowe z plików tekstowych;
- zbadać eksperymentalnie wpływ temperatury i wyboru modelu na odpowiedzi, sterując nimi z poziomu interfejsu.

Instrukcja laboratoryjna/ćwiczeniowa

Zadanie 8.1. Nowy projekt i środowisko wirtualne

Aplikacja tworzona jest od początku, w nowym katalogu projektu, niezależnie od katalogów z poprzednich laboratoriów. Docelowa struktura projektu wygląda następująco:

```
asystent_streamlit/
|-- src/
|   |-- app_ui.py          # interfejs Streamlit (Zadanie 8.5)
```

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

```
|   |-- demo_rerun.py      # demonstracja mechanizmu rerun
(Zadanie 8.4)
|   |-- models.py          # warstwa modeli (Zadanie 8.3)
|   `-- test_client.py     # klient terminalowy (Zadanie 8.3)
|-- prompts/
|   |-- system_note_assistant.txt
|   `-- system_translator.txt
|-- .env                   # klucz API (poza repozytorium!)
|-- .gitignore
`-- requirements.txt
```

Treść zadania:

1. Utwórz nowy katalog roboczy, np. `C:\GenAI_Lab\asystentAI` i otwórz go w VS Code (**File ▸ Open Folder**).
2. W katalogu projektu utwórz podkatalogi `src` oraz `prompts`.
3. Otwórz wbudowany terminal (**Terminal ▸ New Terminal**), utwórz i aktywuj nowe środowisko wirtualne:

```
# utworzenie izolowanego środowiska wirtualnego
python -m venv .venv

# aktywacja  (Windows PowerShell)
.venv\Scripts\Activate.ps1
# aktywacja  (Linux / macOS)
source .venv/bin/activate
```

4. Zainstaluj wymagane biblioteki i utrwal listę zależności projektu:

```
pip install -U streamlit groq python-dotenv
pip freeze > requirements.txt
```

5. Zweryfikuj instalację Streamlit poleceniem `streamlit --version` – w terminalu powinna pojawić się nazwa i numer wersji biblioteki.

Środowisko wirtualne nowego projektu jest niezależne od środowisk z poprzednich laboratoriów. Zawiera tylko te biblioteki, których projekt rzeczywiście potrzebuje.

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

Dzięki plikowi `requirements.txt` projekt można odtworzyć na innym komputerze jednym poleceniem `pip install -r requirements.txt`.

Zadanie 8.2. Wygenerowanie nowego klucza API i konfiguracja .env

Dla nowego projektu generowany jest nowy klucz API – nie należy kopiować klucza z poprzednich laboratoriów. Osobny klucz dla każdej aplikacji ułatwia diagnozowanie limitów zapytań i pozwala w razie wycieku unieważnić tylko jeden klucz.

Treść zadania:

1. Wejdź na `console.groq.com` i zaloguj się kontem Google lub GitHub.
2. Wybierz zakładkę **API Keys** ▸ **Create API Key**, nadaj kluczowi nazwę wskazującą projekt (np. `asystentAI`) i skopiuj go – pełna wartość klucza jest wyświetlana tylko raz.
3. W katalogu głównym projektu utwórz plik `.env` i zapisz w nim klucz (bez cudzysłowów, bez spacji wokół znaku =):

```
GROQ_API_KEY=twoj_wygenerowany_klucz_api
```

4. Utwórz plik `.gitignore` i dodaj wpisy chroniące klucz oraz pliki środowiska przed publikacją w repozytorium:

```
.env
.venv/
__pycache__/
```

Zadanie 8.3. Warstwa modeli (models.py) i test terminalowy

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

Zanim powstanie interfejs, przygotowana zostanie warstwa komunikacji z API – moduł `models.py` z trzema funkcjami: `get_client()` (utworzenie klienta Groq), `zapytaj()` (jedno wywołanie modelu) oraz `load_system_prompt()` (wczytanie promptu systemowego z pliku). Interfejs w Zadaniu 8.5 będzie wyłącznie korzystał z tego modułu.

Treść zadania:

1. W katalogu `src` utwórz plik `models.py` o następującej treści:

```
import os

from dotenv import load_dotenv
from groq import Groq

# wczytanie zmiennych środowiskowych z pliku .env
load_dotenv()

# dostępne modele w jednym miejscu - łatwiej je podmienić,
# gdy dostawca wycofa model (console.groq.com/docs/deprecations)
MODELE = ['llama-3.3-70b-versatile', 'llama-3.1-8b-instant']
DOMYSLNY_PROMPT = 'prompts/system_note_assistant.txt'


def get_client():
    """Tworzy klienta Groq na podstawie klucza z pliku .env."""
    api_key = os.getenv('GROQ_API_KEY')
    if not api_key:
        raise RuntimeError(
            'Nie znaleziono klucza. Sprawdź, czy plik .env
znajduje '
            'się w katalogu projektu i czy zawiera GROQ_API_KEY.')
    return Groq(api_key=api_key)


def zapytaj(client, model, messages, temperature=0.7,
            max_tokens=1000):
    """Jedno wywołanie modelu czatu; zwraca tekst odpowiedzi."""
```

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

```
    response = client.chat.completions.create(
        model=model,
        messages=messages,
        temperature=temperature,
        max_tokens=max_tokens,
    )
    return response.choices[0].message.content


def load_system_prompt(path=DOMYSLNY_PROMPT):
    """Wczytuje treść promptu systemowego z pliku tekstowego."""
    try:
        with open(path, 'r', encoding='utf-8') as plik:
            return plik.read().strip()
    except FileNotFoundError:
        return ('Jesteś pomocnym asystentem. Odpowiadaj '
                'zwięźle, w języku polskim.')
```

2. W katalogu `prompts` utwórz dwa pliki z promptami systemowymi. Plik `system_note_assistant.txt`:

```
Jesteś asystentem do notatek i nauki dla studentów informatyki.
Pomagasz porządkować notatki, streszczać materiały i wyjaśniać
pojęcia. Odpowiadaj zwięźle i rzeczowo, w języku polskim.
Gdy wyjaśniasz pojęcie, podawaj krótki przykład.
```

oraz plik system_translator.txt:

```
Jesteś tłumaczem tekstów technicznych z języka angielskiego
na polski. Tłumaczysz precyzyjnie, zachowując terminologię
informatyczną (np. thread = wątek; commit pozostaje bez
tłumaczenia). Zwracaj wyłącznie tłumaczenie, bez komentarzy.
```

3. W katalogu `src` utwórz plik `test_client.py` – prosty klient terminalowy do sprawdzenia warstwy modeli, zanim powstanie UI:

```
import sys

from models import MODELE, get_client, load_system_prompt, zapytaj
```


**Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00**

```
# poprawne wyświetlanie polskich znaków w konsoli Windows
sys.stdout.reconfigure(encoding='utf-8')

client = get_client()
SYSTEM_PROMPT = load_system_prompt()

print('Klient testowy. Wpisz "koniec", aby zakończyć.')
while True:
    user_input = input('\nTy: ')
    if user_input.lower() in ['koniec', 'exit', 'quit']:
        print('Do widzenia!')
        break
    odpowiedz = zapytaj(
        client,
        MODELE[0],
        [
            {'role': 'system', 'content': SYSTEM_PROMPT},
            {'role': 'user', 'content': user_input},
        ],
    )
    print('Asystent:', odpowiedz)
```

4. Uruchom klienta z katalogu głównego projektu poleceniem `python src/test_client.py` i zadaj modelowi 2–3 pytania. Uruchomienie z katalogu głównego jest istotne, ścieżka `prompts/...` jest względna wobec bieżącego katalogu.

P: Dołącz zrzut ekranu sesji klienta terminalowego z co najmniej dwoma pytaniami i poprawnym zakończeniem pętli słowem „koniec”.

Zadanie 8.4. Pierwsza aplikacja Streamlit – mechanizm rerun

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

Przed budową właściwego asystenta warto zrozumieć na krótkim przykładzie, jak Streamlit wykonuje skrypt i dlaczego zwykłe zmienne nie zachowują wartości między interakcjami.

Treść zadania:

1. W katalogu `src` utwórz plik `demo_rerun.py` o następującej treści:

```
import streamlit as st

st.title('Pierwsza aplikacja Streamlit')

# licznik A: zwykła zmienna - resetuje się przy każdym przebiegu
licznik_zwykly = 0
if st.button('Dodaj 1 (zwykła zmienna)'):
    licznik_zwykly += 1
st.write('Licznik zwykły:', licznik_zwykly)

# licznik B: st.session_state - wartość przetrwa kolejne przebiegi
if 'licznik' not in st.session_state:
    st.session_state.licznik = 0
if st.button('Dodaj 1 (session_state)'):
    st.session_state.licznik += 1
st.write('Licznik w session_state:', st.session_state.licznik)
```

2. Uruchom aplikację z katalogu głównego projektu poleceniem `streamlit run src/demo_rerun.py`. W przeglądarce otworzy się strona pod adresem `http://localhost:8501`.
3. Kliknij oba przyciski kilkukrotnie i porównaj zachowanie liczników. Licznik oparty na zwykłej zmiennej nigdy nie przekroczy wartości 1 – przy każdym kliknięciu cały skrypt wykonuje się od nowa, a zmienna jest ponownie inicjalizowana zerem. Licznik w `session_state` rośnie poprawnie.
4. Zatrzymaj serwer w terminalu za pomocą skrótu Ctrl+C.

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

P: Dołącz zrzut ekranu aplikacji po kilku kliknięciach obu przycisków i wyjaśnij w 2–3 zdaniach zaobserwowaną różnicę, odwołując się do mechanizmu rerun.

Zadanie 8.5. Interfejs asystenta (app_ui.py)

Budowa interfejsu asystenta. Aplikacja zawiera panel boczny z ustawieniami (wybór modelu, suwak temperatury, tryb asystenta), pole wejściowe z przyciskiem wysłania oraz historię interakcji zachowywaną w `st.session_state.history` i wyświetlaną na stronie. W tej wersji każde pytanie jest wysyłane do modelu osobno. Lista `messages` zawiera wyłącznie prompt systemowy oraz bieżące pytanie. Historia służy do wyświetlania przebiegu pracy z asystentem.

Treść zadania:

1. W katalogu `src` utwórz plik `app_ui.py` o następującej treści:

```
import streamlit as st

from models import MODELE, get_client, load_system_prompt, zapytaj

# --- konfiguracja strony ---
st.set_page_config(page_title='Asystent do notatek i nauki')
st.title('Asystent do notatek i nauki')

# --- panel boczny: ustawienia ---
st.sidebar.header('Ustawienia')
model = st.sidebar.selectbox('Wybierz model:', MODELE)
temperature = st.sidebar.slider('Temperatura:', 0.0, 1.5, 0.7,
0.1)
tryb = st.sidebar.selectbox(
    'Tryb asystenta:',
    ['system_note_assistant.txt', 'system_translator.txt'])
system_prompt = load_system_prompt(f'prompts/{tryb}')

# --- historia interakcji w st.session_state ---
if 'history' not in st.session_state:
    st.session_state.history = []
```

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

**Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00**

```
# --- pole wejściowe i wysłanie pytania ---
user_input = st.text_input('Zadaj pytanie:')

if st.button('Wyślij') and user_input:
    # do modelu trafia TYLKO bieżące pytanie (bez historii)
    messages = [
        {'role': 'system', 'content': system_prompt},
        {'role': 'user', 'content': user_input},
    ]
    client = get_client()
    odpowiedz = zapytaj(client, model, messages,
                        temperature=temperature)
    st.session_state.history.append((user_input, odpowiedz))

# --- wyświetlenie historii interakcji ---
for pytanie, odpowiedz in st.session_state.history:
    st.markdown(f'**Ty:** {pytanie}')
    st.markdown(f'**Asystent:** {odpowiedz}')
    st.divider()
```

2. Zapisz plik (Ctrl+S) i uruchom aplikację z katalogu głównego projektu poleceniem `streamlit run src/app_ui.py`.
3. Przetestuj wszystkie elementy interfejsu: zadaj pytanie, zmień model, przesuń suwak temperatury, przełącz tryb asystenta na tłumacza i poproś o przetłumaczenie zdania technicznego. Zwróć uwagę, że historia interakcji nie znika mimo mechanizmu rerun – jest zachowywana w `st.session_state`.
4. Sprawdź „pamięć" modelu: zadaj pytanie „Wyjaśnij w 2 zdaniach, czym jest rekurencja.", a następnie „Podaj przykład w Pythonie do tego, co właśnie wyjaśniłeś." Zaobserwuj, że model nie wie, o co chodzi, nie otrzymał wcześniejszej informacji.
5. Prześledź przepływ danych w kodzie: skąd pochodzi wartość zmiennej `temperature` w wywołaniu `zapytaj()`? Co dokładnie trafia do listy `messages` przy każdym pytaniu?

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

Uwaga: aplikacja zachowuje pamięć rozmowy (historię) na poziomie interfejsu w `st.session_state` ale model jej nie otrzymuje, więc nie potrafi nawiązać do wcześniejszych pytań.

> **P:** Dołącz zrzuty ekranu: (1) rozmowy w trybie asystenta notatek, (2) rozmowy w trybie tłumacza, (3) reakcji modelu na pytanie nawiązujące do poprzedniej wymiany (punkt 4).

Zadanie 8.6. Eksperyment: temperatura i wybór modelu z poziomu UI

Eksperyment pokazuje, jak suwak temperatury i wybór modelu w panelu bocznym zmieniają odpowiedzi asystenta. Wyniki należy przedstawić w tabeli 8.1.

Treść zadania:

1. **Scenariusz A – temperatura 0.0.** Wybierz model `llama-3.1-8b-instant`, ustaw temperaturę na 0.0 i zadaj to samo pytanie dwukrotnie: „Zaproponuj tytuł notatki o pętlach w Pythonie." Porównaj obie odpowiedzi.
2. **Scenariusz B – temperatura 1.5.** Powtórz scenariusz A przy temperaturze 1.5.
3. **Scenariusz C – większy model.** Ustaw temperaturę 0.7, wybierz model `llama-3.3-70b-versatile` i zadaj pytanie: „Wyjaśnij w 3 zdaniach, czym jest złożoność obliczeniowa." Następnie przełącz na `llama-3.1-8b-instant` i zadaj to samo pytanie. Porównaj jakość i czas odpowiedzi.
4. Uzupełnij tabelę 8.1 na podstawie obserwacji.

Tabela 8.1. Wyniki eksperymentu z temperaturą i wyborem modelu

Scenariusz	Model	Temperatura	Obserwacja (2–3 zdania)
A	llama-3.1-8b-instant	0.0	
B	llama-3.1-8b-instant	1.5	
C	oba modele	0.7	

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

**Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00**

P: Zamieść uzupełnioną tabelę. Sformułuj wnioski: (1) jakie wartości temperatury wybrać do zadań wymagających powtarzalności, a jakie do kreatywnych? (2) kiedy warto używać większego modelu mimo dłuższego czasu odpowiedzi?

Ćwiczenia samodzielne

Ćwiczenie 8.1 (podstawowe). Trzeci tryb asystenta

Dodaj do aplikacji trzeci tryb asystenta np. recenzenta kodu (`system_code_reviewer.txt`) albo generatora pytań kontrolnych do nauki. Utwórz plik promptu w katalogu `prompts`, dodaj go do listy w `selectbox` i przetestuj nowy tryb na 2–3 pytaniach. W sprawozdaniu zamieść treść promptu i zrzut ekranu rozmowy.

Ćwiczenie 8.2 (średnie). Suwak max_tokens w panelu bocznym

Dodaj do panelu bocznego suwak „Limit tokenów odpowiedzi” (np. 100–2000, krok 100) i przekaż jego wartość do funkcji `zapytaj()` jako parametr `max_tokens`. Sprawdź działanie: przy małym limicie (np. 100) poproś o dłuższe wyjaśnienie i zaobserwuj ucięcie odpowiedzi. W sprawozdaniu wyjaśnij, do czego w praktyce służy ograniczanie liczby tokenów wyjściowych (koszt, czas odpowiedzi).

Ćwiczenie 8.3 (trudne). Statystyki tokenów ostatniej odpowiedzi

Rozbuduj funkcję `zapytaj()` tak, aby oprócz treści odpowiedzi zwracała także liczbę tokenów wejściowych i wyjściowych (pole `usage` odpowiedzi API, znane z Laboratorium 2). Wyświetl te wartości w panelu bocznym przez `st.sidebar.metric`. Dodaj również przycisk „Wyczyść historię”, który wyczyści listę `st.session_state.history`.

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

Pytania pomocnicze

1. Czym jest Streamlit i jakim poleceniem uruchamia się aplikację? Pod jakim adresem jest ona domyślnie dostępna?
2. Na czym polega mechanizm rerun w Streamlit? Jakie ma to konsekwencje dla zwykłych zmiennych w Pythonie?
3. Do czego służy `st.session_state` i dlaczego inicjalizuje się go warunkiem `if 'history' not in st.session_state`?
4. Dlaczego w zbudowanej aplikacji model nie potrafi nawiązać do poprzednich pytań, mimo że historia jest widoczna na stronie? Co dokładnie trafia do listy `messages`?
5. Jak parametr `temperature` wpływa na odpowiedzi modelu?
6. Dlaczego dla nowego projektu wygenerowano nowy klucz API zamiast użyć klucza z poprzednich laboratoriów? Jak klucz jest chroniony przed publikacją?
7. Jakie korzyści daje rozdzielenie aplikacji na moduły `models.py` i `app_ui.py` oraz przechowywanie promptów systemowych w osobnych plikach?

Podsumowanie

Po ukończeniu laboratorium student dysponuje działającą aplikacją webową asystenta zbudowaną od podstaw w nowym projekcie. W szczególności:

1. ma skonfigurowany nowy projekt z izolowanym środowiskiem wirtualnym, listą zależności oraz nowym, dedykowanym kluczem API przechowywanym w pliku `.env` i chronionym wpisem w `.gitignore`;
2. rozumie model wykonania Streamlit (rerun) i rolę `st.session_state` w zachowywaniu stanu aplikacji, w tym historii rozmowy;
3. potrafi zbudować interfejs z panelem ustawień: wyborem modelu, suwakiem temperatury i trybami asystenta ładowanymi z plików promptów;
4. stosuje separację warstw (UI / warstwa modeli / prompty w plikach), która ułatwi rozbudowę aplikacji;

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

WSKAZÓWKA: zachowaj cały katalog projektu wraz ze środowiskiem i kluczem API. Aplikacja zbudowana w tym laboratorium zostanie w Laboratorium 9 przekształcona w pełnego chatbota z pamięcią konwersacyjną, a następnie będzie rozwijana w laboratoriach 10 i 11. Warto również dopisać do biblioteki promptów (pliku prompts.md lub programu AI Prompt Lab, prowadzonych od Laboratorium 2) prompty systemowe dla trybów asystenta wraz z oceną ich działania.

UWAGA dotycząca bezpieczeństwa: klucza API nie wolno udostępniać publicznie (np. w repozytorium GitHub ani w sprawozdaniu na zrzutach ekranu). Plik `.env` musi być zawsze ujęty w pliku `.gitignore`. W przypadku ujawnienia klucza należy niezwłocznie unieważnić go w konsoli GroqCloud i wygenerować nowy.

Literatura

[1]. Caelen O., Blete M.-A., Tworzenie aplikacji z wykorzystaniem GPT-4 i ChatGPT. Buduj inteligentne chatboty, generatory treści i fascynujące projekty, Helion, Gliwice 2024.
[2]. Streamlit – dokumentacja oficjalna, https://docs.streamlit.io/ (dostęp: 2026).
[3]. Streamlit, Session State – dokumentacja, https://docs.streamlit.io/develop/concepts/architecture/session-state (dostęp: 2026).
[4]. Streamlit, Input widgets – dokumentacja API, https://docs.streamlit.io/develop/api-reference/widgets (dostęp: 2026).
[5]. Groq, Quickstart – pierwsze żądanie do API, https://console.groq.com/docs/quickstart (dostęp: 2026).
[6]. python-dotenv – dokumentacja pakietu, https://pypi.org/project/python-dotenv/ (dostęp: 2026).
[7]. Groq, Text/Chat Generation – dokumentacja API, https://console.groq.com/docs/text-chat (dostęp: 2026).
[8]. Groq, Model deprecations (harmonogram wyłączania modeli), https://console.groq.com/docs/deprecations (dostęp: 2026).
[9]. Phoenix J., Taylor M., Skuteczna inżynieria promptów. Przyszłościowe rozwiązania dla rzetelnych wyników generatywnej AI, Helion, Gliwice 2025.

Fundusze Europejskie dla Rozwoju Społecznego | Rzeczpospolita Polska | Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

[10]. Tunstall L., von Werra L., Wolf T., Przetwarzanie języka naturalnego z wykorzystaniem transformerów, Helion, Gliwice 2023.

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

POLITECHNIKA LUBELSKA

Materiały do przedmiotu:

GENERATYWNA SZTUCZNA INTELIGENCJA I PROMPT ENGINEERING

Laboratorium nr 9.
Tworzenie chatbota wykorzystującego AI

Autor:

dr inż. Krzysztof Dziedzic

dr Marcin Barszcz

Katedra Informatyki

Fundusze Europejskie dla Rozwoju Społecznego

Rzeczpospolita Polska

Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

POLLUB z nami nowoczesne technologie

POLITECHNIKA LUBELSKA

Fundusze Europejskie dla Rozwoju Społecznego

Rzeczpospolita Polska

Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

POLITECHNIKA LUBELSKA

Spis treści

Wprowadzenie, tematyka zajęć ---------- 173
Cel ćwiczenia/laboratorium ---------- 174
Instrukcja laboratoryjna/ćwiczeniowa ---------- 175
Ćwiczenia samodzielne ---------- 183
Pytania pomocnicze ---------- 184
Podsumowanie ---------- 184
Literatura ---------- 185

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

Wprowadzenie, tematyka zajęć

Niniejsze laboratorium stanowi bezpośrednią kontynuację Laboratorium 8, w którym zbudowano aplikację asystenta w Streamlit: nowy projekt z kluczem API Groq, warstwę modeli (`models.py`) oraz interfejs z wyborem modelu, suwakiem temperatury i trybami asystenta. Aplikacja ta jednak odpowiada na każde pytanie osobno. Model nie zna wcześniejszych wymian i nie potrafi do nich nawiązać. W tym laboratorium ta sama aplikacja zostanie przekształcona w funkcjonalnego chatbota; otrzyma komponenty czatu, pamięć konwersacyjną przekazywaną do modelu, obsługę błędów API oraz strumieniowanie odpowiedzi „na żywo" [1].

Chatbot AI to aplikacja prowadząca z użytkownikiem rozmowę w języku naturalnym, w której odpowiedzi generuje duży model językowy. O jakości chatbota decydują nie tylko możliwości modelu, ale przede wszystkim warstwa aplikacji: prompt systemowy definiujący rolę, zarządzanie historią rozmowy, obsługa sytuacji błędnych oraz wygoda interfejsu [1, 2].

Bezstanowość API i pamięć konwersacyjna. Modele językowe udostępniane przez API są bezstanowe. Każde żądanie jest przetwarzane niezależnie, a model nie pamięta poprzednich zapytań. Wrażenie ciągłej rozmowy uzyskuje się, dołączając do każdego żądania pełną historię wymian w liście `messages` (role: `system`, `user`, `assistant`). Pamięć konwersacyjna jest więc realizowana przez aplikację, nie przez model. Ma to swoją cenę: z każdą wymianą rośnie liczba tokenów wejściowych, a więc koszt i czas odpowiedzi, a cała historia musi mieścić się w oknie kontekstu modelu. Dlatego w chatbotie pamięć będzie przełączalna, a użytkownik otrzyma przycisk do czyszczenia rozmowy [3].

Komponenty czatu w Streamlit. Do budowy interfejsu konwersacyjnego służą dwa dedykowane komponenty: `st.chat_input()` – pole wejściowe przypięte do dołu strony, zwracające tekst po zatwierdzeniu klawiszem Enter, oraz `st.chat_message(rola)` – dymek wiadomości z ikoną roli (`user` lub `assistant`), używany jako menedżer kontekstu (`with`). Historia rozmowy będzie

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

przechowywana w `st.session_state.history` jako lista słowników w formacie zgodnym z API: `{'role': ..., 'content': ...}`. Dzięki temu tę samą listę można bez żadnej konwersji dokleić do listy `messages` wysyłanej do modelu [4, 5].

Strumieniowanie odpowiedzi (streaming). Domyślnie API zwraca odpowiedź dopiero po wygenerowaniu całości, co przy dłuższych tekstach oznacza kilkusekundowe oczekiwanie przed pustym ekranem. Po ustawieniu parametru `stream=True` serwer przesyła odpowiedź w miarę jej powstawania, w postaci fragmentów (ang. chunk) zawierających kolejne porcje tekstu w polu `delta.content`. W Streamlit fragmenty te wyświetla funkcja `st.write_stream()`, która przyjmuje generator, pokazuje tekst „piszący się" na żywo i zwraca sklejoną całość, gotową do zapisania w historii. Strumieniowanie nie skraca całkowitego czasu generacji, ale radykalnie skraca czas do pojawienia się pierwszego słowa, co jest standardem we współczesnych chatbotach [6, 7].

Niezawodność chatbota. Aplikacja udostępniana użytkownikom nie może kończyć się nieczytelnym wyjątkiem, gdy zabraknie klucza API albo zostanie przekroczony limit zapytań (błąd 429). Wywołania modelu należy objąć obsługą wyjątków: brak klucza (`RuntimeError` z własnej warstwy modeli) oraz błędy usługi (`APIError` z pakietu `groq`) będą przechwytywane i prezentowane użytkownikowi jako czytelny komunikat `st.error()`, a wykonanie skryptu zatrzyma funkcja `st.stop()` [8].

Cel ćwiczenia/laboratorium

Głównym celem laboratorium jest utworzenie funkcjonalnego chatbota AI z interfejsem konwersacyjnym, pamięcią rozmowy przekazywaną do modelu, obsługą błędów i strumieniowaniem odpowiedzi. Po ukończeniu zajęć student potrafi:

- wyjaśnić, czym jest chatbot AI i w jaki sposób aplikacja realizuje pamięć konwersacyjną bezstanowego modelu;

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

**Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00**

- zbudować interfejs czatu z komponentów `st.chat_message` i `st.chat_input` oraz przechowywać historię w formacie zgodnym z API;
- zaimplementować pamięć rozmowy dołączaną do żądań wraz z przełącznikiem włączania/wyłączania i przyciskiem czyszczenia czatu;
- obsłużyć błędy API (brak klucza, limit zapytań) czytelnym komunikatem zamiast awarii aplikacji;
- zaimplementować strumieniowanie odpowiedzi (`stream=True`, `st.write_stream`) i wyjaśnić, co poprawia ono w działaniu chatbota;
- zbadać eksperymentalnie wpływ pamięci rozmowy i temperatury na spójność dialogu oraz ocenić koszt pamięci w tokenach.

Laboratorium wymaga ukończonego projektu z Laboratorium 8 (katalog `asystentAI` z działającą warstwą modeli i kluczem API). Zbudowany chatbot będzie rozwijany w laboratoriach 10 i 11.

Instrukcja laboratoryjna/ćwiczeniowa

Zadanie 9.1. Przebudowa interfejsu na komponenty czatu

Interfejs z Laboratorium 8 (pole `st.text_input`, przycisk, historia jako lista krotek) zostanie przebudowany na komponenty czatu. Historia zmienia format na listę słowników `{'role': ..., 'content': ...}` identyczny z formatem listy `messages` wysyłanej do API. Model wciąż otrzymuje wyłącznie bieżące pytanie.

Treść zadania:

1. Otwórz projekt `asystent_streamlit` z Laboratorium 8 w VS Code i aktywuj środowisko wirtualne projektu (`.venv`).
2. Zastąp zawartość pliku `src/app_ui.py` następującym kodem (panel boczny pozostaje bez zmian, zmienia się obsługa wejścia i wyświetlanie rozmowy):

```
import streamlit as st

from models import MODELE, get_client, load_system_prompt, zapytaj
```

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

```
# --- konfiguracja strony ---
st.set_page_config(page_title='Chatbot - asystent do notatek')
st.title('Chatbot - asystent do notatek i nauki')

# --- panel boczny: ustawienia ---
st.sidebar.header('Ustawienia')
model = st.sidebar.selectbox('Wybierz model:', MODELE)
temperature = st.sidebar.slider('Temperatura:', 0.0, 1.5, 0.7,
0.1)
tryb = st.sidebar.selectbox(
    'Tryb asystenta:',
    ['system_note_assistant.txt', 'system_translator.txt'])
system_prompt = load_system_prompt(f'prompts/{tryb}')

# --- historia rozmowy w formacie zgodnym z API ---
if 'history' not in st.session_state:
    st.session_state.history = []

# --- wyświetlenie dotychczasowej rozmowy ---
for wiadomosc in st.session_state.history:
    with st.chat_message(wiadomosc['role']):
        st.markdown(wiadomosc['content'])

# --- obsługa nowej wiadomości ---
user_input = st.chat_input('Napisz wiadomość...')

if user_input:
    with st.chat_message('user'):
        st.markdown(user_input)

    # UWAGA: do modelu trafia tylko bieżące pytanie (bez historii)
    messages = [
        {'role': 'system', 'content': system_prompt},
        {'role': 'user', 'content': user_input},
    ]
    client = get_client()
    odpowiedz = zapytaj(client, model, messages,
```

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

```
                        temperature=temperature)

    with st.chat_message('assistant'):
        st.markdown(odpowiedz)

    st.session_state.history.append(
        {'role': 'user', 'content': user_input})
    st.session_state.history.append(
        {'role': 'assistant', 'content': odpowiedz})
```

3. Uruchom aplikację poleceniem `streamlit run src/app_ui.py` i porównaj wygląd z wersją z Laboratorium 8: wiadomości pojawiają się w dymkach z ikonami ról, a pole wejściowe jest przypięte do dołu strony i zatwierdzane klawiszem Enter.
4. **Test braku pamięci.** Napisz: „Wyjaśnij w 2 zdaniach, czym jest rekurencja.", a następnie: „Podaj przykład w Pythonie do tego, co właśnie wyjaśniłeś." Model nie potrafi nawiązać do pierwszej wymiany – mimo że jest ona widoczna na ekranie, nie została wysłana w żądaniu.
5. Prześledź w kodzie, dlaczego tak się dzieje: co dokładnie zawiera lista `messages` w chwili drugiego pytania?

P: Dołącz zrzut ekranu rozmowy z punktu 4, który pokazuje, że model nie nawiązał do wcześniejszej wymiany, wraz z 1–2 zdaniami wyjaśnienia.

Zadanie 9.2. Pamięć konwersacyjna chatbota

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

Chatbot otrzyma pamięć rozmowy. Przed wysłaniem żądania do listy `messages` doklejana będzie cała dotychczasowa historia z `st.session_state.history`. Ponieważ historia ma format zgodny z API, wystarczy jedno wywołanie metody `messages.extend()`. Pamięć będzie przełączalna, a panel boczny zyska przycisk do czyszczenia czatu.

Treść zadania:

1. W sekcji ustawień panelu bocznego (pod wyborem trybu asystenta) dodaj przełącznik pamięci:

```
use_memory = st.sidebar.checkbox('Pamiętaj kontekst rozmowy',
                                 value=True)
```

2. Pod inicjalizacją historii dodaj przycisk czyszczenia rozmowy – funkcja `st.rerun()` wymusza natychmiastowy ponowny przebieg skryptu, dzięki czemu wyczyszczona rozmowa znika z ekranu od razu:

```
if st.sidebar.button('Wyczyść czat'):
    st.session_state.history = []
    st.rerun()
```

3. Zmień budowanie listy `messages` tak, aby przy włączonej pamięci zawierała całą historię rozmowy:

```
messages = [{'role': 'system', 'content': system_prompt}]
if use_memory:
    messages.extend(st.session_state.history)
messages.append({'role': 'user', 'content': user_input})
```

4. Zapisz plik i powtórz test z Zadania 9.1 (rekurencja + pytanie nawiązujące). Tym razem model poprawnie nawiązuje do wcześniejszego wyjaśnienia – chatbot prowadzi spójną rozmowę.
5. Przetestuj przycisk „Wyczyść czat” oraz zachowanie przy wyłączonym przełączniku pamięci.

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

Uwaga: historia jest zapisywana do `session_state` także przy wyłączonej pamięci – po to, aby po ponownym włączeniu przełącznika kontekst rozmowy był kompletny. Przełącznik decyduje wyłącznie o tym, czy historia ma być dołączana do żądania wysyłanego do modelu.

> **P:** Dołącz dwa zrzuty ekranu tej samej pary pytań: z pamięcią włączoną i wyłączoną. Wyjaśnij w 2–3 zdaniach, jak jedna linia kodu `messages.extend(...)` zmienia zachowanie chatbota.

Zadanie 9.3. Obsługa błędów API

Funkcjonalny chatbot musi przewidywać sytuacje błędne: brak pliku `.env` z kluczem, nieprawidłowy klucz czy przekroczenie limitu zapytań darmowej warstwy (błąd 429). Wywołanie modelu zostanie objęte obsługą wyjątków z czytelnym komunikatem dla użytkownika.

Treść zadania:

1. Na początku pliku `app_ui.py` dodaj import klasy wyjątku:

```
from groq import APIError
```

2. Fragment wywołujący model (utworzenie klienta i wywołanie `zapytaj()`) obejmij obsługą wyjątków:

```
try:
    client = get_client()
    odpowiedz = zapytaj(client, model, messages,
                        temperature=temperature)
except RuntimeError as e:      # brak klucza w .env
    st.error(str(e))
    st.stop()
except APIError as e:          # np. błąd 429 - limit zapytań
    st.error(f'Błąd Groq API: {e}')
    st.stop()
```

3. **Test obsługi błędu.** Zatrzymaj aplikację, zmień tymczasowo nazwę pliku `.env` (np. na `.env.bak`), uruchom aplikację ponownie i wyślij wiadomość.

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

Zamiast awarii aplikacji powinien pojawić się czerwony komunikat o braku klucza. Przywróć nazwę pliku `.env`.

P: Dołącz zrzut ekranu komunikatu błędu z punktu 3. Wyjaśnij, jaką rolę pełni `st.stop()` i co zobaczyłby użytkownik bez obsługi wyjątków.

Zadanie 9.4. Strumieniowanie odpowiedzi

Chatbot zyska efekt „pisania na żywo": warstwa modeli otrzyma funkcję `zapytaj_stream()` zwracającą generator fragmentów odpowiedzi, a interfejs wyświetli je funkcją `st.write_stream()`, która dodatkowo zwraca sklejoną całość – gotową do zapisania w historii rozmowy.

Treść zadania:

1. Na końcu pliku `src/models.py` dodaj funkcję strumieniującą:

```
def zapytaj_stream(client, model, messages, temperature=0.7,
                   max_tokens=1000):
    """Wywołanie modelu ze strumieniowaniem; zwraca generator
    kolejnych fragmentów (chunków) odpowiedzi."""
    stream = client.chat.completions.create(
        model=model,
        messages=messages,
        temperature=temperature,
        max_tokens=max_tokens,
        stream=True,
    )
    for chunk in stream:
        fragment = chunk.choices[0].delta.content
        if fragment:
            yield fragment
```

2. W pliku `app_ui.py` zmień import warstwy modeli oraz blok wywołania modelu. Po zmianach kompletny plik `app_ui.py` (końcowa wersja chatbota) wygląda następująco:

```
import streamlit as st
```

```
from groq import APIError
from models import (MODELE, get_client, load_system_prompt,
                    zapytaj_stream)

# --- konfiguracja strony ---
st.set_page_config(page_title='Chatbot - asystent do notatek')
st.title('Chatbot - asystent do notatek i nauki')

# --- panel boczny: ustawienia ---
st.sidebar.header('Ustawienia')
model = st.sidebar.selectbox('Wybierz model:', MODELE)
temperature = st.sidebar.slider('Temperatura:', 0.0, 1.5, 0.7,
0.1)
tryb = st.sidebar.selectbox(
    'Tryb asystenta:',
    ['system_note_assistant.txt', 'system_translator.txt'])
system_prompt = load_system_prompt(f'prompts/{tryb}')
use_memory = st.sidebar.checkbox('Pamiętaj kontekst rozmowy',
                                 value=True)

# --- pamięć rozmowy w st.session_state ---
if 'history' not in st.session_state:
    st.session_state.history = []

if st.sidebar.button('Wyczyść czat'):
    st.session_state.history = []
    st.rerun()

# --- wyświetlenie dotychczasowej rozmowy ---
for wiadomosc in st.session_state.history:
    with st.chat_message(wiadomosc['role']):
        st.markdown(wiadomosc['content'])

# --- obsługa nowej wiadomości ---
user_input = st.chat_input('Napisz wiadomość...')
```

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

```
if user_input:
    with st.chat_message('user'):
        st.markdown(user_input)

    messages = [{'role': 'system', 'content': system_prompt}]
    if use_memory:
        messages.extend(st.session_state.history)
    messages.append({'role': 'user', 'content': user_input})

    try:
        client = get_client()
        with st.chat_message('assistant'):
            odpowiedz = st.write_stream(
                zapytaj_stream(client, model, messages,
                               temperature=temperature))
    except RuntimeError as e:      # brak klucza w .env
        st.error(str(e))
        st.stop()
    except APIError as e:          # np. błąd 429 - limit zapytań
        st.error(f'Błąd Groq API: {e}')
        st.stop()

    st.session_state.history.append(
        {'role': 'user', 'content': user_input})
    st.session_state.history.append(
        {'role': 'assistant', 'content': odpowiedz})
```

3. Uruchom aplikację i poproś o dłuższą odpowiedź, np.: „Opisz w 10 punktach dobre praktyki pisania notatek z wykładu.” Zaobserwuj, że tekst pojawia się stopniowo, w miarę generowania.
4. Upewnij się, że po zakończeniu strumieniowania odpowiedź trafia do historii i jest poprawnie wyświetlana po kolejnym uruchomieniu skryptu (np. po wysłaniu następnej wiadomości).

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

P: Dołącz końcowe wersje plików `app_ui.py` i `models.py` oraz zrzut ekranu rozmowy. Wyjaśnij: (1) co zwraca funkcja `zapytaj_stream()` i czym różni się od `zapytaj()`? (2) skąd `st.write_stream` „wie", jaki tekst zapisać do zmiennej `odpowiedz`?

Zadanie 9.5. Eksperyment: pamięć rozmowy i temperatura

Eksperyment pokazuje, jak przełącznik pamięci oraz suwak temperatury zmieniają zachowanie chatbota. Wyniki należy zebrać w tabeli 9.1.

Treść zadania:

1. **Scenariusz A – pamięć włączona.** Ustaw temperaturę 0.7, włącz pamięć. Napisz: „Wyjaśnij w 2 zdaniach, czym jest lista w Pythonie.", a następnie: „A czym różni się od niej krotka? Odwołaj się do swojego poprzedniego wyjaśnienia." Zapisz, czy druga odpowiedź poprawnie nawiązała do pierwszej.
2. **Scenariusz B – pamięć wyłączona.** Wyczyść czat, wyłącz pamięć i powtórz obie wiadomości. Zaobserwuj, jak model radzi sobie z drugim pytaniem bez dostępu do wcześniejszej wymiany.
3. **Scenariusz C – temperatura 0.0.** Wyczyść czat, włącz pamięć, ustaw temperaturę 0.0 i napisz dwukrotnie tę samą wiadomość: „Zaproponuj tytuł notatki o pętlach w Pythonie." Porównaj obie odpowiedzi.
4. **Scenariusz D – temperatura 1.5.** Powtórz scenariusz C przy temperaturze 1.5.
5. Uzupełnij tabelę 9.1 na podstawie obserwacji.

Tabela 9.1. Wyniki eksperymentu z pamięcią rozmowy i temperaturą

Scenariusz	Pamięć	Temperatura	Obserwacja (2–3 zdania)
A	włączona	0.7	
B	wyłączona	0.7	
C	włączona	0.0	
D	włączona	1.5	

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

P: Zamieść uzupełnioną tabelę 9.1 oraz zrzuty ekranu scenariuszy A i B. Sformułuj wnioski: (1) w jaki sposób aplikacja realizuje „pamięć” bezstanowego modelu i jaki jest tego koszt w tokenach? (2) jakie wartości temperatury wybrać dla chatbota-asystenta do nauki i dlaczego?

Ćwiczenia samodzielne

Ćwiczenie 9.1 (podstawowe). Powitanie i awatary chatbota

Spraw, aby po uruchomieniu (i po wyczyszczeniu czatu) chatbot rozpoczynał rozmowę wiadomością powitalną, np. „Cześć! Jestem asystentem do notatek i nauki. W czym mogę pomóc?” – dodaj ją do historii podczas inicjalizacji `session_state`. Następnie nadaj wiadomościom własne awatary, przekazując do `st.chat_message` parametr `avatar` (np. emoji „🧑” i „📄”).

Ćwiczenie 9.2 (średnie). Statystyki tokenów sesji

Rozbuduj warstwę modeli tak, aby zliczać tokeny całej sesji: w wywołaniu strumieniującym przekaż dodatkowy parametr `stream_options={'include_usage': True}`. Ostatni fragment strumienia zawiera wtedy pole `usage` ze statystykami (`prompt_tokens`, `completion_tokens`). Sumuj te wartości w `st.session_state` i wyświetlaj w panelu bocznym przez `st.sidebar.metric`. Zaobserwuj i opisz w sprawozdaniu, jak włączona pamięć zwiększa liczbę tokenów wejściowych z każdą kolejną wymianą.

Ćwiczenie 9.3 (trudne). Ograniczenie pamięci i eksport rozmowy

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

Dodaj do panelu bocznego suwak „Długość pamięci” (np. 2–20 wiadomości) i dołączaj do żądania tylko N ostatnich wiadomości historii (wycinek listy `st.session_state.history[-n:]`). Następnie dodaj przycisk `st.download_button`, który eksportuje całą rozmowę do pliku Markdown (z kolejnymi wiadomościami oznaczeniami roli). Dlaczego ograniczanie pamięci jest istotne podczas długich rozmów (koszt, okno kontekstu) i jaki jest kompromis między długością pamięci a spójnością rozmowy?

Pytania pomocnicze

1. Czym jest chatbot AI i jakie elementy – poza samym modelem językowym – składają się na jego działanie?
2. Dlaczego modele językowe udostępniane przez API nazywamy bezstanowymi i jak aplikacja realizuje pamięć konwersacyjną?
3. Dlaczego historia rozmowy jest przechowywana w formacie `{'role': ..., 'content': ...}`, identycznym z formatem listy `messages`? Jaką korzyść daje to w kodzie?
4. Czym różni się `st.chat_input` od `st.text_input`, a `st.chat_message` od zwykłego `st.markdown`?
5. Jaką rolę pełni wywołanie `st.rerun()` po wyczyszczeniu historii rozmowy?
6. Jaki jest koszt utrzymywania pamięci rozmowy (tokeny, okno kontekstu) i jakie mechanizmy chatbota pozwalają go kontrolować?
7. Na czym polega strumieniowanie odpowiedzi?

Podsumowanie

Po ukończeniu laboratorium student dysponuje funkcjonalnym chatbotem AI zbudowanym na fundamencie z Laboratorium 8. W szczególności:

1. potrafi zbudować interfejs konwersacyjny z komponentów `st.chat_message` i `st.chat_input`, z historią w formacie zgodnym z API;

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

2. implementuje pamięć konwersacyjną jako historię wiadomości dołączaną do żądań, z przełącznikiem i czyszczeniem rozmowy, i rozumie jej koszt w tokenach;
3. zabezpiecza aplikację przed typowymi błędami API (brak klucza, limit zapytań) czytelnymi komunikatami dla użytkownika;
4. stosuje strumieniowanie odpowiedzi (`stream=True`, `st.write_stream`) i potrafi wyjaśnić jego wpływ na odbiór chatbota;
5. świadomie dobiera ustawienia chatbota (pamięć, temperatura, model) do charakteru zadania na podstawie własnych eksperymentów.

WSKAZÓWKA: zachowaj projekt – chatbot zbudowany w tym laboratorium będzie rozwijany w laboratoriach 10 i 11 o kolejne funkcjonalności. Warto dopisać do biblioteki promptów (pliku prompts.md lub programu AI Prompt Lab) prompty systemowe dla trybów chatbota oraz zanotować ustawienia (model, temperatura, pamięć), które najlepiej sprawdziły się w eksperymentach.

Literatura

[1]. Caelen O., Blete M.-A., Tworzenie aplikacji z wykorzystaniem GPT-4 i ChatGPT. Buduj inteligentne chatboty, generatory treści i fascynujące projekty, Helion, Gliwice 2024.
[2]. Phoenix J., Taylor M., Skuteczna inżynieria promptów. Przyszłościowe rozwiązania dla rzetelnych wyników generatywnej AI, Helion, Gliwice 2025.
[3]. Streamlit, Session State – dokumentacja, https://docs.streamlit.io/develop/concepts/architecture/session-state (dostęp: 2026).
[4]. Streamlit, Build a basic LLM chat app, https://docs.streamlit.io/develop/tutorials/chat-and-llm-apps/build-conversational-apps (dostęp: 2026).
[5]. Streamlit, Chat elements – dokumentacja API, https://docs.streamlit.io/develop/api-reference/chat (dostęp: 2026).
[6]. Streamlit, st.write_stream – dokumentacja, https://docs.streamlit.io/develop/api-reference/write-magic/st.write_stream (dostęp: 2026).
[7]. Groq, Streaming – dokumentacja API, https://console.groq.com/docs/text-chat (dostęp: 2026).

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

[8]. Groq, Rate limits (limity i przydziały zapytań), https://console.groq.com/docs/rate-limits (dostęp: 2026).
[9]. Groq, Model deprecations (harmonogram wyłączania modeli), https://console.groq.com/docs/deprecations (dostęp: 2026).
[10]. Mizrahi G., Inżynieria promptów bez tajemnic. Sztuka kreatywnego generowania tekstów, Helion, Gliwice 2025.

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

POLITECHNIKA LUBELSKA

Materiały do przedmiotu:

GENERATYWNA SZTUCZNA INTELIGENCJA I PROMPT ENGINEERING

Laboratorium nr 10.
Tworzenie asystenta AI do wspomagania nauki

Autor:

dr inż. Krzysztof Dziedzic

dr Marcin Barszcz

Katedra Informatyki

Fundusze Europejskie dla Rozwoju Społecznego

Rzeczpospolita Polska

Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

POLLUB z nami nowoczesne technologie

POLITECHNIKA LUBELSKA

Fundusze Europejskie dla Rozwoju Społecznego

Rzeczpospolita Polska

Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

Spis treści

Wprowadzenie, tematyka zajęć ---------- 189
Cel ćwiczenia/laboratorium---------- 190
Instrukcja laboratoryjna/ćwiczeniowa----------191
Ćwiczenia samodzielne ---------- 201
Pytania pomocnicze ---------- 202
Podsumowanie---------- 202
Literatura ---------- 203

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

Wprowadzenie, tematyka zajęć

Niniejsze laboratorium stanowi kontynuację Laboratoriów 8 i 9, w których zbudowano chatbota w Streamlit: interfejs konwersacyjny, pamięć rozmowy, obsługę błędów i strumieniowanie odpowiedzi. Chatbot ma jednak istotne ograniczenie jako narzędzie do nauki. Odpowiada wyłącznie na podstawie wiedzy zamkniętej w wagach modelu. Nie zna materiałów konkretnego przedmiotu, wykładów prowadzącego, instrukcji laboratoryjnych ani własnych notatek studenta, więc nie pomoże w powtórce „z tego, co było na zajęciach", a dopytany o szczegóły potrafi zmyślać (halucynować). W tym laboratorium chatbot zostanie przekształcony w asystenta wspomagającego naukę, który odpowiada na podstawie materiałów wgranych przez studenta, dokumentów PDF oraz zdjęć odręcznych notatek (JPG/PNG), dzięki technice RAG [1].

RAG (Retrieval-Augmented Generation), czyli generowanie wspomagane wyszukiwaniem, to technika łącząca model językowy z zewnętrzną bazą wiedzy [2]. Odpowiedź powstaje w trzech krokach: (1) wyszukanie w bazie fragmentów najbardziej pasujących do pytania (ang. retrieval), (2) doklejenie znalezionych fragmentów do promptu (augmentacja), (3) wygenerowanie odpowiedzi opartej na dostarczonym kontekście (generacja). Model nie jest w żaden sposób dotrenowywany. Całą „nową wiedzę" otrzymuje w prompcie. RAG ogranicza halucynacje, pozwala odpowiadać na pytania o prywatne lub aktualne dane oraz umożliwia wskazanie źródła odpowiedzi [3].

Dlaczego nie wkleić całych materiałów do promptu? Okno kontekstu modelu jest ograniczone, a każdy token wejściowy kosztuje czas (a w wersji płatnej – pieniądze). Skrypt z kilku wykładów to dziesiątki tysięcy tokenów wysyłanych przy każdym pytaniu, z których większość nie ma z tym tematem związku. Dlatego do promptu trafia tylko kilka najlepiej dopasowanych fragmentów, co wymaga podziału materiałów na fragmenty i mechanizmu wyszukiwania.

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

Podział na fragmenty (chunking). Wczytany tekst jest dzielony na fragmenty (ang. chunk) o stałym rozmiarze, np. 800 znaków. Kolejne fragmenty nakładają się na siebie (zakładka, ang. overlap, np. 200 znaków), aby zdanie przecięte granicą fragmentu wystąpiło w całości w co najmniej jednym z nich. Rozmiar fragmentu to kompromis. Małe fragmenty są precyzyjne, ale gubią kontekst; duże niosą kontekst, ale rozmywają wyszukiwanie i zużywają tokeny [3, 4].

Wyszukiwanie fragmentów. W laboratorium zastosowana zostanie klasyczna, lekka metoda, reprezentacja TF-IDF i podobieństwo kosinusowe (biblioteka scikit-learn). TF-IDF zamienia każdy fragment tekstu w wektor wag słów. Słowo jest tym ważniejsze, im częściej występuje w danym fragmencie i im rzadsze jest w całej bazie. Pytanie użytkownika zamieniane jest na taki sam wektor, a miarą dopasowania jest kosinus kąta między tymi wektorami (1 – identyczna tematyka, 0 – brak wspólnych słów). W systemach produkcyjnych zamiast TF-IDF stosuje się embeddingi czyli gęste wektory znaczeniowe z osobnego modelu, które znajdują fragmenty podobne znaczeniowo, a nie tylko zawierające te same słowa [5, 6].

Modele wizyjne (multimodalne). Notatki studenta często mają postać zdjęć zeszytu lub tablicy. Modele wizyjne przyjmują w wiadomości, obok tekstu, również obraz, przekazywany jako zakodowany w base64 adres danych (`...`). W laboratorium model wizyjny z usługi GroqCloud posłuży do transkrypcji zdjęcia notatek na tekst, który następnie trafi do bazy wiedzy tak samo jak tekst z PDF. Lista dostępnych modeli wizyjnych zmienia się. Aktualne nazwy należy sprawdzać w dokumentacji `console.groq.com/docs/models` [7, 8].

Prywatność materiałów. Treść wgranych dokumentów i notatek jest wysyłana do zewnętrznego API (fragmenty w promptach, obrazy w całości). W przypadku materiałów poufnych lub danych osobowych należy rozważyć model lokalny (Ollama).

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

Cel ćwiczenia/laboratorium

Głównym celem laboratorium jest rozbudowanie chatbota z Laboratorium 9 do asystenta wspomagającego naukę, który odpowiada na podstawie materiałów studenta (PDF, JPG/PNG) z wykorzystaniem techniki RAG. Po ukończeniu zajęć student potrafi:

- wyjaśnić, czym jest RAG, z jakich kroków się składa i jakie problemy modelu językowego rozwiązuje (halucynacje, brak wiedzy prywatnej, okno kontekstu);
- wczytać tekst z dokumentu PDF (biblioteka `pypdf`) i podzielić go na nakładające się fragmenty o zadanym rozmiarze;
- zbudować prostą bazę wiedzy z wyszukiwaniem fragmentów metodą TF-IDF i podobieństwa kosinusowego (scikit-learn);
- wykorzystać model wizyjny do transkrypcji zdjęcia notatek (obraz przekazany w wiadomości jako base64);
- zintegrować bazę wiedzy z chatbotem: wgrywanie plików (`st.file_uploader`), doklejanie znalezionych fragmentów do promptu i wskazywanie źródła odpowiedzi;
- ocenić eksperymentalnie różnicę między odpowiedziami z RAG i bez RAG oraz rozpoznać sytuacje, w których model powinien przyznać brak informacji.

Laboratorium wymaga ukończonego projektu z Laboratorium 9 (katalog `asystent_streamlit` z działającym chatbotem i kluczem API).

Instrukcja laboratoryjna/ćwiczeniowa

Zadanie 10.1. Rozbudowa środowiska i struktury projektu

Projekt zostanie rozszerzony o moduł `rag.py` (wczytywanie i wyszukiwanie materiałów) oraz katalog `materialy/` na pliki do testów terminalowych. Docelowa struktura projektu:

```
asystent_streamlit/
```

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

```
|-- src/
|   |-- app_ui.py          # chatbot (rozbudowa w Zadaniu 10.5)
|   |-- models.py          # warstwa modeli (+ model wizyjny,
Zadanie 10.4)
|   |-- rag.py             # baza wiedzy RAG (Zadania 10.2-10.3)
|   |-- test_rag.py        # testy terminalowe RAG
|   `-- ...                # pliki z Laboratoriow 8-9
|-- prompts/
|-- materialy/             # przykladowe materialy do nauki
|   |-- wyklad.pdf
|   |-- labolatorium.pdf
|   `-- notatki.jpg
|-- .env
`-- requirements.txt
```

Treść zadania:

1. Otwórz projekt `asystentAI` z Laboratorium 9 w VS Code i aktywuj środowisko wirtualne projektu (`.venv`).
2. Doinstaluj biblioteki do obsługi PDF i wyszukiwania oraz zaktualizuj listę zależności:

```
pip install -U pypdf scikit-learn
pip freeze > requirements.txt
```

3. Utwórz katalog `materialy` i umieść w nim: (a) dokumenty PDF z treścią przedmiotową, np. instrukcję laboratoryjną oraz prezentację z wykładu (zapisz je pod nazwami `labolatorium.pdf` oraz `wyklad.pdf`), oraz (b) zdjęcie własnych odręcznych notatek w formacie JPG lub PNG (zapisz jako `notatki.jpg`). Jeśli nie masz notatek pod ręką, zapisz na kartce kilka punktów o dowolnym pojęciu z kursu i sfotografuj ją.

Uwaga: nie kopiuj katalogu projektu wraz z folderem .venv. Środowisko wirtualne nie jest przenośne, jego skrypty zawierają bezwzględne ścieżki do pierwotnej lokalizacji, więc w skopiowanym projekcie polecenia pip i python mogą korzystać z różnych środowisk (typowy objaw: „ModuleNotFoundError” mimo udanej instalacji

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

pakietu). Jeśli przenosisz projekt, skopiuj wyłącznie pliki źródłowe, a środowisko utwórz od nowa poleceniem python -m venv .venv i zainstaluj zależności z pliku requirements.txt.

Zadanie 10.2. Wczytywanie PDF i podział na fragmenty

W pierwszym kroku powstanie pierwsza część modułu `rag.py`, funkcja wczytująca tekst z dokumentu PDF oraz funkcja dzieląca tekst na nakładające się fragmenty. Każdy fragment przechowuje też nazwę pliku źródłowego. Dzięki temu asystent będzie mógł wskazać, skąd pochodzi odpowiedź.

Treść zadania:

1. W katalogu `src` utwórz plik `rag.py` o następującej treści:

```
from pypdf import PdfReader

ROZMIAR_FRAGMENTU = 800   # znaki
ZAKLADKA = 200            # nakladanie sie sasiednich fragmentow

def wczytaj_pdf(plik):
    """Zwraca pelny tekst dokumentu PDF.
    Przyjmuje sciezke albo obiekt pliku (np. z file_uploader)."""
    reader = PdfReader(plik)
    strony = [strona.extract_text() or '' for strona in
reader.pages]
    return '\n'.join(strony)

def podziel_na_fragmenty(tekst, zrodlo):
    """Dzieli tekst na nakladajace sie fragmenty (chunki)."""
    fragmenty = []
    krok = ROZMIAR_FRAGMENTU - ZAKLADKA
    for start in range(0, len(tekst), krok):
        czesc = tekst[start:start + ROZMIAR_FRAGMENTU].strip()
        if czesc:
            fragmenty.append({'zrodlo': zrodlo, 'tekst': czesc})
    return fragmenty
```

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

2. W katalogu `src` utwórz plik `test_rag.py` i sprawdź wczytywanie na przygotowanym dokumencie:

```
import sys

from rag import podziel_na_fragmenty, wczytaj_pdf

# poprawne wyswietlanie polskich znakow w konsoli Windows
sys.stdout.reconfigure(encoding='utf-8')

fragmenty = []
for nazwa in ['wyklad.pdf', 'labolatorium.pdf']:
    tekst = wczytaj_pdf(f'materialy/{nazwa}')
    fragmenty += podziel_na_fragmenty(tekst, nazwa)
print(f'Fragmentów łącznie: {len(fragmenty)}')
print('--- fragment 1 ---')
print(fragmenty[0]['tekst'])
```

3. Uruchom test z katalogu głównego projektu poleceniem `python src/test_rag.py`. Sprawdź, czy liczba fragmentów jest sensowna (długość tekstu podzielona przez ok. 600 znaków kroku) i czy pierwszy fragment zawiera czytelny tekst dokumentu.

P: Dołącz zrzut ekranu wyniku działania `test_rag.py` (liczba znaków, liczba fragmentów, pierwszy fragment) oraz odpowiedz: po co sąsiednie fragmenty nakładają się na siebie?

Zadanie 10.3. Baza wiedzy i wyszukiwanie fragmentów

Druga część modułu `rag.py`: klasa `BazaWiedzy` przechowująca fragmenty i wyszukująca k najlepiej dopasowanych do pytania. Wyszukiwanie oparte jest na reprezentacji TF-IDF i podobieństwie kosinusowym z biblioteki scikit-learn.

Treść zadania:

1. Na początku pliku `rag.py` dodaj importy scikit-learn:

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
```

2. Na końcu pliku `rag.py` dodaj klasę bazy wiedzy:

```
class BazaWiedzy:
    """Prosta baza wiedzy: fragmenty + wyszukiwanie TF-IDF."""

    def __init__(self):
        self.fragmenty = []
        self._vectorizer = None
        self._macierz = None

    def dodaj(self, nowe_fragmenty):
        """Dodaje fragmenty i przelicza reprezentacje TF-IDF."""
        self.fragmenty.extend(nowe_fragmenty)
        teksty = [f['tekst'] for f in self.fragmenty]
        self._vectorizer = TfidfVectorizer()
        self._macierz = self._vectorizer.fit_transform(teksty)

    def szukaj(self, pytanie, k=3):
        """Zwraca k fragmentow najlepiej pasujacych do pytania."""
        if not self.fragmenty:
            return []
        wektor = self._vectorizer.transform([pytanie])
        podobienstwa = cosine_similarity(wektor, self._macierz)[0]
        kolejnosc = podobienstwa.argsort()[::-1][:k]
        return [self.fragmenty[i] for i in kolejnosc]
```

3. Rozszerz `test_rag.py` – dopisz na końcu wyszukiwanie i uruchom ponownie (treść pytania dopasuj do swojego dokumentu):

```
from rag import BazaWiedzy

baza = BazaWiedzy()
baza.dodaj(fragmenty)
```

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

```
PYTANIE = 'Omów zaliczenie przedmiotu Wprowadzenie do
informatyki?'   # dopasuj do tresci dokumentu
print(f'\nPytanie: {PYTANIE}')
for i, f in enumerate(baza.szukaj(PYTANIE, k=3), start=1):
    print(f'--- wynik {i} (źródło: {f["zrodlo"]}) ---')
    print(f['tekst'][:200], '\n')
```

4. Oceń wyniki: czy pierwszy zwrócony fragment rzeczywiście dotyczy pytania? Zadaj też pytanie o temat, którego na pewno nie ma w dokumencie, i zaobserwuj, że wyszukiwarka i tak zwróci „coś" – najbliższe fragmenty, nawet jeśli pasują słabo. Wniosek: to model, nie wyszukiwarka, musi ocenić, czy kontekst zawiera odpowiedź (rozwiąże ten problem prompt w Zadaniu 10.5).

P: Dołącz zrzut ekranu wyników wyszukiwania dla dwóch pytań: trafnego i spoza treści dokumentu. Skomentuj w 2–3 zdaniach jakość dopasowania.

Zadanie 10.4. Zdjęcia notatek – transkrypcja modelem wizyjnym

Warstwa modeli zostanie rozszerzona o funkcję `opisz_obraz()`, która wysyła zdjęcie notatek do modelu wizyjnego i zwraca transkrypcję. Obraz przekazywany jest w wiadomości użytkownika jako adres danych base64, obok polecenia tekstowego.

Treść zadania:

1. Sprawdź w dokumentacji `console.groq.com/docs/models` aktualną nazwę modelu wizyjnego (obsługującego obrazy) i w razie potrzeby podmień wartość stałej `MODEL_WIZYJNY` w kodzie poniżej.
2. Na końcu pliku `src/models.py` dodaj (import `base64` umieść na górze pliku, obok pozostałych importów):

```
import base64
```

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

**Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00**

```
MODEL_WIZYJNY = 'meta-llama/llama-4-scout-17b-16e-instruct'


def opisz_obraz(client, plik_obrazu, model=MODEL_WIZYJNY):
    """Transkrybuje tresc notatek ze zdjecia (JPG/PNG) na
tekst."""
    dane = base64.b64encode(plik_obrazu.read()).decode('utf-8')
    response = client.chat.completions.create(
        model=model,
        messages=[{
            'role': 'user',
            'content': [
                {'type': 'text',
                 'text': 'Przepisz wiernie całą treść notatek '
                         'widocznych na obrazie. Zachowaj
strukturę '
                         '(nagłówki, punkty, wzory). Zwróć
wyłącznie '
                         'przepisany tekst, bez komentarzy.'},
                {'type': 'image_url',
                 'image_url': {
                     'url': f'{dane}'}},
            ],
        }],
        temperature=0.0,
        max_tokens=2000,
    )
    return response.choices[0].message.content
```

3. Przetestuj transkrypcję w terminalu – dopisz na końcu `test_rag.py` i uruchom:

```
from models import get_client, opisz_obraz

client = get_client()
with open('materialy/notatki.jpg', 'rb') as plik:
    transkrypcja = opisz_obraz(client, plik)
print('--- transkrypcja notatek ---')
```

```
print(transkrypcja)
```

4. Porównaj transkrypcję z oryginałem zdjęcia: czy model odczytał całość?

Uwaga: modele wizyjne mają limity rozmiaru obrazu (typowo kilka MB dla obrazu base64). Jeśli żądanie zostanie odrzucone, zmniejsz zdjęcie przed wysłaniem. Pamiętaj też, że obraz trafia w całości do zewnętrznego API. Nie wysyłaj materiałów zawierających dane osobowe.

P: Dołącz zdjęcie notatek (lub ich czytelny wycinek) oraz zrzut ekranu transkrypcji. Oceń wierność transkrypcji w skali 1–5 i wypisz zauważone błędy. Wyjaśnij, dlaczego dla transkrypcji stosuje się temperaturę 0.0.

Zadanie 10.5. Integracja RAG z chatbotem – asystent do nauki

Elementy z zadań 10.2–10.4 zostaną połączone w aplikacji: panel boczny zostanie uzupełniony o sekcję wgrywania materiałów (`st.file_uploader`), baza wiedzy zostanie zachowana w st.session_state, a przed każdym pytaniem do promptu systemowego doklejane będą znalezione fragmenty wraz z poleceniem odpowiadania na ich podstawie oraz wskazaniem źródła.

Treść zadania:

1. W pliku `src/app_ui.py` rozszerz importy o nowe funkcje i moduł RAG:

```
from models import (MODELE, get_client, load_system_prompt,
                    opisz_obraz, zapytaj_stream)
from rag import BazaWiedzy, podziel_na_fragmenty, wczytaj_pdf
```

2. Pod sekcją ustawień panelu bocznego dodaj sekcję materiałów: wgrywanie plików, przełącznik trybu RAG, bazę wiedzy w stanie sesji oraz przetwarzanie nowo wgranych plików (PDF – ekstrakcja tekstu; obraz – transkrypcja modelem wizyjnym):

```
# --- materiały do nauki (RAG) ---
st.sidebar.header('Materiały do nauki')
pliki = st.sidebar.file_uploader(
    'Dodaj materiały (PDF, JPG, PNG):',
```

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

**Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00**

```
    type=['pdf', 'jpg', 'jpeg', 'png'],
    accept_multiple_files=True)
uzyj_rag = st.sidebar.checkbox(
    'Odpowiadaj na podstawie materiałów', value=True)

if 'baza' not in st.session_state:
    st.session_state.baza = BazaWiedzy()
    st.session_state.wczytane = set()

for plik in pliki or []:
    if plik.name in st.session_state.wczytane:
        continue          # plik już przetworzony
    try:
        if plik.name.lower().endswith('.pdf'):
            tekst = wczytaj_pdf(plik)
        else:             # zdjęcie notatek -> transkrypcja
            tekst = opisz_obraz(get_client(), plik)
        st.session_state.baza.dodaj(
            podziel_na_fragmenty(tekst, plik.name))
        st.session_state.wczytane.add(plik.name)
        st.sidebar.success(f'Wczytano: {plik.name}')
    except Exception as e:
        st.sidebar.error(f'Błąd pliku {plik.name}: {e}')
```

3. W bloku `if user_input:` po wyświetleniu wiadomości użytkownika, a przed zbudowaniem listy `messages` dodaj augmentację promptu systemowego znalezionymi fragmentami:

```
    # --- RAG: doklejenie fragmentów materiałów do promptu ---
    if uzyj_rag and st.session_state.baza.fragmenty:
        znalezione = st.session_state.baza.szukaj(user_input, k=3)
        kontekst = '\n\n'.join(
            f'[Źródło: {f["zrodlo"]}]\n{f["tekst"]}'
            for f in znalezione)
        system_prompt = (
            system_prompt
            + '\n\nOdpowiadaj przede wszystkim na podstawie '
            'poniższych fragmentów materiałów studenta. Jeśli '
```


Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

```
            'fragmenty nie zawierają odpowiedzi, powiedz to '
            'wprost. Na końcu odpowiedzi wskaż źródło (nazwę '
            'pliku).\nFRAGMENTY:\n"""\n' + kontekst + '\n"""')
```

4. Uruchom aplikację (`streamlit run src/app_ui.py`), wgraj w panelu bocznym pliki `wyklad.pdf`, labolatorium.pdf oraz `notatki.jpg` i poczekaj na komunikaty o wczytaniu.
5. Zadaj pytanie o konkretny szczegół z wgranego dokumentu (np. o definicję, wartość parametru, treść zadania). Sprawdź, czy odpowiedź jest zgodna z materiałem i czy asystent wskazał źródło. Następnie zadaj pytanie o treść sfotografowanych notatek.
6. Zadaj pytanie spoza materiałów (np. o pojęcie, którego w nich nie ma) i sprawdź, czy asystent, zgodnie z poleceniem w prompcie, przyznaje, że materiały nie zawierają odpowiedzi.

Uwaga: fragmenty są doklejane do promptu systemowego tylko na czas jednego żądania. Zmienna `system_prompt` jest nadpisywana lokalnie przy każdym przebiegu skryptu, więc historia rozmowy w `session_state` pozostaje czysta (bez powielonych fragmentów). Zwróć też uwagę na współdziałanie RAG z pamięcią rozmowy z Laboratorium 9. Model otrzymuje jednocześnie historię wymian i fragmenty materiałów.

P: Dołącz końcowe wersje plików `app_ui.py`, `models.py` i `rag.py` oraz zrzuty ekranu: (1) panelu z wczytanymi materiałami, (2) odpowiedzi na pytanie o treść PDF ze wskazanym źródłem, (3) odpowiedzi na pytanie o treść notatek ze zdjęcia, (4) odpowiedzi na pytanie spoza materiałów.

Zadanie 10.6. Eksperyment: asystent z RAG i bez RAG

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

**Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00**

Eksperyment pokazuje, jak technika RAG wpływa na jakość i wiarygodność odpowiedzi. Te same pytania zostaną zadane z włączonym i wyłączonym przełącznikiem „Odpowiadaj na podstawie materiałów". Wyniki należy zebrać w tabeli 10.1.

Treść zadania:

1. **Scenariusz A.** Włącz RAG i zadaj pytanie o szczegół obecny wyłącznie w Twoich materiałach (np. o treść konkretnego zadania z instrukcji). Zapisz odpowiedź i wskazane źródło.
2. **Scenariusz B.** Wyczyść czat, wyłącz RAG i zadaj to samo pytanie. Zaobserwuj, czy model odpowiada ogólnikami, przyznaje brak wiedzy, czy zmyśla szczegóły (halucynacja).
3. **Scenariusz C.** Włącz RAG i zadaj pytanie spoza materiałów. Sprawdź, czy asystent zgodnie z instrukcją przyznaje, że materiały nie zawierają odpowiedzi.
4. **Scenariusz D.** Z włączonym RAG poproś: „Przygotuj 3 pytania kontrolne do powtórki z moich materiałów wraz z odpowiedziami." Oceń, czy pytania rzeczywiście dotyczą wgranych treści.
5. Uzupełnij tabelę 10.1 na podstawie obserwacji.

Tabela 10.1. Porównanie odpowiedzi asystenta z RAG i bez RAG

Scenariusz	RAG	Pytanie (skrót)	Obserwacja (2–3 zdania)
A	włączony		
B	wyłączony		
C	włączony		
D	włączony		

P: Zamieść uzupełnioną tabelę. Sformułuj wnioski: (1) jak RAG wpływa na wiarygodność odpowiedzi i możliwość ich weryfikacji? (2) skąd biorą się halucynacje w scenariuszu B i dlaczego polecenie „powiedz wprost, jeśli nie wiesz" w prompcie jest istotne? (3) jaki jest koszt RAG w tokenach wejściowych względem zwykłego pytania?

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

Ćwiczenia samodzielne

Ćwiczenie 10.1 (podstawowe). Zarządzanie materiałami

Rozbuduj sekcję materiałów w panelu bocznym: wyświetl listę wczytanych plików (zawartość zbioru `st.session_state.wczytane`) wraz z liczbą fragmentów w bazie oraz dodaj przycisk „Wyczyść materiały", który opróżnia bazę wiedzy i listę wczytanych plików (analogicznie do przycisku „Wyczyść czat"). W sprawozdaniu zamieść zrzut ekranu panelu przed i po wyczyszczeniu.

Ćwiczenie 10.2 (średnie). Podgląd źródeł odpowiedzi

Po każdej odpowiedzi asystenta wyświetl pod nią rozwijany panel `st.expander('Źródła')` z fragmentami użytymi w prompcie (nazwa pliku + początek tekstu fragmentu). Zapisz ostatnio użyte fragmenty w `st.session_state`, aby przetrwały podczas ponownego uruchomienia skryptu. W sprawozdaniu wyjaśnij, dlaczego prezentowanie źródeł zwiększa zaufanie do odpowiedzi asystenta i ułatwia naukę.

Ćwiczenie 10.3 (średnie). Parametry RAG: rozmiar fragmentu i k

Zbadaj wpływ parametrów na jakość odpowiedzi: przetestuj trzy konfiguracje – (a) `ROZMIAR_FRAGMENTU=200`, k=3, (b) `ROZMIAR_FRAGMENTU=800`, k=3, (c) `ROZMIAR_FRAGMENTU=800`, k=8 – na tym samym zestawie 3 pytań o Twoje materiały. Zestaw wyniki w tabeli (konfiguracja, trafność odpowiedzi 1–5, orientacyjna liczba tokenów kontekstu) i sformułuj wniosek o kompromisie między precyzją, kontekstem a kosztem.

Ćwiczenie 10.4 (trudne). Wyszukiwanie semantyczne (embeddingi)

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

Zastąp TF-IDF wyszukiwaniem semantycznym. Zainstaluj bibliotekę `sentence-transformers`, użyj wielojęzycznego modelu embeddingów (np. `paraphrase-multilingual-MiniLM-L12-v2`) do zamiany fragmentów i pytań na wektory oraz porównuj je za pomocą podobieństwa kosinusowego. Porównaj oba podejścia na pytaniu sformułowanym innymi słowami niż w tekście materiałów (np. synonimami) – TF-IDF wymaga wspólnych słów, a embeddingi znajdują podobieństwo znaczeniowe. Zamieść porównanie wyników wyszukiwania obu metod dla 2 pytań oraz wniosek, kiedy warto sięgnąć po embeddingi.

Pytania pomocnicze

1. Czym jest RAG i z jakich trzech kroków składa się generowanie odpowiedzi w tej technice?
2. Jakie problemy modelu językowego rozwiązuje RAG? Dlaczego samo „wklejenie wszystkich materiałów do promptu" nie jest dobrym rozwiązaniem?
3. Na czym polega podział tekstu na fragmenty (chunking) i po co stosuje się zakładkę (overlap) między fragmentami?
4. Jak działa wyszukiwanie fragmentów metodą TF-IDF i podobieństwa kosinusowego? Jakie jest główne ograniczenie tej metody względem embeddingów?
5. W jaki sposób obraz przekazywany jest do modelu wizyjnego w liście `messages`? Do czego służy kodowanie base64?
6. Dlaczego transkrypcję notatek wykonuje się przy temperaturze 0.0?
7. Dlaczego fragmenty materiałów dokleja się do promptu systemowego tylko na czas jednego żądania, zamiast zapisywać je w historii rozmowy?
8. Skąd biorą się halucynacje modelu i jak RAG oraz odpowiednie polecenie w prompcie („powiedz wprost, jeśli materiały nie zawierają odpowiedzi") ograniczają to zjawisko?
9. Jakie kwestie prywatności wiążą się z wysyłaniem materiałów studenta do zewnętrznego API i jaka jest alternatywa?

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

Podsumowanie

Po ukończeniu laboratorium student dysponuje asystentem AI do wspomagania nauki, który odpowiada na podstawie własnych materiałów. W szczególności:

1. wczytuje materiały PDF, dzieli je na nakładające się fragmenty i buduje prostą bazę wiedzy z wyszukiwaniem TF-IDF;
2. transkrybuje zdjęcia odręcznych notatek modelem wizyjnym i włącza je do bazy wiedzy;
3. integruje RAG z chatbotem: wgrywanie plików, doklejanie kontekstu do promptu, wskazywanie źródeł odpowiedzi;
4. potrafi eksperymentalnie wykazać różnicę między odpowiedziami z RAG i bez RAG oraz rozpoznać halucynacje modelu.

WSKAZÓWKA: zachowaj projekt – asystent zbudowany w tym laboratorium będzie rozwijany w Laboratorium 11. Dopisz do biblioteki promptów (plik `prompts.md` lub program AI Prompt Lab) prompt augmentacyjny RAG wraz z oceną jego skuteczności oraz zanotuj dobrane parametry (rozmiar fragmentu, zakładka, k), które najlepiej sprawdziły się na Twoich materiałach.

Literatura

[1]. Alto V., Moch W., Generatywna sztuczna inteligencja z ChatGPT i modelami OpenAI. Podnieś swoją produktywność i innowacyjność za pomocą GPT-3 i GPT-4, Helion, Gliwice 2024.

[2]. Lewis P. i in., Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks, arXiv:2005.11401, 2020.

[3]. Gao Y. i in., Retrieval-Augmented Generation for Large Language Models: A Survey, arXiv:2312.10997, 2023.

[4]. pypdf – dokumentacja biblioteki, https://pypdf.readthedocs.io/ (dostęp: 2026).

Fundusze Europejskie dla Rozwoju Społecznego | Rzeczpospolita Polska | Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

[5]. scikit-learn, TfidfVectorizer – dokumentacja, https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html (dostęp: 2026).
[6]. Tunstall L., von Werra L., Wolf T., Przetwarzanie języka naturalnego z wykorzystaniem transformerów, Helion, Gliwice 2023.
[7]. Groq, Vision – modele wizyjne, dokumentacja API, https://console.groq.com/docs/vision (dostęp: 2026).
[8]. Groq, Models – lista dostępnych modeli, https://console.groq.com/docs/models (dostęp: 2026).
[9]. Streamlit, st.file_uploader – dokumentacja API, https://docs.streamlit.io/develop/api-reference/widgets/st.file_uploader (dostęp: 2026).
[10]. Sentence-Transformers – dokumentacja biblioteki, https://www.sbert.net/ (dostęp: 2026).
[11]. Phoenix J., Taylor M., Skuteczna inżynieria promptów. Przyszłościowe rozwiązania dla rzetelnych wyników generatywnej AI, Helion, Gliwice 2025.

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

POLITECHNIKA LUBELSKA

Materiały do przedmiotu:

GENERATYWNA SZTUCZNA INTELIGENCJA I PROMPT ENGINEERING

Laboratorium nr 11.
Tworzenie asystenta AI do programowania

Autor:

dr inż. Krzysztof Dziedzic

dr Marcin Barszcz

Katedra Informatyki

Fundusze Europejskie dla Rozwoju Społecznego

Rzeczpospolita Polska

Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

POLLUB z nami nowoczesne technologie

POLITECHNIKA LUBELSKA

Fundusze Europejskie dla Rozwoju Społecznego

Rzeczpospolita Polska

Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

Spis treści

Wprowadzenie, tematyka zajęć ---------- 207
Cel ćwiczenia/laboratorium ---------- 209
Instrukcja laboratoryjna/ćwiczeniowa ---------- 209
Ćwiczenia samodzielne ---------- 217
Pytania pomocnicze ---------- 218
Podsumowanie ---------- 218
Literatura ---------- 219

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

Wprowadzenie, tematyka zajęć

Niniejsze laboratorium zamyka cykl zajęć rozpoczęty w Laboratorium 8. Zbudowana dotychczas aplikacja to chatbot z pamięcią rozmowy i strumieniowaniem (Laboratorium 9), rozszerzony o technikę RAG i asystenta do wspomagania nauki na materiałach studenta (Laboratorium 10). W tym laboratorium powstanie asystent AI do programowania: narzędzie wspierające pisanie, wyjaśnianie, debugowanie, przeglądanie (code review) i testowanie kodu [1].

Rozszerzać czy budować od nowa? Przed rozbudową aplikacji o kolejną funkcję warto zadać pytanie architektoniczne: czy dokładać ją do istniejącego interfejsu, czy zbudować osobny, wyspecjalizowany program? W tym laboratorium przyjęto rozwiązanie pośrednie: **wspólny projekt i wspólne warstwy, ale osobny interfejs**. Asystent programowania powstanie w tym samym katalogu projektu i będzie korzystał z tej samej warstwy modeli (`models.py`), klucza API, środowiska wirtualnego i wzorców poznanych w Laboratoriach 8–10 ale otrzyma własny plik interfejsu `app_kod.py`. Uzasadnienie: asystent do nauki i asystent do programowania różnią się promptem systemowym, domyślnymi parametrami (kod wymaga niskiej temperatury), sposobem podawania kontekstu (całe pliki z kodem zamiast wyszukiwania fragmentów) i zestawem trybów pracy. Umieszczenie obu ról w jednym interfejsie oznaczałoby gąszcz przełączników. Budowa zupełnie nowego projektu oznacza natomiast powielenie kodu, drugiego klucza i środowiska bez żadnej korzyści.

Asystenci AI do programowania. Narzędzia takie jak GitHub Copilot, Claude Code czy Cursor na stałe weszły do warsztatu programisty [2, 3]. Ich typowe zadania to: generowanie kodu na podstawie opisu, wyjaśnianie cudzego kodu, lokalizowanie i poprawianie błędów, przegląd kodu (code review), generowanie testów jednostkowych oraz dokumentacji. Badania nad modelami do kodu (np. Codex [4]) pokazały, że jakość odpowiedzi w dalszym ciągu silnie zależy od jakości kontekstu i precyzji polecenia – czyli od inżynierii promptów [5].

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

Prompt systemowy dla kodu. Dobry asystent programowania potrzebuje wyraźnej roli i zasad: kod w blokach markdown z oznaczeniem języka (Streamlit wyświetla je z kolorowaniem składni), zgodność z konwencjami (dla Pythona – PEP 8), wskazywanie linii i przyczyny błędu przed podaniem poprawki, zwięzłość oraz przyznawanie się do niepewności. Oprócz stałej roli asystent otrzyma **tryby pracy** (wyjaśnianie, szukanie błędów, review, testy) wybierane w interfejsie doprecyzowanie zadania, doklejane do wiadomości użytkownika. To ta sama idea struktury promptu (rola – zadanie – kontekst – format), którą wprowadzono w Laboratorium 2 [5, 6].

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

Kontekst: całe pliki zamiast RAG. W Laboratorium 10 materiały dzielono na fragmenty i wyszukiwano najlepiej dopasowane. Przy pracy z kodem takie podejście bywa zawodne: fragment wyrwany z pliku traci definicje, importy i zależności, a model analizujący niekompletny kod łatwo wyciąga błędne wnioski. Pliki źródłowe w projektach studenckich są przy tym na tyle małe, że mieszczą się w całości w oknie kontekstu. Dlatego asystent programowania będzie dołączał do żądania **cały wskazany plik**, oddzielony od polecenia ogranicznikami (delimitery """ – Laboratorium 2). Wyszukiwanie fragmentów (RAG) pozostaje właściwym narzędziem dla dużych repozytoriów. Tak działają indeksy kodu w narzędziach produkcyjnych.

Temperatura przy pracy z kodem. Kod jest wrażliwy na losowość. Zmieniona nazwa zmiennej czy „twórczo" dobrany argument psują program. Dlatego dla zadań programistycznych stosuje się niskie temperatury (0.0–0.3), które premiują odpowiedzi najbardziej prawdopodobne i powtarzalne. Suwak temperatury w nowym interfejsie będzie miał domyślną wartość 0.2 [7].

Weryfikacja wygenerowanego kodu. Kod z modelu językowego jest zawsze propozycją, nie gotowym produktem: może zawierać błędy logiczne, pomijać przypadki brzegowe albo używać nieistniejących funkcji. Odpowiedzialność za działanie kodu ponosi programista. Podstawową bramą jakości są testy jednostkowe (pytest – Laboratorium 2). Wygenerowany kod uruchamia się najpierw na testach, a wyniki błędów zawraca do asystenta w kolejnej iteracji. Nigdy nie należy uruchamiać wygenerowanego kodu bez jego przeczytania, zwłaszcza operacji na plikach, sieci i poleceń systemowych [8].

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

**Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00**

Cel ćwiczenia/laboratorium

Głównym celem laboratorium jest zbudowanie wyspecjalizowanego asystenta AI do programowania jako drugiego interfejsu istniejącego projektu, z trybami pracy, kontekstem z plików źródłowych i weryfikacją generowanego kodu testami. Po ukończeniu zajęć student potrafi:

- uzasadnić decyzję architektoniczną: rozszerzenie wspólnych warstw projektu o wyspecjalizowany interfejs zamiast budowy aplikacji od zera lub przeciążania jednego interfejsu;
- zaprojektować prompt systemowy asystenta programowania oraz tryby pracy (wyjaśnianie, szukanie błędów, code review, generowanie testów) doprecyzowujące zadanie;
- zbudować interfejs czatu dla kodu z niską domyślną temperaturą i kolorowaniem składni w blokach markdown;
- dołączać do żądania całe pliki źródłowe oddzielone ogranicznikami i wyjaśnić, dlaczego przy kodzie pełny plik bywa lepszy od wyszukiwania fragmentów (RAG);
- prowadzić cykl generowania i weryfikacji kodu: wygenerowany kod i testy pytest, uruchomienie testów, zawrócenie błędów do asystenta;
- przeprowadzić przegląd (code review) plików własnego projektu i krytycznie ocenić uwagi asystenta.

Laboratorium wymaga ukończonego projektu z Laboratoriów 8–10 (katalog `asystent_streamlit` z działającą warstwą modeli i kluczem API).

Instrukcja laboratoryjna/ćwiczeniowa

Zadanie 11.1. Prompt systemowy asystenta programowania

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

Nowy asystent zostanie umieszczony w istniejącym projekcie. Struktura po rozbudowie (nowe elementy oznaczono strzałką):

```
asystent_streamlit/
|-- src/
|   |-- app_ui.py            # asystent do nauki (Lab 8-10)
|   |-- app_kod.py           # <- asystent programowania (Zadanie
11.2)
|   |-- models.py            # wspolna warstwa modeli
|   |-- rag.py               # baza wiedzy (Lab 10)
|   `-- ...
|-- prompts/
|   |-- system_note_assistant.txt
|   |-- system_translator.txt
|   `-- system_code_assistant.txt   # <- prompt programisty
|-- .env
`-- requirements.txt
```

Treść zadania:

1. Otwórz projekt `asystentAI` w VS Code i aktywuj środowisko wirtualne (`.venv`). Nowe biblioteki nie są potrzebne. To jedna z korzyści rozbudowy istniejącego projektu.
2. W katalogu `prompts` utwórz plik `system_code_assistant.txt` o następującej treści:

```
Jesteś doświadczonym programistą Python i cierpliwym mentorem.
Pomagasz studentom informatyki w nauce programowania.
Zasady:
- kod umieszczaj w blokach markdown z oznaczeniem języka;
- pisz kod zgodny z PEP 8, z czytelnymi nazwami i docstringami;
- wyjaśniaj krótko i konkretnie, bez zbędnych wstępów;
- gdy znajdujesz błąd, najpierw wskaż miejsce i przyczynę,
  dopiero potem podaj poprawioną wersję;
- jeśli brakuje kontekstu albo nie masz pewności, powiedz to
  wprost zamiast zgadywać.
```

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

3. Przeanalizuj prompt: które zasady odpowiadają za formę odpowiedzi (format), które za jakość kodu, a które ograniczają halucynacje? Możesz dopisać własną zasadę – np. preferowany styl komentarzy.

Zadanie 11.2. Wyspecjalizowany interfejs (app_kod.py)

Interfejs asystenta programowania powtarza szkielet chatbota z Laboratorium 9 (komponenty czatu, pamięć, obsługa błędów, strumieniowanie), ale różni się szczegółami dopasowanymi do pracy z kodem: promptem systemowym, domyślną temperaturą 0.2 oraz słownikiem trybów pracy TRYBY_ZADAN, z którego wybrane polecenie doklejane jest przed wiadomością użytkownika.

Treść zadania:

1. W katalogu src utwórz plik app_kod.py o następującej treści:

```
import streamlit as st

from groq import APIError
from models import (MODELE, get_client, load_system_prompt,
                    zapytaj_stream)

# tryby pracy: polecenie doklejane przed wiadomoscia uzytkownika
TRYBY_ZADAN = {
    'Rozmowa o kodzie': '',
    'Wyjaśnij kod': (
        'Zadanie: wyjaśnij krok po kroku, co robi podany kod '
        'i do czego służą jego elementy.'),
    'Znajdź błąd': (
        'Zadanie: znajdź błędy w podanym kodzie, wskaż miejsca '
        'i przyczyny, a następnie podaj poprawioną wersję.'),
    'Code review': (
        'Zadanie: wykonaj przegląd podanego kodu względem: '
        'poprawności, czytelności, zgodności z PEP 8 i obsługi '
        'błędów. Wypunktuj uwagi w formacie: fragment | problem '
```


Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

```
        '| proponowana poprawka.'),
    'Wygeneruj testy': (
        'Zadanie: przygotuj testy jednostkowe pytest dla podanego
'
        'kodu, obejmujące przypadki typowe i brzegowe.'),
}

# --- konfiguracja strony ---
st.set_page_config(page_title='Asystent AI do programowania')
st.title('Asystent AI do programowania')

# --- panel boczny: ustawienia ---
st.sidebar.header('Ustawienia')
model = st.sidebar.selectbox('Wybierz model:', MODELE)
temperature = st.sidebar.slider('Temperatura:', 0.0, 1.5, 0.2,
0.1)
tryb_zadania = st.sidebar.selectbox('Tryb pracy:',
                                    list(TRYBY_ZADAN))

system_prompt =
load_system_prompt('prompts/system_code_assistant.txt')

# --- pamięć rozmowy (osobna od asystenta do nauki) ---
if 'history_kod' not in st.session_state:
    st.session_state.history_kod = []

if st.sidebar.button('Wyczyść czat'):
    st.session_state.history_kod = []
    st.rerun()

# --- wyświetlenie dotychczasowej rozmowy ---
for wiadomosc in st.session_state.history_kod:
    with st.chat_message(wiadomosc['role']):
        st.markdown(wiadomosc['content'])

# --- obsługa nowej wiadomości ---
user_input = st.chat_input('Napisz wiadomość lub wklej kod...')
```

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

```
if user_input:
    with st.chat_message('user'):
        st.markdown(user_input)

    # doklejenie polecenia wybranego trybu pracy
    tresc = user_input
    zadanie = TRYBY_ZADAN[tryb_zadania]
    if zadanie:
        tresc = zadanie + '\n\n' + tresc

    messages = [{'role': 'system', 'content': system_prompt}]
    messages.extend(st.session_state.history_kod)
    messages.append({'role': 'user', 'content': tresc})

    try:
        client = get_client()
        with st.chat_message('assistant'):
            odpowiedz = st.write_stream(
                zapytaj_stream(client, model, messages,
                               temperature=temperature))
    except RuntimeError as e:
        st.error(str(e))
        st.stop()
    except APIError as e:
        st.error(f'Błąd Groq API: {e}')
        st.stop()

    st.session_state.history_kod.append(
        {'role': 'user', 'content': user_input})
    st.session_state.history_kod.append(
        {'role': 'assistant', 'content': odpowiedz})
```

2. Uruchom asystenta poleceniem `streamlit run src/app_kod.py`. Zwróć uwagę, że asystent do nauki (`app_ui.py`) pozostaje osobną, w pełni

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

działającą aplikacją. Oba interfejsy dzielą warstwę modeli, ale mają osobne historie rozmów w `session_state`.

3. Przetestuj tryby pracy: (a) w trybie **Wyjaśnij kod** wklej krótki fragment, np. `wynik = [x**2 for x in range(10) if x % 2 == 0]`; (b) w trybie **Znajdź błąd** wklej celowo zepsuty kod, np. `def srednia(liczby): return sum(liczby) / len(liczby)` z pytaniem, co się stanie dla pustej listy.
4. Zaobserwuj, że bloki kodu w odpowiedziach są wyświetlane z kolorowaniem składni. Jest to efekt bloków markdown wymuszonych w prompcie systemowym i renderowanych przez `st.markdown`.

P: Dołącz plik `app_kod.py` oraz zrzuty ekranu odpowiedzi w trybach „Wyjaśnij kod" i „Znajdź błąd". Wyjaśnij, jak polecenie trybu pracy trafia do modelu i dlaczego nie umieszczono go w prompcie systemowym.

Zadanie 11.3. Kontekst z plików źródłowych

Asystent zyska możliwość pracy na całych plikach: wgrany plik źródłowy będzie doklejany do bieżącego żądania między ogranicznikami. Plik trafia wyłącznie do bieżącego żądania, a do historii zapisywana jest tylko wiadomość użytkownika – inaczej treść pliku powielałaby się w każdej kolejnej wymianie, szybko zwiększając liczbę tokenów (koszt pamięci – Laboratorium 9).

Treść zadania:

1. W panelu bocznym `app_kod.py`, pod wyborem trybu pracy, dodaj wgrywanie pliku z kodem:

```
plik_kodu = st.sidebar.file_uploader(
    'Dołącz plik z kodem:',
    type=['py', 'txt', 'js', 'java', 'c', 'cpp'])
```

2. W bloku `if user_input:` – po dodaniu polecenia trybu, a przed zbudowaniem listy messages – dodaj dodawanie zawartości pliku. Metoda `getvalue()` (zamiast `read()`) zwraca całą zawartość niezależnie od tego, ile razy skrypt wykonał się ponownie:

```
    # doklejenie calego pliku zrodlowego (tylko biezace zadanie)
```

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

```
    if plik_kodu is not None:
        kod = plik_kodu.getvalue().decode('utf-8',
                                          errors='ignore')
        tresc += (f'\n\nPlik {plik_kodu.name}:\n"""\n'
                  + kod + '\n"""')
```

3. Uruchom aplikację ponownie, wgraj w panelu bocznym plik `src/rag.py` ze swojego projektu i w trybie **Wyjaśnij kod** zapytaj: „Co robi klasa BazaWiedzy i jak działa metoda szukaj?" Sprawdź, czy odpowiedź odnosi się do rzeczywistej zawartości pliku.
4. Zadaj pytanie kontrolne wymagające całości pliku, np. „Które funkcje w tym pliku korzystają ze stałej ROZMIAR_FRAGMENTU?". Odpowiedź wymaga jednoczesnego spojrzenia na definicję stałej i wszystkie funkcje, co ilustruje przewagę pełnego pliku nad wyrwanym fragmentem.

P: Dołącz zrzuty ekranu obu odpowiedzi (punkty 3 i 4). Wyjaśnij: (1) dlaczego przy pracy z kodem dokleja się cały plik zamiast stosować wyszukiwanie fragmentów jak w Laboratorium 10? (2) dlaczego treści pliku nie zapisuje się w historii rozmowy?

Zadanie 11.4. Generowanie kodu weryfikowanego testami

Pełny cykl pracy z asystentem programowania: wygenerowanie funkcji wraz z testami jednostkowymi, uruchomienie testów i w razie niepowodzeń zwrócenie komunikatów o błędach do asystenta. Kod z modelu traktowany jest jako propozycja, a testy pełnią rolę bramy jakościowej.

Treść zadania:

1. W trybie **Rozmowa o kodzie** (temperatura 0.2) wyślij polecenie: „Napisz funkcję `waliduj_indeks(numer: str) -> bool`, która sprawdza numer albumu studenta: dokładnie 6 cyfr, bez innych znaków, funkcja nie może zgłaszać wyjątków dla błędnych danych. Dodaj docstring z przykładami."

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

2. Zapisz wygenerowaną funkcję w pliku `walidacja.py` w katalogu głównym projektu. Przeczytaj kod przed zapisaniem. Upewnij się, że rozumiesz każdą linię.
3. Przełącz tryb pracy na **„Wygeneruj testy”**, wgraj plik `walidacja.py` w panelu bocznym i wyślij: „Testy dla funkcji waliduj_indeks z załączonego pliku, co najmniej 8 przypadków, w tym: poprawny numer, za krótki, za długi, litery, znaki specjalne, pusty łańcuch, spacje, None.” Zapisz testy do pliku `test_walidacja.py`.
4. Zainstaluj framework testowy (jeśli nie ma go w środowisku) i uruchom testy:

```
pip install pytest
pytest test_walidacja.py -v
```

5. Jeśli którykolwiek test nie przechodzi, skopiuj komunikat błędu z terminala, przełącz tryb na **Znajdź błąd**, wgraj plik `walidacja.py` i wklej komunikat z pytaniem o poprawkę. Wprowadź poprawkę i uruchom testy ponownie. Powtarzaj to do skutku. Jeśli wszystkie testy przeszły za pierwszym razem, sprawdź odporność testów. Zepsuj celowo funkcję (np. zmień 6 na 5) i upewnij się, że testy to wykryją, a następnie przywróć poprawną wersję.

P: Dołącz pliki `walidacja.py` i `test_walidacja.py`, zrzut ekranu wyniku `pytest -v` oraz opis przebiegu.

Zadanie 11.5. Analiza przypadku: code review własnego projektu

Asystent zostanie użyty do przeglądu kodu napisanego w trakcie kursu oraz do zbadania wpływu temperatury na generowanie kodu. Uwagi z przeglądu należy ocenić krytycznie. Model bywa nadgorliwy (zgłasza pozorne problemy) albo pomija sedno. Wyniki dokumentuje tabela 11.1.

Treść zadania:

1. **Scenariusz A – code review.** W trybie **Code review** wgraj plik `src/rag.py` i wyślij: „Wykonaj przegląd załączonego pliku.” Z listy uwag wybierz trzy

**Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00**

najważniejsze i oceń każdą z nich: czy jest zasadna? czy poprawka jest bezpieczna?

2. **Scenariusz B – wdrożenie poprawki.** Wybierz jedną zasadną uwagę, wprowadź poprawkę w kodzie projektu i sprawdź, że aplikacja z Laboratorium 10 nadal działa (uruchom `streamlit run src/app_ui.py` i zadaj pytanie z RAG).
3. **Scenariusz C – temperatura a kod.** W trybie **Rozmowa o kodzie** ustaw temperaturę 0.0 i dwukrotnie (w osobnych, wyczyszczonych czatach) wyślij to samo polecenie wygenerowania funkcji `waliduj_indeks` z zadania 11.4. Porównaj oba wyniki. Powtórz przy temperaturze 1.2 i oceń różnorodność oraz poprawność kodu.
4. Uzupełnij tabelę 11.1 na podstawie obserwacji.

Tabela 11.1. Wyniki analizy przypadku (przegląd kodu i temperatura)

Scenariusz	Obserwacja / uwaga	Ocena (zasadna? poprawna?)
A – uwaga 1		
A – uwaga 2		
A – uwaga 3		
C – temp. 0.0		
C – temp. 1.2		

P: Zamieść uzupełnioną tabelę 11.1, opis wdrożonej poprawki (scenariusz B) oraz wnioski: (1) które uwagi asystenta były zasadne, a które nadgorliwe, po czym to poznać? (2) jak temperatura wpłynęła na powtarzalność i poprawność generowanego kodu i jaka wartość jest właściwa dla asystenta programowania?

Ćwiczenia samodzielne

Ćwiczenie 11.1 (podstawowe). Pobieranie wygenerowanego kodu

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

Dodaj pod rozmową przycisk `st.download_button`, który pozwala pobrać ostatnią odpowiedź asystenta jako plik (np. `odpowiedz.md`). Wersja rozszerzona: jeśli odpowiedź zawiera blok kodu Python, wytnij sam kod (fragment między znacznikami ` ```python ` i ` ``` `) i zaproponuj pobranie jako plik `.py`.

Ćwiczenie 11.2 (średnie). Tryb tłumaczenia między językami

Dodaj tryb pracy „Przetłumacz kod" wraz z dodatkowym polem wyboru języka docelowego (np. JavaScript, C++, Java). Polecenie trybu powinno wymagać zachowania logiki, nazw i komentarzy oraz wskazania różnic wynikających ze specyfiki języka docelowego. Przetestuj na funkcji `waliduj_indeks` (Python → JavaScript) i oceń w sprawozdaniu poprawność tłumaczenia: czy obsłużono wszystkie przypadki brzegowe z wersji źródłowej?

Ćwiczenie 11.3 (trudne). Automatyczna pętla popraw kodu

Zbuduj w aplikacji przycisk „Popraw automatycznie", uruchamiający pętlę samodoskonalenia znaną z Laboratorium 5: (1) asystent-krytyk ocenia ostatnio wygenerowany kod względem jawnej listy kryteriów (walidacja wejścia, docstring, nazwy, adnotacje typów), (2) asystent-korektor poprawia kod zgodnie z krytyką, (3) oba kroki (krytyka i poprawiona wersja) są wyświetlane w rozmowie. Wykorzystaj dwa osobne wywołania `zapytaj()` z różnymi promptami.

Pytania pomocnicze

1. Jakie typowe zadania realizują asystenci AI do programowania?
2. Jak działają tryby pracy asystenta i dlaczego polecenie trybu dokleja się do wiadomości użytkownika, a nie do promptu systemowego?
3. Dlaczego przy pracy z kodem dokleja się do żądania cały plik źródłowy, a nie wyszukane fragmenty (RAG)? Kiedy RAG wraca do gry?
4. Dlaczego dla zadań programistycznych stosuje się niskie wartości temperatury?

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

5. Na czym polega cykl weryfikacji wygenerowanego kodu testami i dlaczego kod z modelu należy traktować jako propozycję?
6. Jakie zagrożenia wiążą się z uruchamianiem wygenerowanego kodu bez jego przeczytania?

Podsumowanie

Laboratorium zamyka cykl zajęć 8–11. Zbudowany projekt zawiera kompletną aplikację z dwoma wyspecjalizowanymi asystentami (do nauki (RAG na materiałach studenta) i do programowania) pracującymi na wspólnych warstwach. Po ukończeniu laboratorium student:

1. podejmuje i uzasadnia decyzje architektoniczne przy rozbudowie aplikacji AI, wykorzystując separację warstw (interfejsy / warstwa modeli / prompty w plikach);
2. projektuje prompt systemowy i tryby pracy asystenta programowania zgodnie ze strukturą rola – zadanie – kontekst – format;
3. dołącza pliki źródłowe jako kontekst żądania z ogranicznikami i świadomie wybiera między pełnym plikiem a wyszukiwaniem fragmentów;
4. prowadzi cykl generowania i weryfikacji kodu testami jednostkowymi oraz iteracyjnego poprawiania na podstawie komunikatów błędów;

WSKAZÓWKA: uzupełnij bibliotekę promptów prowadzoną od Laboratorium 2 (plik `prompts.md` lub program AI Prompt Lab): dopisz prompt systemowy programisty, polecenia trybów pracy oraz najskuteczniejsze polecenia z zadań 11.4–11.5.

UWAGA dotycząca bezpieczeństwa: klucza API nie wolno udostępniać publicznie, a plik `.env` musi być ujęty w `.gitignore`. Ponadto wygenerowanego kodu nie wolno uruchamiać bez przeczytania w szczególności operacji na plikach, poleceń systemowych i kodu sieciowego. Pamiętaj też, że wgrywane pliki źródłowe trafiają do zewnętrznego API. Nie wysyłaj kodu objętego tajemnicą (np. z pracy zawodowej).

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

Literatura

[1]. Caelen O., Blete M.-A., Tworzenie aplikacji z wykorzystaniem GPT-4 i ChatGPT. Buduj inteligentne chatboty, generatory treści i fascynujące projekty, Helion, Gliwice 2024.

[2]. GitHub, Copilot – dokumentacja, https://docs.github.com/copilot (dostęp: 2026).

[3]. Anthropic, Claude Code – dokumentacja, https://docs.claude.com/en/docs/claude-code/overview (dostęp: 2026).

[4]. Chen M. i in., Evaluating Large Language Models Trained on Code, arXiv:2107.03374, 2021.

[5]. Phoenix J., Taylor M., Skuteczna inżynieria promptów. Przyszłościowe rozwiązania dla rzetelnych wyników generatywnej AI, Helion, Gliwice 2025.

[6]. van Rossum G., Warsaw B., Coghlan A., PEP 8 – Style Guide for Python Code, https://peps.python.org/pep-0008/ (dostęp: 2026).

[7]. Mizrahi G., Inżynieria promptów bez tajemnic. Sztuka kreatywnego generowania tekstów, Helion, Gliwice 2025.

[8]. pytest – dokumentacja frameworka testowego, https://docs.pytest.org/ (dostęp: 2026).

[9]. Streamlit, st.download_button – dokumentacja API, https://docs.streamlit.io/develop/api-reference/widgets/st.download_button (dostęp: 2026).

[10]. Groq, Text/Chat Generation – dokumentacja API, https://console.groq.com/docs/text-chat (dostęp: 2026).

[11]. Groq, Model deprecations (harmonogram wyłączania modeli), https://console.groq.com/docs/deprecations (dostęp: 2026).

Fundusze Europejskie dla Rozwoju Społecznego

Rzeczpospolita Polska

Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

POLITECHNIKA LUBELSKA

Materiały do przedmiotu:

GENERATYWNA SZTUCZNA INTELIGENCJA I PROMPT ENGINEERING

Laboratorium nr 12.
Tworzenie aplikacji wspomaganych AI do automatyzacji zadań

Autor:

dr inż. Krzysztof Dziedzic

dr Marcin Barszcz

Fundusze Europejskie dla Rozwoju Społecznego

Rzeczpospolita Polska

Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

POLITECHNIKA LUBELSKA

Katedra Informatyki

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

Spis treści

Wprowadzenie, tematyka zajęć ----------222
Cel ćwiczenia/laboratorium----------223
Instrukcja laboratoryjna/ćwiczeniowa----------224
Ćwiczenia samodzielne ----------234
Pytania pomocnicze ----------236
Podsumowanie----------236
Literatura ----------237

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

Wprowadzenie, tematyka zajęć

Dotychczasowe aplikacje budowane na labolatoriach – chatbot, asystent do nauki z RAG, asystent programowania (Laboratoria 8–11) – miały wspólną cechę: w centrum każdej z nich znajdowała się rozmowa, a człowiek inicjował każde wywołanie modelu. W praktyce informatycznej, biznesowej i technicznej równie ważna jest druga klasa zastosowań: **aplikacje automatyzujące zadania**, które przetwarzają dane bez okna czatu np. setki opinii klientów, zgłoszeń serwisowych czy raportów technicznych, jedną po drugiej, według stałego schematu. W niniejszym laboratorium utworzony zostanie automat analizujący opinie klientów sklepu internetowego, który klasyfikuje je, wyciąga z nich dane w ustalonym formacie i generuje raport dla kierownictwa.

Automatyzacja zadań z użyciem LLM. W aplikacji automatyzującej model językowy przestaje być rozmówcą, a staje się **komponentem programu** – wyspecjalizowaną „funkcją", która przyjmuje tekst nieustrukturyzowany (opinię, zgłoszenie, fragment dokumentacji) i zwraca wynik w ustalonym formacie. Cały przepływ – skąd pochodzą dane, w jakiej kolejności są przetwarzane, co dzieje się z wynikami – definiuje programista w kodzie. Modele językowe otwierają tu nową klasę automatyzacji: zadania, których nie dało się zautomatyzować klasycznym kodem, bo wymagają rozumienia języka naturalnego (klasyfikacja treści, ekstrakcja informacji, streszczanie, redagowanie).

Automatyzacja a systemy agentowe. W aplikacji automatyzującej sekwencję kroków wyznacza **program**: przepływ jest z góry zdefiniowany, deterministyczny i taki sam dla każdego rekordu, a model wykonuje pojedyncze, wąsko określone zadania. W **systemach agentowych** to model planuje i wybiera kolejne akcje, decydując samodzielnie o przebiegu pracy. Podejście potokowe (ang. pipeline) ma przy tym ważne zalety praktyczne: jest przewidywalne, łatwe do przetestowania i tanie w utrzymaniu – i dlatego w zastosowaniach produkcyjnych zaleca się je wszędzie tam, gdzie zadanie da się opisać stałą sekwencją kroków.

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

Ustrukturyzowane wyjście – format JSON. Aby program mógł niezawodnie przetwarzać odpowiedzi modelu, muszą one mieć stały, maszynowo czytelny format. Standardem jest JSON: prompt precyzyjnie definiuje wymagane pola i dopuszczalne wartości („zwróć WYŁĄCZNIE poprawny JSON o polach…"), a program parsuje odpowiedź (`json.loads`) i **waliduje** ją – sprawdza obecność pól i poprawność wartości. Taki schemat pełni rolę uzgodnionego formatu między modelem a programem. Ponieważ zależy nam na powtarzalności, a nie kreatywności, zadania ekstrakcji i klasyfikacji wykonuje się przy `temperature=0.0` (Laboratorium 2).

Przetwarzanie wsadowe (batch). Automat przetwarza wiele rekordów w pętli, dlatego musi być odporny na pojedyncze niepowodzenia: błąd parsowania JSON czy chwilowy błąd API nie może przerywać całego przebiegu – rekord trafia do listy błędów, a przetwarzanie biegnie dalej. Trzeba też uwzględnić **limity zapytań** darmowej warstwy API (błąd 429 – Laboratoria 2 i 9): między wywołaniami stosuje się krótką pauzę. Koszt przetwarzania rośnie liniowo z liczbą rekordów (tokeny jednego rekordu × liczba rekordów), więc przed uruchomieniem automatu na dużych danych szacuje się koszt metodami z Laboratorium 2.

Podział odpowiedzialności: liczby liczy program, język pisze model. Modele językowe zawodnie wykonują arytmetykę i agregacje na wielu rekordach – potrafią pominąć wiersz albo „dopowiedzieć" wartość. Dlatego w dobrze zaprojektowanym automacie statystyki (zliczenia, sumy, listy) wyznacza deterministycznie program, a model otrzymuje gotowe, policzone dane i odpowiada wyłącznie za warstwę językową raportu. Drugą zasadą jest **człowiek w pętli** (ang. human-in-the-loop): wyniki wpływające na decyzje – np. zgłoszenia o wysokim priorytecie – przegląda człowiek, zanim zostaną wykorzystane. Automatyzacja nie zdejmuje z autora odpowiedzialności za skutki działania aplikacji.

Cel ćwiczenia/laboratorium

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

Głównym celem laboratorium jest **samodzielne zbudowanie od zera aplikacji wspomaganej AI, automatyzującej analizę danych tekstowych**: od formatu JSON, przez przetwarzanie wsadowe z obsługą błędów, po raport i prosty interfejs. Po ukończeniu zajęć student potrafi:

- wyjaśnić różnicę między aplikacją automatyzującą (przepływ definiuje program) a systemem agentowym (przepływ planuje model) oraz wskazać zalety podejścia potokowego;
- zaprojektować prompt z formatem JSON: wymagane pola, dopuszczalne wartości, zakaz dodatkowego tekstu;
- zaimplementować funkcję analityczną z parsowaniem i walidacją odpowiedzi modelu oraz temperaturą 0.0;
- zbudować przetwarzanie wsadowe odporne na błędy pojedynczych rekordów, z pauzą między wywołaniami i logowaniem postępu;
- rozdzielić odpowiedzialności w raportowaniu: statystyki liczone przez program, tekst raportu generowany przez model;
- zbudować prosty interfejs automatu w Streamlit (wgrywanie CSV, pasek postępu, tabela wyników, pobieranie raportu);
- oszacować koszt i niezawodność automatyzacji oraz wskazać miejsca wymagające udziału człowieka (human-in-the-loop).

Laboratorium wykorzystuje umiejętności z Laboratoriów 1–2 (API Groq, tokeny i koszty) oraz 8–9 (struktura projektu, Streamlit). Aplikacja tworzona jest w nowym katalogu projektu.

Instrukcja laboratoryjna/ćwiczeniowa

Zadanie 12.1. Nowy projekt automatyzacji i dane wejściowe

Docelowa struktura aplikacji jest nastepująca:

```
automat_opinie/
|-- src/
|   |-- analiza.py          # LLM jako funkcja (Zadanie 12.2)
```

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

```
|   |-- przetwarzaj.py     # przetwarzanie wsadowe (Zadanie 12.3)
|   |-- raport.py          # raport z wynikow (Zadanie 12.4)
|   `-- app_automat.py     # interfejs Streamlit (Zadanie 12.5)
|-- dane/
|   `-- opinie.csv         # dane wejsciowe
|-- wyniki/                # pliki wynikowe automatu
|-- .env                   # klucz API (poza repozytorium!)
|-- .gitignore
`-- requirements.txt
```

Treść zadania:

1. Utwórz nowy katalog roboczy, np. `C:\GenAI_Lab\automat_opinie`, otwórz go w VS Code i utwórz podkatalogi `src`, dane oraz `wyniki`.
2. Utwórz i aktywuj środowisko wirtualne, zainstaluj biblioteki i utrwal zależności:

```
python -m venv .venv
.venv\Scripts\Activate.ps1      # Windows PowerShell
pip install -U groq python-dotenv streamlit
pip freeze > requirements.txt
```

3. Wygeneruj w konsoli GroqCloud nowy klucz API dla projektu (zasady z Laboratorium 8), zapisz go w pliku `.env` (`GROQ_API_KEY=twoj_klucz`) i utwórz `.gitignore` z wpisami `.env`, `.venv/`, `__pycache__/`, `wyniki/`.
4. W katalogu dane utwórz plik `opinie.csv` (separator średnika, kodowanie UTF-8) o następującej treści:

```
id;opinia
1;Zamówienie dotarło dzień wcześniej, świetnie zapakowane.
2;Kurier zostawił paczkę na deszczu, pudełko przemokło.
3;Laptop działa, ale wentylator głośno szumi przy pracy.
4;Płatność pobrana dwa razy, na zwrot czekam drugi tydzień!
5;Miła obsługa na czacie, szybko pomogli dobrać ładowarkę.
6;Produkt zgodny z opisem, opakowanie przeciętne, jest ok.
7;Słuchawki padły po tygodniu, żądam zwrotu pieniędzy.
8;Nikt nie odpisuje na maile od pięciu dni, fatalny kontakt.
9;Dostawa punktualna, kurier zadzwonił przed przyjazdem.
10;Strona zawiesza się przy płatności BLIK, ledwo skończyłem.
```

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

Zadanie 12.2. LLM jako funkcja programu – format JSON

Serce automatu: funkcja `analizuj_opinie()`, która przyjmuje tekst opinii i zwraca słownik o stałych polach. Prompt wymusza czysty JSON i definiuje dopuszczalne wartości; program parsuje odpowiedź i waliduje format. Zwróć uwagę na `temperature=0.0` – w ekstrakcji liczy się powtarzalność, nie kreatywność.

Treść zadania:

1. W katalogu `src` utwórz plik `analiza.py` o następującej treści:

```
import json
import os

from dotenv import load_dotenv
from groq import Groq

load_dotenv()

MODEL = 'llama-3.3-70b-versatile'
KATEGORIE = ['DOSTAWA', 'PRODUKT', 'OBSLUGA', 'PLATNOSC', 'INNE']

PROMPT_ANALIZY = (
    'Przeanalizuj opinię klienta sklepu internetowego. '
    'Zwróć WYŁĄCZNIE poprawny JSON, bez żadnego innego tekstu, '
    'o dokładnie tych polach: '
    '{"sentyment": "POZYTYWNY" | "NEUTRALNY" | "NEGATYWNY", '
    '"kategoria": "DOSTAWA" | "PRODUKT" | "OBSLUGA" | '
    '"PLATNOSC" | "INNE", '
    '"priorytet": "WYSOKI" | "NORMALNY", '
    '"streszczenie": "jedno zdanie po polsku"}. '
    'Priorytet WYSOKI nadawaj, gdy klient zgłasza poważny '
    'problem lub zapowiada rezygnację z zakupów.'
)

def get_client():
    """Tworzy klienta Groq na podstawie klucza z pliku .env."""
    api_key = os.getenv('GROQ_API_KEY')
```

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

```
    if not api_key:
        raise RuntimeError('Brak klucza GROQ_API_KEY w .env.')
    return Groq(api_key=api_key)

def analizuj_opinie(client, opinia):
    """Analiza jednej opinii; zwraca slownik zgodny z formatem
    albo zglasza ValueError przy naruszeniu formatu."""
    response = client.chat.completions.create(
        model=MODEL,
        messages=[{'role': 'user',
                   'content': PROMPT_ANALIZY
                   + '\nOpinia: """' + opinia + '"""'}],
        temperature=0.0,
        max_tokens=300,
    )
    tekst = response.choices[0].message.content.strip()
    try:
        dane = json.loads(tekst)
    except json.JSONDecodeError:
        raise ValueError(f'Niepoprawny JSON: {tekst[:80]}')
    # walidacja formatu
    for pole in ('sentyment', 'kategoria', 'priorytet',
                 'streszczenie'):
        if pole not in dane:
            raise ValueError(f'Brak pola: {pole}')
    if dane['kategoria'] not in KATEGORIE:
        dane['kategoria'] = 'INNE'
    return dane
```

2. Przetestuj funkcję na jednej opinii – w katalogu `src` utwórz plik `test_analiza.py` i uruchom go z katalogu głównego poleceniem `python src/test_analiza.py`:

```
import sys

from analiza import analizuj_opinie, get_client

sys.stdout.reconfigure(encoding='utf-8')
```

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

```
client = get_client()
OPINIA = ('Kurier zostawił paczkę na deszczu, pudełko całe mokre.
'
          'Jeśli to się powtórzy, rezygnuję z zakupów.')
wynik = analizuj_opinie(client, OPINIA)
print(wynik)
print('Sentyment:', wynik['sentyment'],
      '| Priorytet:', wynik['priorytet'])
```

3. Sprawdź odpowiedź: czy JSON został poprawnie sparsowany, czy kategoria to DOSTAWA, a priorytet WYSOKI (klient zapowiada rezygnację)? Uruchom test dwukrotnie – przy temperaturze 0.0 wyniki powinny być identyczne lub niemal identyczne.

P: Dołącz plik `analiza.py` oraz zrzut ekranu dwóch uruchomień testu. Wyjaśnij, które elementy promptu tworzą „format" z programem i co by się stało, gdyby model dopisał zdanie komentarza przed JSON-em.

Zadanie 12.3. Przetwarzanie wsadowe z obsługą błędów

Automat przetworzy cały plik CSV w pętli: każda opinia trafia do funkcji analitycznej, wynik do listy, a błąd pojedynczego rekordu – do listy błędów, bez przerywania przebiegu. Pauza między wywołaniami chroni przed limitem zapytań darmowej warstwy (błąd 429).

Treść zadania:

1. W katalogu `src` utwórz plik `przetwarzaj.py` o następującej treści:

```
import csv
import sys
import time

from groq import APIError

from analiza import analizuj_opinie, get_client
```

```
sys.stdout.reconfigure(encoding='utf-8')

PLIK_WE = 'dane/opinie.csv'
PLIK_WY = 'wyniki/wyniki.csv'
PAUZA = 1.0   # sekundy miedzy wywolaniami (limity API)

client = get_client()
wyniki, bledy = [], []

with open(PLIK_WE, encoding='utf-8') as f:
    wiersze = list(csv.DictReader(f, delimiter=';'))

for i, wiersz in enumerate(wiersze, start=1):
    print(f'[{i}/{len(wiersze)}] analiza opinii '
          f'id={wiersz["id"]}...')
    try:
        dane = analizuj_opinie(client, wiersz['opinia'])
        dane['id'] = wiersz['id']
        wyniki.append(dane)
    except (APIError, ValueError, RuntimeError) as e:
        print(f'  BLAD: {e}')
        bledy.append({'id': wiersz['id'], 'blad': str(e)})
    time.sleep(PAUZA)

with open(PLIK_WY, 'w', encoding='utf-8', newline='') as f:
    pola = ['id', 'sentyment', 'kategoria', 'priorytet',
            'streszczenie']
    writer = csv.DictWriter(f, fieldnames=pola, delimiter=';')
    writer.writeheader()
    writer.writerows(wyniki)

print(f'\nZapisano {len(wyniki)} wyników do {PLIK_WY}; '
      f'błędów: {len(bledy)}')
```

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

2. Uruchom automat z katalogu głównego poleceniem `python src/przetwarzaj.py` i obserwuj log postępu. Przetworzenie 10 opinii z pauzami zajmie ok. 1–2 minut.
3. Otwórz plik `wyniki/wyniki.csv` (np. w VS Code lub arkuszu kalkulacyjnym) i oceń jakość klasyfikacji: czy kategorie i sentymenty zgadzają się z Twoją oceną? Zanotuj rozbieżności.
4. **Test odporności.** Dopisz na końcu pliku `dane/opinie.csv` wiersz `11;` (pusta opinia), uruchom automat ponownie i sprawdź, że błąd jednego rekordu nie przerwał przetwarzania pozostałych. Usuń testowy wiersz.

P: Dołącz plik `przetwarzaj.py`, zrzut ekranu logu przebiegu oraz zawartość `wyniki.csv`. Odpowiedz: (1) dlaczego błędy pojedynczych rekordów zbiera się do listy zamiast przerywać program? (2) po co pauza między wywołaniami i od czego zależy jej długość?

Zadanie 12.4. Raport: statystyki z programu, język z modelu

Raport powstaje w dwóch krokach o jasno rozdzielonych odpowiedzialnościach: statystyki (zliczenia, lista zgłoszeń o wysokim priorytecie) wyznacza deterministycznie program, a model – na podstawie wyłącznie policzonych danych – pisze zwięzłe podsumowanie w stylu biznesowym.

Treść zadania:

1. W katalogu `src` utwórz plik `raport.py` o następującej treści:

```
import csv
import sys
from collections import Counter

from analiza import MODEL, get_client

sys.stdout.reconfigure(encoding='utf-8')

# 1) statystyki liczy PROGRAM (deterministycznie)
with open('wyniki/wyniki.csv', encoding='utf-8') as f:
    wyniki = list(csv.DictReader(f, delimiter=';'))
```


**Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00**

```
sentymenty = Counter(w['sentyment'] for w in wyniki)
kategorie = Counter(w['kategoria'] for w in wyniki)
wysokie = [w for w in wyniki if w['priorytet'] == 'WYSOKI']

statystyki = (
    f'Liczba opinii: {len(wyniki)}\n'
    f'Sentyment: {dict(sentymenty)}\n'
    f'Kategorie: {dict(kategorie)}\n'
    'Zgłoszenia o wysokim priorytecie:\n' +
    '\n'.join(f'- id={w["id"]}: {w["streszczenie"]}'
              for w in wysokie)
)
print(statystyki)

# 2) tekst raportu pisze MODEL - tylko z policzonych danych
client = get_client()
response = client.chat.completions.create(
    model=MODEL,
    messages=[{'role': 'user', 'content':
        'Na podstawie poniższych statystyk napisz krótki raport '
        '(3 akapity, styl biznesowy, po polsku) dla kierownika '
        'sklepu: ogólny obraz, główne problemy, rekomendacje. '
        'Nie wymyślaj danych spoza statystyk.\n"""\n'
        + statystyki + '\n"""'}],
    temperature=0.3,
    max_tokens=700,
)
raport = response.choices[0].message.content

with open('wyniki/raport.md', 'w', encoding='utf-8') as f:
    f.write('# Raport z analizy opinii klientów\n\n')
    f.write('## Statystyki\n\n' + statystyki + '\n\n')
    f.write('## Podsumowanie\n\n' + raport + '\n')

print('\nZapisano wyniki/raport.md')
```

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

2. Uruchom skrypt poleceniem `python src/raport.py` i otwórz plik `wyniki/raport.md`. Zweryfikuj raport względem statystyk: czy wszystkie liczby w tekście zgadzają się z sekcją statystyk? Czy rekomendacje odnoszą się do rzeczywiście policzonych problemów?
3. **Kontrprzykład.** Aby zobaczyć, dlaczego agregacji nie powierza się modelowi, wyślij w dowolnym interfejsie (np. `test_analiza.py` lub czacie webowym) surową listę 10 opinii z prośbą „policz, ile jest opinii negatywnych i wypisz ich identyfikatory". Porównaj wynik z własnym zliczeniem – modele często mylą się w takich zadaniach, zwłaszcza przy dłuższych listach.

P: Dołącz plik `raport.md` oraz wynik kontrprzykładu. Sformułuj zasadę podziału odpowiedzialności między program a model w aplikacjach automatyzujących i uzasadnij ją obserwacjami.

Zadanie 12.5. Interfejs automatu w Streamlit

Automat otrzyma prosty interfejs dla użytkownika nietechnicznego: wgranie pliku CSV, przycisk uruchomienia, pasek postępu, tabela wyników i pobranie pliku wynikowego. To inny wzorzec niż chatbot z Laboratorium 9 – jedna akcja zamiast rozmowy.

Treść zadania:

1. W katalogu `src` utwórz plik `app_automat.py` o następującej treści:

```
import csv
import io
import time

import streamlit as st
from groq import APIError

from analiza import analizuj_opinie, get_client
```

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

**Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00**

```
st.set_page_config(page_title='Automat analizy opinii')
st.title('Automat do analizy opinii klientów')

plik = st.file_uploader('Wgraj plik CSV (kolumny: id;opinia):',
                        type=['csv'])

if plik is not None and st.button('Przetwórz'):
    tekst = plik.getvalue().decode('utf-8')
    wiersze = list(csv.DictReader(io.StringIO(tekst),
                                  delimiter=';'))
    client = get_client()
    pasek = st.progress(0.0, text='Przetwarzanie...')
    wyniki, bledy = [], 0
    for i, wiersz in enumerate(wiersze, start=1):
        try:
            dane = analizuj_opinie(client, wiersz['opinia'])
            dane['id'] = wiersz['id']
            wyniki.append(dane)
        except (APIError, ValueError) as e:
            bledy += 1
        pasek.progress(i / len(wiersze),
                       text=f'Przetworzono {i}/{len(wiersze)}')
        time.sleep(1.0)
    st.success(f'Gotowe: {len(wyniki)} wyników, '
               f'błędów: {bledy}')
    st.dataframe(wyniki)

    bufor = io.StringIO()
    writer = csv.DictWriter(bufor, delimiter=';', fieldnames=[
        'id', 'sentyment', 'kategoria', 'priorytet',
        'streszczenie'])
    writer.writeheader()
    writer.writerows(wyniki)
    st.download_button('Pobierz wyniki (CSV)', bufor.getvalue(),
                       file_name='wyniki.csv')
```

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

2. Uruchom aplikację poleceniem `streamlit run src/app_automat.py`, wgraj plik `dane/opinie.csv` i kliknij **Przetwórz**. Obserwuj pasek postępu, a po zakończeniu – tabelę wyników.
3. Pobierz plik wynikowy przyciskiem i porównaj go z wynikami z zadania 12.3.
4. Zastanów się nad różnicą wzorców: chatbot (Laboratorium 9) utrzymywał historię w `session_state` i reagował na każdą wiadomość; automat wykonuje jedną, dłuższą akcję na przycisk. Które elementy (pasek postępu, komunikaty, pobieranie pliku) zastępują tu rozmowę?

P: Dołącz plik `app_automat.py` oraz zrzuty ekranu: w trakcie przetwarzania (pasek postępu) i po zakończeniu (tabela + przycisk pobierania).

Zadanie 12.6. Eksperyment: niezawodność, koszty i granice automatyzacji

Eksperyment ocenia automat jak produkt: jak często zawodzi, ile kosztuje na większej skali i gdzie potrzebny jest człowiek. Wyniki dokumentuje tabela 12.1.

Treść zadania:

1. **Scenariusz A – niezawodność formatu.** Zanotuj z przebiegu zadania 12.3 liczbę rekordów przetworzonych poprawnie i liczbę błędów. Następnie w pliku analiza.py zmień na próbę temperature=0.0 na 1.2, uruchom przetwarzaj.py ponownie i porównaj liczbę błędów parsowania oraz spójność wyników. Przywróć wartość 0.0.
2. **Scenariusz B – koszt na skali.** Odczytaj z jednej odpowiedzi pole usage (metoda z Laboratorium 2: prompt_tokens, completion_tokens) i oszacuj koszt przetworzenia 10 000 opinii w płatnej warstwie API (ceny modelu z cennika Groq). Oszacuj też czas przebiegu przy pauzie 1 s.
3. **Scenariusz C – człowiek w pętli.** Przejrzyj ręcznie wszystkie rekordy z priorytetem WYSOKI w wyniki.csv: czy każdy z nich rzeczywiście wymaga pilnej reakcji? Czy automat pominął jakiś poważny problem (fałszywie NORMALNY)? Zanotuj pomyłki.
4. Uzupełnij tabelę 12.1 na podstawie obserwacji.

Fundusze Europejskie dla Rozwoju Społecznego | Rzeczpospolita Polska | Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

Tabela 12.1. Ocena niezawodności, kosztów i granic automatu

Scenariusz	Pomiar / obserwacja	Wniosek (2–3 zdania)
A – temp. 0.0		
A – temp. 1.2		
B – koszt i czas (10 000 rekordów)		
C – przegląd priorytetów WYSOKI		

P: Zamieść uzupełnioną tabelę 12.1 i obliczenia ze scenariusza B. Sformułuj wnioski: (1) dlaczego niska temperatura jest warunkiem niezawodności formatu JSON? (2) które etapy automatu powinny pozostać pod nadzorem człowieka i dlaczego? (3) czym ta aplikacja różni się od systemu agentowego – kto steruje przepływem?

Ćwiczenia samodzielne

Ćwiczenie 12.1 (podstawowe). Rozszerzenie formatu

Rozszerz format JSON o pole `propozycja_odpowiedzi` – jedno uprzejme zdanie odpowiedzi do klienta, dopasowane do treści opinii. Zaktualizuj prompt, walidację, listę pól w plikach wynikowych i tabelę w interfejsie. W sprawozdaniu zamieść zaktualizowany prompt i 3 przykładowe propozycje odpowiedzi wraz z oceną, czy nadają się do wysłania bez poprawek.

Ćwiczenie 12.2 (średnie). Drugi scenariusz danych

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

Sprawdź uniwersalność potoku: przygotuj własny plik CSV z 8–10 rekordami z innej dziedziny – zgłoszenia działu IT (scenariusz informatyczny) albo uwagi z przeglądów maszyn (scenariusz techniczny) – i dostosuj wyłącznie prompt oraz listę kategorii w `analiza.py`, nie zmieniając kodu pętli, raportu ani interfejsu. W sprawozdaniu zamieść nowy prompt, dane i wyniki oraz wniosek: które elementy aplikacji okazały się niezależne od dziedziny danych?

Ćwiczenie 12.3 (średnie). Automat według harmonogramu

Uruchom automat cyklicznie, bez udziału człowieka: zmodyfikuj `raport.py` tak, aby nazwa pliku raportu zawierała bieżącą datę (moduł `datetime`, np. `raport_2026-07-10.md`), a następnie skonfiguruj automatyczne uruchamianie skryptu przetwarzania i raportu raz dziennie: w Windows – Harmonogram zadań (Task Scheduler), w Linux/macOS – cron. W sprawozdaniu zamieść zrzut ekranu konfiguracji harmonogramu i wygenerowany plik raportu z datą oraz odpowiedz: jakie ryzyka niesie automat działający bez nadzoru i jak je ograniczyć (logi, limity, powiadomienia)?

Ćwiczenie 12.4 (trudne). Próg pewności i przegląd człowieka

Wbuduj mechanizm human-in-the-loop: rozszerz format o pole `pewnosc` (liczba 0–100 – jak bardzo model jest pewny klasyfikacji), dodaj w interfejsie suwak progu pewności i podziel wyniki na dwie tabele: zatwierdzone automatycznie (pewność ≥ próg) oraz wymagające przeglądu (poniżej progu). Tabelę przeglądu wyświetl w edytowalnym komponencie `st.data_editor`, pozwalając człowiekowi poprawić kategorię i priorytet przed eksportem. W sprawozdaniu pokaż działanie dla dwóch różnych progów i skomentuj: jak wybór progu równoważy oszczędność czasu i ryzyko błędnej klasyfikacji?

Ćwiczenie 12.5 (trudne). Powtórzenie wiadomości

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

Na podstawie materiałów zdobytych na laboratoriach zaprojektuj i utwórz aplikację wspomaganą AI. Temat dowolny po ustaleniu z prowadzącym zajęcia.

Pytania pomocnicze

1. Czym aplikacja automatyzująca różni się od chatbota z Laboratorium 9 pod względem roli użytkownika i przepływu sterowania?
2. Wyjaśnij granicę między automatyzacją (potokiem) a systemem agentowym: kto wyznacza sekwencję kroków w każdym z podejść? Jakie zalety praktyczne ma potok?
3. Na czym polega format JSON między modelem a programem i jakie elementy promptu go tworzą?
4. Dlaczego zadania ekstrakcji i klasyfikacji wykonuje się przy temperaturze 0.0? Co pokazał scenariusz A eksperymentu?
5. Dlaczego walidacja odpowiedzi modelu jest konieczna, skoro prompt „zabrania" innego formatu?
6. Jak automat obsługuje błędy w pojedynczych rekordach i dlaczego nie przerywa całego przebiegu? Po co pauza między wywołaniami?
7. Dlaczego program liczy statystyki i agregacje, a nie model?
8. Jak oszacować koszt i czas automatycznego przetworzenia dużego zbioru rekordów przed jego uruchomieniem?
9. Czym jest podejście human-in-the-loop i które etapy zbudowanego automatu powinny mu podlegać?

Podsumowanie

Po ukończeniu laboratorium student dysponuje kompletną, zbudowaną od zera aplikacją automatyzującą analizę danych tekstowych. W szczególności:

1. rozróżnia aplikacje automatyzujące (przepływ definiuje program) od systemów agentowych (przepływ planuje model) i zna zalety podejścia potokowego;

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

2. projektuje format JSON w prompcie oraz implementuje parsowanie i walidację odpowiedzi modelu przy temperaturze 0.0;
3. buduje przetwarzanie wsadowe odporne na błędy, z pauzami chroniącymi przed limitami API i logowaniem postępu;
4. stosuje podział odpowiedzialności: statystyki z programu, warstwa językowa raportu z modelu;
5. udostępnia automat użytkownikowi nietechnicznemu przez prosty interfejs Streamlit;
6. ocenia niezawodność i koszty automatyzacji oraz wskazuje miejsca wymagające nadzoru człowieka.

WSKAZÓWKA: dopisz do biblioteki promptów (plik `prompts.md`) prompt analityczny z formatem JSON wraz z listą pól i dopuszczalnych wartości – takie prompty-formaty są wielokrotnego użytku: wymiana dziedziny danych (ćwiczenie 12.2) sprowadza się do zmiany kategorii i opisu zadania. Tematyka samodzielnego planowania działań przez model (systemy agentowe) rozwijana jest w ramach osobnego przedmiotu.

UWAGA dotyczqca bezpieczeństwa i danych: klucza API nie wolno udostępniać publicznie, a plik `.env` musi być ujęty w `.gitignore`. Rzeczywiste opinie i zgłoszenia klientów mogą zawierać dane osobowe – przed wysłaniem do zewnętrznego API należy je zanonimizować (usunąć imiona, adresy, numery) albo użyć modelu lokalnego (Ollama, Laboratorium 2). Wyników automatu nie wolno wykorzystywać do decyzji wobec osób bez weryfikacji przez człowieka.

Literatura

[1]. Anthropic, Building Effective Agents (rozróżnienie: przepływy zdefiniowane programowo a agenci), https://www.anthropic.com/research/building-effective-agents (dostęp: 2026).
[2]. Groq, Text/Chat Generation – dokumentacja API (pole usage), https://console.groq.com/docs/text-chat (dostęp: 2026).

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00

[3]. Groq, Structured outputs – wymuszanie formatu odpowiedzi, https://console.groq.com/docs/structured-outputs (dostęp: 2026).
[4]. Groq, Rate limits (limity i przydziały zapytań), https://console.groq.com/docs/rate-limits (dostęp: 2026).
[5]. Groq, Pricing (cennik), https://groq.com/pricing/ (dostęp: 2026).
[6]. Python Software Foundation, moduł csv – dokumentacja, https://docs.python.org/3/library/csv.html (dostęp: 2026).
[7]. Streamlit, st.progress, st.dataframe, st.download_button – dokumentacja API, https://docs.streamlit.io/develop/api-reference (dostęp: 2026).
[8]. Phoenix J., Taylor M., Skuteczna inżynieria promptów. Przyszłościowe rozwiązania dla rzetelnych wyników generatywnej AI, Helion, Gliwice 2025.
[9]. Mizrahi G., Inżynieria promptów bez tajemnic. Sztuka kreatywnego generowania tekstów, Helion, Gliwice 2025.

Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

Projekt współfinansowany ze środków Unii Europejskiej w ramach:
FUNDUSZE EUROPEJSKIE DLA ROZWOJU SPOŁECZNEGO 2021-2027 (FERS)
"POLLUB z nami nowoczesne technologie" FERS.01.05-IP.08-0319/23-00